

CYBER ELIAS ACADEMY

TEACHING MANUAL

Web Development

Session Roadmap · Lesson Notes · Concept Diagrams · Answer Keys

READY TO TEACH

© 2026 Cyber Elias Academy · cea.ng

Table of Contents

Course Roadmap

Session 01 — JAVASCRIPT SETUP, VARIABLES AND THE CONSOLE

Session 02 — MAKING DECISIONS: FUNCTIONS, CONDITIONS AND IF/ELSE

Session 03 — ARRAYS, OBJECTS AND LOOPS

Session 04 — THE DOM: SELECTING AND CHANGING THE PAGE

Session 05 — EVENTS AND FORMS BASICS

Session 06 — FORM VALIDATION AND LOCAL STORAGE

Session 07 — FETCHING DATA AND JSON

Session 08 — BUILDING THE SHOPPING CART WITH LOCALSTORAGE

Session 09 — MULTI-PAGE INTEGRATION AND RESPONSIVE DESIGN

Session 10 — DEBUGGING, VALIDATING AND OPTIMISING

Session 11 — PUBLISHING YOUR SITE WITH GITHUB PAGES

Session 12 — CAPSTONE BUILD, DEPLOY AND PRESENT

OVERVIEW

Course Roadmap

What we cover in every session

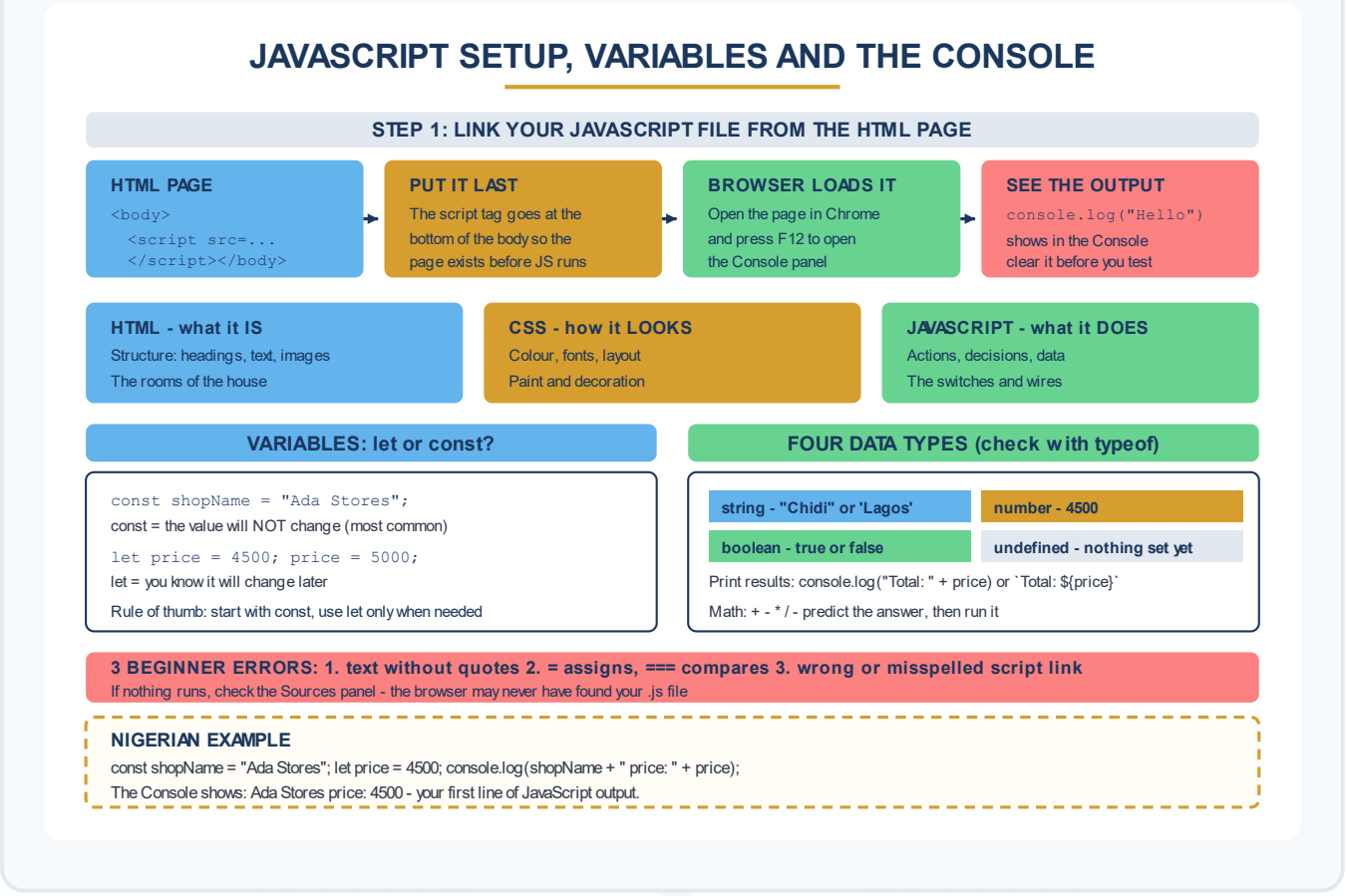
#	SESSION TOPIC	WEEK / MODULE	DURATION
01	JAVASCRIPT SETUP, VARIABLES AND THE CONSOLE	Week 1 — Module 1: JavaScript Foundations	105 minutes
02	MAKING DECISIONS: FUNCTIONS, CONDITIONS AND IF/ELSE	Week 1 — Module 1: JavaScript Foundations	105 minutes
03	ARRAYS, OBJECTS AND LOOPS	Week 1 — Module 1: JavaScript Foundations	105 minutes
04	THE DOM: SELECTING AND CHANGING THE PAGE	Week 2 — Module 2: DOM & Events	105 minutes
05	EVENTS AND FORMS BASICS	Week 2 — Module 2: DOM and Events	105 minutes
06	FORM VALIDATION AND LOCAL STORAGE	Week 2 — Module 2: DOM and Events (storage introduction)	105 minutes
07	FETCHING DATA AND JSON	Week 3 — Module 3: Data & Real Apps	105 minutes
08	BUILDING THE SHOPPING CART WITH LOCALSTORAGE	Week 3 — Module 3: Data & Real Apps	105 minutes
09	MULTI-PAGE INTEGRATION AND RESPONSIVE DESIGN	Week 3 — Module 3: Data and App	105 minutes
10	DEBUGGING, VALIDATING AND OPTIMISING	Week 4 — Module 4: Polish and Deploy	105 minutes
11	PUBLISHING YOUR SITE WITH GITHUB PAGES	Week 4 — Module 4: Polish and Deploy	105 minutes
12	CAPSTONE BUILD, DEPLOY AND PRESENT	Week 4 — Module 4: Polish and Deploy	105 minutes

How to use this book: Each session below is a complete, ready-to-teach lesson note. Section A tells you what to prepare, B lists the outcomes, D–F are the teaching content, G–H are your demos and practicals, and the knowledge check with answer key is at the end. The concept diagram for each session is embedded as a visual aid.

SESSION 01 OF 12

JAVASCRIPT SETUP, VARIABLES AND THE CONSOLE

CONCEPT DIAGRAM — SESSION 01



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 1 — JAVASCRIPT SETUP, VARIABLES AND THE CONSOLE

A. SESSION IDENTIFICATION

Item	Detail
Session number	1 of 12
Session title	JavaScript Setup, Variables and the Console
Week / Module	Week 1 — Module 1: JavaScript Foundations
Recommended duration	105 minutes

Item	Detail
Difficulty level	Beginner (first contact with programming)
Prerequisite knowledge	HTML basics (tags, elements, attributes) and CSS basics (selectors, colours, simple layout). If a learner cannot write a basic HTML page with a heading and a paragraph, give them the 20-minute refresher in Appendix 1A before this session.
Connection to previous work	In the Web Design course learners built a static page — it looked good but did nothing when clicked. This session explains *why* pages can react, and introduces JavaScript, the language that makes a page alive.
Preparation before class	Academy PCs on and logged in; Google Chrome installed; Visual Studio Code installed; a practice folder per learner on the Desktop named `webdev-practice`; ensure the projector/screen shares the instructor's Chrome DevTools.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Explain in their own words** the difference between HTML, CSS and JavaScript using a real-world comparison.
2. **Write a valid HTML page** that links an external JavaScript file using the `<script>` tag — without copying the instructor.
3. **Open the Chrome DevTools Console** and read, clear and use it to view output.
4. **Declare variables** using `let` and `const`, and justify which one to use in a given situation.
5. **Identify the four primitive types** used this week — string, number, boolean and `undefined` — and use `typeof` to inspect them.
6. **Print output** to the console using `console.log()`, including combining text and variables with both concatenation (`+`) and template literals (`${ }`).
7. **Perform arithmetic** with `+ - * /` on number variables and predict the result before running it.
8. **Diagnose three beginner errors** — an unquoted string, confusing `=` with `==`, and a missing/misspelled script link — and correct them.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). A shared computer between two learners is acceptable; pair them so one types and one reads errors aloud, then swap halfway.
- Instructor computer connected to a projector or large screen.

Software

- Google Chrome (any recent version). The Console and DevTools described here exist in all modern versions; the wording of the tabs is stable.
- Visual Studio Code (VS Code). Any version from 2022 onwards works.
- Windows 10 or Windows 11.

Accounts

- None required. All work this session is offline.

Sample files

- The instructor prepares and shares a file `starter/index.html` and an empty `starter/js/app.js` (contents given in Section G). Learners may also type these from scratch — recommended for muscle memory.

Worksheets

- Worksheet 1.1 — "Name the Layer" (printed, 1 page).
- Worksheet 1.2 — "My First Variables" (in the learner workbook, Part 3).

Tools

- Chrome DevTools (built into Chrome — press `F12`).

Internet

- Not required for this session. If the academy has internet, the instructor may optionally show the MDN page for `console.log` at the very end, but do not depend on it.

Alternative for learners who lack a facility

- Any laptop with Chrome and VS Code works. If a learner has only a phone, pair them with a desktop user for the session and ask them to take photos of the code and practise typing the examples into a free online editor (e.g., a phone browser tab) afterwards.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open a simple product page the class already recognises from Web Design — for example a page showing "*Hair Cream — ~~₦2,500~~*" with a styled "Add to Cart" button. Do not open any code yet; show only the browser.

What the instructor says:

> *"Look at this page. It has a heading, a price and a nice button. If I click the button right now, what happens? … Nothing. The page just sits there. But when you shop on Jumia or pay with your bank app, you click a button and the number changes, a message appears, money moves. So what is the difference between this page and a page that reacts?*
>
> *The difference is: this page was built with HTML for the content and CSS for the look. But it has no brain. Today we add the brain. Its name is JavaScript."*

Expected learner response: Learners may say "we need a backend", "the server does it", or stay silent. Accept all answers. Confirm that some reactions do involve a server, but many happen right in the browser — and that is what they will do today.

Then demonstrate the hook: In the projector browser, press `F12`, click the **Console** tab, type the following and press Enter:

```
2 + 3
```

The console prints `5`. Say:

> *"This black panel is the Console. It is where JavaScript talks to us. In a moment, you will each type your own instructions here and watch the computer obey. That is the beginning of programming."*

Why this opener works: It creates a gap between what learners already built (static pages) and what they now want to build (interactive pages), and it instantly gives them a taste of running code, so the session starts with doing, not listening.

E. PREREQUISITE REVIEW (7 minutes)

Run this quickly as an oral round-robin. Put the three questions on the projector.

Question 1: "In HTML, what is the tag for the largest heading?"

- Expected answer: `<h1>`.
- Common gap: learners remember the concept but write `<H1>` or forget to close it. Remind them HTML tags are usually lowercase in this course and must be closed.

Question 2: "In CSS, what do we call `.price { color: red; }` — what is `.price`?"

- Expected answer: a selector (a class selector). It picks the element(s) with `class="price"` and applies the colour red.
- Common gap: confusing class selector `.` with id selector `#`. Re-state: dot = class, hash = id.

Question 3: "Where does the browser get the text the user sees — from the HTML file or the CSS file?"

- Expected answer: HTML. CSS only styles it.
- Common gap: some learners think CSS "holds" the text. Clarify with the analogy below.

Bridge to today (say this):

> *"Think of building a house. HTML is the blocks and walls — the structure and the things inside. CSS is the paint, tiles and curtains — the appearance. JavaScript is the electricity and plumbing — the things that do something when you turn them on. A house with only walls and paint is a static house: beautiful but dead. Today we switch on the electricity."*

If more than a third of the class cannot answer Question 1, spend 10 extra minutes on the Appendix 1A HTML refresher before continuing.

F. FULL LECTURE NOTES

F.1 What JavaScript actually is

The idea. A website is read by a piece of software called a **browser** (Chrome, Firefox, Edge, Safari). The browser understands three languages:

1. **HTML** — the structure and content. *What is on the page.*
2. **CSS** — the presentation. *How it looks.*
3. **JavaScript** — the behaviour. *How it responds.*

Plain-English definition. JavaScript is a **programming language** that runs inside the browser and lets a web page *do things*: react to clicks, check what a user typed, change text, remember choices, load data from the internet and show it on the page.

Why it exists. HTML and CSS cannot *decide*. They cannot say "if the password is wrong, show a warning" or "add this item's price to a running total". A language capable of making decisions and calculations was needed, and it had to run in the browser so the page could react immediately without waiting for a server. JavaScript is that language.

What it does. It stores information (variables), makes decisions (conditions), repeats work (loops), reacts to events (clicks, typing), and changes the page (the DOM — later in this course).

Real-life comparison. Think of a **printing press versus a computer**. A printed flyer (HTML + CSS) shows exactly one version of the information; it cannot change. A computer screen (HTML + CSS + JavaScript) can show different things depending on what you type, click or ask for. JavaScript is what turns the printed flyer into an interactive screen.

Important limitation / misconception. "JavaScript" and "Java" are **not the same language**. The names are similar by historical marketing accident. They are as different as *car* and *carpet*. Do not mix them up in a job interview.

How a learner recognises it. In any page's source code, JavaScript appears inside `<script> ... </script>` tags, or is linked via `<script src="..."></script>`, or is written in code blocks that use words like `let`, `function`, `if`, `const` and `console.log`.

F.2 How JavaScript gets onto a page — the `<script>` tag

The browser only runs JavaScript it has been told about. We connect a JavaScript file to an HTML page using the `<script>` tag.

Two ways to include JavaScript:

Way 1 — internally (inside the HTML file):

```
<script>
  console.log("Hello from inside the HTML file");
</script>
```

Way 2 — externally (separate `.js` file — the professional way, and the way we use in this course):

```
<script src="js/app.js"></script>
```

Why we use the external file (the professional way):

- It keeps HTML about structure and JavaScript about behaviour — each file does one job.
- The same `.js` file can be reused by many pages.
- The browser can **cache** (save) the `.js` file, so pages load faster on repeat visits.

Where to put the `<script>` tag — and why. Put it at the **bottom of the `<body>`**, just before `</body>`.

Reason: the browser reads a page from top to bottom. If the script is in the `<head>` and tries to find an element that appears later in the page, that element does not exist yet, and the script fails with errors like "*Cannot read properties of null*". Placing the script at the bottom means all the HTML elements already exist when the script runs.

> ***Deeper note for the curious learner (optional):*** *A more advanced option is to keep the script in the `<head>` and add the word `defer`, e.g. `<script src="js/app.js" defer></script>`. The `defer` attribute tells the browser: "download this file now, but wait until the whole page is built before running it." Both approaches are correct. In this course we use the bottom-of-body approach because it is the easiest to reason about as a beginner.*

Worked example — a complete, correct starting page:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CEA Web Dev Practice</title>
</head>
<body>
  <h1>My First JavaScript Page</h1>
  <p>Open the Console (press F12) to see the output.</p>

  <script src="js/app.js"></script>
</body>
</html>
```

And in `js/app.js`:

```
console.log("JavaScript is connected and running!");
```

What the learner sees: a normal web page with a heading and a sentence. In the Console, the line `JavaScript is connected and running!`.

Consequence of getting it wrong: If the path in `src` is misspelled or points to the wrong folder, the page still shows, but the Console stays empty and the Console tab shows a red error such as `app.js:1 Failed to load resource: net::ERR_FILE_NOT_FOUND`. We troubleshoot this in Section K.

F.3 The Console — your programming window

The idea. The **Console** is a panel in the browser where JavaScript can print messages, and where you can type JavaScript directly to test it.

Why it matters. As a beginner, you cannot "see" variables on the screen yet. The Console is how you look inside your program. It is your number one debugging tool for the whole course.

How to open it: Press `F12` in Chrome (or right-click the page → **Inspect** → **Console** tab). On some keyboards, `Ctrl + Shift + J` opens it directly.

What it does:

- **Shows output** from `console.log(...)` in your code.
- **Shows errors** in red — very important.
- **Lets you type commands** and press Enter to run them immediately, like a calculator and a scratchpad combined.

The command: `console.log()`

- `console` is the browser's message window.
- `.log()` means "write this message".

- Everything inside the parentheses is the *message*.

```
console.log("Hello, Ada");
console.log(2 + 2);
console.log("The year is", 2026);
```

Real-life comparison. `console.log` is like a **loudspeaker announcement**. You are not changing anything on the page yet; you are simply announcing "here is the value right now" so you can hear (see) it.

Common misconception. Learners think that if they do not see output, their code "is not working". Often the code ran fine, but the Console is filtered or closed, or the script is not connected. Section K covers the checks.

Try it live (instructor on projector): type in the Console directly:

```
console.log("I can type commands here too");
```

F.4 Variables — naming and storing information

The idea. A **variable** is a named box that holds a value so you can use it again later.

Plain-English definition. Instead of writing the price `2500` five times in a program, you store it in a box labelled `price` and use `price` everywhere. If the price changes, you change it in one place.

Why they exist. Programs need to remember information: a user's name, a total, whether someone is logged in. Variables are how a program remembers.

Analogy. A variable is like a **labelled jar in a kitchen**. The jar is labelled "sugar". You can put sugar in it, take sugar out, or replace the sugar with something else. The label stays the same; the contents can change (or be fixed — see `const` below).

Declaring a variable. "Declaring" means creating the box and giving it a value.

```
let name = "Adaeze";
let age = 22;
const school = "Cyber Elias Academy";
```

Line by line:

- `let` — "create a box that may change later".
- `name` — the label of the box.
- `=` — "put the value on the right into the box".
- `"Adaeze"` — the value, in quotes because it is text.
- `;` — the end of the instruction (a full stop for JavaScript).

`let` versus `const`:

Keyword	Meaning	Use it when
`let`	A box whose contents **can change**.	The value will be updated later, e.g. a shopping cart total.
`const`	A box whose contents **must not change** after it is set.	The value stays fixed, e.g. the shop name, the tax rate.

Rule of thumb for beginners: *Use `const` by default. Only use `let` when you know the value must change.* This prevents accidental mistakes.

```
let total = 0;           // will change as items are added
total = total + 2500;    // now 2500
total = total + 1200;    // now 3700, fine — it is a let

const shopName = "Adaeze Fashion";  // will never change
```

What happens if you break a `const`:

```
const shopName = "Adaeze Fashion";
```

```
shopName = "Ada Collections";   // ERROR
```

The Console shows: `Uncaught TypeError: Assignment to constant variable.` This is JavaScript protecting you.

Naming rules and good habits:

- Start with a letter, `_` or `$`. Not a number.
- No spaces. Use **camelCase**: `firstName`, `cartTotal`, `isLoggedIn`.
- Names are **case-sensitive**: `total` and `Total` are different boxes.
- Use **meaningful names**: `price` is better than `p`; `studentName` is better than `x`.
- Do not use reserved words like `let`, `const`, `if`, `function` as names.

Common misconception. Learners write `let name = Ada;` without quotes. JavaScript then thinks `Ada` is a *variable* named Ada (which does not exist) and throws `ReferenceError: Ada is not defined`. Text needs quotes. This is one of the three errors we diagnose later.

F.5 Data types — what kind of value is in the box?

JavaScript cares about the *kind* of value. The four we use this week:

Type	Meaning	Examples	Notes
string	Text	`"Adaeze"`, `"₦2,500"`, `"hello world"`	Always in quotes.
number	A numeric value	`22`, `2500`, `-8`, `3.14`	No quotes. Used for maths.
boolean	True or false only	`true`, `false`	Exactly two possible values.
undefined	A box declared but not yet given a value	`let x;` → `x` is `undefined`	Means "no value yet".

`typeof` — asking JavaScript what type something is:

```
let name = "Adaeze";
let age = 22;
let isOpen = true;
let emptyBox;

console.log(typeof name);     // "string"
console.log(typeof age);      // "number"
console.log(typeof isOpen);   // "boolean"
console.log(typeof emptyBox); // "undefined"
```

Why `typeof` matters. In real work, a common bug is storing a number as text. `"2500"` (string) plus `"1200"` (string) gives `"25001200"` — the computer glued the text together instead of adding! `typeof` helps you catch this.

Worked example — the text-versus-number trap:

```
let a = 2500;      // number
let b = 1200;      // number
console.log(a + b);            // 3700  (maths — correct)

let c = "2500";    // string (note the quotes)
let d = "1200";    // string
console.log(c + d);           // "25001200" (text joined — usually a bug!)
```

Say this to the class:

> *"A number in quotes is not a number to the computer; it is a word made of digits. '2500' is just as much text to JavaScript as 'Ada' is. This is one of the most common real-world bugs, so always check the type."*

F.6 Working with strings — concatenation and template literals

The idea. We often need to join text and variables to form a sentence, e.g. *"Hello Adaeze, your total is ₦3,700."* JavaScript gives us two ways.

Way 1 — Concatenation (the old way) with `+`:

```
let name = "Adaeze";
let total = 3700;
console.log("Hello " + name + ", your total is ₦" + total);
// Hello Adaeze, your total is ₦3700
```

Note the spaces: `"Hello "` has a space at the end so the words do not run together. Beginners often forget it and get `HelloAdaeze`.

Way 2 — Template literals (the modern, preferred way):

```
let name = "Adaeze";
let total = 3700;
console.log(`Hello ${name}, your total is ₦${total}`);
// Hello Adaeze, your total is ₦3700
```

Three things to notice:

1. The text is wrapped in **backticks** (`` ` ``), not normal quotes. On most Nigerian keyboards the backtick is the key above `Tab`, often to the left of the `1` key. If unsure, the instructor will show it physically.
2. To insert a variable you write `${variableName}`.
3. No messy spaces or `+` signs — the sentence reads naturally.

Why we prefer template literals: they are easier to read, harder to get wrong, and let you insert several variables cleanly.

Worked example — a receipt line:

```
const item = "Hair Cream";
const qty = 2;
const price = 2500;
const total = qty * price;

console.log(`${qty} x ${item} @ ₦${price} = ₦${total}`);
// 2 x Hair Cream @ ₦2500 = ₦2500... wait:
```

Careful — `2 * 2500` is `5000`, so the correct output is:

```
2 x Hair Cream @ ₦2500 = ₦5000
```

Let the class catch this by predicting the result before you run it. If the projector shows `5000`, praise the learners who predicted correctly.

F.7 Arithmetic — making the computer calculate

JavaScript does maths with familiar symbols:

Symbol	Meaning	Example	Result
`+`	Add	`10 + 5`	`15`
`-`	Subtract	`10 - 5`	`5`
`*`	Multiply	`10 * 5`	`50`
`/`	Divide	`10 / 5`	`2`
`%`	Remainder (modulo)	`10 % 3`	`1`

The `%` operator explained simply: it gives the *remainder* after division. `10 % 3` is `1` because 3 goes into 10 three times (9) with 1 left over. It is useful later for things like "is this number even?" (`number % 2 == 0`).

Student Manual Page 1 of 235

Order of operations. JavaScript follows normal maths rules: multiplication and division before addition and subtraction. Use brackets to be clear.

```
console.log(2 + 3 * 4);    // 14, because 3*4=12 first, then +2
console.log((2 + 3) * 4);  // 20, because brackets first
```

Comments — notes for humans that the computer ignores:

```
// This is a single-line comment. JavaScript ignores it.
let price = 2500; // comment can go at the end of a line too

/*
  This is a multi-line comment.
  Use it to explain larger blocks later.
*/
```

Why comments matter in real work: In three months, you will not remember why you wrote a line. A short comment saves future-you (and your teammates) a lot of time. But do not over-comment obvious lines.

F.8 Summary of the session

- HTML = structure, CSS = appearance, **JavaScript = behaviour**.
- Connect JavaScript with `<script src="js/app.js"></script>` at the **bottom of the body**.
- The **Console** (`F12`) shows output and errors — your main tool.
- **Variables** store values: `const` for fixed values, `let` for values that change.
- **Types**: string, number, boolean, undefined. Use `typeof` to check.
- Join text with **template literals** `` `Hello ${name}` `` (preferred) or `+` .
- Do maths with `+ - * / %` .

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- A folder on the Desktop: `demo-webdev/week1/` containing:
  - `index.html`
  - a subfolder `js/` containing `app.js`
- Both files open in VS Code.
- Chrome open to the page, DevTools Console open.

G.2 Demonstration 1 — connecting the script (5 minutes)

Say: "Watch how the HTML page and the JavaScript file talk to each other."

Type (or paste) into `index.html` :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Tip Calculator Demo</title>
</head>
<body>
  <h1>Mama Ngozi's Restaurant — Tip Calculator</h1>
  <p>Open the Console with F12 to see the calculation.</p>
  <script src="js/app.js"></script>
</body>
</html>
```

Type into `js/app.js`:

```
console.log("App started");
```

Do this on the projector:

1. Save both files (`Ctrl + S`).
2. Open `index.html` in Chrome (double-click it).
3. Press `F12`, click **Console**.
4. Point at the printed line `App started`.
5. Ask the class: *"How did the HTML page know to run this file?"* → Because of the `<script src="js/app.js"></script>` line.

Then deliberately break it: change `src` to `js/wrong.js`, save, refresh.

- Expected result: nothing printed, and a **red error** in the Console: `Failed to load resource: net::ERR_FILE_NOT_FOUND`.
- **Say:** "This is the single most common beginner mistake — a wrong file name or folder. Now you know what it looks like."
- Fix it back to `js/app.js`, save, refresh. Output returns.

Questions to ask during the demo:

- "What would happen if I put the `<script>` line in the `<head>` instead?" (It would run before the HTML below exists — a real problem once we start changing the page.)
- "Why do we not write the JavaScript inside the HTML?" (Separation of concerns, reuse, caching.)

G.3 Demonstration 2 — variables and a running total (10 minutes)

Say: "Now let's do something useful — a restaurant bill with a tip."

Type into `js/app.js`:

```
// Mama Ngozi's tip calculator — demo version

const restaurant = "Mama Ngozi's Kitchen";
let foodPrice = 4500;   // naira
let tipPercent = 10;    // percent

let tipAmount = (foodPrice * tipPercent) / 100;
let billTotal = foodPrice + tipAmount;

console.log("Restaurant:", restaurant);
console.log("Food price: ₦" + foodPrice);
console.log("Tip (" + tipPercent + "%): ₦" + tipAmount);

// Modern template-literal version:
console.log(`Total to pay: ₦${billTotal}`);
```

Walk through it line by line on the projector:

1. `const restaurant` — the name will not change, so `const`.
2. `let foodPrice = 4500` — this could change tomorrow, so `let`.
3. `tipAmount` — the formula: price × percent ÷ 100. Explain *why* divide by 100: a percentage is "out of 100".
4. `billTotal` — food plus tip.
5. The last `console.log` uses backticks and `${ }`.

Expected output in the Console:

```
Restaurant: Mama Ngozi's Kitchen
Food price: ₦4500
Tip (10%): ₦450
Total to pay: ₦4950
```

Ask learners to predict `tipAmount` before you run it. (450.)

Now demonstrate the type trap live: change `foodPrice` to `"4500"` (in quotes), save, refresh.

- Output becomes: `Food price: ₦4500` (looks fine!) but `Total to pay: ₦4500450` — wrong!

- **Say:** "See how a number in quotes breaks the maths. This is why `typeof` exists." Run `console.log(typeof foodPrice)` to show `string`.
- Change it back to a real number (no quotes).

Common errors to show and correct during the demo:

- Omitting a semicolon — usually still runs, but teach consistency.
- Using `=` instead of `==` inside a comparison — not in this session, but mention it is coming.
- A missing backtick for the template literal → red syntax error.

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding step by step, learners create their own working page with an external JavaScript file, declare variables and print a personal introduction.

Starting materials: A new folder on each learner's Desktop called `webdev-practice`, containing an empty `index.html` and an empty `js/app.js` (or learners create them; instructor shows both).

Procedure (instructor calls out each numbered step; learners follow on their machines; instructor walks the room):

1. Inside `webdev-practice`, create a folder named `js`.
2. Open VS Code. Open the `webdev-practice` folder (**File → Open Folder**).
3. Create `index.html` in the main folder with this content:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Intro</title>
</head>
<body>
  <h1>About Me</h1>
  <p>Press F12 and open the Console.</p>
  <script src="js/app.js"></script>
</body>
</html>
```

4. Create `js/app.js`. Type:

```
const fullName = "Blessing Okoro";   // use YOUR own name
let age = 21;                        // use YOUR own age
let city = "Port Harcourt";
let isStudent = true;
```

5. Add three lines to print:

```
console.log("My name is " + fullName);
console.log(`I am ${age} years old and I live in ${city}.`);
console.log("Am I a student?", isStudent);
```

6. Save both files (`Ctrl + S`).
7. Open `index.html` in Chrome. Press `F12`, Console tab.
8. **Pause and predict:** "What will the three lines look like before you run?" Have learners write their prediction on paper. Then refresh and compare.

Expected result:

```
My name is Blessing Okoro
I am 21 years old and I live in Port Harcourt.
Am I a student? true
```


Points to pause on and predict:

- After step 4: "*What type is* `isStudent`*?*" → boolean.
- After step 5: "*Will the second line print the number 21 or the text '21'?*" → With `${age}` the number is inserted as text in the sentence, but `typeOf age` is still number.

Troubleshooting while walking the room:

- Console empty → check the `src` path and that both files are saved.
- `ReferenceError: Blessing is not defined` → the name was typed without quotes. Add quotes.
- Strange characters (₦ shown as `?`) → file encoding. Instructor: save the file as UTF-8. (Explained in Section K.)

Completion checklist (learner ticks each):

- ☐ `index.html` links `js/app.js`
- ☐ `app.js` declares at least 4 variables
- ☐ At least one `const` and one `let` used
- ☐ Three `console.log` lines print
- ☐ Template literal `` ` `` used at least once
- ☐ Full name, age, city and boolean all appear in the Console

---

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Build a small "shop receipt" calculator on your own, without copying the instructor's restaurant example.

Scenario: You work at *Chinedu Electronics* in Port Harcourt. A customer buys **3 phone chargers** at **₦1,800 each** and a **power bank** at **₦12,000**. The shop adds a **7.5% VAT** to the total.

Starting information (given):

- Charger unit price: `1800`
- Charger quantity: `3`
- Power bank price: `12000`
- VAT percent: `7.5`

Requirements:

1. Create a new page `receipt.html` and a `js/receipt.js` file linked at the bottom of the body.
2. Store the five values above in sensibly named variables. Use `const` where the value will not change.
3. Calculate:
   - `chargerTotal` = unit price × quantity
   - `subtotal` = chargerTotal + power bank price
   - `vat` = subtotal × VAT ÷ 100
   - `grandTotal` = subtotal + vat
4. Print a receipt to the Console using **template literals**, similar to:

```
===== CHINEDU ELECTRONICS =====
3 x Charger @ ₦1800 = ₦5400
1 x Power Bank @ ₦12000 = ₦12000
Subtotal: ₦17400
VAT (7.5%): ₦1305
TOTAL TO PAY: ₦18705
```

Constraints:

- All prices as **numbers**, not strings.
- At least one `const` and one `let`.
- No copying the restaurant code text — write it yourself.

Time allowance: 15 minutes.

Success criteria:

- The Console shows the exact five calculated figures.
- `subtotal` = 17400, `vat` = 1305, `grandTotal` = 18705.
- The script is linked correctly and there are no red errors.

Hints available if stuck:

- Hint 1: To multiply, use `*`. So `chargerTotal = chargerPrice * quantity`.
- Hint 2: VAT of 7.5% means `subtotal * 7.5 / 100`.
- Hint 3: For the receipt lines, use a template literal, e.g. `console.log(`Subtotal: ₦${subtotal}`);`.

Instructor answer key (do not show learners):

```
// receipt.js — Chinedu Electronics
const shopName = "CHINEDU ELECTRONICS";
const chargerPrice = 1800;
const chargerQty = 3;
const powerBankPrice = 12000;
const vatPercent = 7.5;

let chargerTotal = chargerPrice * chargerQty;   // 5400
let subtotal = chargerTotal + powerBankPrice;   // 17400
let vat = (subtotal * vatPercent) / 100;        // 1305
let grandTotal = subtotal + vat;                // 18705

console.log(`===== ${shopName} =====`);
console.log(`${chargerQty} x Charger @ ₦${chargerPrice} = ₦${chargerTotal}`);
console.log(`1 x Power Bank @ ₦${powerBankPrice} = ₦${powerBankPrice}`);
console.log(`Subtotal: ₦${subtotal}`);
console.log(`VAT (${vatPercent}%): ₦${vat}`);
console.log(`TOTAL TO PAY: ₦${grandTotal}`);
```

Marking guidance (out of 10):

- Correct variables and types (2)
- Correct subtotal (2)
- Correct VAT (2)
- Correct grand total (2)
- Clean template-literal receipt output (1)
- No console errors / correct linking (1)

J. REAL-WORLD CASE STUDY

Situation. *Blessing* runs a small cake business from home, *Blessing Bakes*. She takes orders on WhatsApp and always calculates prices by hand. Two weeks ago she quoted a customer ₦18,000 for a cake that actually cost her ₦21,000 to make — because she forgot to add the cost of the cake board and candles. She lost money on the order.

Relevant facts.

- Blessing's base cake price formula is: `(flavour cost + decoration cost) + delivery`.
- She keeps adding extra items (candles, board, delivery) and forgets some.
- She is not a programmer, but she has just learned variables.

Decision point. How can a tiny calculator reduce her mistakes?

Possible approaches.

1. Keep calculating by hand but write a paper checklist.
2. Build a simple JavaScript calculator that takes the costs as variables and prints the total, including all extras.
3. Pay someone to build a full app.

Recommended solution (for this stage). Approach 2. Blessing creates a page where each cost is a clearly named variable, and the program prints a full breakdown and the total every time. This is exactly the skill learned today.

Example solution:

```
const cakeFlavourCost = 9000;
const decorationCost = 6000;
const board = 1500;
const candles = 800;
const delivery = 2500;

let total = cakeFlavourCost + decorationCost + board + candles + delivery;

console.log("===== BLESSING BAKES QUOTE =====");
console.log(`Flavour: ₦${cakeFlavourCost}`);
console.log(`Decoration: ₦${decorationCost}`);
console.log(`Board: ₦${board}`);
console.log(`Candles: ₦${candles}`);
console.log(`Delivery: ₦${delivery}`);
console.log(`TOTAL QUOTE: ₦${total}`);
```

Output: `TOTAL QUOTE: ₦19800`.

Trade-offs.

- Approach 1 is free and immediate but still relies on memory — the very thing that failed.
- Approach 2 takes 20 minutes to build but removes the forgetfulness permanently, and each line is named so a missing cost is visible.
- Approach 3 is overkill and expensive at this stage; it may be worthwhile later once Blessing adds online ordering.

Professional connection. This is exactly why junior developers write small "internal tools" — short programs that remove human errors in repetitive tasks. The skill of naming variables clearly (`cakeFlavourCost`) and printing a readable breakdown is directly used in real entry-level work.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this table as the session's live troubleshooting guide. When a learner hits an issue, do not give the answer immediately — ask them to check the relevant row.

K.1 Nothing appears in the Console

Symptoms: Page loads, but the Console is empty; no `console.log` output. **Likely causes:** (a) The script file is not linked. (b) The `src` path is wrong. (c) The file was not saved. **How to investigate:** Press `F12` → Console. Look for a red error mentioning the file name. Also check the **Sources** tab to confirm the file is loaded. Ensure the `src` matches the real folder and file name exactly. **Safe corrective steps:** Fix the path; save the file; refresh with `Ctrl + F5` (hard refresh). **How to confirm resolved:** Your `console.log` line now appears. **How to prevent recurrence:** Put the script at the bottom of the body; keep the `.js` inside a `js/` folder and copy the exact path.

K.2 `ReferenceError: Ada is not defined`

Symptoms: Red error naming a word you wrote. **Likely cause:** You wrote text without quotes, so JavaScript treated it as a variable name (confirmed cause once you see no quotes in your code). **How to investigate:** Look at the line number in the error; find that line; check whether text is in quotes. **Safe corrective steps:** Add quotes around text values: `let name = "Ada";`. **How to confirm resolved:** The error disappears and the value prints. **Prevention:** Whenever the value is a word, not a number, wrap it in quotes.

K.3 The maths is wrong — numbers joined instead of added

Symptoms: `2500 + 1200` shows `25001200` instead of `3700`. **Likely cause:** The values are **strings** (they have quotes). **How to investigate:** Run `console.log(typeof yourVariable)`. **Corrective steps:** Remove the quotes if you meant numbers, or convert with `Number("2500")`. **Prevention:** Keep prices and quantities as numbers; only quote text.

K.4 `Uncaught TypeError: Assignment to constant variable`

Symptoms: Red error when reassigning. **Likely cause:** You used `const` but then changed the value. **Corrective steps:** Change `const` to `let` if the value truly changes; otherwise do not reassign. **Prevention:** Default to `const`, switch to `let` only when needed.

K.5 Template literal not working / "Unexpected token"

Symptoms: Red syntax error near a backtick or `${ }`. **Likely cause:** You used normal quotes `"` or `'` instead of backticks around a `${ }` expression, or a missing closing backtick. **Corrective steps:** Replace the outer quotes with backticks `` ` `` and ensure they are matched. **Prevention:** Practise finding the backtick key (above Tab).

K.6 Strange symbols like `?` instead of ₦

Symptoms: Naira sign shows as `?` or a box. **Likely cause:** File saved in a non-UTF-8 encoding. **Corrective steps:** In VS Code bottom-right, click the encoding, choose **Save with Encoding → UTF-8**, and include `<meta charset="UTF-8">` in the HTML head. **Prevention:** Always keep the `<meta charset="UTF-8">` line.

Distinguish likely vs confirmed: In every case above, the *likely* cause becomes a *confirmed* cause only after you check it (e.g., see the exact red error text, or run `typeof`). Teach learners to confirm before fixing.

L. SESSION ASSIGNMENT

Assignment title: "My Personal Tip Calculator"

Purpose: Practise variables, types, arithmetic and console output in a context the learner owns.

Realistic scenario: You are a waiter at a small restaurant in your area. Customers ask you to add a tip to their bill. You will build a tiny calculator that takes a food price and a tip percentage and prints the tip and the total.

Detailed instructions:

1. Create a folder `assignment1` with `index.html` and `js/app.js`.
2. In `app.js`, declare:
   - `const restaurantName` — your restaurant's name (invent one).
   - `let foodPrice` — choose any realistic amount (number, not string).
   - `let tipPercent` — choose 5, 10 or 15.
3. Calculate `tipAmount` and `totalToPay`.
4. Print a receipt with template literals, including the restaurant name, food price, tip percent, tip amount and total.
5. Add a second scenario by changing `foodPrice` and `tipPercent` and printing a second receipt below the first. (Tip: you may reassign the `let` variables and print again.)

Required deliverables:

- `assignment1/index.html`
- `assignment1/js/app.js`
- The Console output pasted (or photographed) into your workbook.

Submission format: Files zipped or shared in the class folder; screenshot/photo of the Console in the workbook.

Deadline: Before Session 2.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
Correct file structure & linking	3	External `app.js` linked at bottom of body; no errors
Variables & types	4	Sensible names; prices as numbers; `const` / `let` used correctly
Correct calculations	5	Tip and total correct in both scenarios
Template literals used	3	Receipt reads cleanly with `${ }`
Two scenarios shown	3	Two different receipts printed
Presentation / no errors	2	Clear output, no red errors

Common mistakes to avoid: quoting numbers, forgetting to reassign with `let`, missing backticks; wrong file path.

Instructor reference solution:

```
const restaurantName = "Taste of Naija Kitchen";

let foodPrice = 5000;
let tipPercent = 10;
let tipAmount = (foodPrice * tipPercent) / 100;
let totalToPay = foodPrice + tipAmount;

console.log(`===== ${restaurantName} =====`);
console.log(`Food: ₦${foodPrice}`);
console.log(`Tip (${tipPercent}%): ₦${tipAmount}`);
console.log(`TOTAL: ₦${totalToPay}`);

// Second scenario
foodPrice = 8200;
tipPercent = 15;
tipAmount = (foodPrice * tipPercent) / 100;
totalToPay = foodPrice + tipAmount;

console.log(`===== ${restaurantName} =====`);
console.log(`Food: ₦${foodPrice}`);
console.log(`Tip (${tipPercent}%): ₦${tipAmount}`);
console.log(`TOTAL: ₦${totalToPay}`);
```

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. In one sentence each, what does HTML, CSS and JavaScript do on a web page?

Q2. Multiple choice. Which is the correct way to link an external JavaScript file called `app.js` inside a `js` folder? a) `<link rel="js" href="js/app.js">` b) `<script src="js/app.js"></script>` c) `<script href="js/app.js">` d) `<js src="app.js">`

Q3. Explain in your own words. Why do we usually place the `<script>` tag at the bottom of the `<body>` instead of in the `<head>`?

Q4. Practical identification. What data type is each value? Write string, number, boolean or undefined. a) `"Port Harcourt"` b) `2500` c) `false` d) `let x;` (x)

Q5. Error detection. Find and fix the mistake: `let price = 2500; price = "two thousand";` then `console.log(price * 2);`

Q6. Scenario. A shop sells a bag for ₦7,500 and gives a 5% discount. Write (or describe) the two lines of JavaScript that compute the discount amount and the final price.

INSTRUCTOR ANSWER KEY (Q1–Q6)

A1. HTML gives structure/content; CSS gives appearance/styling; JavaScript adds behaviour/interactivity. **A2.** (b) `<script src="js/app.js"></script>`. **A3.** Because the browser reads top to bottom; if the script runs before the HTML elements exist, it cannot find them and errors. Placing it at the bottom ensures all elements exist first. (Also accept the `defer` explanation.) **A4.** a) string, b) number, c) boolean, d) undefined. **A5.** The bug is that `price` ends up as the string `"two thousand"` (if that was only meant to be a comment) — `price * 2` gives `NaN` (Not a Number) because you cannot multiply text. Fix: keep the numeric value for calculations (`let price = 2500; let doubled = price * 2;`). If the string was accidental, remove the quotes. (Accept any answer identifying the string/number mismatch and its consequence.) **A6.**

```
let bag = 7500;
let discount = (bag * 5) / 100;   // 375
let finalPrice = bag - discount;  // 7125
```

N. REFLECTION AND RECAP

Plain-English summary. Today you met JavaScript, the language that gives a web page a brain. You connected a `.js` file to an HTML page with the `<script>` tag, opened the browser Console to see output, stored information in variables (`const` and `let`), checked the type of a value with `typeof`, joined text and variables with template literals, and did arithmetic. You also learned to spot real bugs such as numbers written as text and a mis-linked script file.

Five important takeaways:

1. HTML = structure, CSS = look, JavaScript = behaviour.
2. Link JavaScript with `<script src="js/app.js"></script>` at the bottom of the body.
3. `const` for fixed values, `let` for values that change.
4. Text needs quotes; numbers do not. Use `typeof` when unsure.
5. The Console (`F12`) is where you see output and errors.

Questions to help you recognise what you can now do:

- Can I create a page and link a JavaScript file without help?
- Can I open the Console and read an error message?
- Can I explain why `"2500" + "1200"` gives text and not `3700`?
- Can I write one template literal from memory?

Connection to the next session: Right now our programs run straight down, line by line, doing the same thing every time. Next session we teach the program to **make decisions** — `if` this, then that — using conditions and `if/else`. That is the beginning of programs that respond intelligently.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner links the script correctly on the first or second try without being told the path.
- Learner opens the Console unaided and reads the exact error text rather than guessing.
- Learner chooses `const` vs `let` with a reason ("this total will change").
- Learner predicts the output of simple arithmetic before running it.
- Learner uses a template literal and can point to the backtick key.

Common misconceptions to watch for and correct:

- "If I don't see output, my code is broken." → Often the script is not linked or not saved. Always check the Console's red errors.
- "JavaScript and Java are the same." → Correct immediately; they are unrelated.
- "All numbers are numbers, even `"500"`." → Run `typeof` to prove a quoted number is a string.
- "Quotes are optional." → Explain why text needs them.

Suggested remedial exercises for learners who struggle:

1. Trace-the-output drills: give five `console.log` lines including `typeof` and ask learners to write the expected output on paper, then run to check.
2. "Fix the broken page" drill: hand out a page where `src` is misspelled and a string is unquoted; ask the learner to find and fix both.
3. Const/let sorting: give ten one-line examples and ask which keyword belongs.

Differentiation:

- **Level 1 (supported):** provide a partially filled `app.js` with variable names already chosen and one `console.log` completed.
- **Level 2 (standard):** the exercise as written.
- **Level 3 (challenge):** add a `discountPercent` and compute the final total after both VAT and discount; also print how much VAT and discount separately in naira.

Timing guidance: If the room is slow, drop the case study (Section J) to a 3-minute mention and protect the independent exercise time — the practical is the priority.

APPENDIX 1A — 20-MINUTE HTML REFRESHER (only if needed)

If learners cannot produce a basic page, run this before Section D.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Refresher</title>
</head>
<body>
  <h1>Largest heading</h1>
  <p>A paragraph of text.</p>
  <button>Click me</button>
</body>
</html>
```

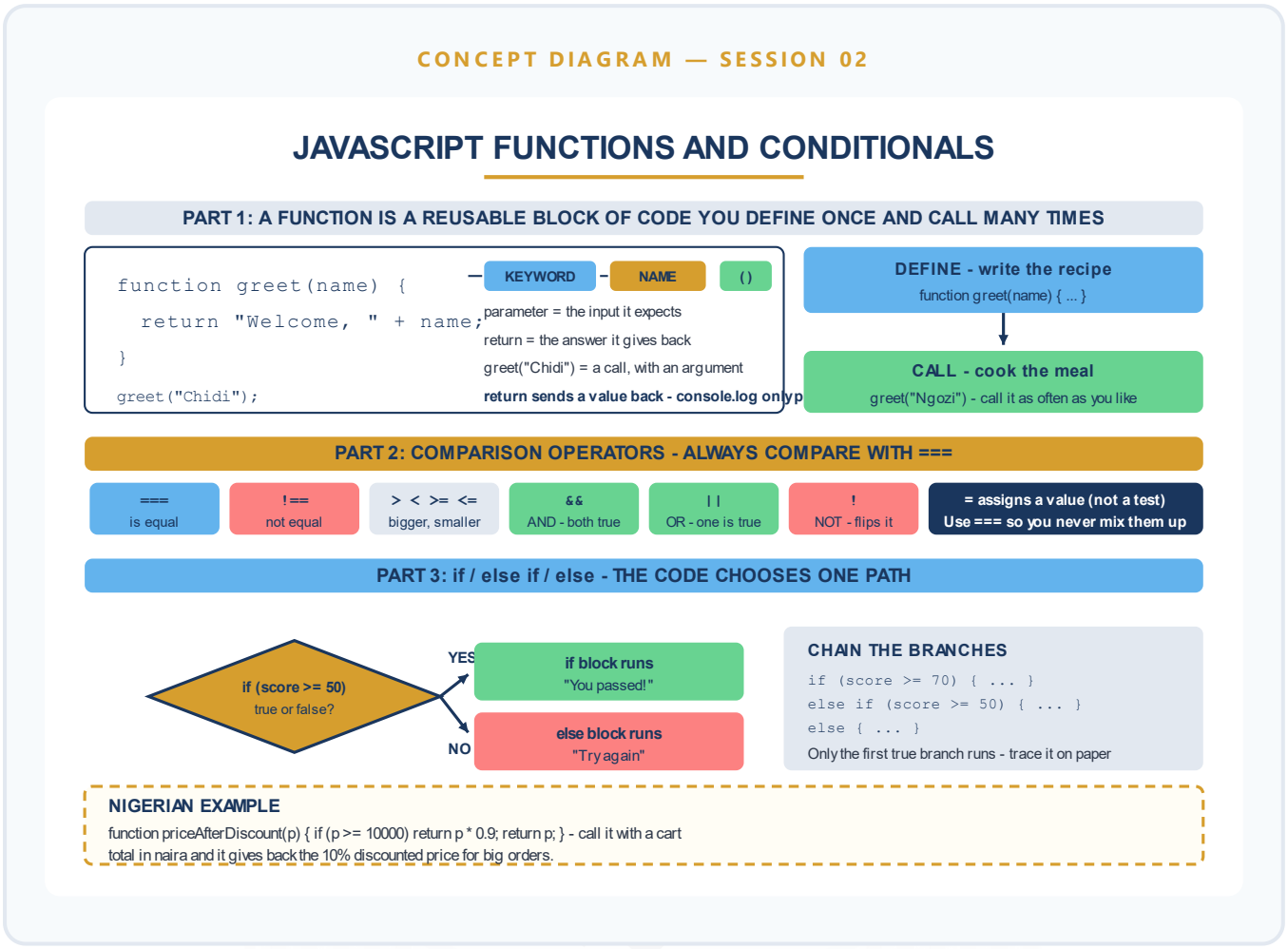
Key points: every tag opens `<tag>` and closes `</tag>`; the page lives inside `<html>`; visible content goes in `<body>`; `class` and `id` attributes let CSS target elements. Confirm each learner can save and open this page before proceeding.

END OF SESSION 1

Next: Session 2 — Making Decisions: Functions, Conditions and If/Else.

SESSION 02 OF 12

MAKING DECISIONS: FUNCTIONS, CONDITIONS AND IF/ELSE



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 2 — MAKING DECISIONS: FUNCTIONS, CONDITIONS AND IF/ELSE

A. SESSION IDENTIFICATION

Item	Detail
Session number	2 of 12
Session title	Making Decisions: Functions, Conditions and If/Else
Week / Module	Week 1 — Module 1: JavaScript Foundations
Recommended duration	105 minutes

Item	Detail
Difficulty level	Beginner
Prerequisite knowledge	Session 1 — variables (`let` / `const`), data types (string, number, boolean, undefined), `console.log` , template literals, arithmetic. Learner must be able to open the Console and link a script file.
Connection to previous session	In Session 1 programs ran straight down, doing the same thing every time. Today programs learn to **decide** and to **repeat a named action** (a function). This is the point where code starts to feel intelligent.
Preparation before class	Learner practice folders from Session 1 available; projector showing VS Code + Chrome Console; a whiteboard or flipchart for drawing decision branches.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Define a function** using the `function` keyword and **call** it, explaining the difference between defining and calling.
2. **Pass arguments** into a function and **return** a value, and explain why `return` is different from `console.log` .
3. **Write comparison expressions** using `===` , `!==` , `>` , `<` , `>=` , `<=` and **explain why** `===` **is preferred over** `==` .
4. **Combine conditions** with `&&` (AND), `||` (OR) and `!` (NOT).
5. **Write** `if` **,** `else if` **and** `else` **branches** that choose different actions based on a value.
6. **Trace the execution** of a multi-branch program on paper and predict which branch runs.
7. **Diagnose and fix** common errors: `=` vs `===` , missing parentheses/braces, and a function that runs but returns nothing (`undefined`).

C. REQUIRED MATERIALS

Hardware: One computer per learner; instructor computer with projector. **Software:** VS Code; Google Chrome. No new software needed. **Accounts:** None. **Sample files:** Continue in the learner's `webdev-practice` folder. Instructor prepares `demo-week1-s2/` with `index.html` and `js/decide.js` . **Worksheets:**

- Worksheet 2.1 — "Trace the Branch" (given a program with several branches, learners circle which line runs).
- Worksheet 2.2 — "Turn a Rule into Code" (English rules → `if/else`). **Tools:** Chrome DevTools Console. **Internet:** Not required. **Alternative for limited facilities:** Everything runs offline; a shared machine works with the pair-swap method from Session 1.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector Console, type the following and ask the class to predict the result before pressing Enter:

```
let age = 20;
console.log(age >= 18);
```

Instructor asks: "Before I press Enter — will this print `true` or `false` ? Think about what `>=` means."

Press Enter → prints `true` .

Then say:

> *"Yesterday our programs did the same thing every single time. They could not think. But look — the computer just answered a question: 'Is age greater than or equal to 18?' and answered `true` . Answering yes/no questions is the first half*

> *of thinking. The second half is acting on the answer: if the person is 18 or older, show the adult content; else, show a warning. Today we build programs that make decisions — and we package actions into reusable blocks called functions, so we do not repeat ourselves."*

Expected learner response: Learners will predict true/false and may argue. Accept both; the point is that the machine evaluates the question.

Then hook with a real rule: Ask, "At your church's youth event, who is allowed to register for the adult session — anyone aged 18 and above, right? That is a rule. Rules become `if` statements in code." This sets up the day's work with a familiar Nigerian institution.

E. PREREQUISITE REVIEW (7 minutes)

Q1. "What is the difference between `let` and `const`?"

- Expected: `let` can be changed; `const` cannot.
- Gap: learners forget and reassign a `const`, producing `Assignment to constant variable`. Remind them.

Q2. "What data type does `console.log(typeof 42)` print?"

- Expected: `number`.
- Gap: some say "integer" — accept but clarify JavaScript just says `number` for all numbers.

Q3. "Write the template literal that prints `Total is ₦5000` using a variable `total`."

- Expected: `console.log(`Total is ₦${total}`);`
- Gap: using normal quotes. Remind that `${ }` only works inside backticks.

Q4. "What does `2 + 3 * 4` print, and why?"

- Expected: `14`, because multiplication happens before addition.
- Gap: `20` (left-to-right). Correct using the order-of-operations review.

Bridge: "Good — you can store and calculate. Now we ask questions (`if`) and reuse actions (functions)."

If the class is shaky, give a 5-minute pair activity: each pair writes three `console.log` lines using variables and types, then swaps with another pair to check.

F. FULL LECTURE NOTES

F.1 Functions — packaging an action

The idea. A **function** is a named block of code that performs a task. You write it once and *call* it (run it) as many times as you like.

Plain-English definition. A function is like a **recipe with a name**. You write the recipe once — "how to make jollof rice" — and then whenever you want jollof rice you just say "make jollof rice" instead of repeating every step.

Why functions exist. Without functions, if you needed the same calculation in five places and later a price changed, you would edit five copies and might miss one. A function keeps one copy. This is called **DRY** — *Don't Repeat Yourself* — a principle real developers live by.

Anatomy of a function:

```
function greet() {
  console.log("Hello and welcome!");
}

greet();   // calling the function
```

Line by line:

- `function` — the keyword that says "I am defining a function".
- `greet` — the name (use a verb: `greet`, `calculateTotal`, `showMenu`).
- `()` — the parentheses, where inputs would go (empty here).
- `{ }` — the **body**: the instructions that run when called.
- `greet();` — **calling** the function; this actually runs the body.

Fact to emphasise: Defining a function does **not** run it. Many beginners write a function and see nothing in the Console because they forgot to call it. Definition = cooking the recipe on paper. Calling = actually cooking.

Analogy. A function is a **wrapped gift**. Writing the function is wrapping the gift. Calling the function is giving and opening it.

F.2 Parameters and arguments — giving a function information

A function becomes far more useful when you can hand it information.

```
function greetUser(name) {
  console.log("Hello, " + name + "!");
}

greetUser("Adaeze");   // Hello, Adaeze!
greetUser("Chinedu");  // Hello, Chinedu!
```

- `name` inside the parentheses is a **parameter** — a placeholder.
- `"Adaeze"` passed in when calling is the **argument** — the real value.
- Analogy: the parameter is an empty cup; the argument is the tea you pour in.

Multiple parameters:

```
function showPrice(item, price) {
  console.log(`${item} costs ₦${price}`);
}

showPrice("Power Bank", 12000);
showPrice("Charger", 1800);
```

Common misconception: Learners swap the order. If a function expects `(item, price)` and you call `showPrice(12000, "Power Bank")`, the output is nonsense. Order matters.

F.3 `return` — sending a value back

`console.log` *shows* a value to a human. `return` *sends* a value back to wherever the function was called, so the program can keep using it.

```
function add(a, b) {
  return a + b;
}

let sum = add(2, 3);   // sum is now 5
console.log(sum);      // 5
console.log(add(10, 20) * 2);  // 30 * 2 = 60
```

Why `return` matters: Without `return`, the function's result is trapped inside and cannot be used elsewhere.

```
function addWrong(a, b) {
  console.log(a + b);   // prints 5
}

let result = addWrong(2, 3);
console.log(result);    // undefined  ← the value did not come back!
```

Say to the class: "`console.log` is a person shouting a number across the room. `return` is a person handing the number to you. If you need the number later, you need it *handed* to you — that is `return`."

Rule of thumb: A function that just *reports* may use `console.log`. A function that *calculates something to be used elsewhere* must `return`.

Key fact: `return` also **stops** the function immediately. Any code after a `return` in the same block does not run.

F.4 Comparison operators — asking yes/no questions

Every decision starts with a question that has a `true` / `false` answer.

Operator	Meaning	Example	Result
`===`	equal to (strict)	`5 === 5`	`true`
`!==`	not equal to	`5 !== 3`	`true`
`>`	greater than	`10 > 5`	`true`
`<`	less than	`10 < 5`	`false`
`>=`	greater than or equal	`5 >= 5`	`true`
`<=`	less than or equal	`4 <= 5`	`true`

The critical `=` vs `===` lesson:

- `=` means **assign**: "put this value in the box." `let x = 5;`
- `===` means **compare**: "are these the same?" `x === 5`
- Using `=` inside an `if` is a classic bug.

Why `===` and not `==`? JavaScript has both. `==` is *loose* and tries to convert types, which causes surprises: `"5" == 5` is `true` with `==` but `"5" === 5` is `false`. Professionals use `===` (strict) almost always to avoid these traps. Rule for this course: **always use `===` and `!==`.**

Analogy for `===`: `===` is like a strict passport officer — everything must match exactly, including type. `==` is a lenient officer who might let things slide and cause problems later.

F.5 Logical operators — combining questions

Real rules are often compound: "18 or older AND resident in Nigeria."

Operator	Name	Meaning	Example
`&&`	AND	Both must be true	`age >= 18 && hasID === true`
`||`	OR	At least one must be true	`isStudent || isTeacher`
`!`	NOT	Reverses the value	`!isClosed` (true if not closed)

Truth table for AND (`&&`):

Left	Right	Result
true	true	true
true	false	false
false	true	false
false	false	false

Truth table for OR (`||`):

Left	Right	Result
true	true	true

Left	Right	Result
true	false	true
false	true	true
false	false	false

Worked example:

```
let age = 20;
let hasVoterCard = true;

console.log(age >= 18 && hasVoterCard);   // true: both satisfied
console.log(age >= 18 || hasVoterCard);   // true: at least one
console.log(!hasVoterCard);               // false
```

F.6 `if`, `else if`, `else` — choosing a path

The idea. An `if` statement runs a block of code **only when** a condition is true.

```
let balance = 5000;

if (balance > 0) {
  console.log("You have money available.");
}
```

If the condition is false, the block is skipped entirely.

Adding `else` — the "otherwise" path:

```
let balance = 0;

if (balance > 0) {
  console.log("You have money available.");
} else {
  console.log("Your balance is empty. Please top up.");
}
```

Adding `else if` — several possibilities:

```
let score = 72;

if (score >= 70) {
  console.log("Grade: A");
} else if (score >= 60) {
  console.log("Grade: B");
} else if (score >= 50) {
  console.log("Grade: C");
} else if (score >= 40) {
  console.log("Grade: D");
} else {
  console.log("Grade: F");
}
```

How the computer reads it — top to bottom, first match wins:

1. Is 72 >= 70? Yes → print "Grade: A" → **stop; skip the rest**.
2. (Never reached.)

Critical rule: order matters. If you put `score >= 50` first, a score of 72 would print "Grade: C" — wrong. Always list the strictest (largest threshold first, for pass marks) or the most specific condition first.

Say to the class: "The computer is lazy but honest — it stops at the first door that opens. So put the most demanding door first."

Braces `{ }`: They group the instructions belonging to each branch. Missing braces are a very common beginner bug.

Analogy. `if/else if/else` is like a **security post with rules**: "If you are staff, enter here. Else if you are a visitor, sign the book. Else, please step aside."

F.7 Putting it together — functions plus decisions

The two ideas combine powerfully: a function decides what to do.

```
function gradeStudent(score) {
  if (score >= 70) {
    return "A";
  } else if (score >= 60) {
    return "B";
  } else if (score >= 50) {
    return "C";
  } else if (score >= 40) {
    return "D";
  } else {
    return "F";
  }
}

console.log(`Adaeze scored 72 → ${gradeStudent(72)}`);
console.log(`Chinedu scored 45 → ${gradeStudent(45)}`);
```

Output:

```
Adaeze scored 72 → A
Chinedu scored 45 → D
```

Why this pattern is professional: The grading rule lives in exactly one place (`gradeStudent`). If the school changes the pass mark, you edit one function, not fifty scattered lines.

F.8 Summary of the session

- A **function** packages a reusable action; define it once, **call** it to run.
- **Parameters** receive **arguments**; `return` sends a value back (different from `console.log`).
- Comparisons (`===` , `!==` , `>` , `<` , `>=` , `<=`) give `true` / `false` ; prefer `===` .
- Combine conditions with `&&` , `||` , `!` .
- `if` / `else if` / `else` choose the first matching path; order matters.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- `demo-week1-s2/index.html` linking `js/decide.js` at the bottom of the body.
- Whiteboard with a drawn branching diagram (three doors).

G.2 Demonstration 1 — a function that decides pass/fail (7 minutes)

Say: "Let's build a function that tells a student whether they passed a test."

Type into `js/decide.js` :

```
function checkResult(score) {
  if (score >= 50) {
    return "PASS";
  } else {
    return "FAIL";
  }
```

```
}

console.log("Ada:", checkResult(72));
console.log("Ngozi:", checkResult(45));
```

Do on the projector:

1. Save, open the page, press `F12`.
2. Point at `PASS` and `FAIL`.
3. **Pause and ask:** "Before I refresh — what will Ngozi print?" (FAIL.)
4. Then change `checkResult(45)` to `checkResult(50)` and ask: "Pass or fail now, and why?" (PASS — because `>=` includes 50.)

Reasoning to explain aloud: We `return` a value so `console.log` can display it; the decision uses `if/else`; the boundary case `>=` is a deliberate choice ("50 and above passes").

Common error to inject: Change `if (score >= 50)` to `if (score = 50)`. Save, refresh.

- The Console may show `PASS` for both, or behave unpredictably, and there is no error shown for the `=` — **this is why it is dangerous.**
- **Say:** "`=` assigns; `===` / `>=` compare. A single `=` here is a silent bug. Always check your comparison operators."

G.3 Demonstration 2 — multi-branch grading with combined conditions (8 minutes)

Type:

```
function grade(score) {
  if (score < 0 || score > 100) {
    return "INVALID SCORE";
  } else if (score >= 70) {
    return "A";
  } else if (score >= 60) {
    return "B";
  } else if (score >= 50) {
    return "C";
  } else if (score >= 40) {
    return "D";
  } else {
    return "F";
  }
}

console.log(grade(85));   // A
console.log(grade(65));   // B
console.log(grade(50));   // C
console.log(grade(120));  // INVALID SCORE
console.log(grade(-5));   // INVALID SCORE
```

Draw the branches on the whiteboard as doors, and trace `grade(120)` together: first door "score < 0 OR score > 100" → true → INVALID, stop.

Ask: "Why did we put the INVALID check first?" (Because it is the most specific/edge case and `120` should not fall into "A".)

Learners should observe: the first matching branch wins, and `||` lets us catch two bad ranges in one line.

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: Build a "Transport Fare" advisor that decides the fare based on distance and whether the passenger is a student.

Starting materials: New file `js/fare.js` linked to a page `fare.html`.

Procedure:

1. Create `fare.html` linking `js/fare.js`.
2. In `fare.js`, write a function:

```
function transportFare(distanceKm, isStudent) {
  let fare;

  if (distanceKm <= 5) {
    fare = 300;
  } else if (distanceKm <= 15) {
    fare = 600;
  } else {
    fare = 1000;
  }

  if (isStudent) {
    fare = fare - (fare * 0.2);   // 20% student discount
  }

  return fare;
}
```

3. Test it:

```
console.log(transportFare(3, false));   // 300
console.log(transportFare(10, true));   // 600 - 20% = 480
console.log(transportFare(25, false));  // 1000
console.log(transportFare(25, true));   // 1000 - 20% = 800
```

4. **Pause and predict** before running: "What is `transportFare(10, true)`?" (480.)
5. Run and compare.

Expected result: 300, 480, 1000, 800.

Points to pause and predict: After the distance branches; after the discount branch.

Troubleshooting while walking the room:

- `undefined` printed → the function is missing `return fare;`.
- Wrong fare → likely `>` vs `>=` or branch order.
- Syntax error → missing brace `}`.

Completion checklist:

- [ ] Function accepts two parameters
- [ ] Three distance branches
- [ ] Student discount applied
- [ ] Returns (not just logs) the fare
- [ ] All four tests match expected values

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Build a "Movie Ticket Price" calculator for a cinema.

Scenario: *Genesis Cinema* in Port Harcourt charges by age group, and gives a discount on weekday mornings.

- Children under 12: ₦1,000
- Ages 12 to 17: ₦1,500
- Adults 18 and above: ₦2,500
- If it is a weekday morning (`isWeekdayMorning === true`), subtract ₦300 from any price.
- If the age is below 0 or above 120, return `"INVALID AGE"`.

Starting information:

- A variable `age` (number)
- A variable `isWeekdayMorning` (boolean)

Requirements:

1. A function `ticketPrice(age, isWeekdayMorning)` that returns the final price as a number, or `"INVALID AGE"`.
2. Use `if / else if / else`.
3. Validate the age **first**.
4. Print three test cases: a 9-year-old on a weekday morning, a 15-year-old in the evening, and an adult (₦2,500) on a weekday morning.

Constraints: All prices as numbers; use `&&` / `||` or nested `if` for the discount; do not copy the transport example structure word-for-word (design your own branch order).

Time allowance: 15 minutes.

Success criteria:

- `ticketPrice(9, true)` → `700`
- `ticketPrice(15, false)` → `1500`
- `ticketPrice(30, true)` → `2200`
- `ticketPrice(200, false)` → `"INVALID AGE"`
- No red errors.

Hints:

- Hint 1: check `if (age < 0 || age > 120) return "INVALID AGE";` first.
- Hint 2: use `let price` then adjust for the morning discount before returning.
- Hint 3: the discount only applies if `isWeekdayMorning` is `true`.

Instructor answer key:

```
function ticketPrice(age, isWeekdayMorning) {
  if (age < 0 || age > 120) {
    return "INVALID AGE";
  }

  let price;
  if (age < 12) {
    price = 1000;
  } else if (age < 18) {
    price = 1500;
  } else {
    price = 2500;
  }

  if (isWeekdayMorning) {
    price = price - 300;
  }

  return price;
}

console.log(ticketPrice(9, true));    // 700
console.log(ticketPrice(15, false));  // 1500
console.log(ticketPrice(30, true));   // 2200
console.log(ticketPrice(200, false)); // INVALID AGE
```

Marking guidance (out of 10): validation first (2); correct three age branches (3); weekday discount (2); returns number/string correctly (1); all tests correct (2).

J. REAL-WORLD CASE STUDY

Situation. *Mr. Emeka* runs a cooperative savings group in his church. Members contribute monthly. At the end of the year, the group shares the profit based on each member's total contribution. Mr. Emeka has been calculating the share on paper, and last year there were complaints about mistakes.

Relevant facts.

- Total profit to share: ₦600,000.

- Total contributions of all members combined: ₦3,000,000.
- A member's share = (their contribution ÷ total contribution) × total profit.
- Members who paid all 12 months on time get a 5% loyalty bonus on their share.

Decision point. How can a function make the yearly calculation trustworthy and repeatable?

Possible approaches.

1. Continue by hand and double-check with a calculator.
2. Build a function `memberShare(contribution, totalContribution, profit, paidAllYear)` that returns the share, added to a loyalty bonus where earned.

Recommended solution. Approach 2. The formula and the bonus rule live in one place; every member is treated identically; the result can be printed for the meeting.

Example solution:

```
function memberShare(contribution, totalContribution, profit, paidAllYear) {
  if (contribution <= 0) {
    return 0;
  }
  let share = (contribution / totalContribution) * profit;
  if (paidAllYear) {
    share = share * 1.05;   // +5% loyalty
  }
  return share;
}

console.log(memberShare(150000, 3000000, 600000, true));   // 30000 * 1.05 = 31500
console.log(memberShare(150000, 3000000, 600000, false));  // 30000
```

Trade-offs.

- Approach 1 relies on the same person who is also handling money — a conflict and an error risk.
- Approach 2 takes one hour to build and removes the dispute risk; the church cooperative can re-run it any year. It uses exactly the skills taught today.

Professional connection. Cooperative and small-business accounting tools are built exactly like this — a rule expressed once as a function, applied consistently. Junior developers are often asked to "automate this spreadsheet formula" — this is that task.

K. COMMON MISTAKES AND TROUBLESHOOTING

K.1 The function prints nothing

Symptoms: Page loads, Console empty. **Likely causes:** (a) The function was defined but never **called**. (b) The script file is not linked (see Session 1, K.1). **Investigate:** Search your code for the function name followed by `()`. If it only appears once (at the definition), it was never called. **Fix:** Add a call, e.g. `checkResult(70);`. **Confirm:** Output appears. **Prevent:** Always write at least one test call.

K.2 `undefined` instead of a value

Symptoms: `console.log(myFunction(5))` prints `undefined`. **Likely cause:** The function has no `return`, or `return` is on a path that was not taken. (Confirmed by reading the function.) **Fix:** Ensure every branch returns a value. A common bug is a `console.log` where a `return` was needed. **Prevent:** Decide whether a function *reports* (log) or *produces a value* (return).

K.3 The wrong branch runs

Symptoms: Score 72 prints "C" instead of "A". **Likely cause:** Branch order — a looser condition appears before a stricter one. **Fix:** Reorder so the strictest condition is first. **Prevent:** When using ranges, go from highest/lowest threshold inward and test boundary values.

K.4 `=` used instead of `===` in a condition

Symptoms: Wrong result with **no error message** (the most dangerous kind). **Likely cause:** `if (x = 5)` assigns instead of compares. **Fix:** Change to `===` (or the intended comparison). **Prevent:** Read conditions aloud as a question: "is x equal to five?" → `===`.

K.5 `else` without `if` / unexpected token

Symptoms: Red syntax error near `else` or `}`. **Likely cause:** Missing or extra brace `{ }`; or `else` detached from its `if` by a stray semicolon. **Fix:** Count braces; ensure `else` follows `if` / `else if` directly. **Prevent:** Indent consistently so nesting is visible.

K.6 `"5" === 5` is false but the learner expected true

Symptoms: A comparison gives unexpected `false` / `true`. **Likely cause:** Type mismatch — a string compared with a number. **Investigate:** `console.log(typeof x)`. **Fix:** Convert with `Number("5")` or use the correct type from the start. **Prevent:** Be deliberate: are you comparing text or numbers?

L. SESSION ASSIGNMENT

Assignment title: "Electricity Bill Estimator"

Purpose: Apply functions, conditions and comparison operators to a realistic recurring household task.

Scenario: A local estate calculates monthly electricity charges on a tiered system. Write a function that returns a customer's bill based on units consumed.

Rules:

- First 100 units: ₦50 per unit.
- Next 100 units (101–200): ₦70 per unit.
- Above 200 units: ₦100 per unit.
- If units are negative, return `"INVALID READING"`.

Detailed instructions:

1. Write a function `electricityBill(units)` that calculates and returns the total in naira.
2. Test with four cases: 80 units, 150 units, 260 units, and -10 units.
3. Print each result with a clear label.

Required deliverables: `assignment2/index.html`, `assignment2/js/bill.js`, and the Console output recorded in the workbook.

Submission format: Files in the class folder; Console output pasted/photographed.

Deadline: Before Session 3.

Marking rubric (20 marks):

Criterion	Marks	Strong answer
Validation (negative)	3	Returns "INVALID READING" first
Tier 1 calculation	4	80 units → 4000
Tier 2 calculation	4	150 units → 100×50 + 50×70 = 8500
Tier 3 calculation	4	260 units → 100×50 + 100×70 + 60×100 = 18000
Function returns value	3	Uses `return`, not just log
Clean output / no errors	2	Clear labels, no red errors

Common mistakes to avoid: charging the *whole* bill at one tier instead of *splitting* units across tiers; forgetting the invalid check; returning nothing.

Instructor reference solution:

```
function electricityBill(units) {
  if (units < 0) {
    return "INVALID READING";
  }

  let bill = 0;
  if (units <= 100) {
    bill = units * 50;
  } else if (units <= 200) {
    bill = (100 * 50) + ((units - 100) * 70);
  } else {
    bill = (100 * 50) + (100 * 70) + ((units - 200) * 100);
  }
  return bill;
}

console.log(`80 units: ₦${electricityBill(80)}`);     // 4000
console.log(`150 units: ₦${electricityBill(150)}`);   // 8500
console.log(`260 units: ₦${electricityBill(260)}`);   // 18000
console.log(`-10 units: ${electricityBill(-10)}`);    // INVALID READING
```

M. KNOWLEDGE CHECK

Q1. Short answer. What is the difference between **defining** a function and **calling** it?

Q2. Multiple choice. Which function correctly returns the larger of two numbers? a) `function max(a,b){ console.log(a>b?a:b); }` b) `function max(a,b){ return a>b?a:b; }` c) `function max(a,b){ return a>b; }` d) `function max(a,b){ a>b?a:b; }`

Q3. Explain in your own words. Why is `===` preferred over `==` in professional JavaScript?

Q4. Practical identification. What does each print? a) `console.log(10 > 5 && 3 > 7);` b) `console.log(10 > 5 || 3 > 7);` c) `console.log(!(10 > 5));`

Q5. Error detection. Find and fix the bug:

```
function applyDiscount(price, percent) {
  discount = (price * percent) / 100;
  console.log(price - discount);
}
let finalPrice = applyDiscount(5000, 10);
console.log(finalPrice);
```

Q6. Scenario. A data bundle plan: first 2GB free; each additional GB costs ₦250; if the total bill would exceed ₦2,000, cap it at ₦2,000. Write the `if/else` logic (or full function) that returns the cost given `extraGb`.

INSTRUCTOR ANSWER KEY (Q1–Q6)

A1. Defining writes the recipe and does not run it; calling executes the body. A defined-but-uncalled function does nothing. **A2.** (b) — it `return`s the value; (a) only logs, (c) returns a boolean, (d) returns `undefined`. **A3.** `==` converts types and causes surprises (e.g. `"5" == 5` is true); `===` checks value **and** type, so bugs are caught instead of hidden. **A4.** a) `false` (3>7 is false), b) `true`, c) `false`. **A5.** The function uses `console.log` instead of `return`, so `finalPrice` is `undefined`. Fix:

```
function applyDiscount(price, percent) {
  const discount = (price * percent) / 100;
  return price - discount;
}
console.log(applyDiscount(5000, 10));   // 4500
```

A6.

```
function dataCost(extraGb) {
  if (extraGb <= 0) return 0;
  let cost = extraGb * 250;
  if (cost > 2000) {
```

```
    cost = 2000;
  }
  return cost;
}
```

N. REFLECTION AND RECAP

Plain-English summary. Today you learned to give your programs a brain. Functions let you write an action once and reuse it, with parameters as inputs and `return` to send a result back. Comparison operators turn questions into `true` / `false`, and logical operators (`&&`, `||`, `!`) combine questions. `if` / `else if` / `else` let a program take different paths, and the first matching branch wins.

Five important takeaways:

1. Define a function once; call it as often as you like.
2. `return` sends a value out; `console.log` only shows it.
3. Use `===` to compare; `=` assigns.
4. `&&` needs both; `||` needs one; `!` flips.
5. In `if/else if`, the first true branch runs — order matters.

Questions to help you recognise what you can now do:

- Can I write a function with two parameters and a `return`?
- Can I combine two conditions with `&&`?
- Can I explain why `"5" === 5` is false?
- Can I trace which branch runs for a given input?

Connection to the next session: We can now store values and make single decisions. Next session we handle **many** values at once — **arrays** and **objects** — and use **loops** to process lists like a shopping cart or a class register.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner writes a function and **calls** it without prompting.
- Learner uses `return` where a value is needed, and can explain the difference from `console.log`.
- Learner reorders branches to fix an incorrect result.
- Learner checks a boundary case (e.g., exactly 50) without being asked.

Common misconceptions:

- "Defining = running." Reinforce with the uncalled-function demo.
- "`console.log` and `return` are the same." Show the `undefined` proof.
- "`=` and `===` are interchangeable." Show the silent-bug demo from G.2.
- "Branches are checked independently." Clarify first-match-wins.

Suggested remedial exercises:

1. Give five `if/else` snippets and ask learners to write the output on paper.
2. Give a function with `console.log` where `return` is required and ask them to fix and test.
3. Boundary drills: test `>=` vs `>` at the exact threshold.

Remedial exercise (mirror): Ask learners to rewrite the transport-fare function for a different city with different prices — same structure, new numbers.

Differentiation:

- **Level 1:** provide the function skeleton with branches stubbed and one test pre-filled.
- **Level 2:** the exercise as written.

- **Level 3:** add a third parameter (e.g., `isNight`) with its own surcharge, and validate multiple invalid inputs.

Timing guidance: The independent exercise is the heart of the session. If time is short, reduce the case study to a 2-minute talk and keep the practical intact.

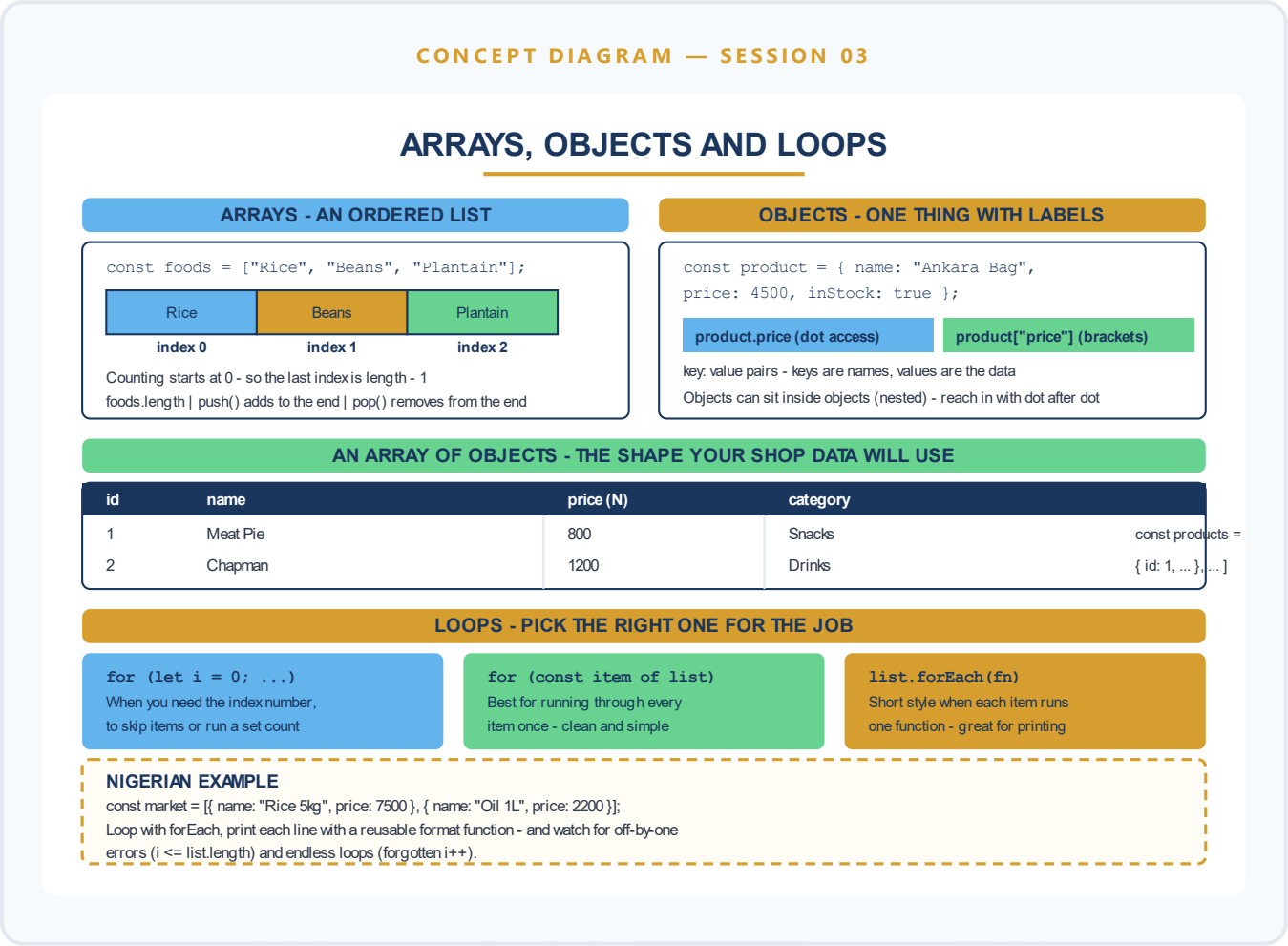
END OF SESSION 2

Next: Session 3 — Arrays, Objects and Loops.

SESSION 03 OF 12

ARRAYS, OBJECTS AND LOOPS

CONCEPT DIAGRAM — SESSION 03



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 3 — ARRAYS, OBJECTS AND LOOPS

A. SESSION IDENTIFICATION

Item	Detail
Session number	3 of 12
Session title	Arrays, Objects and Loops
Week / Module	Week 1 — Module 1: JavaScript Foundations
Recommended duration	105 minutes
Difficulty level	Beginner (a step up from Sessions 1–2 — this is the session where data structures appear)

Item	Detail
Prerequisite knowledge	Session 1 — variables (`let` / `const`), data types, `console.log` , template literals, arithmetic, linking a script file. Session 2 — functions (definition, parameters, `return`) and `if/else` decisions. A learner who cannot write a function with a parameter and a `return` should be paired with a stronger learner for today and given the remedial drill in Section O.
Connection to previous session	In Session 2 a function could handle **one** value well (one score, one fare, one bill). But a shop does not have one product — it has *many*. Today we learn how to store **many** values in one place (arrays and objects) and how to walk through them one at a time with a **loop**, reusing the functions from Session 2 to format each one. This is the exact data shape used for the Adaeze Fashion capstone shop.
Preparation before class	Academy PCs on and logged in; Chrome and VS Code open; learner practice folders from Sessions 1–2 available; projector showing the instructor's VS Code **and** the Chrome Console (Console is essential today). Print Worksheet 3.1 and 3.2. Have a whiteboard or flipchart for drawing "boxes in a row" (array) and "labelled drawers" (object).

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Create an array** using square brackets `[ ]` and **explain what an array is** using a real-world comparison.
2. **Read and update array items** using index numbers, and **explain clearly why indexing starts at 0** (not 1).
3. **Use** `.length` to find how many items an array holds, and **use** `push()` **and** `pop()` to add and remove items from the end.
4. **Create an object** using curly braces `{ }` with key/value pairs, and **read a value** using both dot access (`product.price`) and bracket access (`product["price"]`).
5. **Update an object's value** and **access a nested object** (an object inside an object).
6. **Build an array of objects** — for example a list of products `{ name, price, category }` — which is the data shape used for the capstone shop.
7. **Write and choose between** a classic `for` loop, a `for...of` loop, and `forEach` , and **say when each is the best choice**.
8. **Iterate over an array of objects** and print a formatted product catalogue to the Console, **reusing a function** (from Session 2) to format and print each product.
9. **Diagnose common loop and indexing errors** — off-by-one errors, a loop that never ends, and using `=` instead of `<=` .

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). Pairing is acceptable: one learner types, the other predicts the console output and reads errors aloud; swap halfway.
- Instructor computer connected to a projector or large screen.

Software

- Google Chrome (any recent version) — for the Console and the DevTools.
- Visual Studio Code (VS Code).
- Windows 10 or Windows 11.
- No new software is needed; today is a pure JavaScript session.

Accounts

- None required. All work is offline.

Sample files

- The instructor prepares and shares a folder `demo-week1-s3/` containing `index.html` and `js/catalogue.js` (full contents in Section G).

- Learners continue in their existing `webdev-practice` folder with a new file `js/products.js`.

Worksheets

- Worksheet 3.1 — "Count the Boxes" (given arrays, learners write the index of each item and the `.length`; 1 page).
- Worksheet 3.2 — "Label the Drawers" (given objects, learners write how to read and update each value using dot and bracket access).
- Worksheet 3.3 — "Which Loop?" (three mini-tasks; learners tick the best loop to use and write one sentence why).
- These live in the learner workbook (Part 3).

Tools

- Chrome DevTools Console (built into Chrome — press `F12`). Learners may also type array/Object snippets directly into the Console to experiment, as in Session 1.

Internet

- Not required. If the academy has internet, the instructor may optionally show the MDN pages for `Array` and `Object` at the very end, but do not depend on it.

Alternative resources for learners lacking facilities

- Any laptop with Chrome and VS Code works fully offline. If a learner has only a phone, pair them with a desktop user for the session; ask them to photograph the code and the console output, then practise typing the examples later in a phone browser's code-editor site (for example a free online `HTML/CSS/JS` playground). No learner should be left without a role: phone-only learners can be the "predictor" who guesses each console line before it runs.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open the `demo-week1-s3/index.html` page with the Chrome Console already open. Keep the code hidden at first; show only the Console.

What the instructor says:

> *"Quick question. At Adaeze Fashion shop, suppose I sell hair cream, and I want to print the name and price for each product. With what you learned last week, how would I do it?"*

Let learners answer. They will say: make a variable for each product, then `console.log` it.

Then show the pain (type this live in the Console or in a scratch file):

```
const product1 = "Hair Cream";
const price1 = 2500;
const product2 = "Shea Butter";
const price2 = 1800;
const product3 = "Coconut Oil";
const price3 = 3200;

console.log(`${product1} - ₦${price1}`);
console.log(`${product2} - ₦${price2}`);
console.log(`${product3} - ₦${price3}`);
```

Say:

> *"That worked for three products. But Adaeze has two hundred products. Am I going to write two hundred variables and two hundred `console.log` lines? That is madness. What if I could keep all the products in **one** list, and tell the computer: 'go through the whole list and print each one'? That single container is called an **array**, each product is an **object**, and the instruction 'go through each one' is a **loop**. Today, three products or three thousand — the code is the same length."*

Expected learner response: Learners usually laugh at the "two hundred lines" idea and immediately see the problem. Some may say "use a loop" or "put them in a list" — accept that and say, "Yes! That is exactly the word — a list. In JavaScript a list is called an array."

Why this opener works: It creates a real, felt problem (repetition does not scale) before giving the tool (arrays + loops). The learner now *wants* the array, instead of being told to memorise one.

E. PREREQUISITE REVIEW (7 minutes)

Run this as a fast oral round-robin. Put the four questions on the projector.

Question 1: "Write a function `greet(name)` that **returns** the text `Hello, <name>!`."

- Expected answer:

```
function greet(name) {
  return "Hello, " + name + "!";
}
```

- Common gap: using `console.log` instead of `return`. Remind them: `return` sends the value back so we can use it again — and we will reuse exactly this idea today to format each product.
- Common gap: forgetting quotes. `name` is a parameter, not text.

Question 2: "What is the difference between `let` and `const`?"

- Expected answer: `let` can be reassigned; `const` cannot be reassigned. (Important today: we will use `const` for most of our data containers.)
- Common gap: learners think `const` means the *contents* can never change. We will gently correct this today: a `const` array can still have items added to it — it is the *label* that cannot be repointed.

Question 3: "What does `console.log(typeof "2500")` print, and why does it matter?"

- Expected answer: `"string"` — because it has quotes. It matters because `"2500" + "1800"` glues text (`"25001800"`) instead of adding. Prices must be real numbers.
- Common gap: guessing `number`. Re-run `typeof` live to prove it.

Question 4: "What does `2 + 3 * 4` print, and why?"

- Expected answer: `14`, because multiplication happens before addition.
- Common gap: `20`. Correct using order of operations.

Bridge to today (say this):

> *"You can store a value, make a decision, and package an action into a function. Today, we store **many** values at once, and we let the computer **repeat an action** for each value automatically. Think of it this way: a function is one worker doing a job. A loop is the supervisor telling that worker, 'do this job for every item on the list.'"*

If more than a third of the class cannot answer Question 1, run the 5-minute pair drill in Section O (remedial) before continuing.

F. FULL LECTURE NOTES

F.1 Arrays — one box that holds a row of values

The idea. An **array** is an ordered list of values stored in a single variable.

Plain-English definition. An array is like a **counter in a market stall**. It has many slots in a row, and each slot holds one item. You can point to the first slot, the second slot, and so on. Instead of naming every item separately (`item1` , `item2` , `item3`), the whole row lives under one name, and you reach the items by their position number.

Why arrays exist. Programs constantly deal with *lists*: a list of products, a list of students, a list of scores, a list of WhatsApp contacts. Repeating a variable for every item does not scale (the opening activity proved that). An array lets you keep all of them together, count them with `.length`, and process them with a loop.

How to create an array — using square brackets `[ ]`:

```
const products = ["Hair Cream", "Shea Butter", "Coconut Oil"];
const prices = [2500, 1800, 3200];
const isAvailable = [true, true, false];
```

Line by line:

- `const products =` — declare a constant variable named `products`.
- `[...]` — the square brackets say "this is an array".
- `"Hair Cream", "Shea Butter", "Coconut Oil"` — the items, separated by **commas**.
- `;` — end of the instruction.

An array can hold strings, numbers, booleans — even other arrays or objects (we will do that soon). In this course we mostly store one *kind* of thing per array.

Real-life comparison. An array is a **numbered row of lockers**. Each locker has a number. The lockers sit in a fixed order. Locker number 0 is the first one.

F.2 Reading array items — indexing, and why it starts at 0

The idea. Each item in an array has a **position number** called its **index**. You read an item with its index in square brackets: `products[0]`.

```
const products = ["Hair Cream", "Shea Butter", "Coconut Oil"];

console.log(products[0]);   // "Hair Cream"
console.log(products[1]);   // "Shea Butter"
console.log(products[2]);   // "Coconut Oil"
console.log(products[3]);   // undefined  ← there is no fourth item
```

Why did the first item use `[0]` and not `[1]`? This confuses almost every beginner, so let us be very clear.

> *In programming, the index is not "how many items come before this one plus one" — it is **how many steps from the start**. The first item is at the start, so you take **zero** steps to reach it. The second item is one step away, so its index is `1`. The third is two steps away: `2`.*

Another way to see it: the index counts **the gap before the item**, like house numbers on a street corner where the first house is gate 0. Or think of a queue at a bank: the very first person is "0 people in front of you." That is why we say the index is a *zero-based* count.

A concrete table for our products array:

Item	Index
"Hair Cream"	0
"Shea Butter"	1
"Coconut Oil"	2

The `undefined` trap. If you ask for `products[3]`, the answer is `undefined` because there is no item at that position. There is no error — just the word `undefined`. This is a very common source of confusion, and it is why counting correctly matters. A good safety habit: the **last valid index is always** `array.length - 1`.

Say to the class:

> *"If you remember one sentence from today: the first item is at `0`, and the last item is at `length - 1`. Write it in your workbook."*

F.3 `.length` — how many items are there?

The idea. Every array knows how many items it holds. That number is stored in a property called `.length`.

```
const products = ["Hair Cream", "Shea Butter", "Coconut Oil"];

console.log(products.length);   // 3
```

Why it matters. A loop (coming in Section F.7) needs to know when to stop. Instead of hard-coding "3" — which breaks the moment Adaeze adds a fourth product — we let the computer check `.length` itself. This is a habit professional developers keep: **never hard-code the count of a list; ask the list.**

Common misconception. New learners sometimes write `products.length()` with brackets, thinking it is a function. It is a **property**, not a function — no brackets. `products.length` is correct.

F.4 Adding and removing items — `push()` and `pop()`

The idea. An array can grow or shrink. Two everyday tools work at the **end** of the array:

- `push(item)` — **adds** an item to the end.
- `pop()` — **removes** the last item and hands it back to you.

```
const products = ["Hair Cream", "Shea Butter"];

products.push("Coconut Oil");
console.log(products);          // ["Hair Cream", "Shea Butter", "Coconut Oil"]

const removed = products.pop();
console.log(removed);           // "Coconut Oil"  (pop returns what it removed)
console.log(products);          // ["Hair Cream", "Shea Butter"]
```

Important surprise for today: `products` was declared with `const`, yet `push()` changed it! Is that not "changing a constant"? No. The rule is subtle and worth explaining carefully:

> *`const` locks the **name** to **one array**. It says: "`products` must always point to this same list." But the contents of that list can still change — you can add or remove items freely. You simply cannot do `products = ["something else"]` (that would repoint the name, and JavaScript would throw `Assignment to constant variable`).*

Analogy. A `const` array is like a **specific basket** you own. You can keep putting things into the basket and taking them out. You just cannot swap your basket for a different basket. The basket stays the same basket; its contents can change.

Why push matters for the capstone: When a shopper clicks "Add to Cart", the cart array grows with `push()`. When they remove an item, we use `pop()` (or a more specific removal we will learn later). You are already using patterns the capstone needs.

F.5 Objects — one box with labelled drawers

The idea. An **object** stores related information as **key/value pairs**. Each value has a *name* (the key) so you can find it later.

Plain-English definition. If an array is a **numbered row of lockers**, an object is a **single chest of drawers where every drawer has a written label**: one drawer labelled "name", another labelled "price", another labelled "category". You do not reach a drawer by its position number — you reach it by its **label**.

Why objects exist. A product is not just a name. It has a name, a price, a category, maybe how many are in stock. These belong together. An array holds a *list of things*; an object describes **one thing** in detail. Used together they are unstoppable: an array of objects is a *list of detailed things*.

How to create an object — using curly braces `{ }`:

```
const product = {
  name: "Hair Cream",
  price: 2500,
  category: "Skincare",
```

```
  inStock: true
};
```

Line by line:

- `const product =` — declare a constant named `product`.
- `{ ... }` — the curly braces say "this is an object".
- `name: "Hair Cream"` — a key/value pair. `name` is the **key** (the label), `"Hair Cream"` is the **value**.
- Note the **colon** `:` between key and value, and the **comma** `,` after each pair (but not after the last one).
- Keys are usually written without quotes (though `"name"` with quotes is also allowed).

F.6 Reading and updating object values — dot and bracket access

The idea. There are two ways to read a value from an object.

Way 1 — Dot access (the common way):

```
console.log(product.name);      // "Hair Cream"
console.log(product.price);     // 2500
```

`product.name` reads: "from the object called `product`, give me the value stored under the key `name`."

Way 2 — Bracket access (the flexible way):

```
console.log(product["name"]);   // "Hair Cream"
console.log(product["price"]);  // 2500
```

Notice the key is written as a **string in quotes** inside the brackets. `product.name` and `product["name"]` give the same answer.

When must you use brackets? When the key is held in a variable, or when the key has a space or special character. For example:

```
const field = "price";
console.log(product[field]);          // 2500  (looks up the key stored in 'field')

const shopItem = { "unit price": 2500 };  // key with a space
console.log(shopItem["unit price"]);      // 2500  (dot access cannot do this)
```

For everyday use, most developers prefer dot access because it is shorter and clearer. Learn both; the capstone and later `fetch` lessons rely on brackets.

Updating a value. Assign a new value to the key, exactly like updating a variable:

```
product.price = 2700;           // price is now 2700
console.log(product.price);     // 2700

product["category"] = "Haircare";  // bracket update also works
console.log(product.category);     // "Haircare"
```

Adding a new key works the same way — assign to a key that does not exist yet, and it is created:

```
product.discount = 10;          // adds a new key 'discount'
console.log(product);           // { name:"Hair Cream", price:2700, category:"Haircare", inStock:true, discount:10 }
```

Common misconception. Learners try `product[price]` (no quotes). JavaScript then looks for a *variable* named `price`, which does not exist — it throws `ReferenceError: price is not defined` (or reads `undefined`). Dot access uses the bare word; bracket access needs a **string in quotes**, unless you deliberately want the variable lookup described above.

F.7 Nested objects — an object inside an object

The idea. A value inside an object can itself be an object. This is called a **nested object**, and it is how real data is usually shaped.

```
const customer = {
  name: "Adaeze",
  phone: "0803 000 0000",
  address: {
    street: "24 Ebony Road",
    city: "Port Harcourt",
    state: "Rivers"
  }
};
```

To reach a nested value, chain the keys with dots:

```
console.log(customer.name);           // "Adaeze"
console.log(customer.address.city);   // "Port Harcourt"
console.log(customer.address.state);  // "Rivers"
```

Real-life comparison. It is a **big chest of drawers where one drawer contains a smaller chest of drawers**. You open the big drawer labelled "address", then inside it open the small drawer labelled "city".

Why this matters in real work: When we later fetch product data from the internet (Session 7), it arrives as a deeply nested object. If you can follow the chain of dots, you can read any real data. The skill starts here.

F.8 Array of objects — the shape of real data

The idea. Put several objects inside one array. Now you have a **list of detailed things** — the single most common data structure in front-end development.

```
const products = [
  { name: "Hair Cream",  price: 2500, category: "Skincare" },
  { name: "Shea Butter", price: 1800, category: "Skincare" },
  { name: "Coconut Oil", price: 3200, category: "Haircare" },
  { name: "Herbal Soap", price: 1200, category: "Bath" }
];
```

Reading it:

- `products` is an array (square brackets).
- Each item inside is an object (curly braces).
- `products.length` is `4`.
- `products[0]` is the **first object**: `{ name: "Hair Cream", price: 2500, category: "Skincare" }`.
- `products[0].name` is `"Hair Cream"` — **first pick the object by index, then pick the value by key**. Read it left to right: "products, item zero, the name."

```
console.log(products[0].name);       // "Hair Cream"
console.log(products[2].price);      // 3200
console.log(products[3].category);   // "Bath"
```

Say to the class:

> *"This is the shape of the capstone. When you build the Adaeze shop, your products will look exactly like this — an array of objects, each with a name, a price and a category. Learn this shape and the capstone becomes easy."*

Common misconception. Learners write `products.name` (forgetting the index). An array does not have a `name` key — only the *objects inside it* do. You must first pick the item: `products[0].name`.

F.9 Loops — doing a job for every item

The idea. A **loop** repeats a block of code. For arrays, the usual job is: *"do this once for each item in the array."*

There are three loops we use in this course. We will learn all three and, crucially, learn **when to use each**.

F.9.1 The classic `for` loop

```
const products = ["Hair Cream", "Shea Butter", "Coconut Oil"];

for (let i = 0; i < products.length; i++) {
  console.log(products[i]);
}
```

Output:

```
Hair Cream
Shea Butter
Coconut Oil
```

Line by line — this is the most important line of the session, so slow down:

- `let i = 0` — **start**: create a counter `i` and set it to `0`. We start at `0` because arrays start at index `0`.
- `i < products.length` — the **condition to continue**: keep looping as long as `i` is less than the array's length. When `i` reaches `3` and length is `3`, `3 < 3` is false, so the loop **stops**.
- `i++` — the **step**: after each pass, add `1` to `i` (`i++` is shorthand for `i = i + 1`).
- `{ ... }` — the **body**: the code that repeats. Here it prints `products[i]`, the item at the current position.

Trace it on the whiteboard with the class:

Pass	`i`	`i < 3`?	Prints
1	0	true	"Hair Cream"
2	1	true	"Shea Butter"
3	2	true	"Coconut Oil"
4	3	false	(loop ends)

Notice the loop ran **three** times but `i` reached `3` before stopping — and we **never** printed `products[3]`. That is correct and it is exactly why the condition is `i < length` (strictly less than), not `i <= length`.

The off-by-one trap (the classic beginner bug):

- If you write `i <= products.length`, the last pass uses `i = 3`, and `products[3]` is `undefined`. You get a stray `undefined` printed at the end. This is called an **off-by-one error**.
- **Rule:** for arrays, the condition is `i < array.length` (not `<=`).

Real-life comparison. A `for` loop is a **teacher calling the register by seat number**: "Seat 0, stand up. Seat 1, stand up. Seat 2..." The teacher knows to stop when there are no more seats (`i < length`).

F.9.2 The `for...of` loop

When you only need the **items** and do **not** care about the index, `for...of` is cleaner:

```
for (const item of products) {
  console.log(item);
}
```

Output — exactly the same:

```
Hair Cream
Shea Butter
Coconut Oil
```

- `const item of products` reads naturally: "for each `item` in `products`".
- On each pass, `item` holds the next product. No counter, no length, no `i++` — so no off-by-one error is even possible.
- Use `for...of` when you want **each value**, not its position.

Prefer `for...of` for reading — it is shorter and safer. Use the classic `for` loop when you specifically need the **index number** (for example, "item 1 of 3").

F.9.3 `forEach` — a loop built into the array

Arrays have a built-in method called `forEach` that runs a **function** once for each item. This connects directly to your Session 2 functions:

```
products.forEach(function (item) {
  console.log(item);
});
```

Output — the same three lines. Explanation:

- `products.forEach(...)` means "for each item in products, run this function".
- The function we pass in receives one argument — here named `item` — which is the current product.
- JavaScript calls that little function automatically for every item, behind the scenes.

You will often see the same thing written more briefly with an **arrow function** (a shorter function style introduced in modern JavaScript):

```
products.forEach((item) => {
  console.log(item);
});
```

`(item) => { ... }` is a compact way of writing `function (item) { ... }`. They mean the same thing for our purposes. Arrow functions are extremely common in real code and in the capstone, so see them today and get comfortable.

`forEach` **gives you the index too, if you want it** — the function can take a **second** parameter for the position:

```
products.forEach((item, index) => {
  console.log(`${index + 1}. ${item}`);
});
```

Output:

```
1. Hair Cream
2. Shea Butter
3. Coconut Oil
```

We added `1` to the index for display, because a human counting a list says "1, 2, 3" — the machine still uses 0, 1, 2 internally. This tiny example shows how the "starts at 0" idea meets the "people count from 1" reality.

F.9.4 Which loop should I use?

Loop	Best when	Watch out for
classic `for`	You need the **index number** (position), or you need full control of start/stop/step.	Off-by-one: use `i < arr.length`.
`for...of`	You just want **each item** and no index.	It gives values, not indexes.
`forEach`	You want to run a **function** for each item; very readable; common in modern code.	You cannot use `break` to stop early; it always runs the whole list.

Say to the class: "Any of the three can print a product list. Choose by *intent*: need positions → classic `for`; just the items → `for...of`; pass a function to each item → `forEach`. Professionals pick the clearest one for the job."

F.10 Putting it together with Session 2 — a function that formats one product

Here is where today meets last session. Instead of writing the print inside the loop, we write **one function** that formats a single product, then call it from the loop. This is exactly the DRY principle from Session 2.

```
function formatProduct(product) {
  return `${product.name} — ₦${product.price} [${product.category}]`;
}

const products = [
  { name: "Hair Cream",  price: 2500, category: "Skincare" },
  { name: "Shea Butter", price: 1800, category: "Skincare" },
  { name: "Coconut Oil", price: 3200, category: "Haircare" },
  { name: "Herbal Soap", price: 1200, category: "Bath" }
];

console.log("===== ADAEZE FASHION — PRODUCT CATALOGUE =====");

for (const product of products) {
  console.log(formatProduct(product));
}
```

Expected Console output:

```
===== ADAEZE FASHION — PRODUCT CATALOGUE =====
Hair Cream — ₦2500 [Skincare]
Shea Butter — ₦1800 [Skincare]
Coconut Oil — ₦3200 [Haircare]
Herbal Soap — ₦1200 [Bath]
```

Why this is the professional pattern:

- `formatProduct` describes **how one product looks**. If Adaeze wants to add the stock status tomorrow, you edit **one** function, not every line.
- The loop describes **how to go through the list**. The two jobs are separated.
- Add a fifth product to the array and the output grows automatically — no code change. That is the whole point of Section D's promise.

An `if` inside the loop (combining Session 2 again). Real catalogues flag special items:

```
for (const product of products) {
  if (product.price > 3000) {
    console.log(`⭐ PREMIUM: ${formatProduct(product)}`);
  } else {
    console.log(formatProduct(product));
  }
}
```

Say to the class: "Inside a loop, you can do anything you already know — call functions, make decisions. A loop is just a 'repeat' wrapper around skills you already have."

F.11 Summary of the session

- An **array** `[ ... ]` is an ordered list; read items by **index**, which **starts at 0**; the last index is `length - 1`.
- `.length` tells you how many items; `push()` adds to the end; `pop()` removes the last.
- An **object** `{ key: value }` describes one thing; read values with **dot** (`obj.key`) or **bracket** (`obj["key"]`) access; update by assigning; nest objects for detailed data.
- An **array of objects** `[{...}, {...}]` is the shape of real data (and the capstone shop).
- Loops repeat work: the classic `for` (need the index), `for...of` (just the items), and `forEach` (pass a function).
- Reuse a **function** to format each item — Session 2 and Session 3 working together.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- A folder `demo-week1-s3/` on the Desktop containing:
  - `index.html` (links `js/catalogue.js` at the bottom of the body)
  - a subfolder `js/` containing `catalogue.js`
- Both files open in VS Code; Chrome open to the page; DevTools Console open on the projector.
- Whiteboard with three things pre-drawn lightly: (1) a row of three boxes numbered 0, 1, 2; (2) a chest of drawers labelled name/price/category; (3) a loop arrow circling the boxes.

G.2 Demonstration 1 — arrays, indexing and `.length` (5 minutes)

Say: "Watch how one variable can hold a whole row of items."

Type into the Console directly (so learners see instant feedback), then later into the file:

```
const products = ["Hair Cream", "Shea Butter", "Coconut Oil"];
console.log(products[0]);
console.log(products[1]);
console.log(products[2]);
console.log(products.length);
```

Do on the projector:

1. Press Enter after each line and point at each result.
2. **Ask before pressing Enter on the last line:** "What will `products[0]` print — 'Hair Cream' or 'Shea Butter'? Remember, we start at 0." (Hair Cream.)
3. Then deliberately type `products[3]` and press Enter → prints `undefined`.
   - **Say:** "There is no item at position 3. Four items would be 0,1,2,3. The last valid index is always `length - 1` — here, `3 - 1 = 2`."

Deliberately break it: type `products.length()` (with brackets) → red error `TypeError: products.length is not a function`.

- **Say:** "`.length` is a property — a plain fact the array knows about itself. No brackets. I write `products.length`."

Questions to ask:

- "What is the index of the *middle* item?" (1.)
- "If I `push` a fourth item, what will `.length` become?" (4.)

G.3 Demonstration 2 — objects and array of objects (5 minutes)

Say: "Now let us describe one product properly, then a whole list of them."

Type into `catalogue.js`:

```
const oneProduct = {
  name: "Hair Cream",
  price: 2500,
  category: "Skincare"
};

console.log(oneProduct.name);        // Hair Cream
console.log(oneProduct["price"]);    // 2500

oneProduct.price = 2700;             // update
console.log(oneProduct.price);       // 2700
```

Walk through each line on the projector:

1. `{ }` with keys and values, colons and commas.
2. Dot access for `name`; bracket access with a quoted string for `price` — both give the same kind of result.
3. Updating `price` by assignment, then reading it again shows `2700`.

Then build the array of objects:

```
const products = [
  { name: "Hair Cream",  price: 2700, category: "Skincare" },
```

```
  { name: "Shea Butter", price: 1800, category: "Skincare" },
  { name: "Coconut Oil", price: 3200, category: "Haircare" }
];

console.log(products[0].name);   // Hair Cream  ← index first, then key
console.log(products[2].price);  // 3200
```

Ask: "In `products[0].name`, which do we do first — pick the object, or pick the key?" (Pick the object by index first, then the key.)

Error to inject: change `products[0].name` to `products.name` → prints `undefined`.

- **Say:** "`products` is the *array*; it has no `name` key. Only the objects inside have names. I must first pick an item with an index."

G.4 Demonstration 3 — the loop and the reused function (5 minutes)

Say: "Now the magic — one short block that handles three products, or three thousand."

Type:

```
function formatProduct(product) {
  return `${product.name} — ₦${product.price} [${product.category}]`;
}

console.log("===== ADAEZE FASHION — PRODUCT CATALOGUE =====");

for (const product of products) {
  console.log(formatProduct(product));
}
```

1. **Pause and predict:** "Before I run it — how many lines will print below the heading?" (Three — one per product.)
2. Run it; confirm the three catalogue lines.
3. Now **push a fourth product** and run the same loop again without touching the loop:

   ```
   products.push({ name: "Herbal Soap", price: 1200, category: "Bath" });
   for (const product of products) {
     console.log(formatProduct(product));
   }
   ```

   - **Say:** "I changed only the data. The loop stayed exactly the same. This is why loops matter."

Now show the classic `for` loop side by side:

```
for (let i = 0; i < products.length; i++) {
  console.log(`${i + 1}. ${formatProduct(products[i])}`);
}
```

- **Say:** "Same result, but now you also get a numbered position in front."

Error to inject (the off-by-one): change `i < products.length` to `i <= products.length`, save, view Console.

- Expected: an extra final line `5. undefined...` — actually `undefined` values appear because `products[4]` does not exist.
- **Say:** "One extra `=` and we get a ghost line. Always use `i < array.length`, never `<=`, when walking an array."

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding step by step, learners build a small product catalogue using an array of objects, a formatting function (from Session 2), and a loop.

Starting materials: In `webdev-practice`, a new page `catalogue.html` linking `js/catalogue.js` at the bottom of the body. The instructor shows the HTML shell; learners type it.

Procedure (instructor calls each numbered step; instructor walks the room as learners follow):

1. Create `catalogue.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Catalogue</title>
</head>
<body>
  <h1>Adaeze Fashion Catalogue</h1>
  <p>Press F12 and open the Console to see the products.</p>
  <script src="js/catalogue.js"></script>
</body>
</html>
```

2. Create `js/catalogue.js`. Start with the products array:

```
const products = [
  { name: "Hair Cream",  price: 2500, category: "Skincare" },
  { name: "Shea Butter", price: 1800, category: "Skincare" },
  { name: "Coconut Oil", price: 3200, category: "Haircare" },
  { name: "Herbal Soap", price: 1200, category: "Bath" },
  { name: "Black Soap",  price: 900,  category: "Bath" }
];
```

3. Add the formatting function from Session 2:

```
function formatProduct(product) {
  return `${product.name} — ₦${product.price} [${product.category}]`;
}
```

4. Add a heading line and a loop:

```
console.log("===== ADAEZE FASHION — PRODUCT CATALOGUE =====");

for (const product of products) {
  console.log(formatProduct(product));
}
```

5. Save both files (`Ctrl + S`). Open `catalogue.html` in Chrome and press `F12`, then the **Console** tab.

Pause-and-predict points:

- **After step 2:** "How many products are in the array? What will `products.length` print?" (5.)
- **After step 4, before running:** "How many lines will print under the heading, and what is the first line?" (Five lines; first is `Hair Cream — ₦2500 [Skincare]`.)
- **Midway:** "What is the index of 'Herbal Soap'?" (3.)

Expected result in the Console:

```
===== ADAEZE FASHION — PRODUCT CATALOGUE =====
Hair Cream — ₦2500 [Skincare]
Shea Butter — ₦1800 [Skincare]
Coconut Oil — ₦3200 [Haircare]
Herbal Soap — ₦1200 [Bath]
Black Soap — ₦900 [Bath]
```

6. **Extension (if the class is fast):** add a decision inside the loop so that products over ₦3000 are flagged:

```
for (const product of products) {
  if (product.price > 3000) {
    console.log(`⭐ PREMIUM — ${formatProduct(product)}`);
  } else {
    console.log(formatProduct(product));
  }
}
```

Expected: Coconut Oil's line starts with `⭐ PREMIUM —`.

Troubleshooting while walking the room:

- `undefined` printed for a product's name → the object is missing a key, or the key is misspelled (`nane` vs `name`). Check the object.
- Only one line prints → braces `{ }` of the loop are missing, so the loop body is just one line. Re-check the braces.
- Nothing prints → the script is not saved or not linked (Session 1, K.1), or the loop was never written.
- `products is not defined` → the file was saved in the wrong folder, or the `<script src>` path is wrong.
- Naira shows as `?` → file encoding. Save as UTF-8 with `<meta charset="UTF-8">`.

Completion checklist (learner ticks each):

- ☐ `catalogue.html` links `js/catalogue.js` at the bottom of the body
- ☐ `products` is an array of at least five objects
- ☐ Each object has `name`, `price`, `category`
- ☐ `formatProduct(product)` returns a formatted string (uses `return`)
- ☐ A loop prints every product (one line each)
- ☐ All prices are **numbers**, not strings
- ☐ No red errors in the Console

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Build a class register and print it on your own — a **new** task, not a copy of the catalogue.

Scenario: You are the secretary of your church's youth fellowship in Port Harcourt. You keep a register of members. Each member has a **name**, an **age**, and whether they have **paid their dues** this month (`true` or `false`). You want a clean printed list, and a flag next to anyone who has *not* paid.

Starting information: Create the data yourself, using these members:

- Chidi, 19, paid
- Ngozi, 22, not paid
- Emeka, 25, paid
- Amaka, 18, not paid
- Tunde, 21, paid

Requirements:

1. Create a new page `register.html` and a `js/register.js` file linked at the bottom of the body.
2. Store the members as an **array of objects**. Each object must have the keys `name`, `age`, and `paid` (a boolean).
3. Write a function `formatMember(member)` that **returns** a string, for example: `Chidi (19) — Dues paid` or `Ngozi (22) — NOT PAID`.
4. Use a **loop** to go through the array and `console.log` the formatted line for each member.
5. Print a heading line `===== YOUTH FELLOWSHIP REGISTER =====` before the list, and a total line after it: `Total members: 5`.

Constraints:

- Use **bracket access** at least once somewhere in your code (to prove you know both access styles).
- Use `paid` inside the `formatMember` function with an `if/else` (Session 2 skill).
- Do **not** copy the instructor's catalogue text word-for-word — design your own output wording.
- All ages as numbers.

Time allowance: 15 minutes.

Success criteria:

- Five member lines print, one per member.
- Ngozi and Amaka's lines say "NOT PAID"; the other three say "Dues paid".
- The heading and `Total members: 5` both appear.
- No red errors.

Hints available if stuck:

- Hint 1: The array looks like `[{ name: "Chidi", age: 19, paid: true }, ...]`.
- Hint 2: Inside `formatMember`, use `if (member.paid) { return ...; } else { return ...; }`.
- Hint 3: A `for...of` loop is the simplest: `for (const member of members) { console.log(formatMember(member)); }`.
- Hint 4: `members.length` gives the total for the last line.

Instructor answer key (do not show learners):

```
const members = [
  { name: "Chidi", age: 19, paid: true },
  { name: "Ngozi", age: 22, paid: false },
  { name: "Emeka", age: 25, paid: true },
  { name: "Amaka", age: 18, paid: false },
  { name: "Tunde", age: 21, paid: true }
];

function formatMember(member) {
  if (member["paid"]) {                    // bracket access demonstrated
    return `${member.name} (${member.age}) — Dues paid`;
  } else {
    return `${member.name} (${member.age}) — NOT PAID`;
  }
}

console.log("===== YOUTH FELLOWSHIP REGISTER =====");

for (const member of members) {
  console.log(formatMember(member));
}

console.log(`Total members: ${members.length}`);
```

Expected Console output:

```
===== YOUTH FELLOWSHIP REGISTER =====
Chidi (19) — Dues paid
Ngozi (22) — NOT PAID
Emeka (25) — Dues paid
Amaka (18) — NOT PAID
Tunde (21) — Dues paid
Total members: 5
```

Marking guidance (out of 10):

- Array of objects, correct keys (2)
- `formatMember` returns a string (not logs) (2)
- `if/else` on `paid` produces correct wording (2)
- Loop prints all five, one per line (2)
- Heading and total line correct (1)
- Bracket access used; no console errors (1)

J. REAL-WORLD CASE STUDY

Situation. *Grace* runs a fruit-and-vegetable stall at Mile 3 Market in Port Harcourt. Every morning she writes her prices on a card so customers can see them, and every evening she rubs them out and writes new ones. When the market price of tomatoes jumps, she often forgets to update the card, or updates it late, and customers complain that the price on the card is not the price she charges.

Relevant facts.

- Grace sells about 12 items each day — for example tomatoes, pepper, onions, okra, plantain.
- The prices change almost daily with the market.
- She already has a small webpage for the stall (from the Web Design course).

- She cannot retype the whole card every time one price changes.

Decision point. How can Grace keep a single "source of truth" for her prices and have the page list them automatically?

Possible approaches.

1. Keep writing the card by hand each day.
2. Store her items in a JavaScript **array of objects** and use a **loop** to print the price list, so she updates a price in exactly one place and the whole list refreshes.
3. Ask a developer to build a whole online shop with a database.

Recommended solution (for this stage). Approach 2. Grace keeps her stock in one array; each item is an object `{ item, price, unit }`; a loop prints the whole list. To change the tomato price, she edits one number. This is exactly today's skill.

Example solution:

```
const marketList = [
  { item: "Tomatoes", price: 1200, unit: "basket" },
  { item: "Pepper",   price: 800,  unit: "paint bucket" },
  { item: "Onions",   price: 1500, unit: "bag" },
  { item: "Okra",     price: 300,  unit: "small bowl" },
  { item: "Plantain", price: 1000, unit: "bunch" }
];

function formatItem(entry) {
  return `${entry.item}: ₦${entry.price} per ${entry.unit}`;
}

console.log("===== GRACE FRESH MARKET — TODAY'S PRICES =====");

for (const entry of marketList) {
  console.log(formatItem(entry));
}

console.log(`Items listed: ${marketList.length}`);
```

Expected output:

```
===== GRACE FRESH MARKET — TODAY'S PRICES =====
Tomatoes: ₦1200 per basket
Pepper: ₦800 per paint bucket
Onions: ₦1500 per bag
Okra: ₦300 per small bowl
Plantain: ₦1000 per bunch
Items listed: 5
```

Trade-offs.

- Approach 1 is free and familiar, but it relies on Grace's memory and is error-prone when prices change fast — which is exactly when mistakes cost her customers.
- Approach 2 takes about 30 minutes once, then makes daily updates a 5-second edit to a single number. It scales: adding okra to the list is one extra object.
- Approach 3 is a real business project (database, hosting, payments) and is far beyond a one-person stall right now. It may make sense later if Grace starts delivering online.

Professional connection. Developers constantly build "data-driven lists": product pages, price checkers, contact lists, booking schedules. The pattern is always the same — store the data as an array of objects, format each item with a function, and render them with a loop. Knowing this pattern is the core of front-end work, and it is exactly what the capstone shop needs.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this table as the live troubleshooting guide. When a learner hits an issue, ask them to find the matching row and *check* before guessing a fix.

K.1 Off-by-one — a stray `undefined` at the end

Symptoms: A loop prints all items, then one extra `undefined` line. **Likely cause:** The loop condition uses `i <= products.length` instead of `i < products.length`; the final pass reads an index that does not exist. **How to investigate:** Look at the `for` line. If it says `<=`, that is the likely cause. Confirm by printing `i` inside the loop and watching it reach `3` when the length is `3`. **Corrective steps:** Change `<=` to `<`. **How to confirm resolved:** The trailing `undefined` disappears. **How to prevent recurrence:** Memorise the rule: for arrays, `i < array.length`; the last index is `length - 1`.

K.2 Asking for an index that does not exist

Symptoms: A value prints as `undefined` with no error. **Likely cause:** The index is out of range — e.g., `products[5]` when the array has five items (valid indexes 0–4). This is a **likely** cause; confirm by checking `.length`. **How to investigate:** Run `console.log(products.length)` and compare with the index you used. **Corrective steps:** Use a valid index, or use a loop so you never hard-code indexes. **How to prevent:** When in doubt, loop instead of indexing by hand.

K.3 Reading an object key that does not exist / is misspelled

Symptoms: `product.nmae` (typo) prints `undefined`; the whole name is missing from the output. **Likely cause:** A spelling mistake in the key, or the key is not quoted correctly in bracket access. **How to investigate:** `console.log(Object.keys(product));` prints all the real keys — compare them with what you typed. **Corrective steps:** Fix the spelling; remember `object["key"]` needs quotes around the key. **How to prevent:** Be consistent; use dot access for known keys and check `Object.keys()` when unsure.

K.4 The loop prints only one line

Symptoms: You have five products but only the first (or last) prints. **Likely cause:** Missing braces `{ }` around the loop body, so only the single line directly under the loop repeats — or the print is *outside* the loop. **How to investigate:** Check indentation and braces. Everything that should repeat must be inside `{ ... }`. **Corrective steps:** Wrap the repeating line(s) in braces directly after the `for (...)`. **How to prevent:** Always use braces, even for a one-line loop body.

K.5 A number stored as a string ruins the price

Symptoms: A "total" glues digits together (`"2500" + "1800"` → `"25001800"`). **Likely cause:** A price in the object was written with quotes: `price: "2500"`. **How to investigate:** `console.log(typeof product.price)`; if it says `"string"`, that is confirmed. **Corrective steps:** Remove the quotes so the price is a real number. **How to prevent:** Prices, ages and quantities are numbers — never quoted.

K.6 The loop never ends (the page freezes)

Symptoms: The browser tab hangs; the Console floods with repeated lines. **Likely cause:** The counter is never updated or the condition is always true — e.g., you wrote `i + 1` in the step instead of `i++`, or you forgot the step entirely, or the condition is `i >= 0` with a decreasing counter. **How to investigate:** Look at the third part of the `for (...)` — is `i` actually changing and moving toward the stop condition? **Corrective steps:** Use `i++` as the step; make sure the condition eventually becomes false. (If the tab already hung, close the tab.) **How to prevent:** Read the `for` header aloud: start, stop, step — all three must be present and correct.

K.7 `products.forEach is not a function`

Symptoms: Red TypeError mentioning `forEach`. **Likely cause:** `products` is not an array — perhaps it is an object, `undefined`, or you misspelled the variable name. **How to investigate:** `console.log(typeof products, products);` and check it is an array. **Corrective steps:** Correct the variable name, or make sure the data is genuinely an array. **How to prevent:** Name arrays with plural nouns (`products`, `members`) as a reminder.

Distinguish likely vs confirmed. In every case, the listed cause is only *likely* until you check it — see the exact red error text, run `typeof`, or print `Object.keys()`. Teach learners to **confirm** the cause before fixing, exactly as in Sessions 1 and 2.

L. SESSION ASSIGNMENT

Assignment title: "Build a Store Price List"

Purpose: Practise arrays, objects, array-of-objects, loops and functions together, in a context the learner owns — the shape they will reuse in the capstone.

Realistic scenario: You own a small provisions store in your area. You want a page that prints today's stock and price list automatically, so you only ever update the data, never the layout.

Detailed instructions:

1. Create a folder `assignment3` with `index.html` and `js/stock.js`.
2. In `stock.js`, create an array called `stock` holding **at least six product objects**. Each object must have the keys:
   - `name` (string)
   - `price` (number, in naira)
   - `quantity` (number, how many you have)
   - `category` (string, e.g., "Drinks", "Food", "Toiletries")
3. Write a function `formatStock(item)` that **returns** a line of text, for example: `Milo Tin — ₦4500 x 12 [Food]`
4. Print a heading `===== <YOUR STORE NAME> — STOCK LIST =====`.
5. Loop through `stock` and print every item using `formatStock`.
6. After the list, print a summary line with the **total number of different items** (use `stock.length`).
7. Add at least one product whose `quantity` is `0`, and inside the loop flag it: if quantity is 0, print `— OUT OF STOCK` at the end of that line (use `if/else`, Session 2).

Constraints:

- Prices and quantities must be numbers, not strings.
- Use a **function with a** `return` to format each line (do not format inside the loop).
- Do not copy the register example; design your own wording.
- Use at least one `const` for the array.

Required deliverables:

- `assignment3/index.html`
- `assignment3/js/stock.js`
- The Console output pasted (or photographed) into your workbook.

Submission format: Files in the class folder; Console output pasted/photographed in the workbook.

Deadline: Before Session 4.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
File structure & linking	2	`index.html` links `js/stock.js` at bottom of body; no errors
Array of ≥6 objects, correct keys	4	Every object has name, price, quantity, category
Correct types	2	Prices/quantities are numbers; names/categories are strings
`formatStock` returns (not logs)	3	Function returns a formatted string
Loop prints all items	4	One line per item, no off-by-one errors
Summary total via `.length`	2	Correct count printed
Out-of-stock flag	2	`if/else` correctly flags quantity 0
Presentation / no errors	1	Clean output, no red errors

Common mistakes to avoid: quoting numbers; printing inside the loop instead of using the function; `<=` in the loop condition; forgetting the `return`; misspelling object keys.

Instructor reference solution:

```
const storeName = "Blessed Provisions Store";

const stock = [
  { name: "Milo Tin",    price: 4500, quantity: 12, category: "Food" },
  { name: "Peak Milk",   price: 1200, quantity: 20, category: "Food" },
```

```
  { name: "Coke (50cl)", price: 300,  quantity: 0,  category: "Drinks" },
  { name: "Bournvita",   price: 3800, quantity: 8,  category: "Food" },
  { name: "Colgate",     price: 1500, quantity: 5,  category: "Toiletries" },
  { name: "Dettol",      price: 2200, quantity: 0,  category: "Toiletries" }
];

function formatStock(item) {
  let line = `${item.name} — ₦${item.price} x ${item.quantity} [${item.category}]`;
  if (item.quantity === 0) {
    line = line + " — OUT OF STOCK";
  }
  return line;
}

console.log(`===== ${storeName} — STOCK LIST =====`);

for (const item of stock) {
  console.log(formatStock(item));
}

console.log(`Total different items: ${stock.length}`);
```

Expected Console output:

```
===== Blessed Provisions Store — STOCK LIST =====
Milo Tin — ₦4500 x 12 [Food]
Peak Milk — ₦1200 x 20 [Food]
Coke (50cl) — ₦300 x 0 [Drinks] — OUT OF STOCK
Bournvita — ₦3800 x 8 [Food]
Colgate — ₦1500 x 5 [Toiletries]
Dettol — ₦2200 x 0 [Toiletries] — OUT OF STOCK
Total different items: 6
```

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. In one sentence each, what is the difference between an **array** and an **object**?

Q2. Multiple choice. What does `products[2]` give you? a) The second item in the array b) The third item in the array c) An error, because indexes start at 1 d) The number 2

Q3. Explain in your own words. Why does array indexing start at `0` instead of `1`?

Q4. Practical identification. Given:

```
const product = { name: "Shea Butter", price: 1800, category: "Skincare" };
```

Write the code to print the price using (a) dot access and (b) bracket access.

Q5. Error detection. Find and fix the bug:

```
const items = ["Rice", "Beans", "Garri"];
for (let i = 0; i <= items.length; i++) {
  console.log(items[i]);
}
```

Q6. Scenario. You have an array `scores` of numbers. Write a loop that prints each score, and prints `PASS` next to scores of 50 or above and `FAIL` otherwise. (Hint: combine today's loop with Session 2's `if/else`.)

INSTRUCTOR ANSWER KEY (Q1–Q6)

A1. An array is an **ordered list of items** accessed by position number (index); an object is a **single collection of key/value pairs** accessed by key name. (Accept: array = list by number; object = one thing with labelled parts.) **A2.** (b) the third item — indexes are 0, 1, 2. **A3.** The index counts the number of steps from the start / the gap before an item. The first item is at the start, so it is

0 steps away. (Accept any correct explanation of zero-based counting.) **A4.** (a) `console.log(product.price);` (b) `console.log(product["price"]);` **A5.** The bug is `i <= items.length`: it uses `<=` so the loop runs a fourth time and prints `items[3]`, which is `undefined`. Fix: change `<=` to `<`:

```
for (let i = 0; i < items.length; i++) {
  console.log(items[i]);
}
```

A6.

```
for (const score of scores) {
  if (score >= 50) {
    console.log(`${score} — PASS`);
  } else {
    console.log(`${score} — FAIL`);
  }
}
```

N. REFLECTION AND RECAP

Plain-English summary. Today you learned to hold **many** values in one place. An **array** is an ordered list you read by index — and the first index is `0`, because the index counts steps from the start. `.length` tells you how many items there are; `push` adds and `pop` removes from the end. An **object** describes one thing using labelled key/value pairs, read with dot or bracket access. Put objects inside an array and you get an **array of objects** — the shape of the shop data in your capstone. Finally, **loops** repeat a job for every item: the classic `for` (when you need the index), `for...of` (just the items), and `forEach` (pass a function). You reused your Session 2 function to format each item once, cleanly.

Five important takeaways:

1. Arrays use `[ ]` and are read by index; indexing **starts at 0**, and the last index is `length - 1`.
2. `.length` counts items; `push()` adds to the end; `pop()` removes the last.
3. Objects use `{ key: value }`; read with `obj.key` or `obj["key"]`; update by assignment.
4. An array of objects is the real-world data shape — and the capstone shape.
5. Loops (`for`, `for...of`, `forEach`) repeat work; pair them with a function to format each item once.

Questions to help you recognise what you can now do:

- Can I create an array and safely read its last item without knowing its length in advance?
- Can I build one product object and read a value two different ways?
- Can I explain to a friend why `products[0]` is the first product?
- Can I print a whole list with a loop, adding no code when the list grows?
- Can I reuse a function to format every item in the list?

Connection to the next session: Everything today happened in the **Console**. The page itself still shows the same static HTML from Web Design. Next session we take this product array and make it appear **on the actual page** — using the **DOM** to find HTML elements, change their text and styles, and even create new elements from our data. In other words: you will finally see the catalogue *on the screen*, not just in the Console.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner reaches the last array item using `length - 1`, not by guessing.
- Learner chooses dot access for a known key and can explain that bracket access needs a quoted string.
- Learner writes a loop that prints exactly the items — no trailing `undefined`.
- Learner keeps prices as numbers and spots a quoted number as a likely bug.

- Learner refactors a formatting line into `formatProduct(product)` and calls it from the loop, mirroring Session 2.
- Learner predicts how many lines a loop will print *before* running it.

Common misconceptions to watch for and correct:

- "Arrays start at 1." → Re-draw the numbered row starting at 0; ask for the last index (`length - 1`).
- " `const` means the array can never change." → Demonstrate `push` on a `const` array; clarify it is the *name* that is locked, not the contents.
- " `products.name` should give the name." → Only the objects inside have keys; you must index first (`products[0].name`).
- " `.length()` is a method." → It is a property; no brackets.
- " `for...of` and a classic `for` are identical." → The classic gives the index; `for...of` gives only the item.
- "A loop printing once means the data is wrong." → Usually missing braces around the loop body.

Suggested remedial exercises for learners who struggle:

1. **Index drill:** give a five-item array and three index questions ("what is at 0?", "what is the last index?", "what is at 5?"), answer on paper, then verify in the Console.
2. **Access drill:** give one object and ask for each value twice — once dot, once bracket.
3. **Loop trace:** hand out a `for` loop and a table (pass, `i` , prints) and ask learners to complete the trace by hand before running it.
4. **Mirror exercise:** rewrite the fruit market list for a different shop (a phone-repair kiosk), same structure, new data.

Differentiation:

- **Level 1 (supported):** provide the array of objects already written, plus a `formatProduct` skeleton with a `return` line partially filled; learners complete the loop only.
- **Level 2 (standard):** the independent exercise as written.
- **Level 3 (challenge):** require (a) a **total value** of stock (sum of `price * quantity`) printed at the end using a loop and an accumulator variable — for example `let total = 0;` before the loop and `total = total + item.price * item.quantity;` inside; and (b) use `forEach` with `(item, index)` to number the lines.

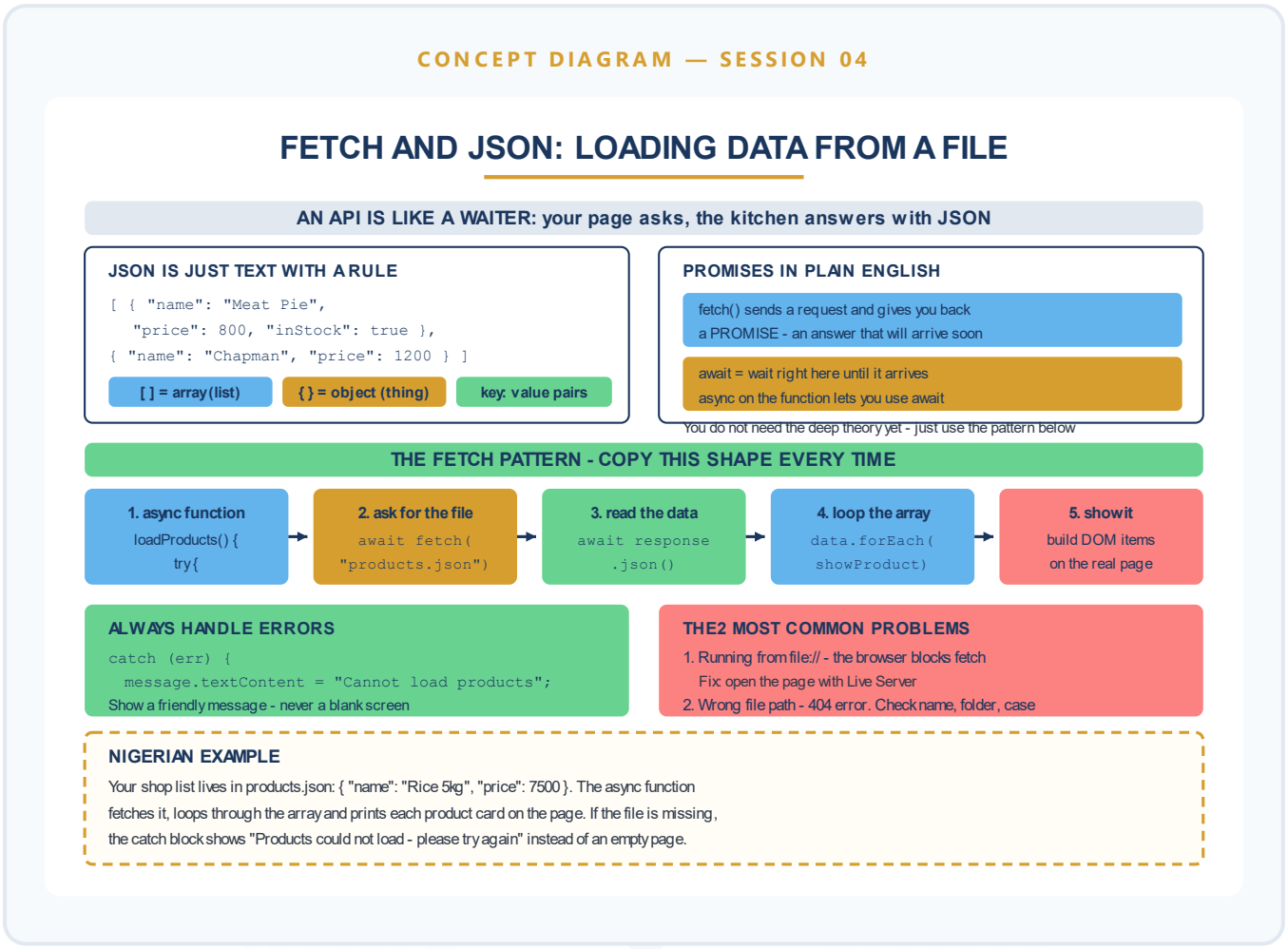
Timing guidance. The loop and the array-of-objects pattern are the heart of the session and the backbone of the capstone. If the room is slow: shorten the case study (Section J) to a 2-minute talk, keep the guided practical (H) and the independent exercise (I) intact. The opening "two hundred products" demonstration is cheap to run and very motivating — do not cut it.

END OF SESSION 3

Next: Session 4 — The DOM: Selecting and Changing the Page.

SESSION 04 OF 12

THE DOM: SELECTING AND CHANGING THE PAGE



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 4 — THE DOM: SELECTING AND CHANGING THE PAGE

A. SESSION IDENTIFICATION

Item	Detail
Session number	4 of 12
Session title	The DOM: Selecting and Changing the Page
Week / Module	Week 2 — Module 2: DOM & Events
Recommended duration	105 minutes

Item	Detail
Difficulty level	Beginner (first time JavaScript changes what is *on* the screen)
Prerequisite knowledge	Session 1 — linking an external script at the **bottom of the body**, variables, `console.log`. Session 2 — functions, `if/else`. Session 3 — arrays, objects, arrays of objects, loops. A learner who cannot create an array of objects and loop through it should be paired for today and given the remedial loop drill in Section O.
Connection to previous session	In Session 3 everything happened in the **Console** — the products printed as text that only developers see. The actual web page never changed. Today we finally connect our data to the **page itself**: we make JavaScript *find* HTML elements, *change* their text and appearance, and *create* new elements so the product catalogue appears on screen for real users. This is the bridge from "code that runs" to "a page that lives".
Preparation before class	Academy PCs on and logged in; Chrome and VS Code open; learner `webdev-practice` folders available (from Sessions 1–3); projector showing VS Code **and** Chrome side by side (learners must see code change the page live). Have the learner's HTML/CSS shop page from Web Design available so the "before and after" is obvious. Print Worksheets 4.1 and 4.2. Whiteboard for drawing the DOM as a family tree.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Explain what the DOM is** in plain English, using the "family tree of elements" comparison, and **explain why JavaScript can change the page at all**.
2. **Select a single element** using `document.getElementById()` and `document.querySelector()`.
3. **Select many elements** using `document.querySelectorAll()`, and **loop over the result**.
4. **Read and change text** using `.textContent`, and **explain the difference and the safety warning** for `.innerHTML`.
5. **Change styles** using `.style`, and **add, remove and toggle classes** with `classList.add/remove/toggle`.
6. **Create a new element** with `document.createElement()` and **insert it into the page** with `appendChild()`.
7. **Explain why the `<script>` tag must be at the bottom of the `<body>`** (recap from Session 1) now that the script touches page elements.
8. **Render an array of objects** (the Session 3 products) into the real HTML page as a list, replacing the old static content.
9. **Diagnose common DOM errors** — `Cannot read properties of null`, a script running before the elements exist, and unsafe use of `innerHTML`.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). Pairing is fine: one learner edits the HTML/JS, the other watches the browser and predicts what will change; swap at the halfway point.
- Instructor computer connected to a projector or large screen, able to show Chrome and VS Code together.

Software

- Google Chrome (any recent version) — DevTools Console and the **Elements** panel are both used today.
- Visual Studio Code (VS Code).
- Windows 10 or Windows 11.
- No new software. (The optional VS Code "Live Server" extension is part of the course toolset; if it is already installed, the instructor may use it, but double-clicking the file in Chrome works fine and is what we rely on.)

Accounts

- None required. All work is offline.

Sample files

- The instructor prepares and shares `demo-week2-s4/` containing:
  - `index.html` (a page with a heading, a paragraph, and an empty `<ul id="product-list">`)
  - `css/style.css` (basic styling, including a `.highlight` class and a `.low-stock` class — full contents in Section G)
  - `js/render.js` (the Session 3 products plus the DOM rendering code — full contents in Section G)
- Learners continue in `webdev-practice` with `shop.html`, `css/style.css`, and `js/render.js`.

Worksheets

- Worksheet 4.1 — "Find the Element" (given an HTML page, learners write the correct selector for each named element; 1 page).
- Worksheet 4.2 — "Text or Style?" (given intended effects, learners tick whether `.textContent`, `.style`, or `.classList` is the right tool).
- These live in the learner workbook (Part 3).

Tools

- Chrome DevTools: the **Console** (F12) for output and errors, and the **Elements** panel for inspecting the DOM live.

Internet

- Not required. If the academy has internet, the instructor may optionally show the MDN pages for `document.querySelector` and `Element.classList` at the very end, but do not depend on it.

Alternative resources for learners lacking facilities

- Everything runs offline on any laptop with Chrome and VS Code. A phone-only learner can pair with a desktop user as the "browser watcher" — their job is to say out loud what changed on the page after each save and refresh. Ask them to photograph the before/after page and the code, and to practise tomorrow in a free phone browser code playground. The pair-swap method from Session 1 keeps every learner active.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open the learner's own HTML/CSS shop page from the Web Design course (static — shows products but nothing reacts), with Chrome DevTools already open on the **Elements** panel. Do not open any JavaScript yet.

What the instructor says:

> *"Remember this page? You built it in Web Design. It looks fine. But watch — I am going to change the heading without touching the HTML file, and without reloading the page."*

Then, live in the Elements panel, double-click the `<h1>` text and type `LIVE EDITED`. The heading on the page changes instantly.

> *"How did that happen? I did not open VS Code. I edited the page while it was running. That is possible because the browser does not use your HTML file directly while you view it. It builds its own **live copy** of the page in memory — a working model of the elements. That model is called the **DOM** — short for Document Object Model. And here is the key idea of today: **JavaScript is allowed to edit that live model too.** When JavaScript changes the DOM, the screen updates immediately — exactly like what I just did by hand."*

Then prove the model is separate from the file: press `Ctrl + R` to refresh.

> *"Now the page reloads and my edit is gone — because I changed the live model, not the file. JavaScript changes the model for the user right now; the file on disk stays as it is. That is normal and it is how every interactive website works."*

Expected learner response: Learners usually say "wow" or "so that is how buttons work". Some will think the edit was saved to the file — clarify with the refresh demonstration. Confirm that the file is untouched.

Why this opener works: It makes the invisible visible. Learners have used websites all their lives; now they see the mechanism (the DOM) that lets code change a page, and they see it in two minutes by hand before any syntax appears.

E. PREREQUISITE REVIEW (7 minutes)

Run this as a fast oral round-robin. Put the four questions on the projector.

Question 1: "Write the line of HTML that links a JavaScript file called `render.js` from a `js` folder — and say where it goes."

- Expected answer: `<script src="js/render.js"></script>`, placed at the **bottom of the** `<body>`, just before `</body>`.
- Common gap: putting it in the `<head>`. Remind them today this *really* matters: a script in the head runs before the elements exist, and every selector will fail. (We will prove this in the demo.)
- Common gap: using `<link>` instead of `<script>`. Remind: `<link>` is for CSS.

Question 2: "What is an **array of objects**, and give the shape for one product?"

- Expected answer: a list of objects, e.g. `[{ name: "Hair Cream", price: 2500, category: "Skincare" }, ...]`.
- Common gap: describing an object alone. Clarify: the array is the *list*; each object is *one item*.
- Why it matters today: this is the exact data we will render onto the page.

Question 3: "In a loop, what does `products[i].name` mean, step by step?"

- Expected answer: take the array `products`, get the item at index `i`, then read its `name` key.
- Common gap: reading `products.name` directly. Remind: index first, then key.

Question 4: "What does a function's `return` do, compared with `console.log`?"

- Expected answer: `return` sends the value back so it can be used elsewhere; `console.log` only shows it to a human.
- Why it matters today: we will build small functions that return elements or strings to insert into the page.

Bridge to today (say this):

> *"You can store a list of products and loop through them — but only you, the developer, could see the result, in the Console. Today we take that same list and let the **browser put it on the page**, where your customer can see it. To do that, JavaScript must find the elements and change them. That is the DOM."*

If more than a third of the class cannot answer Question 2, run the 5-minute loop drill in Section O before continuing.

F. FULL LECTURE NOTES

F.1 What the DOM actually is

The idea. When a browser loads an HTML file, it does not keep the raw text. It **reads** the HTML and builds a live, in-memory model of the page — a tree of objects, one object per element. That model is the **DOM** (Document Object Model).

Plain-English definition. The DOM is the browser's **working model of the page while it is open**. Every tag in your HTML becomes a node in this model: the `<html>`, the `<head>`, the `<body>`, every `<h1>`, every `<p>`, every `<li>`. JavaScript can look up any of these nodes and read or change it, and the browser immediately repaints the screen to match.

Why it exists. HTML alone is fixed text. To let code *react* and *change* a page, the browser needed to expose the page as something a program can walk through and edit. The DOM is that bridge between code and screen.

Real-life comparison — the family tree. Picture a family tree. At the top is the great-grandparent `<html>`. Under it are two children, `<head>` and `<body>`. Under `<body>` are more children — headings, paragraphs, lists — and those can have their own children (a `<ul>` has `<li>` children). This is why we call it a **tree**, and why we speak of **parent** and **child** elements. Every element is a member of one big family, and they are all related by nesting.

Important limitation / misconception. The DOM is **not** your HTML file. The file is the recipe; the DOM is the dish currently on the table. You can add salt to the dish (change the DOM) without changing the recipe (the file). Refresh the page and the browser cooks a fresh dish from the recipe again. This explains the opening activity perfectly.

Terms defined plainly:

- **Node / element** — one item in the tree (e.g., a `<p>`).
- **Select** — find an element in the DOM so you can work with it.
- **Render** — put content on the page so the user can see it.
- **Parent / child** — the element that contains another / the element contained inside.

F.2 `document` — the entry point

The idea. JavaScript reaches the DOM through a built-in object called `document`. It represents the whole page.

Every selection command starts with `document`, followed by a dot, followed by the method:

```
document.getElementById("title");
document.querySelector(".price");
```

Think of `document` as the **front door of the house**. You always enter through it, then move to the room (element) you want.

Say to the class: "If you ever forget how to start a DOM command, remember: it begins with the word `document`. The document *is* the page."

F.3 Selecting one element — `getElementById()` and `querySelector()`

The goal. Before you can change anything, you must *find* it. There are two common ways to find a single element.

Way 1 — `document.getElementById("...")`

This finds the element whose `id` attribute matches. IDs are unique, so this always returns at most one element.

HTML:

```
<h1 id="title">Adaeze Fashion</h1>
```

JavaScript:

```
const title = document.getElementById("title");
```

- `getElementById` — "get the element by its id".
- `"title"` — the id **without** the `#` sign. (This is the most common beginner mistake — see Section K.)

Way 2 — `document.querySelector("...")`

This finds the **first** element that matches a **CSS selector** — the same selectors you already learned in Web Design. This is the most flexible and most used method today.

```
const title = document.querySelector("#title");     // by id (with #)
const firstPrice = document.querySelector(".price"); // by class (with .)
const firstPara = document.querySelector("p");       // by tag name
```

Notice the difference in style:

- `getElementById("title")` — id **without** `#`.
- `querySelector("#title")` — id **with** `#`, because it takes a CSS selector.

Say to the class: "You already know CSS selectors from Web Design. `querySelector` lets you reuse that exact skill to find elements. Dot for class, hash for id — same as CSS."

Why two methods? `getElementById` is direct and fast for ids. `querySelector` is more flexible: it can find by class, tag, attribute, or any complex CSS rule, and it matches the way you already think about styling.

Common misconception. `querySelector` returns only **one** element — the first match. If there are three `.price` elements, you get the first one only. To get all of them, you need `querySelectorAll` (next section).

F.4 Selecting many elements — `querySelectorAll()`

The idea. To get **all** matching elements, use `querySelectorAll`.

```
const allPrices = document.querySelectorAll(".price");
console.log(allPrices.length);   // for example, 4
```

- It returns a **list of elements** (called a `NodeList` — you can treat it like an array for our purposes).
- You read its `.length`, and you can **loop** over it exactly like the arrays from Session 3:

```
allPrices.forEach(function (priceEl) {
  console.log(priceEl.textContent);
});
```

Why this matters. This is how you will loop through many products on a page. Session 3 taught you to loop over an *array of data*; today you loop over an *array of elements*. The loop skill transfers directly.

Common misconception. Learners try to use `.style` or `.textContent` directly on the *result of* `querySelectorAll`, e.g. `allPrices.textContent = "..."`. That does not work because `allPrices` is a **list**, not a single element. You must loop and act on each `priceEl` inside.

F.5 Reading and changing text — `.textContent` vs `.innerHTML`

The idea. Once you have an element, you can read or set its text.

`.textContent` — the safe, plain-text way

```
const title = document.querySelector("#title");

console.log(title.textContent);   // reads: "Adaeze Fashion"

title.textContent = "Adaeze Fashion — Port Harcourt";
```

When you assign to `.textContent`, the element's text is replaced with **plain text**. If that text contains angle brackets, they are shown as literal characters, not treated as HTML. This is exactly what you want for names, prices, and messages — and it is **safe by default**.

`.innerHTML` — the "interpret as HTML" way

```
const box = document.querySelector("#box");

box.innerHTML = "<strong>Big sale!</strong> today";
```

Here the `<strong>` is **interpreted as HTML**, so "Big sale!" appears **bold**. `innerHTML` reads and writes the HTML *inside* the element.

The safety warning (say this clearly):

> "`.innerHTML` *is powerful but dangerous when the text comes from a* ***user*** *— for example a name typed into a form. If a user types something that looks like HTML, the browser may run it as real code. This is called a* ***script injection*** *or* ***XSS*** *attack. Rule for this course:* ***use*** `.textContent` ***for anything a user typed, and reserve*** `.innerHTML` ***for HTML you wrote yourself.***"

Worked example showing the difference:

```
const el = document.querySelector("#demo");

el.textContent = "<b>Hello</b>";
// Page shows the literal text:  <b>Hello</b>

el.innerHTML = "<b>Hello</b>";
// Page shows bold:  Hello
```

Real-life comparison. `.textContent` is like reading a note out loud exactly as written — the words are just words. `.innerHTML` is like a builder who *builds* whatever the note describes — so if the note says "make a door", a door appears. Powerful, but you would not let a stranger's note tell your builder what to build.

When to use which:

Situation	Use
Plain words, names, prices, messages	`.textContent` (default, safe)
You need to insert HTML that **you** wrote	`.innerHTML` (use carefully)
Never with unsanitised user input	`.innerHTML` is forbidden here

F.6 Changing how things look — `.style` and `classList`

JavaScript can change appearance in two ways.

Way 1 — `.style` (change one property at a time)

```
const title = document.querySelector("#title");

title.style.color = "crimson";
title.style.fontSize = "40px";
title.style.backgroundColor = "#f0f0f0";
```

Notes:

- In CSS the property is `font-size` (with a hyphen). In JavaScript you write it in **camelCase**: `fontSize`, `backgroundColor`, `borderRadius`. The hyphen is not allowed in a dot property, so we join the words and capitalise.
- The value must be a **string** (in quotes): `"40px"`, not `40px`.

Way 2 — `classList` (swap whole CSS classes) — the professional way

Instead of changing many style rules in JavaScript, keep the styles in your CSS file and just add or remove a **class**:

CSS:

```
.highlight {
  background-color: yellow;
  font-weight: bold;
}

.low-stock {
  color: red;
}
```

JavaScript:

```
const item = document.querySelector("#first-item");

item.classList.add("highlight");      // add the class
item.classList.remove("highlight");   // remove it
item.classList.toggle("highlight");   // if present, remove; if absent, add
```

- `classList.add("name")` — adds a class.
- `classList.remove("name")` — removes it.
- `classList.toggle("name")` — flips it on or off. Perfect for a "dark mode" button or an "active" menu item.

Why prefer `classList` over `.style`? Because your design lives in **one place** — the CSS file. If the academy wants to change the highlight colour tomorrow, you edit the CSS once; you do not hunt through JavaScript. This is the same "keep it in one place" principle you met with functions in Session 2.

Real-life comparison. `.style` is putting on one piece of clothing at a time. `classList.add("highlight")` is putting on a complete, pre-planned *outfit* that the designer (your CSS) already chose.

Say to the class: "Use `.style` for a quick one-off tweak; use `classList` when the look is something your CSS should own."

F.7 Creating elements — `createElement()` and `appendChild()`

The idea. So far we changed elements that already existed in the HTML. But when a page shows a **list from data** (your products), there is no list yet — JavaScript must **build** the elements.

The two-step pattern:

1. **Create** the element: `document.createElement("li")` makes a new `<li>` that is **not yet on the page**.
2. **Insert** it: `parent.appendChild(newElement)` puts it at the end of a parent element.

```
// 1. Find the parent that will hold the list
const list = document.querySelector("#product-list");   // a <ul>

// 2. Create a new list item
const newItem = document.createElement("li");

// 3. Give it some text
newItem.textContent = "Hair Cream — ₦2500";

// 4. Put it into the page
list.appendChild(newItem);
```

Analogy. `createElement` is carving a wooden spoon in the workshop. `appendChild` is **nailing it to the shelf** so people can see and use it. Creating alone does nothing visible; you must insert it.

Why this matters: This is the moment a **static page becomes a dynamic page**. Until now the products only existed in the Console. Now they exist on the screen, created from data.

F.8 The `.appendChild()` position — a quick note

`appendChild` always adds to the **end** of the parent's children. That is normally exactly right for a list: first product, then second, then third. Later in the course you will meet ways to insert at a specific position, but for building lists and carts, adding at the end is precisely what you want.

A useful habit: before appending, clear last time's items if you are re-rendering (we will see why in Section K). One common way is `list.textContent = "";` at the start of a render, which empties the parent. You will use this in the capstone when filters change the list.

F.9 Why the script goes at the bottom of the `<body>` (recap, now it truly matters)

In Session 1 we put `<script>` at the bottom of the body as a good habit. **Today it becomes essential.**

Recall that the browser reads the page **top to bottom**, building the DOM as it goes. If your script runs **before** an element exists in the DOM, then when the script tries to find that element, the search returns **nothing** — `null`. Trying to use `null` throws the famous error:

```
Uncaught TypeError: Cannot read properties of null (reading 'textContent')
```

Worked comparison. Suppose the page is:

```
<head>
  <script src="js/render.js"></script>   <!-- WRONG place -->
</head>
<body>
  <h1 id="title">Adaeze Fashion</h1>
</body>
```

And `js/render.js` starts with:

```
const title = document.querySelector("#title");
```

```
title.textContent = "New title";   // crashes: title is null
```

When the script runs, the `<h1>` has **not been created yet** (it appears later in the page), so `querySelector` finds nothing, `title` is `null`, and `.textContent` on `null` throws. Move the script to the bottom of the body and the `<h1>` exists by the time the code runs — no error.

Say to the class: "The DOM is built as the browser reads down the page. If your script runs too early, the elements are not there yet. Put the script at the bottom of the body — then the whole page already exists."

> ***Deeper note for the curious learner (optional):*** *an alternative is to keep the script in the `<head>` with the `defer` attribute, `<script src="js/render.js" defer></script>`, which waits until the page is built. We use the bottom-of-body approach because it is easiest to reason about.*

F.10 Putting it all together — rendering the Session 3 products onto the page

Here is the full picture. We reuse the **array of objects** from Session 3 and use a **loop** to build a `<li>` for each product, using `createElement` + `appendChild`, and we reuse a **formatting function** from Session 2.

HTML (the shopping page):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Shop</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1 id="title">Adaeze Fashion Shop</h1>
  <p id="intro">Our current products are listed below.</p>

  <ul id="product-list">
    <!-- OLD static items (we will let JavaScript replace these) -->
    <li>Old product 1</li>
    <li>Old product 2</li>
  </ul>

  <script src="js/render.js"></script>
</body>
</html>
```

JavaScript (`js/render.js`):

```
// The same product data shape from Session 3
const products = [
  { name: "Hair Cream",  price: 2500, category: "Skincare" },
  { name: "Shea Butter", price: 1800, category: "Skincare" },
  { name: "Coconut Oil", price: 3200, category: "Haircare" },
  { name: "Herbal Soap", price: 1200, category: "Bath" }
];

// A function to format ONE product into a text line (Session 2 + 3)
function formatProduct(product) {
  return `${product.name} — ₦${product.price} [${product.category}]`;
}

// 1. Find the list on the page
const list = document.querySelector("#product-list");

// 2. Clear the old static items
list.textContent = "";

// 3. Loop over the data and build a list item for each product
for (const product of products) {
  const li = document.createElement("li");
  li.textContent = formatProduct(product);
```

```
  // 4. Flag expensive items with a CSS class
  if (product.price > 3000) {
    li.classList.add("highlight");
  }

  // 5. Insert it into the page
  list.appendChild(li);
}

// 6. Update the heading and intro using the DOM
document.querySelector("#title").textContent = "Adaeze Fashion Shop — Fresh Stock";
document.querySelector("#intro").textContent =
  `We have ${products.length} products in stock today.`;
```

Expected on-page result: the `<ul>` no longer shows the two "Old product" lines. Instead it shows four lines:

```
Hair Cream — ₦2500 [Skincare]
Shea Butter — ₦1800 [Skincare]
Coconut Oil — ₦3200 [Haircare]     ← highlighted background (price > 3000)
Herbal Soap — ₦1200 [Bath]
```

The heading now reads **"Adaeze Fashion Shop — Fresh Stock"**, and the paragraph reads **"We have 4 products in stock today."**

Say to the class: "Look carefully. The HTML still has the old static list in the file. But the user sees the fresh list, because JavaScript **replaced the DOM**. This is the whole point of the DOM: the file is the starting point; the live model is what the user experiences, and our code controls it."

F.11 Summary of the session

- The **DOM** is the browser's live tree/model of the page; JavaScript edits it and the screen updates.
- **Select**: `document.getElementById("id")` (no `#`), `document.querySelector("css")` (first match, uses CSS selectors), `document.querySelectorAll("css")` (all matches, loop over them).
- **Text**: `.textContent` for plain text (safe); `.innerHTML` interprets HTML — **avoid with user input** (XSS).
- **Look**: `.style.color` for one-off tweaks; `classList.add/remove/toggle` to swap whole CSS classes (preferred).
- **Build**: `document.createElement("li")` then `parent.appendChild(li)`.
- Keep the `<script>` at the **bottom of the body** so elements exist before the script runs.
- **Render data** by looping an array of objects and creating elements for each.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- `demo-week2-s4/index.html`, `css/style.css`, `js/render.js` open in VS Code; Chrome open to the page; DevTools **Console** ready and the **Elements** panel reachable.
- Whiteboard with a small DOM tree drawn: `html → body → (h1, p, ul → li, li)`.

G.2 Demonstration 1 — select and change text/style (5 minutes)

Say: "First, let us find an element and change it — the simplest DOM action."

Type into `render.js`:

```
const title = document.querySelector("#title");
console.log(title);                 // shows the <h1> element

title.textContent = "Welcome to Adaeze Fashion!";
title.style.color = "crimson";
```

Do on the projector:

1. Save (`Ctrl + S`). Refresh the page (`Ctrl + R`).
2. Point at the heading: the text changed and it is now red.
3. Open Console: `console.log(title)` shows the `<h1>` element (hovering highlights it in the page).
4. **Pause and ask:** "If I put this script in the `<head>` instead, what would happen?" Then *actually move it* to the head, save, refresh → red error `Cannot read properties of null`. Move it back.
    - **Say:** "This is why the script lives at the bottom of the body. The element must exist before we search for it."

Error to inject: change `querySelector("#title")` to `querySelector("#titel")` (typo) → `title` is `null` → red error `Cannot read properties of null (reading 'textContent')`.

- **Say:** "The `#` means id, and the id must match the HTML exactly. A single wrong letter gives `null`. Check the spelling and the `#`."

Questions to ask:

- "Why `#title` and not `.title`?" (Because in the HTML it is `id="title"`, and `#` selects by id.)
- "Why is the value `'40px'` in quotes?" (Style values are strings.)

G.3 Demonstration 2 — `querySelectorAll` and looping (4 minutes)

Say: "Now let us find *many* elements at once."

First, temporarily add some prices to the HTML so there is something to select:

```
<span class="price">₦2500</span>
<span class="price">₦1800</span>
<span class="price">₦3200</span>
```

Type in `render.js`:

```
const prices = document.querySelectorAll(".price");
console.log(prices.length);          // 3

prices.forEach(function (priceEl) {
  priceEl.style.color = "green";
});
```

- Save, refresh → all three price spans turn green, and the Console prints `3`.
- **Ask:** "Why did I use `querySelectorAll` and not `querySelector`?" (Because I want all three; `querySelector` gives only the first.)
- **Error to inject:** try `prices.style.color = "green";` (no loop) → nothing turns green and often an error. **Say:** "A list does not have a single style — you must loop and act on each element."

G.4 Demonstration 3 — render the products (6 minutes)

Say: "This is the big one — turning your Session 3 data into a real list on the page."

Type (or paste) the full `render.js` from Section F.10, and make sure the HTML has an empty `<ul id="product-list">`.

Walk through the code aloud, one block at a time:

1. The products array — "exactly what you built in Session 3."
2. `formatProduct` — "a function that returns one line of text; we reuse it here."
3. `document.querySelector("#product-list")` — "find the empty list."
4. `list.textContent = "";` — "clear anything already in it, so we do not duplicate."
5. The loop with `createElement("li")`, `textContent`, `classList.add`, `appendChild` — "create a row, write its text, maybe add a class, then attach it to the list."

Pause and predict before the first run: "How many list items will appear, and which one is highlighted?" (Four items; Coconut Oil, because its price is over ₦3000.)

Expected on-page result: four products listed; Coconut Oil highlighted. The heading and intro paragraphs also update.

Then demonstrate a real-world interaction manually (preview of next session): in the Console, type:

```
products.push({ name: "New Balm", price: 1500, category: "Skincare" });
```

and re-run the loop by refreshing. Point out that the extra product appears — because the page is built from the data.

Common errors to show and correct:

- Forgetting `appendChild` → the created `<li>` never appears (create ≠ insert).
- Using `.textContent` on the `querySelectorAll` result directly → error; must loop.
- Script still in the `<head>` → `null` errors.
- Using `innerHTML` with user text → mention the safety rule; keep `.textContent` today.

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding step by step, learners take a static HTML page and use the DOM to (1) change existing text and style, and (2) build a list from an array of objects.

Starting materials: In `webdev-practice`, a page `shop.html`, a `css/style.css`, and an empty `js/render.js`. The instructor shows each file's content.

Procedure (instructor calls each numbered step; learners follow; walk the room):

1. Create `shop.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Shop</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1 id="title">My Shop</h1>
  <p id="intro">Loading products...</p>
  <ul id="product-list"></ul>
  <script src="js/render.js"></script>
</body>
</html>
```

2. Create `css/style.css`:

```
body {
  font-family: Arial, sans-serif;
  margin: 20px;
}

.highlight {
  background-color: #fff3b0;
  font-weight: bold;
}

.low-stock {
  color: red;
}
```

3. Create `js/render.js`. Start with the data and a formatting function:

```
const products = [
  { name: "Hair Cream",  price: 2500, category: "Skincare", stock: 8 },
  { name: "Shea Butter", price: 1800, category: "Skincare", stock: 0 },
  { name: "Coconut Oil", price: 3200, category: "Haircare", stock: 4 },
  { name: "Herbal Soap", price: 1200, category: "Bath",     stock: 12 }
];

function formatProduct(product) {
```

```
  return `${product.name} — ₦${product.price} [${product.category}]`;
}
```

4. Change the heading text and style:

```
const title = document.querySelector("#title");
title.textContent = "Adaeze Fashion — Today's Products";
title.style.color = "darkgreen";
```

5. Build the list:

```
const list = document.querySelector("#product-list");
list.textContent = "";   // clear in case we re-run

for (const product of products) {
  const li = document.createElement("li");
  li.textContent = formatProduct(product);

  if (product.stock === 0) {
    li.classList.add("low-stock");
  } else if (product.price > 3000) {
    li.classList.add("highlight");
  }

  list.appendChild(li);
}
```

6. Update the intro with the number of products:

```
document.querySelector("#intro").textContent =
  `We have ${products.length} products today.`;
```

7. Save all three files. Open `shop.html` in Chrome. Watch the page (no need for the Console this time — the change is on the page).

Pause-and-predict points:

- **After step 5, before saving:** "Which product's line will be red? Which will be highlighted? How many lines total?" (Shea Butter red — stock 0; Coconut Oil highlighted — price > 3000; four lines total.)
- **After step 6:** "What will the paragraph say?" (We have 4 products today.)

Expected on-page result: an `<h1>` reading **"Adaeze Fashion — Today's Products"** in dark green; an intro paragraph reading **"We have 4 products today."**; and a four-item list:

```
Hair Cream — ₦2500 [Skincare]
Shea Butter — ₦1800 [Skincare]        (red text)
Coconut Oil — ₦3200 [Haircare]        (yellow highlighted background)
Herbal Soap — ₦1200 [Bath]
```

Troubleshooting while walking the room:

- Nothing appears / list empty → the script is not at the bottom of the body, or `appendChild` is missing, or the file was not saved.
- `Cannot read properties of null` → a selector id/class does not match the HTML. Check `#title`, `#intro`, `#product-list`.
- Old items duplicate on refresh → the `list.textContent = "";` clear step is missing.
- Colours not showing → the class name in `classList.add(...)` does not match the class in `style.css`, or the CSS file is not linked in the `<head>`.
- Naira shows as `?` → save the file as UTF-8; keep `<meta charset="UTF-8">`.

Completion checklist (learner ticks each):

- ☐ `shop.html` links `css/style.css` in the head and `js/render.js` at the bottom of the body
- ☐ Heading text and colour changed via the DOM
- ☐ Intro paragraph updated with the product count

- [ ] Four list items appear, built with `createElement` + `appendChild`
- [ ] A loop builds the list from the data (not typed by hand)
- [ ] `.highlight` and `.low-stock` classes applied correctly
- [ ] No red errors in the Console

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Build a **"Student Results Board"** on your own — a **new** DOM task, not a copy of the product shop.

Scenario: You teach a small computer class at a community centre in Port Harcourt. You keep each student's name and score, and you want a webpage that shows the class results automatically, with a colour flag for those who failed (below 50).

Starting information: Use these students:

- Adaeze — 78
- Chidi — 45
- Ngozi — 91
- Emeka — 52
- Amaka — 38

Requirements:

1. Create `results.html`, `css/results.css`, and `js/results.js`.
2. `results.html` must have:
   - an `<h1 id="heading">` (start with any text)
   - a `<ul id="results-list">` (empty)
   - the CSS linked in the head and the JS linked at the **bottom of the body**.
3. In `results.js`, store the students as an **array of objects**, each with `name` and `score` (numbers).
4. Write a function `formatStudent(student)` that **returns** a string such as `Adaeze — 78 (PASS)` or `Chidi — 45 (FAIL)` (reuse your Session 2 `if/else`).
5. Use `document.querySelector("#heading").textContent = ...` to set the heading to `"Community Centre — Results"`.
6. Loop over the students, create an `<li>` for each with `createElement` + `appendChild`, and set its text with `formatStudent`.
7. Add a CSS class `.fail` (red background or red text) and apply it with `classList.add("fail")` to any student who scored **below 50**.

Constraints:

- Build the list **from the data** with a loop — do **not** type the `<li>` items by hand in the HTML.
- Use `.textContent` (not `.innerHTML`) for the student text — remember the safety rule.
- All scores must be numbers.
- Do **not** copy the products example word-for-word; design your own output wording.

Time allowance: 15 minutes.

Success criteria:

- The heading reads "Community Centre — Results".
- Five list items appear, one per student.
- Chidi and Amaka (48 and 38 — below 50) show the `.fail` style; the others do not.
- The list is built by JavaScript, and the HTML `<ul>` starts empty.
- No red errors.

Hints available if stuck:

- Hint 1: In `formatStudent`, use `if (student.score >= 50) { return ...PASS... } else { return ...FAIL... }`.
- Hint 2: Create each item with `const li = document.createElement("li"); li.textContent = formatStudent(student);`.
- Hint 3: Apply the class inside the same loop: `if (student.score < 50) { li.classList.add("fail"); }`.
- Hint 4: Remember to `document.querySelector("#results-list").appendChild(li);` — creating alone is not enough.

Instructor answer key (do not show learners):

```
const students = [
  { name: "Adaeze", score: 78 },
  { name: "Chidi",  score: 45 },
  { name: "Ngozi",  score: 91 },
  { name: "Emeka",  score: 52 },
  { name: "Amaka",  score: 38 }
];

function formatStudent(student) {
  if (student.score >= 50) {
    return `${student.name} — ${student.score} (PASS)`;
  } else {
    return `${student.name} — ${student.score} (FAIL)`;
  }
}

document.querySelector("#heading").textContent = "Community Centre — Results";

const list = document.querySelector("#results-list");
list.textContent = "";

for (const student of students) {
  const li = document.createElement("li");
  li.textContent = formatStudent(student);

  if (student.score < 50) {
    li.classList.add("fail");
  }

  list.appendChild(li);
}
```

Corresponding `css/results.css` (minimum):

```
.fail {
  color: white;
  background-color: #d9534f;
  padding: 4px;
}
```

Expected on-page result: heading "Community Centre — Results"; five items, with Chidi's and Amaka's lines shown in the `.fail` style.

Marking guidance (out of 10):

- Correct files and correct script placement (2)
- Array of objects with name/score numbers (2)
- `formatStudent` returns PASS/FAIL correctly (2)
- List built with a loop + `createElement` / `appendChild` (2)
- `.fail` class applied only to scores below 50 (1)
- Heading set via `.textContent`; no console errors (1)

J. REAL-WORLD CASE STUDY

Situation. *Chief Nwosu* runs a small private school in Port Harcourt. On result day, he prints a notice board showing each of the 60 pupils' scores and whether they passed. Last term the person typing the board made three typing mistakes, and two parents complained that the board did not match their child's report sheet. Chief Nwosu wants one reliable source of the results, with the board generated from it — and a clear colour flag for anyone who did not pass.

Relevant facts.

- Each pupil has a name and a score.
- The pass mark is 50.

- The school already has a basic webpage for announcements (built with HTML/CSS).
- The scores change only twice a term, but the board must always match the master list exactly.

Decision point. How can the school make the notice board *derive* from one list of scores, so a typo in the board generator cannot happen?

Possible approaches.

1. Keep typing the board by hand each term.
2. Store the pupils as an **array of objects** in JavaScript and use the **DOM** to render the board automatically: heading, one list item per pupil, and a `.fail` class for anyone below 50.
3. Hire a developer to build a full results-management system with a database.

Recommended solution (for this stage). Approach 2. One array is the "single source of truth". The page is built by the DOM, so the board can never disagree with the data, and passing vs failing is decided by code (`>= 50`), not by a tired typist.

Example solution:

```
const schoolName = "Chief Nwosu Memorial School";

const pupils = [
  { name: "Adaeze O.",  score: 78 },
  { name: "Chidi N.",   score: 45 },
  { name: "Ngozi E.",   score: 91 },
  { name: "Emeka A.",   score: 52 },
  { name: "Amaka U.",   score: 38 },
  { name: "Tunde B.",   score: 67 }
];

function formatPupil(pupil) {
  const status = pupil.score >= 50 ? "PASS" : "FAIL";
  return `${pupil.name} — ${pupil.score} (${status})`;
}

document.querySelector("#school").textContent = schoolName;
document.querySelector("#board-title").textContent = "End of Term Results";

const board = document.querySelector("#board");
board.textContent = "";

for (const pupil of pupils) {
  const li = document.createElement("li");
  li.textContent = formatPupil(pupil);

  if (pupil.score < 50) {
    li.classList.add("fail");
  }

  board.appendChild(li);
}

document.querySelector("#summary").textContent =
  `Total pupils: ${pupils.length}`;
```

Trade-offs.

- Approach 1 costs nothing but keeps the typo risk — exactly the problem Chief Nwosu is trying to remove.
- Approach 2 is a 45-minute build; after that, updating results means editing numbers in one array. The board and the master list cannot drift apart, because the board *is* the master list, rendered.
- Approach 3 (a full system with a database and logins) is a genuine business project and overkill for a 60-pupil notice board. It may make sense if the school grows and needs parents to log in online.

Professional connection. "Data in, page out" is the core loop of modern front-end work. News sites, bank statements, e-commerce listings, dashboards — all render a list of objects to the DOM with a loop. The skill you practised today is exactly what junior developers are paid to do: turn data into a visible, accurate interface. It is also the exact technique the capstone shop uses to show its products.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this table as the live troubleshooting guide. When a learner hits a problem, ask them to find the matching row and **confirm** the cause before fixing.

K.1 `Cannot read properties of null (reading '...')`

Symptoms: A red TypeError mentioning `null`; the page does not update. **Likely causes:** (a) The selector does not match any element (wrong id/class/tag, or a typo, or missing `#` / `.`). (b) The script is running **before** the element exists (script in the head, or placed above the element). **How to investigate:** Check the selector against the HTML character by character. Check the script is at the **bottom of the body**. In the Console, run `document.querySelector("#yourid")` — if it prints `null`, the selector is wrong (confirmed). **Corrective steps:** Fix the id/class spelling; use `#` for ids and `.` for classes; move the script to the bottom of the body. **How to confirm resolved:** Run the query again — it prints the element, not `null`. **How to prevent recurrence:** Copy the id from the HTML; keep the script at the bottom of the body.

K.2 The new element never appears

Symptoms: You called `createElement` but nothing shows on the page. **Likely cause:** You created the element but never **inserted** it (missing `appendChild`). Creating alone is invisible. **How to investigate:** Look for `.appendChild(` after `createElement(`. If it is absent, that is confirmed. **Corrective steps:** Add `parent.appendChild(newElement);`. **How to confirm resolved:** The element appears on the page. **How to prevent:** Always remember the two-step pattern: **create**, then **attach**.

K.3 List items duplicate every time you re-run / refresh

Symptoms: The list grows: 4 items, then 8, then 12. **Likely cause:** You append new items on top of old ones without clearing the parent first. **How to investigate:** Check whether the render starts by emptying the parent. **Corrective steps:** Add `list.textContent = "";` at the start of the render (before the loop). **How to confirm resolved:** The list stays at the correct count on repeated runs. **How to prevent:** Make "clear, then build" a habit for any re-render.

K.4 `.forEach is not a function` on a selection

Symptoms: Red TypeError when you call `.forEach` on something from the DOM. **Likely cause:** You used `querySelector` (single element) but tried to loop, or you called `.forEach` on a non-list. (Confirmed by checking which selector you used.) **How to investigate:** `console.log(theThing)` — is it one element or a list? If it is one element, `forEach` is wrong. **Corrective steps:** Use `querySelectorAll` to get all matches, then loop. **How to prevent:** Ask yourself: "Do I want one element or many?" One → `querySelector`; many → `querySelectorAll` + loop.

K.5 Text shows HTML tags as literal text, or (worse) a script runs

Symptoms: You wanted bold but saw `<b>...</b>`; or a name typed by a user ran as code. **Likely cause:** The opposite of what you need — using `.textContent` when you wanted HTML, or using `.innerHTML` with user input. **How to investigate:** Check which property you assigned. If the text came from a user, `.innerHTML` is the dangerous culprit. **Corrective steps:** Use `.textContent` for plain text and any user input. Use `.innerHTML` only for HTML you wrote yourself. **How to confirm resolved:** The text displays exactly as intended, and no unexpected HTML is created. **How to prevent:** Default to `.textContent`; treat `.innerHTML` as a sharp tool.

K.6 Style change does nothing

Symptoms: You set `element.style.color = "red"` but nothing changes. **Likely causes:** (a) The selector found the wrong element or `null`. (b) The CSS you wrote is being overridden by a stronger rule. (c) For `classList`, the class name does not match the CSS file. **How to investigate:** `console.log(element)` and hover it in the page. Check the **Elements** panel → **Styles** to see which rule wins. Confirm the class name spelling matches the CSS. **Corrective steps:** Fix the selector; fix the class name; or strengthen the CSS rule. Remember camelCase in `.style` (`backgroundColor`, not `background-color`). **How to prevent:** Prefer `classList` with classes defined in the stylesheet; verify the class name is identical in both files.

K.7 Setting `.style.fontSize = 40` (missing unit) does nothing

Symptoms: Font size does not change. **Likely cause:** Values that need a unit must include it, as a **string**: `"40px"`. `40` alone is invalid. **How to investigate:** Check whether the value is quoted and includes `px`, `%`, `em`, etc. **Corrective steps:** Write `element.style.fontSize = "40px";`. **How to prevent:** Remember: style values are strings, units included.

CYBER ELIAS ACADEMY

Distinguish likely vs confirmed. As in earlier sessions, the listed cause is only *likely* until you check it — run `document.querySelector(...)` to see `null`, inspect the **Elements** panel, or hover the logged element. Confirm first, then fix.

L. SESSION ASSIGNMENT

Assignment title: "My Shop Page — Rendered from Data"

Purpose: Combine arrays of objects, loops, functions and the DOM to turn a static page into one that builds itself from data — the exact pattern the capstone uses.

Realistic scenario: You run a small online stall (choose any products — fashion, food, phone accessories). You want a page that lists your products automatically from one data array, and flags any item that is out of stock.

Detailed instructions:

1. Create a folder `assignment4` containing:
   - `index.html`
   - `css/style.css`
   - `js/products.js`
2. `index.html` must have:
   - a `<link>` to `css/style.css` in the head
   - an `<h1 id="shop-name">`
   - a `<p id="summary"></p>`
   - a `<ul id="product-list"></ul>` (empty)
   - a `<script src="js/products.js"></script>` at the **bottom of the body**
3. In `js/products.js`:
   - Create an array `products` of **at least five** objects. Each object must have: `name` (string), `price` (number), `category` (string), `inStock` (boolean).
   - Write a function `formatProduct(product)` that **returns** a line of text, for example: `Hair Cream — ₦2500 [Skincare]`.
   - Set the shop name with `document.querySelector("#shop-name").textContent = ...`.
   - Set `#summary` to something like `We have 5 products in our store.` using `.length`.
   - Loop over the products; for each, `createElement("li")`, set its `.textContent` with `formatProduct`, and `appendChild` it to `#product-list`.
   - If `inStock` is `false`, add a class `.out-of-stock` with `classList.add`.
4. In `css/style.css`, define at least `.out-of-stock` (e.g., grey text with a strikethrough) and one more style of your choice.

Constraints:

- Build the list with a loop — do **not** hard-code `<li>` items in the HTML.
- Use `.textContent` for all text (do not use `.innerHTML` with any user-entered text).
- Prices are numbers, not strings.
- The script must be at the bottom of the body (or use `defer`).

Required deliverables:

- `assignment4/index.html`
- `assignment4/css/style.css`
- `assignment4/js/products.js`
- A photo/screenshot of the finished page in your workbook, and the Console output noting any errors (or "no errors").

Submission format: Files in the class folder; a photo of the rendered page in the workbook.

Deadline: Before Session 5.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
Files & linking correct	3	CSS in head, JS at bottom of body, all three files present
Array of ≥ 5 objects, correct keys/types	4	name/price/category/inStock with correct types
`#shop-name` and `#summary` set via DOM	3	Heading and count update on the page
List built by loop with createElement/appendChild	5	No hard-coded `<li>`; correct count of items
`.out-of-stock` class applied when false	3	Styling appears only for out-of-stock items
Clean result / no console errors	2	Page renders correctly, Console clear

Common mistakes to avoid: forgetting `appendChild`; script in the head causing `null` errors; hard-coding list items instead of looping; using `.innerHTML` with data; misspelling a selector id/class.

Instructor reference solution:

```
const shopName = "Adaeze Fashion Online";

const products = [
  { name: "Hair Cream",   price: 2500, category: "Skincare", inStock: true },
  { name: "Shea Butter",  price: 1800, category: "Skincare", inStock: false },
  { name: "Coconut Oil",  price: 3200, category: "Haircare", inStock: true },
  { name: "Herbal Soap",  price: 1200, category: "Bath",     inStock: true },
  { name: "Black Soap",   price: 900,  category: "Bath",     inStock: false }
];

function formatProduct(product) {
  return `${product.name} — ₦${product.price} [${product.category}]`;
}

document.querySelector("#shop-name").textContent = shopName;
document.querySelector("#summary").textContent =
  `We have ${products.length} products in our store.`;

const list = document.querySelector("#product-list");
list.textContent = "";

for (const product of products) {
  const li = document.createElement("li");
  li.textContent = formatProduct(product);

  if (!product.inStock) {
    li.classList.add("out-of-stock");
  }

  list.appendChild(li);
}
```

Corresponding `css/style.css` (minimum):

```
body {
  font-family: Arial, sans-serif;
  margin: 20px;
}

.out-of-stock {
  color: grey;
  text-decoration: line-through;
}
```

Expected on-page result: heading "Adaeze Fashion Online"; summary "We have 5 products in our store."; five list items, with **Shea Butter** and **Black Soap** shown in grey with a strikethrough.

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. In one sentence, what is the DOM?

Q2. Multiple choice. Which line finds the element `<h1 id="title">`? a) `document.querySelector("title")` b) `document.querySelector("#title")` c) `document.getElementById(".title")` d) `document.getElement("title")`

Q3. Explain in your own words. What is the difference between `.textContent` and `.innerHTML`, and when is `.innerHTML` dangerous?

Q4. Practical identification. Given an element stored in a variable `card`, write the code to (a) change its text to `"Sold out"`, and (b) add the CSS class `"sold-out"` to it.

Q5. Error detection. Find and fix the bug:

```
<head>
  <script src="js/app.js"></script>
</head>
<body>
  <h1 id="title">Hello</h1>
</body>
```

```
document.querySelector("#title").textContent = "New heading";
```

Q6. Scenario. You have an array `students` of objects with `name` and `score`. Write the code that creates a list item for each student and adds it to a `<ul id="list">`. (You may assume a function `formatStudent(student)` already exists and returns a string.)

INSTRUCTOR ANSWER KEY (Q1–Q6)

A1. The DOM is the browser's live, in-memory tree/model of the HTML page, which JavaScript can read and change; when it changes, the page updates on screen. (Accept any equivalent wording.) **A2.** (b) `document.querySelector("#title")` — `#` selects by id. (a) is missing `#`; (c) has the `.` in the wrong place for `getElementById`; (d) is not a real method. **A3.** `.textContent` reads/writes **plain text** (tags are shown as literal characters) and is safe; `.innerHTML` reads/writes **HTML**, so tags are interpreted. `.innerHTML` is dangerous when the text comes from a **user**, because a user could enter HTML/script that runs — a security risk (XSS). Use `.textContent` for user input. **A4.**

```
card.textContent = "Sold out";
card.classList.add("sold-out");
```

A5. The script is in the **head**, so it runs before the `<h1 id="title">` exists; `querySelector` returns `null` and `.textContent` throws `Cannot read properties of null`. Fix: move the script to the **bottom of the body** (or add `defer`):

```
<body>
  <h1 id="title">Hello</h1>
  <script src="js/app.js"></script>
</body>
```

A6.

```
const list = document.querySelector("#list");
list.textContent = "";

for (const student of students) {
  const li = document.createElement("li");
  li.textContent = formatStudent(student);
  list.appendChild(li);
}
```

N. REFLECTION AND RECAP

Plain-English summary. Today you connected your code to the page. The browser turns your HTML into a live model called the **DOM**, and JavaScript is allowed to edit that model. You learned to **select** elements with `getElementById`, `querySelector` (one element) and `querySelectorAll` (many), and to **loop** over many. You changed **text** with `.textContent` (and learned why `.innerHTML` is risky with user input), changed **appearance** with `.style` and, better, with `classList`. You **created** elements with `createElement` and **inserted** them with `appendChild`. Most importantly, you took the array of objects from Session 3 and **rendered** it into the real page — turning your static Web Design page into one that builds itself from data. Finally, you saw why the script must sit at the bottom of the body: the elements must exist before your code looks for them.

Five important takeaways:

1. The DOM is the browser's **live model** of the page; change it and the screen updates.
2. `querySelector("#id")` finds **one**; `querySelectorAll(".class")` finds **many** (loop over them).
3. Use `.textContent` for plain text (safe); `.innerHTML` only for HTML you wrote yourself.
4. Prefer `classList.add/remove/toggle` over `.style` — keep your design in CSS.
5. Build a list from data with a loop: `createElement` → set text → `appendChild`.

Questions to help you recognise what you can now do:

- Can I change a heading's text and colour from JavaScript without editing the HTML by hand?
- Can I explain the difference between `querySelector` and `querySelectorAll`?
- Can I build a list on the page from an array of objects, with no hard-coded `<li>`?
- Can I explain why a script in the `<head>` causes a `null` error?
- Can I explain why `.innerHTML` should not be fed a user's typed text?

Connection to the next session: Right now the page changes **once**, when it loads. Nothing yet responds to the user. Next session we add **events** — listening for a **click** on a button, or text being **typed** into a field — and running our DOM code in response. That is when the page stops being a poster and becomes something a customer can actually *use*: buttons that add to a cart, forms that react as you type. You now have the DOM as your paintbrush; next we give you a hand that moves it — the user's hand.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner selects an element, changes its text, and the page updates on refresh without being shown which selector to use.
- Learner distinguishes `querySelector` (one) from `querySelectorAll` (many) and loops when needed.
- Learner builds a list from data using `createElement` + `appendChild`, not by typing `<li>` in the HTML.
- Learner explains, in their own words, why the script belongs at the bottom of the body.
- Learner reaches for `classList` rather than piling up `.style` changes.
- Learner uses `.textContent` by default and can say why `.innerHTML` is risky.

Common misconceptions to watch for and correct:

- "The DOM is my HTML file." → Show that a refresh discards in-page edits; the DOM is a live model built from the file.
- "The script can go anywhere." → Demonstrate the `null` error from the head placement.
- "`createElement` puts the element on the page." → Clarify: create then **append**.
- "`querySelectorAll` returns one element I can style directly." → It is a list; loop over it.
- "`.innerHTML` and `.textContent` do the same thing." → Show the literal-tags versus rendered-HTML difference, and the safety rule.
- "`.length` on a selection is an error because it is a NodeList." → It is fine; NodeLists have `.length`.

Suggested remedial exercises for learners who struggle:

1. **Selector drill:** give a page with several ids and classes; ask learners to write the correct selector for each named element on paper, then verify in the Console with `document.querySelector(...)`.

2. **Create-and-attach drill:** give a three-line skeleton and ask the learner to add the missing `appendChild` so a single `<li>` appears.
3. **Trace-the-render:** hand out a small render script and ask learners to write, on paper, the final list the page will show before running it.
4. **Mirror exercise:** rebuild the product list for a different shop (a phone-accessories kiosk) — same structure, new data and class names.

Differentiation:

- **Level 1 (supported):** provide `index.html` and `style.css` complete, plus a `products.js` where the array and `formatProduct` are written and only the loop body has blanks to fill.
- **Level 2 (standard):** the independent exercise as written.
- **Level 3 (challenge):** add (a) a small search: an input box and a filter so that only products whose name contains the typed text are rendered — you can use `product.name.toLowerCase().includes(query)` (mention or demonstrate the idea; the full search UI arrives later in the course); and (b) number each list item using `querySelectorAll` after building, or a counter variable in the loop.

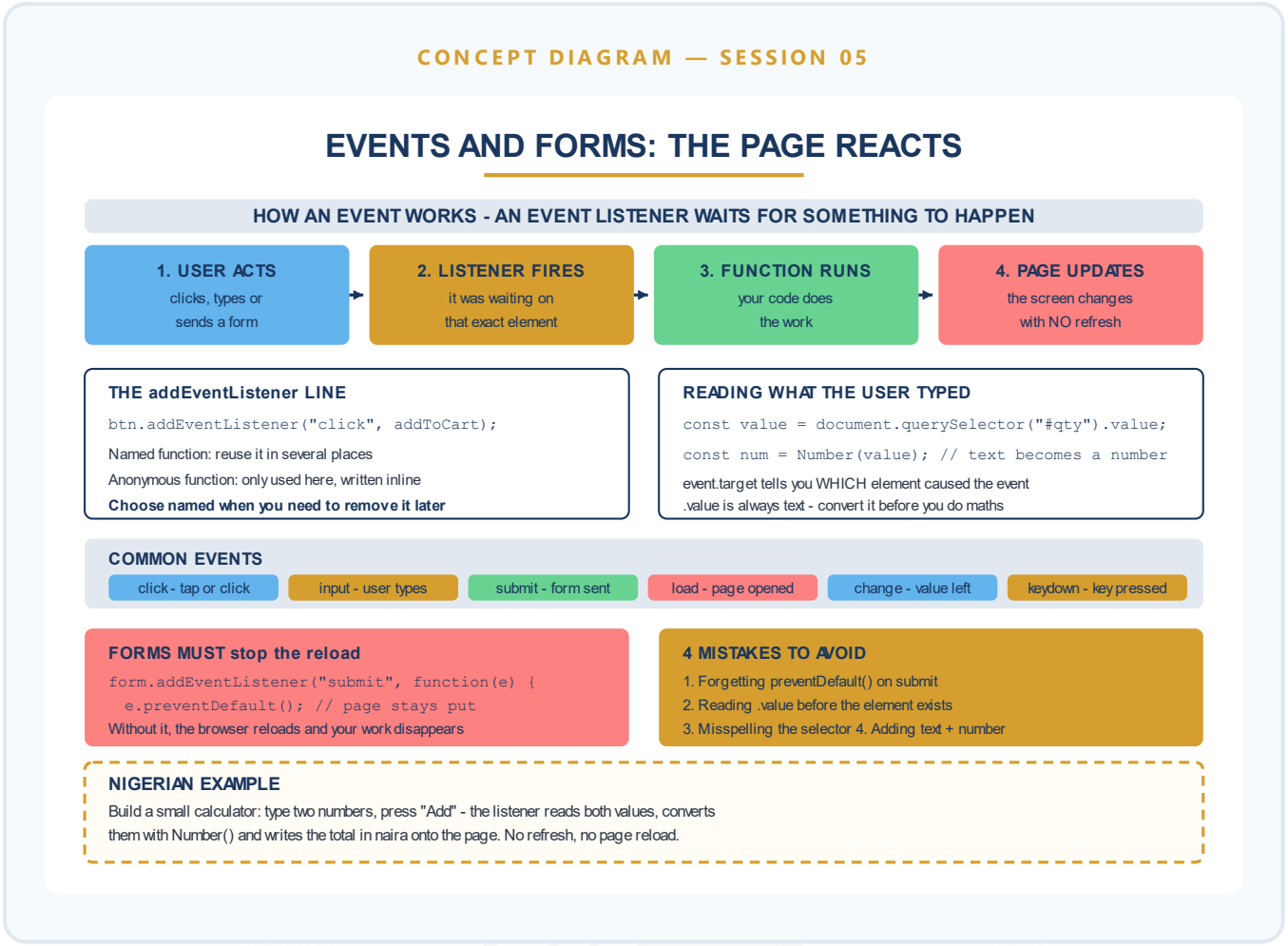
Timing guidance. The independent exercise is the heart of the session and builds directly toward the capstone. If the room is slow: turn the case study (Section J) into a 2-minute mention, and cut Demonstration 2 (`querySelectorAll` loop) to a quick look — but **protect** the guided practical (H) and the independent exercise (I), because rendering a list to the DOM is the single most reused skill in the entire course. If the room is fast, let learners attempt the Level 3 search as a stretch task.

END OF SESSION 4

Next: Session 5 — Events and Forms: Making the Page Respond to the User.

SESSION 05 OF 12

EVENTS AND FORMS BASICS



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 5 — EVENTS AND FORMS BASICS

A. SESSION IDENTIFICATION

Item	Detail
Session number	5 of 12
Session title	Events and Forms Basics
Week / Module	Week 2 — Module 2: DOM and Events
Recommended duration	105 minutes
Difficulty level	Beginner (some prior JavaScript)

Item	Detail
Prerequisite knowledge	Session 4 — DOM basics: selecting elements with `document.querySelector()` / `getElementById()`, reading and changing `textContent`, changing `.style`, and creating/removing elements. Also Sessions 1–3: variables, functions, conditions, arrays and loops. A learner must be able to select an element and change its text; if not, run the 15-minute DOM refresher in Section E before proceeding.
Connection to previous session	In Session 4 we learned to *find* elements and change them, but the change happened the moment the page loaded — the learner had no control over *when*. Today the page waits for the user to **do something** (click, type, submit) and only then reacts. This is the moment pages become truly interactive.
Preparation before class	Learner `webdev-practice` folders from Session 4 available; projector showing VS Code plus Chrome; DevTools Console open; a whiteboard for drawing the "button → listener → function → page updates" flow; instructor demo folder `demo-week2-s5/` prepared with `index.html` and `js/calculator.js` (contents in Section G).

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Define an event** in plain English and list at least four common events (`click` , `input` , `submit` , `load`).
2. **Attach an event listener** with `element.addEventListener("click", someFunction)` and explain what a listener is waiting for.
3. **Distinguish** a named function from an anonymous (inline) function in a listener, and say when each is suitable.
4. **Use the event object** `event.target` to find out which element triggered the event.
5. **Read what a user typed** using `document.querySelector("input").value` , and convert it to a number with `Number()` .
6. **Build a live calculator/counter** where a button click reads input and updates the page without a refresh.
7. **Handle a form's** `submit` **event** and explain why `event.preventDefault()` stops the page from reloading.
8. **Diagnose and fix** four common mistakes: forgetting `preventDefault()` , reading `.value` too early, misspelling a selector, and adding a number to a string.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). Shared machines are acceptable; pair learners so one types and one predicts, then swap halfway.
- Instructor computer connected to a projector or large screen.

Software

- Google Chrome (any recent version). The Console behaviour described here is stable across modern versions.
- Visual Studio Code (VS Code).
- Windows 10 or Windows 11.

Accounts

- None required. All work is offline.

Sample files

- Instructor demo folder `demo-week2-s5/` with `index.html` and `js/calculator.js` (given in Section G).
- A partially completed `starter/calculator-partial/` folder (HTML done, JavaScript stubbed) for Level 1 learners.

Worksheets

- Worksheet 5.1 — "Name the Event" (match a real-life action to a JavaScript event; 1 page).
- Worksheet 5.2 — "Predict the Page" (given a small listener block, write what the page does; in the learner workbook, Part 3).

Tools

- Chrome DevTools (press `F12`). The **Elements** tab is used to confirm the page really changed.

Internet

- Not required. If available, the instructor may optionally show the MDN page for `addEventListener` at the very end, but the session does not depend on it.

Alternative for learners who lack a facility

- Any laptop with Chrome and VS Code works. A learner with only a phone can pair with a desktop learner and photograph the code, then practise later in a phone browser tab. The exercise is short enough to complete on a shared machine.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open a product page with a "Buy Now" button and a small form (a name box and a submit button). Do not open any code yet — show only the browser.

What the instructor says:

> *"In the last session we made the page change by itself the moment it loaded. That is nice, but a real shop cannot guess what the customer wants — the customer has to act first. Look at this page. Watch what happens when I type my name and press the button on this form."*

Slowly type a name into the form and click the button. The page flashes and goes blank, or the text disappears and the URL grows a `?name=...` at the end. (This is the default browser behaviour: the form tries to send data to a server and reloads the page.)

> *"Notice the page reloaded. My typing was thrown away. That is the browser's default. Today you will learn to stop that from happening, read exactly what the user typed, and make the number or message on the page update live — without losing anything."*

Second half of the hook — the counter: On the projector, open the Console and paste this single line, then ask learners to watch the page:

```
let n = 0; document.body.innerHTML = '<button id="b">Clicked 0 times</button>'; document.querySelector("#b").addEventListener("click
```

Click the button several times. The text counts up while the page never reloads.

> *"The page is now listening. It waits. When you click, it counts. That is an event, and that is today's whole lesson."*

Expected learner response: Learners will be surprised the page reacts without reloading. Some will ask, "So the page doesn't go anywhere?" Confirm: yes — the browser can react entirely on its own, before any server is involved.

Why this opener works: It creates a concrete, visible gap (the reload problem) and immediately gives a taste of live interaction, so the session starts with a puzzle the learners want solved.

E. PREREQUISITE REVIEW (7 minutes)

Run this quickly as an oral round-robin. Put the questions on the projector.

Question 1: "What does `document.querySelector(".price")` return?"

- Expected answer: the **first** element on the page with `class="price"` (or `null` if none exists).
- Common gap: learners mix up `.price` (class) with `#price` (id). Re-state: dot = class, hash = id.

Question 2: "How do you change the visible text of a heading element called `titleEl`?"

- Expected answer: `titleEl.textContent = "New text";`.
- Common gap: learners write `titleEl.text = ...` (works sometimes but inconsistent) or forget the `=`. Reinforce `textContent`.

Question 3: "Why do we place the `<script>` at the bottom of the `<body>`?"

- Expected answer: so all the HTML elements exist before the script tries to select them; otherwise `querySelector` returns `null`.
- Common gap: learners forget and get `Cannot read properties of null`. Remind them.

Question 4: "What does `typeof "2500"` print?"

- Expected answer: `"string"`.
- Common gap: confidence. Remind them a quoted number is text, and today we will convert it with `Number()`.

Bridge to today (say this):

> *"You can find an element and change it. But right now your code changes it immediately. Think of a radio: right now you have wired the speaker, but there is no switch. An **event listener** is the switch. Today we install switches — click, type, submit — so the page changes only when the user touches it."*

If more than a third of the class cannot answer Question 1 or 3, spend an extra 10 minutes on the DOM refresher below before continuing.

DOM refresher (only if needed):

```
<h1 id="title">Old title</h1>
<script>
  const t = document.querySelector("#title");
  t.textContent = "New title";
</script>
```

Key points: `querySelector` returns one element; `textContent` reads or writes the text; the script must run *after* the element exists.

F. FULL LECTURE NOTES

F.1 What an event is

The idea. An **event** is something that happens in the browser that JavaScript is allowed to react to. It can be caused by the **user** (a click, a key press, typing, scrolling, submitting a form) or by the **browser itself** (the page finishing loading, an image finishing downloading, the window being resized).

Plain-English definition. An event is simply "a thing that happened." When it happens, the browser makes an announcement: "a click just occurred on this button!" Our job is to tell the browser what to do when it hears that announcement.

Why events exist. Without events, a page can only run its script once, top to bottom, and then sit frozen. Real software must respond to people: a shop reacts when you choose a quantity, a bank app reacts when you press "Send", a form reacts when you press "Submit". Events are how the browser says "the user did something — now react."

Real-life analogy. Think of a **security guard at a gate**. The guard does not chase every car; the guard *waits at the post*. When a car arrives (the event), the guard acts (checks the car). If no car ever comes, the guard stays calm and does nothing. An event listener is the guard; the event is the car arriving.

Common events we use in this course:

Event name	Fires when...	Typical element
`click`	The user clicks (or taps) an element	button, link, image
`input`	The user types or changes a text box (fires on every keystroke)	input, textarea

Event name	Fires when…	Typical element
`change`	The user finishes changing a value (e.g. leaves a select box)	select, input
`submit`	The user submits a form (button or Enter key)	form
`load`	The page (or an image) has finished loading	window, image

Important limitation / misconception. "Interactive" does **not** always mean "connected to a server". A counter, a live total, a colour change — these happen entirely in the browser, with no internet. Server work (like saving an order to a database) is a different topic, covered later in the course.

How a learner recognises an event in code: whenever they see `.addEventListener("…")`, the word inside the quotes is the event being watched.

F.2 Attaching a listener — `addEventListener`

The idea. We tell an element to *watch* for a specific event, and to run a specific function when it happens. That is called **attaching an event listener**.

The pattern:

```
element.addEventListener("click", functionName);
```

Three parts, read left to right:

1. `element` — the element we selected (the switch we are wiring).
2. `.addEventListener(...)` — the instruction "start listening".
3. `"click", functionName` — *what* to listen for (`"click"`), and *what to do* (run `functionName`).

Named function version:

```
const btn = document.querySelector("#calcBtn");

function sayHello() {
  console.log("Button was clicked!");
}

btn.addEventListener("click", sayHello);
```

Read this as: "Button, please start listening for a click. When a click happens, run the function called `sayHello`."

Critical detail — no parentheses after the function name. We write `sayHello`, **not** `sayHello()`. Writing `sayHello()` would *call the function immediately* and pass its result to the listener — which is wrong. The listener needs the *name of the function* so the browser can call it later, at the right moment.

Say to the class: "`sayHello` is like giving the guard the name of a dance move. `sayHello()` is like performing the whole dance now, in front of the guard, and then asking him to copy it later. Give him the name, not the performance."

Anonymous (inline) function version:

```
btn.addEventListener("click", function () {
  console.log("Button was clicked!");
});
```

Here we write the function directly inside the listener with no name. This is called an **anonymous function** (it has no name). It is convenient for short, one-off actions.

Named vs anonymous — when to use each:

Style	Use it when
Named	The action is longer, or you want to run/remove it in more than one place, or you want a clear name for debugging.

Style	Use it when
Anonymous	The action is very short and used only once, right here.

Arrow function (a modern shorthand, optional this session):

```
btn.addEventListener("click", () => {
  console.log("Clicked!");
});
```

`() => { ... }` is a shorter way to write `function () { ... }`. Both work identically here. The course accepts either; for beginners, the `function () { }` form is easier to read at first.

Common misconception. Learners attach the listener but nothing happens. The usual cause: the element was never found (`querySelector` returned `null`), so the error appears in the Console as `Cannot read properties of null (reading 'addEventListener')`. Always check the selector and that the HTML element exists.

F.3 The event object and `event.target`

The idea. When an event fires, the browser hands the listening function a small package of information about what just happened. That package is called the **event object**.

How it arrives. The listening function can receive it as a parameter:

```
btn.addEventListener("click", function (event) {
  console.log(event);
});
```

Here `event` is the parameter — the browser fills it in automatically. The name can be anything (`event`, `e`, `evt`), but the course uses `event` so it reads clearly.

`event.target` — who did it?

`event.target` is the actual element that triggered the event. This matters when many buttons share one listener.

```
<button class="buy" data-item="Hair Cream">Buy Hair Cream</button>
<button class="buy" data-item="Body Lotion">Buy Body Lotion</button>
```

```
const buttons = document.querySelectorAll(".buy");

buttons.forEach(function (button) {
  button.addEventListener("click", function (event) {
    console.log("You clicked: " + event.target.textContent);
  });
});
```

Click the first button → Console prints `You clicked: Buy Hair Cream`. Click the second → `You clicked: Buy Body Lotion`. One piece of code, and `event.target` tells it *which* button was clicked.

Real-life analogy. `event.target` is the **name tag of the person who asked the question**. If ten people are in a room and one shouts "I need help!", `event.target` tells you exactly who shouted.

Other useful pieces of the event object (brief mention):

- `event.type` — the name of the event, e.g. `"click"`.
- `event.preventDefault()` — a method to cancel the browser's default behaviour (used in Section F.6).

Common misconception. Learners think `target` is always the element the listener was attached to. It is not: it is the element the user actually interacted with (which may be a child element inside the one you attached to). For now, the two are usually the same; just know they *can* differ.

F.4 Reading what the user typed — `.value`

The idea. A text box or number box holds what the user typed. We read it with the `.value` property.

```
const nameInput = document.querySelector("#name");
const typed = nameInput.value;
console.log(typed);
```

Key facts about `.value`:

1. It is always a **string**, even if the box is `type="number"`. If the user typed `3`, `.value` is `"3"` (text), not `3` (number).
2. `.value` is read the moment you ask for it. If you read it before the user types, you get an empty string `""`.
3. To *set* the box's contents, you assign: `nameInput.value = "Adaeze";`.

Converting text to a number with `Number()`:

```
const qtyInput = document.querySelector("#qty");
const qty = Number(qtyInput.value);   // "3" becomes 3
```

Why this matters — the string trap from Session 1: if we forget `Number()`, the code treats the value as text, and `"3" * "2500"` gives `7500` by luck (the `*` operator forces a conversion), but `"3" + "2500"` would give `"32500"` — text glued together. Using `Number()` makes the intention clear and the maths reliable.

Worked example:

```
const qtyInput = document.querySelector("#qty");   // user typed 3
const priceInput = document.querySelector("#price"); // user typed 2500

console.log(qtyInput.value + priceInput.value);          // "32500"  ← text joined (bug!)
console.log(Number(qtyInput.value) + Number(priceInput.value)); // 2503  ← numbers added
```

Say to the class: "Anything that comes out of a form field is text — even digits. If you are going to do arithmetic, wrap it in `Number()`."

Trimming whitespace (mentioned, used in Session 6): sometimes a user types spaces by accident. `.value.trim()` removes spaces from the start and end. We do not need it for number inputs, but it is essential for names and emails (Session 6).

Common misconception. "Because the input is `type="number"`, `.value` is a number." False — it is still a string. `typeof` will prove it.

F.5 Putting it together — a live calculator

Here is the whole idea in one small widget: a button that, when clicked, reads two inputs, calculates a total, and writes the result onto the page.

```
<p>
  <input type="number" id="qty" value="2">
  <input type="number" id="price" value="2500">
  <button id="calcBtn">Calculate Total</button>
</p>
<p id="result">Total: ₦0</p>
```

```
const qtyInput = document.querySelector("#qty");
const priceInput = document.querySelector("#price");
const calcBtn = document.querySelector("#calcBtn");
const result = document.querySelector("#result");

function calculateTotal() {
  const qty = Number(qtyInput.value);
  const price = Number(priceInput.value);
  const total = qty * price;
  result.textContent = "Total: ₦" + total;
}

calcBtn.addEventListener("click", calculateTotal);
```

Line by line:

1. Four `querySelector` lines grab the four elements (two inputs, the button, the result paragraph) so we can use them later.
2. `function calculateTotal()` defines the action — but does not run it yet.
3. Inside, `Number(...)` converts each typed value from text to a real number.
4. `total = qty * price` does the maths.
5. `result.textContent = ...` updates the paragraph on the page.
6. The last line wires the button: "when clicked, run `calculateTotal`."

Expected behaviour: Type `3` and `1800`, click the button → the paragraph reads `Total: ₦5400`. Nothing reloads. Nothing appears in the Console unless we add a `console.log`.

Why we do NOT run `calculateTotal()` directly: if we wrote `calculateTotal();` instead of passing it to the listener, it would run once at load time (reading empty/default values) and never again when the button is clicked.

F.6 Forms and the `submit` event — and why `preventDefault()`

The idea. A `<form>` is the element that groups input fields for sending data somewhere. When a form is submitted, the browser's **default** behaviour is to **send the data to a server and reload the page**. For now we do not have a server, and reloading throws away our JavaScript state. So we must **stop the default** with `event.preventDefault()`.

The form element:

```
<form id="orderForm">
  <label for="customer">Your name</label>
  <input type="text" id="customer">

  <label for="qty">Quantity</label>
  <input type="number" id="qty" value="1">

  <button type="submit">Place Order</button>
</form>
<p id="message"></p>
```

Listening for the submit:

```
const orderForm = document.querySelector("#orderForm");

orderForm.addEventListener("submit", function (event) {
  event.preventDefault();
  console.log("Form submitted without reloading!");
});
```

What `event.preventDefault()` does: it tells the browser "do not carry out your normal action for this event." For a form submit, the normal action is "reload the page and send the data". Cancelling it keeps us on the same page.

Important detail — listen on the `<form>`, not the button. The `submit` event belongs to the form. If you attach a `click` listener to the submit button instead, pressing **Enter** inside a field would still submit and reload. Listening for `submit` on the form catches both the button click and the Enter key.

Real-life analogy. The form is like a **courier envelope** ready to be sealed and posted. `preventDefault()` is like catching the courier at the door and saying, "Wait — I still need to add something." Without it, the envelope leaves immediately and you get a fresh, empty page back.

Common misconception. Learners call `event.preventDefault()` but it does nothing. Usually the listener was attached to the wrong element (the button, not the form) or the parameter `event` was not declared in the function, so `event` is undefined. Always write `function (event) { ... }`.

F.7 The running example — Adaeze Fashion's live calculator

Scenario. Adaeze runs *Adaeze Fashion* in Port Harcourt. Her page lists an item at a unit price; the customer chooses a quantity and the page should show the total **live**, without the page jumping or reloading. This is exactly the "quantity × price = total" widget.

Full runnable HTML — `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Price Calculator</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 420px; margin: 40px auto; }
    h1 { color: #8a2b6b; }
    label { display: block; margin-top: 12px; font-weight: bold; }
    input { padding: 8px; width: 100%; box-sizing: border-box; }
    button { margin-top: 15px; padding: 10px 18px; cursor: pointer; }
    #result { margin-top: 18px; font-size: 22px; font-weight: bold; }
  </style>
</head>
<body>
  <h1>Adaeze Fashion</h1>
  <p>Enter the quantity and the unit price, then click Calculate.</p>

  <label for="qty">Quantity</label>
  <input type="number" id="qty" value="1" min="1">

  <label for="price">Unit price (₦)</label>
  <input type="number" id="price" value="2500" min="0">

  <button id="calcBtn">Calculate Total</button>

  <p id="result">Total: ₦0</p>

  <script src="js/calculator.js"></script>
</body>
</html>
```

Full runnable JavaScript — `js/calculator.js`:

```
// Adaeze Fashion — live price calculator

// 1. Grab (select) all the elements we need
const qtyInput = document.querySelector("#qty");
const priceInput = document.querySelector("#price");
const calcBtn = document.querySelector("#calcBtn");
const result = document.querySelector("#result");

// 2. Define the action
function calculateTotal() {
  // Read the typed values (they are strings) and convert to numbers
  const qty = Number(qtyInput.value);
  const price = Number(priceInput.value);

  // Guard against empty or silly input
  if (isNaN(qty) || isNaN(price)) {
    result.textContent = "Please enter numbers in both boxes.";
    return;
  }

  // Do the maths
  const total = qty * price;

  // Update the page
  result.textContent = "Total: ₦" + total;
}

// 3. Wire the button to the action.
//    Note: pass the NAME (calculateTotal), not calculateTotal().
calcBtn.addEventListener("click", calculateTotal);
```

Line-by-line explanation of the JavaScript:

- Lines 4–7: select the two inputs, the button and the result paragraph. If any selector is misspelled, that variable is `null` and we will see an error on the next use.

- Line 10: `function calculateTotal()` — defines the action; nothing runs yet.
- Line 12–13: `.value` reads what is in each box (text), and `Number()` converts it to a real number.
- Line 16–19: `isNaN` means "is Not a Number". If the user cleared a box or typed letters, we show a friendly message and `return` (stop the function early) so we never show a nonsense total.
- Line 22: `total = qty * price`.
- Line 25: write the total into the page with `textContent`.
- Line 30: attach the listener. Every future click runs `calculateTotal` again with the *current* values.

Expected on-page behaviour:

1. Page opens showing `Total: ₦0`.
2. Change quantity to `3`, price to `1800`, click **Calculate Total** → `Total: ₦5400`.
3. Change quantity to `2`, price to `12000`, click again → `Total: ₦24000`.
4. Clear the price box and click → `Please enter numbers in both boxes.`
5. At no point does the page reload; the typed values are never lost.

Possible extension (optional, show if time allows): react while typing instead of only on click, by listening for `input` on both boxes:

```
qtyInput.addEventListener("input", calculateTotal);
priceInput.addEventListener("input", calculateTotal);
```

Now the total updates on every keystroke. This is a great "wow" moment and shows that the same function can serve several listeners.

F.8 Summary of the session

- An **event** is something that happens in the browser (click, input, submit, load); we **react** to it.
- `element.addEventListener("click", functionName)` wires a switch. Pass the function **name** without `()`.
- Use a **named** function for reuse; an **anonymous** function for short one-off actions.
- The **event object** carries details; `event.target` is the element that was actually interacted with.
- Read user input with `.value` (always text); convert with `Number()` for maths.
- For forms, listen for `submit` on the `<form>` and call `event.preventDefault()` to stop the reload.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- `demo-week2-s5/index.html` (the Adaeze Fashion page from Section F.7) and `js/calculator.js`.
- Both files open in VS Code.
- Chrome open to the page, DevTools Console open.
- Whiteboard with the flow drawn: **Button → click → listener → calculateTotal → update page**.

G.2 Demonstration 1 — the click listener (6 minutes)

Say: "Watch how a page can wait for a click and then act."

Do this on the projector:

1. Show `index.html` linking `js/calculator.js` at the bottom of the body.
2. In `js/calculator.js`, type the basic version (variables + `calculateTotal` + one listener).
3. Save both files, open the page, press `F12`.
4. Type quantity `3`, price `1800`, click the button.
5. **Pause and ask before clicking:** "What will the total line show?" (₦5400.)
6. Point at the page updating. Then change the price to `1000` and click again — total becomes ₦3000.

Then inject the classic error — remove the parentheses rule by writing:

```
calcBtn.addEventListener("click", calculateTotal());
```

- Save, refresh. **Expected result:** the total is calculated **immediately on load** and the button does nothing when clicked (or an error appears because the function ran with no user input).
- **Say:** "See the `()`? That called the function *now* and passed nothing to the listener. Remove the parentheses and the button works again."
- Change it back to `calculateTotal` (no parentheses), save, refresh. Button works.

Questions to ask:

- "Why did the total change after I typed a new price?" (Because `.value` reads the box at the moment of the click, not at load.)
- "What would happen if the script were in the `<head>`?" (The elements would not exist yet; the selectors would be `null`.)

G.3 Demonstration 2 — form submit and `preventDefault()` (9 minutes)

Say: "Now the form problem from our opening. Watch the page reload — and then watch me stop it."

Set up a form on the page:

```
<form id="newsletter">
  <label for="email">Your email</label>
  <input type="email" id="email" placeholder="you@example.com">
  <button type="submit">Subscribe</button>
</form>
<p id="thanks"></p>
```

First, show the problem — attach a `click` listener to the button only:

```
document.querySelector("#newsletter button").addEventListener("click", function () {
  alert("Subscribed!");
});
```

- Fill in an email, press **Enter** instead of clicking. **Expected:** the page reloads (the click listener never fired for the Enter key) and the email is lost.
- **Say:** "That is the danger — the form's real event is `submit`, not the button's click."

Now fix it — listen for `submit` on the form and prevent the default:

```
const form = document.querySelector("#newsletter");
const email = document.querySelector("#email");
const thanks = document.querySelector("#thanks");

form.addEventListener("submit", function (event) {
  event.preventDefault();                 // stop the page reload
  const typed = email.value.trim();       // read what was typed
  thanks.textContent = "Thank you! We will contact " + typed;
  form.reset();                           // clear the boxes
});
```

- Save, refresh. Type an email, press **Enter**. **Expected:** no reload; the paragraph shows `Thank you! We will contact you@example.com`; the box clears.
- Press Enter with an empty box to show it still "works" (we will properly validate in Session 6).

Then deliberately break it: remove the `event.preventDefault();` line, save, refresh, press Enter. **Expected:** the page reloads, the email disappears, and the URL grows `?...`. **Say:** "This is the single most common form bug. If your page keeps jumping, you forgot `preventDefault()`." Put it back.

Reasoning to explain aloud: the browser has a *default* action for submit (send and reload); `preventDefault` cancels just that action without breaking the rest of our code; reading `.value` gives us the text; `textContent` writes it back to the page.

Questions to ask during the demo:

- "Why do we attach to the form rather than the button?" (To catch Enter as well as clicks.)
- "What does `event.target` refer to here?" (The `<form>`, because we attached to the form.)
- "What type is `email.value`?" (A string — we trim it and could convert it if it were a number.)

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding step by step, build a **"Counter and Tip" mini-app** — a click counter and a simple tip calculator on one page, both driven by events.

Starting materials: A new folder `webdev-practice/s5/` with an empty `index.html` and an empty `js/tip.js` (or learners create them; instructor shows both).

Procedure (instructor calls out each numbered step; learners follow; instructor walks the room):

1. Create `index.html` with the following structure:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Counter and Tip</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 420px; margin: 40px auto; }
    label { display: block; margin-top: 12px; }
    input, button { padding: 8px; margin-top: 6px; }
    #count { font-size: 24px; font-weight: bold; }
    #tipResult { margin-top: 14px; font-weight: bold; }
  </style>
</head>
<body>
  <h1>Counter &amp;amp; Tip</h1>

  <h2>Counter</h2>
  <p id="count">0 clicks</p>
  <button id="countBtn">Click me</button>

  <h2>Tip Calculator</h2>
  <label for="bill">Bill amount (₦)</label>
  <input type="number" id="bill" value="5000">
  <label for="tipPercent">Tip percent (%)</label>
  <input type="number" id="tipPercent" value="10">
  <button id="tipBtn">Calculate Tip</button>
  <p id="tipResult">Tip: ₦0</p>

  <script src="js/tip.js"></script>
</body>
</html>
```

2. Create `js/tip.js` and start with the counter:

```
// ---- Counter ----
const countBtn = document.querySelector("#countBtn");
const countLabel = document.querySelector("#count");
let clicks = 0;

function addClick() {
  clicks = clicks + 1;
  countLabel.textContent = clicks + " clicks";
}

countBtn.addEventListener("click", addClick);
```

3. Save and test: click the button several times. The counter should rise: `1 clicks`, `2 clicks`, `3 clicks`.

4. Add the tip calculator below the counter in the same file:

```
// ---- Tip Calculator ----
const billInput = document.querySelector("#bill");
const tipInput = document.querySelector("#tipPercent");
const tipBtn = document.querySelector("#tipBtn");
const tipResult = document.querySelector("#tipResult");

function calculateTip() {
  const bill = Number(billInput.value);
  const percent = Number(tipInput.value);
  const tip = (bill * percent) / 100;
  tipResult.textContent = "Tip: ₦" + tip;
}

tipBtn.addEventListener("click", calculateTip);
```

5. Save, refresh, test: bill `5000`, tip `10` → `Tip: ₦500`. Change tip to `15` → `Tip: ₦750`.
6. **Pause and predict:** "Before I click with bill 8000 and tip 7.5, what will the tip be?" (₦600, because 8000 × 7.5 ÷ 100 = 600.)

Expected result:

- Counter increases by one on each click, no page reload.
- Tip updates on each calculation from the current box values.
- Console shows no red errors.

Points to pause on and predict:

- After step 3: "Where is the number `clicks` stored — on the page or in a variable?" (In the variable; the page just displays it.)
- After step 5: "If I read `billInput.value` on page load instead of on click, what would I get?" (`"5000"` , the default HTML value — but it would never update when the user changes it.)

Troubleshooting while walking the room:

- Button does nothing → the function name in the listener has `()` after it, or the selector is misspelled (`null`).
- Counter jumps by a lot → the listener was attached more than once (e.g. pasted twice).
- Tip shows `NaN` → a box was empty; add the `isNaN` guard from Section F.7.
- Everything prints `Total: ₦50005000` → forgot `Number()` ; the values were joined as text.

Completion checklist (learner ticks each):

- ☐ Counter increases correctly and never reloads the page
- ☐ Counter is stored in a variable (`let clicks`)
- ☐ Tip calculator reads both inputs on each click
- ☐ Both inputs converted with `Number()`
- ☐ At least one **named** function and one listener used
- ☐ No red errors in the Console

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Build a **"POS Change Calculator"** on your own, without copying the tip example.

Scenario: You work at *Chidi's Provisions* in Port Harcourt. The cashier enters the **total price** of a customer's goods and the **amount of cash the customer handed over**. The page should show the **change due**, live, when a button is clicked. If the cash handed over is less than the price, show a warning instead of a negative change.

Starting information (given):

- A number input for the goods price.
- A number input for the cash paid.
- A button labelled **Calculate Change**.
- A paragraph for the result.

Requirements:

1. Create `change.html` and `js/change.js`, linked at the bottom of the body.
2. Select the two inputs, the button and the result paragraph.
3. Write a **named** function `calculateChange()`.
4. Inside, convert both values with `Number()`.
5. If the cash paid is **less than** the price, set the result to `"Not enough money — customer still owes ₦X"` where X is the difference.
6. Otherwise set the result to `"Change: ₦X"`.
7. Attach the function to the button's `click` event.
8. Add an `input` listener on the cash box so the change also updates while the cashier types.

Constraints:

- All arithmetic must use numbers, not strings (use `Number()`).
- No page reload at any point.
- Do not copy the tip example line-for-line — write your own structure.

Time allowance: 15 minutes.

Success criteria:

- Price `4500`, cash `5000` → `Change: ₦500`.
- Price `4500`, cash `4000` → `Not enough money — customer still owes ₦500`.
- Typing in the cash box updates the result without clicking (after requirement 8).
- No red errors.

Hints:

- Hint 1: `const price = Number(priceInput.value);`
- Hint 2: for the warning, `if (cash < price) { ... } else { ... }`.
- Hint 3: the shortfall is `price - cash`.

Instructor answer key (do not show learners):

```
// change.js — Chidi's Provisions POS Change Calculator
const priceInput = document.querySelector("#price");
const cashInput = document.querySelector("#cash");
const calcBtn = document.querySelector("#calcBtn");
const result = document.querySelector("#result");

function calculateChange() {
  const price = Number(priceInput.value);
  const cash = Number(cashInput.value);

  if (isNaN(price) || isNaN(cash)) {
    result.textContent = "Please enter both amounts.";
    return;
  }

  if (cash < price) {
    result.textContent = "Not enough money — customer still owes ₦" + (price - cash);
  } else {
    result.textContent = "Change: ₦" + (cash - price);
  }
}

calcBtn.addEventListener("click", calculateChange);
cashInput.addEventListener("input", calculateChange);
```

Marking guidance (out of 10):

- Correct file structure and linking (2)
- Named function and click listener (2)
- `Number()` conversion used correctly (2)
- Correct change when cash is enough (2)
- Warning with correct shortfall when cash is short (1)
- Live update on `input` (1)

J. REAL-WORLD CASE STUDY

Situation. *Adaeze Fashion* has a product page. Customers who want to buy several items of the same product currently have to message Adaeze on WhatsApp to ask, "How much for four?" Adaeze answers the same question dozens of times a day. She wants the page to answer it instantly.

Relevant facts.

- Each product has a fixed unit price (for example, the *Hair Cream* at ₦2,500).
- Customers choose a quantity and want to see the total immediately.
- Adaeze is not a programmer; her page was built in the Web Design course and now has a JavaScript file.

Decision point. How should the page let a customer see the total without messaging anyone?

Possible approaches.

1. Add the price of the item as plain text and let the customer multiply it themselves.
2. Build an event-driven calculator on the page: a quantity box, a button, and a live total paragraph.
3. Integrate a full online payment system with a server and a database.

Recommended solution (for this stage). Approach 2. It removes the repeated WhatsApp questions, uses only the skills from this session, and works offline in the browser.

Example solution (condensed):

```
const qtyInput = document.querySelector("#qty");
const totalLabel = document.querySelector("#total");
const unitPrice = 2500;

function updateTotal() {
  const qty = Number(qtyInput.value);
  totalLabel.textContent = "Total: ₦" + (qty * unitPrice);
}

qtyInput.addEventListener("input", updateTotal);
```

Here the unit price is a `const` (it is fixed on the page), and the total updates on every keystroke — the customer sees `Total: ₦10000` the instant they type `4`.

Trade-offs.

- Approach 1 is free but still burdens the customer and invites arithmetic mistakes (garbage in, garbage out).
- Approach 2 takes about 30 minutes to build and permanently removes the repeated question; it is the professional first step.
- Approach 3 is the eventual goal (it needs a server, payment gateway and security) and is far beyond this week. It is worth doing only once the shop sells online for real.

Professional connection. This exact pattern — a small input, a live total, no reload — appears on almost every e-commerce site (Jumia, Konga, and small shops alike). A junior front-end developer is very often asked to "make this total update when the customer changes the quantity." This exercise is that real task.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this as the live troubleshooting guide. When a learner is stuck, ask them to check the relevant row rather than giving the answer.

K.1 The page reloads / jumps and loses the typed data

Symptoms: After submitting a form, the page refreshes, the boxes clear, and the URL grows `?...`. **Likely cause:** The default form behaviour was not cancelled — missing `event.preventDefault()`, or the listener was attached to the button instead of the form. **Investigate:** Check that `addEventListener("submit", function (event) { ... })` is on the `<form>`, and that

`event.preventDefault();` is the first line inside. **Fix:** Move the listener to the form and add `event.preventDefault();`. **Confirm:** Submitting no longer reloads; the info stays on the page. **Prevent:** Always pair a `submit` listener with `preventDefault()`.

K.2 Button click does nothing

Symptoms: Clicking the button produces no change and no error. **Likely causes:** (a) The listener was never attached. (b) The function name has `()` in the listener. (c) The `querySelector` selector does not match the HTML (`null`). **Investigate:** Add `console.log("listener attached");` just after the `addEventListener` line, and `console.log("clicked");` inside the function. Reload and click. **Fix:** Remove `()` from the function name; correct the selector; make sure the element id in HTML matches. **Prevent:** Use ids consistently; place the script at the bottom of the body.

K.3 `Cannot read properties of null (reading 'addEventListener')`

Symptoms: Red error mentioning `null`. **Likely cause:** `querySelector` found no element — a typo in the selector, or the script ran before the element existed. **Investigate:** In the Console, run the same selector, e.g. `document.querySelector("#calcBtn")`. If it prints `null`, the selector is wrong. **Fix:** Correct the spelling/case (`#calcBtn` ≠ `#calcbtn`) or move the script to the bottom of the body. **Prevent:** Match ids exactly and keep scripts at the bottom.

K.4 The total shows text glued together (e.g. `₦50005000`)

Symptoms: Numbers appear joined instead of added/multiplied. **Likely cause:** `.value` returns a string and `Number()` was not used. **Investigate:** `console.log(typeof billInput.value)`. **Fix:** Wrap the value in `Number(...)` before arithmetic. **Prevent:** Treat every `.value` as text until converted.

K.5 The total shows `NaN`

Symptoms: The result reads `Total: ₦NaN`. **Likely cause:** A box was empty or contained non-numeric text, so `Number("")` is `0` but a cleared required field or letters give `NaN`. **Investigate:** `console.log(qtyInput.value)` before calculating. **Fix:** Add an `isNaN` guard and a friendly message (see Section F.7). **Prevent:** Validate before calculating; do not assume the user typed a number.

K.6 Reading `.value` gives an empty string

Symptoms: Console logs `""` even though the page shows text. **Likely cause:** You read `.value` at load time (before the user typed), or you selected the wrong element (e.g. the `<label>` instead of the `<input>`). **Investigate:** Confirm the selector points at the `<input>`, and that the read happens inside the event function, not at the top level. **Fix:** Move the read inside the click/submit handler. **Prevent:** Read values only when reacting to an event.

Distinguish likely vs confirmed: In every case, the *likely* cause becomes *confirmed* only after you check it (e.g. run the selector in the Console, or `typeof` the value). Teach learners to confirm before fixing.

L. SESSION ASSIGNMENT

Assignment title: "My Interactive Order Form"

Purpose: Practise events, reading input, conversion and `preventDefault()` in a form context the learner controls.

Realistic scenario: A small food business takes orders over a simple form. You will build a page where a customer types their **name**, chooses a **quantity** of a product, and sees a **live order summary** — all without the page reloading.

Detailed instructions:

1. Create a folder `assignment5` with `index.html` and `js/order.js`.
2. In the HTML, include a `<form id="orderForm">` containing:
   - a text input for the customer's name (`id="name"`),
   - a number input for quantity (`id="qty"`, default `1`),
   - a fixed display of the unit price (for example, *"Jollof Rice — ₦3,000 each"*),
   - a submit button,
   - a paragraph `id="summary"` for the output.
3. In `order.js`:

- Select the form, both inputs and the summary paragraph.
- Add a `submit` listener that calls `event.preventDefault()` first.
- Read the name and quantity, convert quantity with `Number()`.
- Calculate `total = qty * 3000`.
- Set the summary to a sentence such as: `"Adaeze, your 2 plates of Jollof Rice cost ₦6000."`
- Clear the form with `form.reset()` after showing the summary.

4. Add a live preview using an `input` listener on the quantity box, so the total also appears while typing.

Required deliverables:

- `assignment5/index.html`
- `assignment5/js/order.js`
- A photo/screenshot of the page showing a completed order summary.

Submission format: Files in the class folder (or zipped); screenshot in the workbook.

Deadline: Before Session 6.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
Correct structure & linking	3	External `order.js` linked at the bottom of the body
`submit` listener on the form	4	Attached to the form, not the button
`event.preventDefault()` used	4	Page never reloads on submit
Reads inputs & converts with `Number()`	3	Name as text, quantity as number
Correct total & clear summary	4	Total correct; sentence reads naturally
Live preview + clean output	2	`input` listener updates as the user types; no red errors

Common mistakes to avoid: forgetting `preventDefault()`; attaching to the button; forgetting `Number()`; misspelling a selector; leaving the script in the head.

Instructor reference solution:

```
// order.js — My Interactive Order Form
const orderForm = document.querySelector("#orderForm");
const nameInput = document.querySelector("#name");
const qtyInput = document.querySelector("#qty");
const summary = document.querySelector("#summary");
const unitPrice = 3000;

function showSummary() {
  const name = nameInput.value.trim() || "Customer";
  const qty = Number(qtyInput.value);
  const total = qty * unitPrice;
  summary.textContent = `${name}, your ${qty} plates of Jollof Rice cost ₦${total}.`;
}

orderForm.addEventListener("submit", function (event) {
  event.preventDefault();     // stop the reload
  showSummary();
  orderForm.reset();          // clear the boxes after showing
});

qtyInput.addEventListener("input", showSummary);   // live preview
```

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. In one sentence, what is an event in a web page?

Q2. Multiple choice. Which line correctly makes `myBtn` run `doThing` when clicked? a) `myBtn.addEventListener("click", doThing());` b) `myBtn.addEventListener("click", doThing);` c) `myBtn.onClick = doThing;` d) `myBtn.addEventListener(doThing, "click");`

Q3. Explain in your own words. Why do we call `event.preventDefault()` inside a form's `submit` listener?

Q4. Practical identification. Given `<input type="number" id="age" value="20">`, what does each print? a) `document.querySelector("#age").value` b) `typeof document.querySelector("#age").value` c) `Number(document.querySelector("#age").value) + 5`

Q5. Error detection. Find and fix the bug:

```
const qty = document.querySelector("#qty");
const total = qty.value * 2500;   // user typed 4
console.log(total);               // sometimes wrong / unexpected
qty.addEventListener("click", function () { console.log("clicked"); });
```

Q6. Scenario. A shop's "Buy" button should show the item name in a message paragraph when clicked, using `event.target`. Describe (or write) the listener that does this for a button `<button class="buy">Sunglasses</button>`.

INSTRUCTOR ANSWER KEY (Q1–Q6)

A1. An event is something that happens in the browser — usually caused by the user (a click, typing, submitting) or by the page loading — that JavaScript can react to.

A2. (b) `myBtn.addEventListener("click", doThing);`. Option (a) calls the function immediately; (c) is not the standard method (and case is wrong); (d) swaps the arguments.

A3. A form's default behaviour is to send the data and reload the page. `preventDefault()` cancels that default so the page stays put, keeping our data and letting our JavaScript handle the information.

A4. a) `"20"` (a string), b) `"string"`, c) `25` (because `Number` converts "20" to 20, then +5).

A5. Two issues: (1) `qty.value` is a string placed at the top level, read once at load (before the user necessarily typed) and not converted, so the maths may be wrong or `NaN`. (2) The learner tried to detect a click on an input, but the intended action is unclear. Corrected version:

```
const qty = document.querySelector("#qty");
function showTotal() {
  const total = Number(qty.value) * 2500;
  console.log(total);
}
qty.addEventListener("input", showTotal);
```

A6.

```
const button = document.querySelector(".buy");
const message = document.querySelector("#message");
button.addEventListener("click", function (event) {
  message.textContent = "You selected: " + event.target.textContent;
});
```

`event.target` is the `<button>`, so `event.target.textContent` reads `"Sunglasses"`.

N. REFLECTION AND RECAP

Plain-English summary. Today your pages learned to *listen*. An **event** is something that happens in the browser — a click, typing, a form submit, a page load. You wired switches with `element.addEventListener("click", functionName)`, passing the function's **name** (never calling it with `()`), and learned the difference between **named** and **anonymous** functions. The **event object** and `event.target` tell you what happened and which element triggered it. You read user input with `.value` (always text)

and converted it with `Number()` for maths, then updated the page live. Finally, you handled forms: listen for `submit` on the `<form>` and call `event.preventDefault()` so the page does not reload and lose the data.

Five important takeaways:

1. An event is "a thing that happened"; a listener is "what to do when it happens".
2. `addEventListener("click", myFunction)` — pass the name, not `myFunction()`.
3. `.value` gives what the user typed, always as text; use `Number()` for maths.
4. `event.target` tells you which element was interacted with.
5. Forms: listen for `submit` on the form and always call `event.preventDefault()`.

Questions to help you recognise what you can now do:

- Can I attach a click listener and update the page without reloading?
- Can I read a number from an input and calculate with it?
- Can I explain the difference between `doThing` and `doThing()` in a listener?
- Can I stop a form from reloading the page, and say why that matters?

Connection to the next session: Right now the page accepts whatever the user types — even empty boxes or a fake email. Next session we make the page **check the input first**, show clear error messages, and only proceed when everything is valid. We will also learn to **remember** the user's details with **localStorage**, so their name is still there when they come back tomorrow.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner attaches a listener and passes the function name without parentheses, unprompted.
- Learner reads a value inside the event function (not at load) and converts it with `Number()`.
- Learner attaches the `submit` listener to the form and adds `preventDefault()` as the first line.
- Learner uses `event.target` or can explain what it would contain.
- Learner tests boundary cases (e.g. empty box, cash less than price) without being told.

Common misconceptions to watch for and correct:

- "`addEventListener("click", doThing())` is fine." → No; the `()` runs it immediately. Demonstrate the bug from Section G.2.
- "`.value` is a number because the input is `type="number"`." → Prove it with `typeof`.
- "The submit button's click is the form event." → It is not; pressing Enter bypasses a click listener. Use `submit`.
- "Events need the internet." → Counters and live totals run entirely offline.

Suggested remedial exercises for learners who struggle:

1. **Switch drill:** give five one-line snippets where the function is called with `()` and ask learners to fix each so it runs only on click.
2. **Trace the flow:** hand out the Adaeze calculator with the numbers filled in and ask learners to write, step by step, what happens from click to updated total.
3. **Number-conversion drill:** give values from `.value` and ask learners to write the correct `Number(...)` line and expected result.

Differentiation:

- **Level 1 (supported):** provide `calculator-partial/` with the HTML done and the JavaScript stubbed (selectors and function shell written, body empty). Learner fills in the conversion, maths and listener.
- **Level 2 (standard):** the guided exercise as written.
- **Level 3 (challenge):** add a **discount** input and a **VAT** input to the calculator, apply both to the subtotal, and display subtotal, discount amount, VAT amount and grand total — all updating live on `input`.

Timing guidance: The independent exercise is the heart of the session. If the room is slow, compress the case study to a 2-minute mention and protect the practical time. If very fast, extend with the "live update on `input`" enhancement in Section F.7.

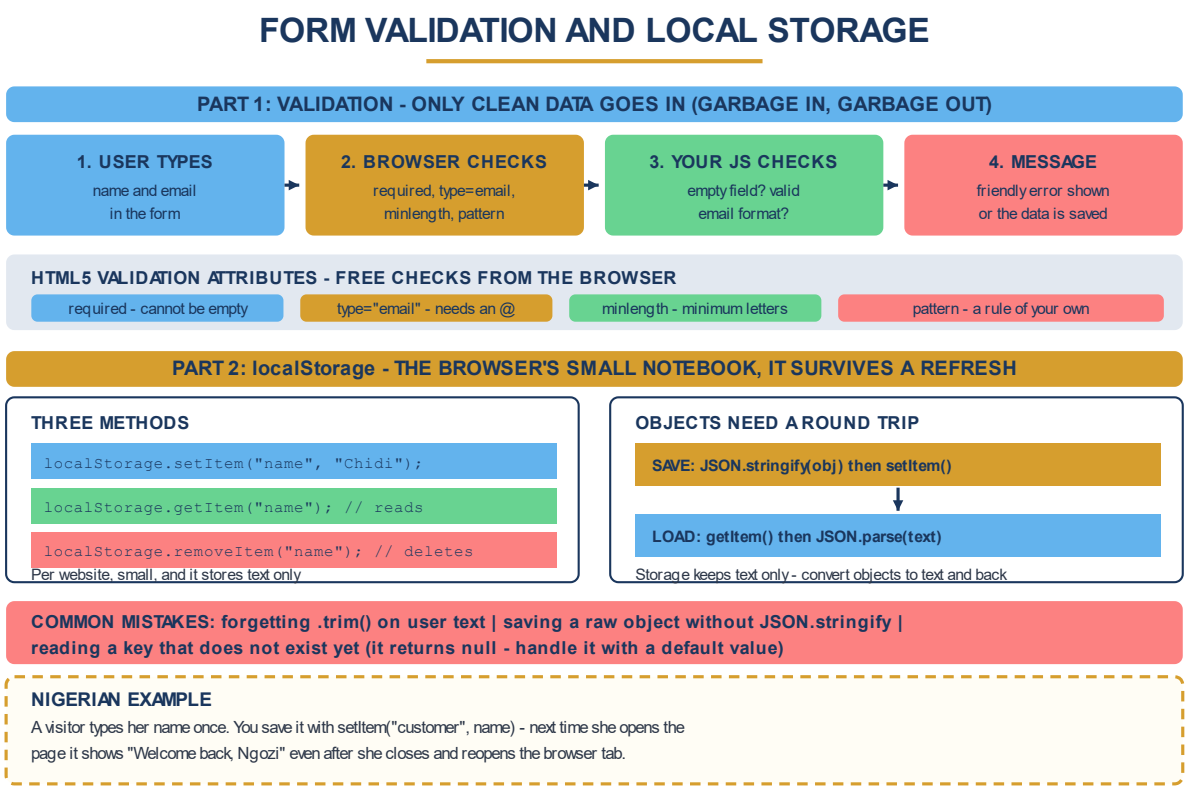
END OF SESSION 5

Next: Session 6 — Form Validation and Local Storage.

SESSION 06 OF 12

FORM VALIDATION AND LOCAL STORAGE

CONCEPT DIAGRAM — SESSION 06



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 6 — FORM VALIDATION AND LOCAL STORAGE

A. SESSION IDENTIFICATION

Item	Detail
Session number	6 of 12
Session title	Form Validation and Local Storage
Week / Module	Week 2 — Module 2: DOM and Events (storage introduction)
Recommended duration	105 minutes
Difficulty level	Beginner (consolidating events and forms)

Item	Detail
Prerequisite knowledge	Session 5 — events (`addEventListener`) for `click` , `input` and `submit` ; reading input with `.value` ; converting with `Number()` ; `event.preventDefault()` ; and Sessions 1–4 (variables, functions, conditions, arrays/objects, DOM selection and `textContent`). A learner must be able to read an input value inside an event function; if not, run the review in Section E before continuing.
Connection to previous session	In Session 5 the page *reacted* to user actions, but it accepted anything the user typed — even an empty name or a fake email. Today we add a **checkpoint**: the page inspects the input, shows a clear message if something is wrong, and only proceeds when the input is good. We also learn how the browser can **remember** details after the page is closed or refreshed, using **localStorage**.
Preparation before class	Learner `webdev-practice` folders from Session 5 available; projector showing VS Code plus Chrome; DevTools open (both the Console and the **Application → Local Storage** panel, so learners can *see* saved data); whiteboard for the validation flow diagram; instructor demo folder `demo-week2-s6/` with `index.html` and `js/validate.js` (contents in Section G). Reinforce: this is the final session of Week 2 and feeds the Week 2 mini-project (a validated form).

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Explain why form validation matters** using the idea of "garbage in, garbage out" and the safety of the user and the business.
2. **Use HTML5 validation attributes** — `required` , `minlength` , `type="email"` and `pattern` — and describe what each one checks.
3. **Write manual JavaScript validation** that checks for an empty field, checks a simple email format, and shows or hides an error message element.
4. **Prevent a form from submitting until it is valid**, and give the user clear, friendly feedback.
5. **Define** `localStorage` as the browser's small, per-website, string-only storage that survives a refresh.
6. **Save, read and delete** values with `localStorage.setItem()` , `.getItem()` and `.removeItem()` .
7. **Store objects and arrays** by converting them with `JSON.stringify()` on save and `JSON.parse()` on load.
8. **Build a page that welcomes a returning user** ("Welcome back, [name]") and saves a small list, both persisting after a refresh.
9. **Diagnose and fix** common mistakes: forgetting to `trim()` , saving data as a raw object without `JSON.stringify()` , and reading a key that does not exist.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). Shared machines acceptable; pair learners and swap halfway.
- Instructor computer connected to a projector or large screen.

Software

- Google Chrome (any recent version).
- Visual Studio Code (VS Code).
- Windows 10 or Windows 11.

Accounts

- None required. All work is offline.

Sample files

- Instructor demo folder `demo-week2-s6/` with `index.html` and `js/validate.js` (given in Section G).
- A partially completed `starter/contact-form-partial/` folder (HTML + error spans done, JavaScript stubbed) for Level 1 learners.

Worksheets

- Worksheet 6.1 — "Spot the Bad Input" (given five pieces of user input, tick whether each is valid and name the rule; 1 page).
- Worksheet 6.2 — "Save or Parse?" (given five lines, decide whether `JSON.stringify` or `JSON.parse` is needed; in the learner workbook, Part 3).

Tools

- Chrome DevTools: the **Console** and the **Application → Local Storage** panel (used to watch data being saved in real time).

Internet

- Not required. If available, the instructor may optionally show the MDN page for `localStorage` at the end, but the session does not depend on it.

Alternative for learners who lack a facility

- Everything runs offline. A learner with only a phone can pair with a desktop learner and photograph the code, then practise in a phone browser tab afterwards. The localStorage examples work in any modern mobile browser too.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open a simple contact form (name, email, message, a "Send" button). Demonstrate two bad submissions using only the browser and the Console — no code yet.

What the instructor says:

> *"Imagine you are the person receiving orders for Adaeze Fashion. A customer fills your new contact form and presses Send. What if the name box is empty? What if the 'email' is just the word `hello`? What if the message is blank? At the moment, in the form we built last week, the page happily accepts all of that. It says 'Thank you!' — for an order it cannot even reply to."*

Type nothing into the fields and submit; then type `hello` in the email field and submit. Both times the page thanks you.

> *"That is called **garbage in, garbage out**. If the page accepts junk going in, the shop has junk coming out — wrong orders, missed customers, wasted time. Today we put a **checkpoint** at the door of the form: nothing gets through until it is good."*

Second half of the hook — memory. Now say:

> *"And here is a second problem. Every time this customer refreshes the page, the form forgets them completely. Would it not be better if the shop's page remembered their name and greeted them, 'Welcome back, Chinedu'? The browser can actually remember — right here on this computer — even after the page is closed. That is the second thing we learn today."*

On the projector, open the Console, type the following and press Enter, then refresh the page and type it again:

```
localStorage.setItem("demoName", "Chinedu");
localStorage.getItem("demoName");
```

The second time (after refresh) it still prints `"Chinedu"`.

> *"It remembered, even through a refresh. That is localStorage."*

Expected learner response: Learners will recognise the empty-form problem from their own experience with websites that reject submissions. Some will ask, "Is this a database?" Explain: no — it is only on this computer, in this browser, for this website.

Why this opener works: It creates two concrete problems — bad data and forgotten details — that the two halves of today's lesson solve, and it gives learners an immediate, visible proof that the browser can remember.

E. PREREQUISITE REVIEW (7 minutes)

Run this quickly as an oral round-robin. Put the questions on the projector.

Question 1: "In Session 5, which event do we listen for on a form, and what is the first line inside the handler?"

- Expected answer: the `submit` event; the first line is `event.preventDefault();`.
- Common gap: learners attach to the button or forget `preventDefault`. Re-state: always listen on the form and prevent the default reload.

Question 2: "How do you read what the user typed in `<input id="email">`?"

- Expected answer: `const email = document.querySelector("#email").value;` — a string.
- Common gap: reading at load time instead of inside the handler. Remind them to read it when reacting to the event.

Question 3: "What does `Number("2500")` return, and what does `Number("abc")` return?"

- Expected answer: `2500`; and `NaN` (Not a Number).
- Common gap: learners think `"abc"` becomes `0`. Clarify: it becomes `NaN`.

Question 4: "What is the difference between `===` and `=`?"

- Expected answer: `===` compares; `=` assigns.
- Common gap: using `=` in an `if`. Reinforce the difference from Session 2.

Bridge to today (say this):

> *"You can react to a submit and read the values. Today we add two new powers: **checking** those values before we trust them, and **remembering** them after the page is gone. Think of a bank teller: they must check your ID before paying out cash (validation), and the bank keeps a record of your account (storage)."*

If more than a third of the class cannot answer Question 1 or 2, spend an extra 10 minutes redoing the form example from Session 5 before continuing.

Quick refresher (only if needed):

```
<form id="f">
  <input id="email">
  <button type="submit">Send</button>
</form>
<script>
  const f = document.querySelector("#f");
  f.addEventListener("submit", function (event) {
    event.preventDefault();
    console.log(document.querySelector("#email").value);
  });
</script>
```

F. FULL LECTURE NOTES

F.1 Why validation matters

The idea. Validation means checking that the information a user typed is correct and complete *before* we accept or use it.

Plain-English definition. Validation is the form's security guard. Before the form is allowed to "go through", the guard checks: Is the name present? Is the email in a sensible shape? Did the user actually write a message?

Why it exists. Two big reasons:

1. **Garbage in, garbage out.** If a shop's order form accepts an empty name or a fake email, the shop cannot contact the buyer, and the order is wasted. Bad input guarantees bad output.
2. **Protecting the user and the business.** Clear validation stops honest mistakes (a forgotten field or a mistyped email) and discourages nonsense or harmful input. It saves time, money and reputation.

Real-life analogy. Think of a **bank teller**. Before paying out money, the teller checks your ID and your account details. A bank that paid anyone who walked in, with any name, would collapse. A form is the same: it must check before it acts.

Two layers of validation — and why we use both:

- **HTML5 validation** — built-in browser checks (attributes like `required` and `type="email"`). Fast and free, but limited and easy to bypass.
- **JavaScript validation** — our own checks. More control, custom messages, and logic the browser cannot do alone (like comparing two fields).

The professional approach is: use HTML5 for simple checks **and** add JavaScript for anything important. Never rely on the browser alone.

Important limitation / misconception. "If the browser checks it, my data is safe." False. HTML validation can be bypassed (a user can edit the page, or a script can post directly). Real-world apps **must** re-check everything on the server too — that is a later topic. For now, our client-side validation protects the *user experience*; note that server validation is the eventual safety net.

How a learner recognises good validation: the form refuses to proceed with an empty or malformed field and shows a clear, specific message next to the offending field.

F.2 HTML5 built-in validation attributes (brief and practical)

Before writing any JavaScript, the browser can already do some checking through special attributes on the input element.

`required` — the field must not be empty:

```
<input type="text" id="name" required>
```

If the user leaves it empty and submits, the browser blocks the submit and shows a small bubble: *"Please fill out this field."*

`minlength` — the text must be at least this many characters:

```
<input type="text" id="name" minlength="2" required>
```

A single letter is rejected. This stops accidental one-character names.

`type="email"` — the browser expects something shaped like an email (it must contain `@` and a domain):

```
<input type="email" id="email" required>
```

If the user types `hello`, the browser blocks it and says *"Please include an '@' in the email address."*

`pattern` — a custom rule using a **regular expression** (a pattern language):

```
<input type="text" id="phone" pattern="[0-9]{11}" title="Enter an 11-digit phone number">
```

`[0-9]{11}` means "eleven characters, each a digit from 0 to 9". The `title` attribute is the message shown when the pattern fails. This is useful for Nigerian phone numbers (11 digits, e.g. `08012345678`).

What HTML5 validation gives you automatically:

- It blocks the submit when a rule fails.
- It shows a small built-in message.
- It styles the field as invalid (`:invalid` in CSS, if you wish).

Limitations of HTML5 validation:

- The messages are short and cannot be fully customised in a consistent way across browsers.
- It cannot compare two fields (e.g. "confirm password").
- A determined user can bypass it.

Say to the class: "HTML5 attributes are the first guard — quick and free. We use them, but we do not *depend* on them. JavaScript is our second, smarter guard."

Common misconception. Learners think putting `required` on the input is enough to stop an empty name. It stops the *browser's default submit*, but if our JavaScript runs first or the browser's validation is disabled, the value can still be empty. That is why we also check in JavaScript.

F.3 Manual JavaScript validation — checking and reporting

Now we do it ourselves so we control the logic and the messages.

The pattern: check the value, then show or hide an error.

```
<form id="contactForm" novalidate>
  <label for="name">Your name</label>
  <input type="text" id="name">
  <small id="nameError" class="error"></small>

  <label for="email">Your email</label>
  <input type="text" id="email">
  <small id="emailError" class="error"></small>

  <button type="submit">Send</button>
</form>
<p id="success"></p>
```

```
const form = document.querySelector("#contactForm");
const nameInput = document.querySelector("#name");
const emailInput = document.querySelector("#email");
const nameError = document.querySelector("#nameError");
const emailError = document.querySelector("#emailError");
const success = document.querySelector("#success");

form.addEventListener("submit", function (event) {
  event.preventDefault();
  let isValid = true;

  // Clear old messages
  nameError.textContent = "";
  emailError.textContent = "";
  success.textContent = "";

  // 1. Check the name is not empty
  if (nameInput.value.trim() === "") {
    nameError.textContent = "Please enter your name.";
    isValid = false;
  }

  // 2. Check the email looks like an email
  const email = emailInput.value.trim();
  if (!email.includes("@") || !email.includes(".")) {
    emailError.textContent = "Please enter a valid email address.";
    isValid = false;
  }

  // 3. Only proceed when everything is valid
  if (isValid) {
    success.textContent = "Thank you, " + nameInput.value.trim() + "! Your message was sent.";
    form.reset();
  }
});
```

Line by line:

- `novalidate` on the form tells the browser *not* to use its own HTML5 bubbles, so our custom messages are the only ones the user sees. (We keep `required` etc. as a backup, but we control the presentation.)
- The three `querySelector` lines grab the fields and their error elements.
- Inside the submit handler: `event.preventDefault()` stops the reload (Session 5).
- `let isValid = true;` is a **flag** — a variable that tracks whether everything passed. We start optimistic and set it to `false` the moment any check fails.
- We clear old messages first, so a resubmission does not show stale errors.
- Check 1: `nameInput.value.trim() === ""` — `.trim()` removes spaces from the start and end, so a name of only spaces is treated as empty. If empty, we put a message in `nameError` and set `isValid = false`.
- Check 2: we take the trimmed email and test it. The simple check is: does it contain `@` **and** a `.`? The `!` means "if it does *not* contain". If either is missing, we show an error. (This is a simple check, not a perfect one — see the misconception below.)
- Check 3: only if `isValid` is still `true` do we show the success message and reset the form.

Why show the error in a separate `<small>` element? Keeping the message in its own element (styled red) is clean, accessible, and easy to clear. It sits right next to the field it refers to.

The email check — honest about its limits: our check is "contains `@` and `.`". It will accept `a@b.c` (technically valid enough) and reject `hello`. It will *not* catch every odd email, but it stops the obvious mistakes. A more precise check uses a **regular expression** (pattern), which we saw with the `pattern` attribute. For beginners, the simple contains-check is a good, honest starting point.

Real-life analogy. A flag (`isValid`) is like a **checklist at an airport gate**. The gate starts "clear". Each check that fails raises a red flag. Only if no red flags remain is the passenger allowed through.

Common misconception. Learners write separate `if` blocks that *show an error* but forget to set `isValid = false`, so the form proceeds even with errors. Always set the flag.

F.4 Giving clear feedback — the UX of validation

Good validation is not just about blocking bad input; it is about **telling the user kindly what to do**.

Principles:

1. **Be specific.** "Please enter your name" is better than "Invalid."
2. **Be close.** Put the message next to the field it concerns.
3. **Be kind.** The user is not an enemy; help them fix it.
4. **Clear old errors** when the user tries again.
5. **Show success clearly** when everything passes.

Styling the error (optional but recommended):

```
.error {
  color: #c0392b;
  font-size: 0.9rem;
  display: block;
  min-height: 1em;
}
```

`min-height: 1em` reserves a little space so the layout does not jump when a message appears.

Optional live feedback: you can also re-check a field on `input`, clearing the error as soon as the user fixes it:

```
nameInput.addEventListener("input", function () {
  if (nameInput.value.trim() !== "") {
    nameError.textContent = "";
  }
});
```

This gives the user the feeling that the form is listening and forgiving.

Common misconception. Learners hide errors by only *appending* a message each time, so the user sees three copies after three attempts. Always **clear first**, then set.

F.5 What is localStorage?

The idea. localStorage is a small storage area that the **browser** gives to each website, living on the user's computer. It keeps information even after the page is refreshed or closed and reopened.

Plain-English definition. Think of localStorage as the **browser's small filing cabinet, one drawer per website**. A website can put labelled notes in its own drawer, read them later, and remove them. No other website can open that drawer.

Key facts — memorise these:

1. **Per website (per origin).** Each website gets its own separate storage. Data saved on `adaezefashion.com` is not visible to another site.
2. **Persists.** It survives page refresh, closing the tab, and restarting the browser. It is *not* deleted when you close the tab. (Only clearing browser data or our own `removeItem` deletes it.)
3. **String-only.** It can only store **text (strings)**. This is the crucial limitation — numbers, objects and arrays must be converted to text first (Section F.7).
4. **Small.** Usually about 5 MB per website — plenty for a name or a small list, not for photos or large data.
5. **On this computer only.** Data saved on this laptop is not on another laptop, and not on any server.

Why it exists. Websites often want to remember simple things between visits: your chosen theme, your name, a shopping cart, a language setting. A database on a server is overkill for a tiny preference, and it may not exist yet. localStorage is the lightweight, instant option that needs no internet.

Real-life analogy. localStorage is a **sticky note stuck to the fridge in your own kitchen**. Anyone in your house can read it later, but the neighbours cannot see it, and it stays there until someone peels it off.

Common misconception. "localStorage is a database / is shared across users / is secure." All false. It is local to one browser on one machine, unencrypted, and visible to anyone using that computer. **Never store passwords, card numbers, or secrets in localStorage.**

How a learner sees it: open DevTools → **Application** tab → **Local Storage** → click the site. The saved key/value pairs appear and update live as you save.

F.6 The three essential localStorage commands

Saving a value — `setItem(key, value)`:

```
localStorage.setItem("name", "Adaeze");
```

- `"name"` is the **key** (the label on the sticky note).
- `"Adaeze"` is the **value** (what we write on it).
- Both must be strings.

Reading a value — `getItem(key)`:

```
const savedName = localStorage.getItem("name");
console.log(savedName);   // "Adaeze"
```

If the key was never saved, `getItem` returns `null`.

```
const missing = localStorage.getItem("somethingNeverSaved");
console.log(missing);   // null
```

Removing a value — `removeItem(key)`:

```
localStorage.removeItem("name");
```

You can also clear everything for this site with `localStorage.clear();`, but use that carefully — it deletes all your keys.

Worked example — save, read, remove:

```
// 1. Save
localStorage.setItem("greeting", "Good morning");

// 2. Read (on the same page or after a refresh)
console.log(localStorage.getItem("greeting"));   // "Good morning"

// 3. Remove
localStorage.removeItem("greeting");
console.log(localStorage.getItem("greeting"));   // null
```

Say to the class: "`setItem` writes the note; `getItem` reads the note; `removeItem` peels it off. Three commands — that is the whole filing cabinet."

Common misconception. Learners expect `getItem` to give the value back as it was saved. It always gives a string or `null`. If they saved a number `25`, `getItem` returns the *string* `"25"`, and `"25" + 5` gives `"255"`. Convert back with `Number()` if you need maths.

F.7 The string limitation — `JSON.stringify()` and `JSON.parse()`

localStorage can only hold strings. But real data is often a **number**, an **object**, or an **array**. To store those, we convert them to a string on the way in, and convert them back on the way out.

What is JSON? JSON (JavaScript Object Notation) is a text format for representing objects and arrays as a string. It looks almost exactly like JavaScript object syntax, but it is a *string*. Example: the object

```
const user = { name: "Chinedu", age: 22 };
```

becomes the JSON string:

```
'{"name":"Chinedu","age":22}'
```

Converting to a string — `JSON.stringify()`:

```
const user = { name: "Chinedu", age: 22 };
localStorage.setItem("user", JSON.stringify(user));
// stored as the string: {"name":"Chinedu","age":22}
```

Converting back to an object — `JSON.parse()`:

```
const raw = localStorage.getItem("user");   // a string (or null)
const userObject = JSON.parse(raw);          // back to an object
console.log(userObject.name);                // "Chinedu"
```

An array example — a saved list:

```
const interests = ["tailoring", "marketing", "design"];
localStorage.setItem("interests", JSON.stringify(interests));

// later…
const savedInterests = JSON.parse(localStorage.getItem("interests"));
console.log(savedInterests[0]);   // "tailoring"
```

Remember it this way:

- `JSON.stringify` = "make this object into a string to save".
- `JSON.parse` = "turn this saved string back into an object".
- `stringify` → **String** → to be **gift**-wrapped for storage.
- `parse` → to **parse** (unpack) it back.

The null trap — a very common beginner bug. If you save nothing, `getItem` returns `null`, and `JSON.parse(null)` does *not* throw but returns `null`, while `JSON.parse("")` throws a red error. Always check before parsing:

```
const raw = localStorage.getItem("interests");
const interests = raw ? JSON.parse(raw) : [];
```

The `raw ? ... : ...` is a short form of "if raw is not null/empty, parse it; otherwise use an empty array." This prevents crashes on a first visit.

Common misconceptions:

- "localStorage can store an object directly." No — `setItem("user", user)` would save the *string* `"[object Object]"`, which is useless. Always `JSON.stringify`.
- "`JSON.parse` never fails." It throws on invalid text, so guard with the null check.
- "I can store numbers and they stay numbers." They come back as strings; convert if needed.

F.8 The running example — "Welcome back" and a saved list

Part A — Welcome back. We save the user's name and greet them on their next visit.

HTML (`index.html`):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Welcome</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 460px; margin: 40px auto; }
    h1 { color: #8a2b6b; }
    input, button { padding: 8px; margin-top: 8px; }
    #greeting { font-size: 20px; font-weight: bold; margin-top: 16px; }
    #list { margin-top: 10px; }
  </style>
</head>
<body>
  <h1>Adaeze Fashion</h1>

  <div id="greeting">Welcome!</div>

  <label for="nameInput">What is your name?</label>
  <input type="text" id="nameInput" placeholder="e.g. Chinedu">
  <button id="saveName">Save my name</button>
  <button id="forgetName">Forget me</button>

  <hr>

  <h2>My Interests</h2>
  <input type="text" id="interestInput" placeholder="e.g. tailoring">
  <button id="addInterest">Add interest</button>
  <ul id="list"></ul>

  <script src="js/validate.js"></script>
</body>
</html>
```

JavaScript (`js/validate.js`):

```
// ---------- Part A: Welcome back ----------
const greeting = document.querySelector("#greeting");
const nameInput = document.querySelector("#nameInput");
const saveName = document.querySelector("#saveName");
const forgetName = document.querySelector("#forgetName");

function showGreeting() {
  const saved = localStorage.getItem("userName");
  greeting.textContent = saved ? "Welcome back, " + saved + "!" : "Welcome!";
}
```

```
saveName.addEventListener("click", function () {
  const name = nameInput.value.trim();
  if (name === "") {
    greeting.textContent = "Please type a name first.";
    return;
  }
  localStorage.setItem("userName", name);
  showGreeting();
  nameInput.value = "";
});

forgetName.addEventListener("click", function () {
  localStorage.removeItem("userName");
  showGreeting();
});

// Run once when the page loads
showGreeting();

// ---------- Part B: Interests list ----------
const interestInput = document.querySelector("#interestInput");
const addInterest = document.querySelector("#addInterest");
const list = document.querySelector("#list");

function loadInterests() {
  const raw = localStorage.getItem("interests");
  const interests = raw ? JSON.parse(raw) : [];

  list.textContent = "";                        // clear the <ul> first
  interests.forEach(function (item) {           // rebuild one <li> per item
    const li = document.createElement("li");
    li.textContent = item;
    list.appendChild(li);
  });
}

addInterest.addEventListener("click", function () {
  const item = interestInput.value.trim();
  if (item === "") {
    return;
  }
  const raw = localStorage.getItem("interests");
  const interests = raw ? JSON.parse(raw) : [];   // read existing list
  interests.push(item);                           // add the new one
  localStorage.setItem("interests", JSON.stringify(interests));  // save it back
  interestInput.value = "";                       // clear the box
  loadInterests();                                // redraw the list
});

loadInterests();   // draw any saved list on load
```

How it behaves — step by step:

1. First visit: the greeting says `Welcome!`. Type `Chinedu`, click **Save my name**. The greeting becomes `Welcome back, Chinedu!`, and the name is stored.
2. **Refresh the page.** The greeting still says `Welcome back, Chinedu!` because `showGreeting()` runs on load and reads from localStorage.
3. On another computer, the greeting would say `Welcome!` — the data is local to this machine.
4. Add interests (`tailoring`, then `marketing`). Each is added to the array, saved as JSON, and the `<ul>` is redrawn. **Refresh** — the list is still there.
5. Click **Forget me** to remove the name, then refresh — the greeting resets to `Welcome!`.

Line-by-line highlights:

- `showGreeting()`: reads `userName`; the `saved ? ... : ...` shows the personal or generic greeting depending on whether a value exists.
- `saveName` handler: trims the name, refuses an empty one, saves with `setItem`, updates the greeting.
- `forgetName` handler: `removeItem` deletes the stored name, then re-renders.

- `showGreeting();` at the bottom runs once on load so a returning user is greeted immediately.
- `loadInterests()` : reads `interests` , parses JSON safely with the null check, clears the `<ul>` , and rebuilds it with a `<li>` per item — reusing the DOM skills from Session 4.
- `addInterest` handler: reads the existing array, pushes the new item, saves the whole array back as JSON, then redraws.
- `loadInterests();` at the bottom draws the saved list on load.

Expected on-page behaviour after refresh: the name and interests are still displayed, proving persistence. DevTools → **Application** → **Local Storage** shows `userName` and `interests` (the latter as a JSON string).

F.9 Summary of the session

- **Validation** stops bad input before it causes bad outcomes ("garbage in, garbage out").
- **HTML5 attributes** (`required` , `minlength` , `type="email"` , `pattern`) give quick, free checks.
- **JavaScript validation** checks `.value` (with `.trim()`), shows/hides an error element, uses an `isValid` flag, and only proceeds when everything passes.
- **localStorage** is the browser's small, per-site, persistent, string-only store.
- `setItem` , `getItem` , `removeItem` save, read and delete; `getItem` returns a string or `null` .
- `JSON.stringify` to save objects/arrays; `JSON.parse` to read them back, guarded with a null check.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- `demo-week2-s6/index.html` and `js/validate.js` (the contact form and the welcome/list example combined).
- Both files open in VS Code.
- Chrome open to the page, with DevTools open. Keep **Console** and **Application** → **Local Storage** visible on the projector (Application may be in a second DevTools dock).
- Whiteboard with the validation flow: **submit → preventDefault → clear errors → check each field → set isValid → if valid, success**.

G.2 Demonstration 1 — the contact form validator (8 minutes)

Say: "Watch how the form refuses to send until the customer gets it right."

Do on the projector:

1. Show the contact form HTML with the two `<small>` error elements and `novalidate` on the form.
2. Type in the JavaScript from Section F.3.
3. Save, refresh, press `F12` .
4. Click **Send** with everything empty. **Expected:** two red messages appear — "Please enter your name." and "Please enter a valid email address." The page does **not** reload, and no success message shows.
5. Type a name but a bad email (`hello`). Send again. **Expected:** only the email error shows, because the name now passes.
6. Type a valid email (`chinedu@example.com`). Send. **Expected:** the success paragraph appears and the boxes clear.

Pause and predict before step 5: "If I type a name but leave the email as `hello` , will we see one error or two?" (One — only the email.)

Inject a classic error: change `nameInput.value.trim() === ""` to `nameInput.value == ""` (removing `.trim()`), save, refresh.

- Type a name of just a few spaces and send. **Expected:** with `.trim()` , spaces count as empty and the error shows; without it, spaces are treated as a valid name and it slips through. Show the bug, then put `.trim()` back.
- **Say:** "Always trim. A name made of spaces is not a name."

Reasoning to explain aloud: the `isValid` flag starts true and is flipped to false by any failed check; we clear old errors first; only a clean pass shows success.

G.3 Demonstration 2 — localStorage persistence (7 minutes)

Say: "Now watch the browser remember."

Do on the projector:

1. In the Console, type `localStorage.setItem("userName", "Adaeze");` and press Enter.
2. Open DevTools → **Application** → **Local Storage** and point at the new `userName` entry.
3. Refresh the page. In the Console, type `localStorage.getItem("userName");` → `"Adaeze"`. **Say:** "It survived the refresh."
4. Now demonstrate the object problem: save an object the wrong way:

```
const user = { name: "Chinedu", age: 22 };
localStorage.setItem("user", user);       // WRONG
localStorage.getItem("user");             // "[object Object]"
```

- **Say:** "See `[object Object]`? That is useless. We must convert first." Fix it:

```
localStorage.setItem("user", JSON.stringify(user));   // RIGHT
JSON.parse(localStorage.getItem("user")).name;        // "Chinedu"
```

5. Show the null trap:

```
JSON.parse(localStorage.getItem("nothingHere"));   // null, no crash
JSON.parse("");                                     // red error
```

- **Say:** "Guard your parse with a null check, as in our example."

6. Finally, click **Forget me** on the page, refresh, and show the greeting has reset — proving `removeItem` works.

Questions to ask during the demo:

- "Will the saved name appear on another learner's computer?" (No — localStorage is per browser, per machine.)
- "What type does `getItem` return for a saved number?" (A string.)
- "Why do we `JSON.stringify` before saving an array?" (Because localStorage only stores strings.)

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding step by step, build a **"Signup Validator with Saved Draft"** — a small form that validates a name and an email, shows errors, greets the user on success, and remembers the last name typed even if the page is refreshed before submitting.

Starting materials: A new folder `webdev-practice/s6/` with an empty `index.html` and an empty `js/signup.js`.

Procedure (instructor calls out each numbered step; learners follow; instructor walks the room):

1. Create `index.html` with the form and error elements:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Signup</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 460px; margin: 40px auto; }
    label { display: block; margin-top: 12px; }
    input { padding: 8px; width: 100%; box-sizing: border-box; }
    button { margin-top: 14px; padding: 10px 18px; }
    .error { color: #c0392b; display: block; min-height: 1em; }
    #success { color: #1e7e34; font-weight: bold; margin-top: 12px; }
  </style>
</head>
<body>
  <h1>Create your account</h1>

  <form id="signupForm" novalidate>
    <label for="name">Full name</label>
    <input type="text" id="name" required>
```

```
    <small id="nameError" class="error"></small>

    <label for="email">Email address</label>
    <input type="text" id="email" required>
    <small id="emailError" class="error"></small>

    <button type="submit">Sign up</button>
  </form>
  <p id="success"></p>

  <script src="js/signup.js"></script>
</body>
</html>
```

2. Create `js/signup.js` and select the elements:

```
const form = document.querySelector("#signupForm");
const nameInput = document.querySelector("#name");
const emailInput = document.querySelector("#email");
const nameError = document.querySelector("#nameError");
const emailError = document.querySelector("#emailError");
const success = document.querySelector("#success");
```

3. Add the validation on submit:

```
form.addEventListener("submit", function (event) {
  event.preventDefault();
  let isValid = true;

  nameError.textContent = "";
  emailError.textContent = "";
  success.textContent = "";

  if (nameInput.value.trim().length < 2) {
    nameError.textContent = "Please enter your full name (at least 2 letters).";
    isValid = false;
  }

  const email = emailInput.value.trim();
  if (!email.includes("@") || !email.includes(".")) {
    emailError.textContent = "Please enter a valid email address.";
    isValid = false;
  }

  if (isValid) {
    const name = nameInput.value.trim();
    success.textContent = "Welcome, " + name + "! Your account is ready.";
    localStorage.setItem("lastSignup", name);
    form.reset();
  }
});
```

4. **Pause and predict:** "If I leave both fields empty, how many errors appear?" (Two.) Then "If I fix the name but the email is still `abc`, how many?" (One.)

5. Add the **saved draft** feature — remember the name as the user types, and restore it on load:

```
// Save the draft name while the user types
nameInput.addEventListener("input", function () {
  localStorage.setItem("draftName", nameInput.value);
});

// On page load, restore any saved draft name
const draft = localStorage.getItem("draftName");
if (draft) {
  nameInput.value = draft;
}
```

6. Save, refresh, and test:

- Type a name but do NOT submit. Refresh the page. The name is still in the box (the draft was restored).
- Submit with everything valid. The success message shows and the form clears.
- Check DevTools → **Application** → **Local Storage** to see `lastSignup` and `draftName`.

Expected result:

- Empty submit → two red errors, no success, no reload.
- Bad email only → one red error.
- Valid inputs → green success message "Welcome, [name]! Your account is ready."
- Refreshing after typing (but before submitting) keeps the typed name in the box.
- localStorage panel shows `draftName` and (after a valid submit) `lastSignup`.

Points to pause on and predict:

- After step 3: "What if I type an email of `ada@`?" (No `.` → error. `ada@x.com` → passes.)
- After step 5: "Does the draft save only on submit, or while typing?" (While typing, because we listen for `input`.)

Troubleshooting while walking the room:

- Page reloads on submit → missing `event.preventDefault()`, or listener on the button.
- Success message shows even with errors → forgot `isValid = false` in a failed check.
- Error never clears → forgot the clear-lines at the top of the handler.
- Draft not restored → script in the head (runs before the input exists) or the key name differs between save and read.
- `[object Object]` in storage → tried to save something that is not a string.

Completion checklist (learner ticks each):

- [ ] `submit` listener on the form with `preventDefault()`
- [ ] Empty name is rejected with a clear message
- [ ] Invalid email is rejected with a clear message
- [ ] Success only shows when both are valid
- [ ] A name is saved to localStorage and restored after refresh
- [ ] `.trim()` used when checking input
- [ ] No red errors in the Console

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Build a **"Nigerian Phone Number Checker"** that validates an 11-digit phone number and remembers the last valid number entered.

Scenario: *Chidi's Provisions* wants to collect customers' phone numbers for delivery. A valid Nigerian phone number is **exactly 11 digits** and **starts with** `0` (for example `08012345678`). The form must reject anything else and show a clear message. When a valid number is entered, it should be saved to localStorage and shown on the next visit.

Starting information (given):

- A form with one text input (`id="phone"`), an error element (`id="phoneError"`), and a result paragraph (`id="phoneResult"`).
- A submit button.

Requirements:

1. Create `phone.html` and `js/phone.js`, linked at the bottom of the body.
2. On submit: call `event.preventDefault()`, clear old messages.
3. Read the phone number and `.trim()` it.
4. Check three things:
   - it is not empty,
   - it is **exactly 11 digits** (hint: use `.length`),
   - it **starts with** `0` (hint: use `.startsWith("0")` or check the first character).
5. If any check fails, show a specific message in the error element and do not proceed.

6. If all checks pass, show `"Saved: [number]"` and store the number under the key `"lastPhone"`.
7. On page load, if `"lastPhone"` exists, display `"Last number: [number]"` in the result paragraph.

Constraints:

- Use the `isValid` flag pattern.
- `.trim()` the input.
- Do not store anything unless the number is valid.
- Do not copy the signup example line-for-line.

Time allowance: 15 minutes.

Success criteria:

- `08012345678` → accepted and saved; `Saved: 08012345678`.
- `8012345678` (10 digits, no leading zero) → rejected (wrong length and missing zero).
- `08012345abc` → rejected (not all digits — treat letters as invalid).
- `080123456` (9 digits) → rejected (wrong length).
- After a valid save, refreshing the page shows `Last number: 08012345678`.
- No red errors.

Hints:

- Hint 1: check length with `phone.length === 11`.
- Hint 2: check the first character with `phone.charAt(0) === "0"` or `phone.startsWith("0")`.
- Hint 3: to ensure every character is a digit, a simple approach is `Number(phone)` — if it is `NaN`, there were letters. (Use this after the length and first-character checks.)
- Hint 4: save with `localStorage.setItem("lastPhone", phone);` and read with `localStorage.getItem("lastPhone");`.

Instructor answer key (do not show learners):

```
// phone.js — Nigerian Phone Number Checker
const form = document.querySelector("#phoneForm");
const phoneInput = document.querySelector("#phone");
const phoneError = document.querySelector("#phoneError");
const phoneResult = document.querySelector("#phoneResult");

form.addEventListener("submit", function (event) {
  event.preventDefault();
  phoneError.textContent = "";
  phoneResult.textContent = "";

  const phone = phoneInput.value.trim();
  let isValid = true;

  if (phone === "") {
    phoneError.textContent = "Please enter your phone number.";
    isValid = false;
  } else if (phone.length !== 11) {
    phoneError.textContent = "A phone number must be exactly 11 digits.";
    isValid = false;
  } else if (phone.charAt(0) !== "0") {
    phoneError.textContent = "A Nigerian phone number must start with 0.";
    isValid = false;
  } else if (isNaN(Number(phone))) {
    phoneError.textContent = "A phone number must contain digits only.";
    isValid = false;
  }

  if (isValid) {
    phoneResult.textContent = "Saved: " + phone;
    localStorage.setItem("lastPhone", phone);
    form.reset();
  }
});

// Show the last saved number on load
const lastPhone = localStorage.getItem("lastPhone");
if (lastPhone) {
```

```
  phoneResult.textContent = "Last number: " + lastPhone;
}
```

Marking guidance (out of 10):

- `submit` listener with `preventDefault()` (2)
- `.trim()` and empty check (2)
- Length check (2)
- Leading-zero check (1)
- Digit-only check (1)
- Save only when valid + display on load (2)

J. REAL-WORLD CASE STUDY

Situation. *Adaeze Fashion* started taking orders through a contact form on her website. Last month, three orders went wrong: one had an **empty name**, so Adaeze could not address the customer; one had the email `myemail` (no `@`), so the confirmation bounced; one customer typed their delivery address into the "name" box. Adaeze spent hours chasing these customers instead of sewing.

Relevant facts.

- The form has fields for name, email and message.
- Adaeze receives the orders by email and by reading the saved data.
- Most customers are honest but make simple typing mistakes.
- The site is built with HTML, CSS and JavaScript; there is no server yet.

Decision point. How should the form be improved so bad orders stop reaching Adaeze?

Possible approaches.

1. Leave the form as it is and rely on careful customers.
2. Add HTML5 attributes **and** JavaScript validation with clear error messages, plus save the last entered details so returning customers do not retype everything.
3. Rebuild the whole site with a server, accounts and a real order database.

Recommended solution (for this stage). Approach 2. It stops the three real problems with tools learned today:

- `required` and a name `minlength` catch the empty or one-letter name.
- A JavaScript email check (`@` and `.`) catches `myemail`.
- Clear inline messages guide customers to the right field.
- Saving the customer's name and email in localStorage means a returning customer sees their details pre-filled.

Example solution (condensed):

```
const name = document.querySelector("#name");
const email = document.querySelector("#email");
const nameError = document.querySelector("#nameError");
const emailError = document.querySelector("#emailError");
const form = document.querySelector("#orderForm");

// Pre-fill from last visit
const savedName = localStorage.getItem("customerName");
if (savedName) name.value = savedName;

form.addEventListener("submit", function (event) {
  event.preventDefault();
  nameError.textContent = "";
  emailError.textContent = "";
  let ok = true;

  if (name.value.trim().length < 2) {
    nameError.textContent = "Please enter your full name.";
    ok = false;
  }
```

```
  const e = email.value.trim();
  if (!e.includes("@") || !e.includes(".")) {
    emailError.textContent = "Please enter a valid email.";
    ok = false;
  }

  if (ok) {
    localStorage.setItem("customerName", name.value.trim());
    // ... proceed with the order
  }
});
```

Trade-offs.

- Approach 1 is free but guarantees repeated mistakes — exactly what already cost Adaeze hours.
- Approach 2 takes under an hour and removes the common errors, plus improves returning-customer experience. It is the right step for now.
- Approach 3 is the long-term answer (proper orders need a server that *also* validates), but it is far beyond Week 2 and expensive at this stage.

Professional connection. Every serious signup, checkout or contact form validates its inputs and remembers useful details. A junior developer is constantly asked to "add validation so we stop getting junk orders" and "pre-fill the form for returning users." Today's skills are exactly those tasks. Note the important professional caveat: client-side validation improves the experience, but sensitive data still needs server-side validation and must **never** be trusted from the browser alone.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this as the live troubleshooting guide. When a learner is stuck, ask them to check the relevant row rather than giving the answer.

K.1 The form submits even though fields are invalid

Symptoms: Clicking Send proceeds (or reloads) despite empty or malformed fields. **Likely causes:** Missing `event.preventDefault()`; listener attached to the button not the form; a failed check does not set `isValid = false`. **Investigate:** Add `console.log(isValid)` just before the final `if (isValid)`. Also confirm the listener is on the form. **Fix:** Add `preventDefault()`; move the listener to the form; set `isValid = false` in every failing branch. **Confirm:** Invalid submissions no longer proceed. **Prevent:** Use one `isValid` flag and check it once at the end.

K.2 A name that is only spaces is accepted

Symptoms: Typing spaces (or leading/trailing spaces) passes validation. **Likely cause:** `.trim()` was not used before checking. **Investigate:** `console.log(JSON.stringify(nameInput.value))` — the quotes reveal the hidden spaces. **Fix:** Use `.trim()` before every emptiness/length check and when saving. **Prevent:** Always trim text inputs (names, emails, messages).

K.3 `[object Object]` appears in localStorage

Symptoms: The saved value in the Application panel reads `[object Object]`. **Likely cause:** Saving an object/array without `JSON.stringify`. **Investigate:** Look at the value shown in DevTools → Application → Local Storage. **Fix:** Wrap the object/array: `localStorage.setItem("user", JSON.stringify(user));`. **Prevent:** Remember: localStorage stores strings only; convert first.

K.4 Red error `Unexpected token ... in JSON at position 0`

Symptoms: A crash when reading saved data. **Likely cause:** `JSON.parse` received invalid text (often an empty string, or a value saved *without* `stringify` earlier). **Investigate:** `console.log(localStorage.getItem("yourKey"))` to see what is actually stored, then clear the bad key. **Fix:** Guard the parse: `const raw = localStorage.getItem("key"); const data = raw ? JSON.parse(raw) : [];`. If old bad data exists, `localStorage.removeItem("key")`. **Prevent:** Always pair `stringify` (save) with `parse` (read), and null-check before parsing.

K.5 A saved number comes back as text

Symptoms: `"25" + 5` gives `"255"` after reading from localStorage. **Likely cause:** localStorage returns strings only. **Investigate:** `console.log(typeof localStorage.getItem("myNumber"))` → `"string"`. **Fix:** Convert on read: `const n = Number(localStorage.getItem("myNumber"));`. **Prevent:** Convert every number you read back from storage.

K.6 Nothing is remembered after refresh

Symptoms: The greeting or list resets on every refresh. **Likely causes:** Data was saved under a different key than it is read; `getItem` was read before `setItem` ran; or the script runs before the elements exist. **Investigate:** Check the key spelling in both places; check the Application panel for the saved key; check the Console for `null`. **Fix:** Use exactly the same key string; ensure the script is at the bottom of the body. **Prevent:** Define your key names once and reuse them; keep scripts at the bottom.

K.7 Error messages pile up instead of replacing

Symptoms: Three error messages appear after three attempts. **Likely cause:** The handler appends without clearing first. **Fix:** Set each error element's `textContent = ""` at the top of the handler before checking. **Prevent:** Clear, then validate, then display.

Distinguish likely vs confirmed: In every case, the *likely* cause becomes *confirmed* only after you check it (look in the Application panel, run `JSON.stringify` on the value, or `typeof` the stored value). Teach learners to inspect before fixing.

L. SESSION ASSIGNMENT

Assignment title: "My Validated Contact Form with Memory"

Purpose: Combine validation and localStorage in one realistic page — the Week 2 mini-project scaffolding.

Realistic scenario: A local business (choose one: a tailor, a bakery, a phone repair shop) needs a contact form on its website. The form must collect a **name**, an **email** and a **message**, reject bad input with clear messages, and remember the customer's name and email so they are pre-filled on the next visit.

Detailed instructions:

1. Create a folder `assignment6` with `index.html` and `js/contact.js`.
2. In the HTML, build a `<form id="contactForm" novalidate>` with:
   - a text input `id="name"`,
   - a text input `id="email"`,
   - a `<textarea id="message">`,
   - a `<small class="error">` element next to each field with ids `nameError`, `emailError`, `messageError`,
   - a submit button,
   - a success paragraph `id="success"`.
3. In `contact.js`:
   - On submit: `preventDefault()`, clear all errors and the success text.
   - Validate: name has at least 2 non-space characters; email contains `@` and `.`; message has at least 5 non-space characters.
   - Use one `isValid` flag.
   - If valid: show a friendly success message, save the name and email to localStorage (`customerName`, `customerEmail`), and reset the form.
4. On page load: if `customerName` exists, pre-fill the name field; if `customerEmail` exists, pre-fill the email field.
5. Add `required` and a suitable `minlength` to the HTML inputs as a second layer.

Required deliverables:

- `assignment6/index.html`
- `assignment6/js/contact.js`
- A screenshot of the form showing (a) errors with bad input and (b) the pre-filled fields after a refresh.

Submission format: Files in the class folder (or zipped); screenshots in the workbook.

Deadline: Before Session 7.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
Structure & linking	3	All fields, error elements, success paragraph; external script at the bottom of the body
`submit` + `preventDefault()`	3	Listener on the form; page never reloads
Name & message validation	4	`.trim()` used; correct minimum lengths; clear messages
Email validation	3	Rejects `myemail`; accepts `a@b.com`
`isValid` flag & success	3	Only proceeds when all checks pass; friendly success message
localStorage save & pre-fill	4	Name and email saved; pre-filled after refresh; no `[object Object]`

Common mistakes to avoid: forgetting `preventDefault()`; forgetting `.trim()`; not setting `isValid = false`; saving a raw object; using a different key name for save and read.

Instructor reference solution:

```
// contact.js — Validated Contact Form with Memory
const form = document.querySelector("#contactForm");
const nameInput = document.querySelector("#name");
const emailInput = document.querySelector("#email");
const messageInput = document.querySelector("#message");
const nameError = document.querySelector("#nameError");
const emailError = document.querySelector("#emailError");
const messageError = document.querySelector("#messageError");
const success = document.querySelector("#success");

// Pre-fill from last visit
const savedName = localStorage.getItem("customerName");
const savedEmail = localStorage.getItem("customerEmail");
if (savedName) nameInput.value = savedName;
if (savedEmail) emailInput.value = savedEmail;

form.addEventListener("submit", function (event) {
  event.preventDefault();

  nameError.textContent = "";
  emailError.textContent = "";
  messageError.textContent = "";
  success.textContent = "";

  let isValid = true;

  const name = nameInput.value.trim();
  const email = emailInput.value.trim();
  const message = messageInput.value.trim();

  if (name.length < 2) {
    nameError.textContent = "Please enter your name (at least 2 letters).";
    isValid = false;
  }
  if (!email.includes("@") || !email.includes(".")) {
    emailError.textContent = "Please enter a valid email address.";
    isValid = false;
  }
  if (message.length < 5) {
    messageError.textContent = "Please write a message of at least 5 characters.";
    isValid = false;
  }

  if (isValid) {
    success.textContent = "Thank you, " + name + "! We will reply to " + email + " soon.";
    localStorage.setItem("customerName", name);
    localStorage.setItem("customerEmail", email);
    form.reset();
    // Re-fill from storage immediately after reset so the user sees their details
    nameInput.value = name;
    emailInput.value = email;
```

```
  }
});
```

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. In one sentence, why do we validate a form before accepting its data?

Q2. Multiple choice. Which attribute makes a browser refuse to submit an empty field? a) `empty` b) `required` c) `mandatory` d) `validate`

Q3. Explain in your own words. Why must we use `JSON.stringify()` before saving an object to localStorage, and `JSON.parse()` after reading it?

Q4. Practical identification. What does each print? a) `localStorage.getItem("unknownKey")` b) `JSON.parse(localStorage.getItem("unknownKey"))` (assume no null guard) c) `typeof localStorage.getItem("savedNumber")` after saving `25`

Q5. Error detection. Find and fix the bugs:

```
const name = document.querySelector("#name");
form.addEventListener("submit", function () {
  if (name.value == "") {
    document.querySelector("#nameError").textContent = "Name required";
  }
  localStorage.setItem("user", { name: name.value });
});
```

Q6. Scenario. A page greets returning users. Describe the two things the page must do: one when the user submits their name, and one when the page loads — and name the localStorage methods involved.

INSTRUCTOR ANSWER KEY (Q1–Q6)

A1. Because bad input produces bad results ("garbage in, garbage out") and validation protects both the user (clear feedback, fewer mistakes) and the business (usable contact details and orders).

A2. (b) `required`.

A3. Because localStorage can only store strings; an object stored directly becomes the useless string `"[object Object]"`. `JSON.stringify` converts the object into a text form that can be saved, and `JSON.parse` converts that text back into a real object when reading.

A4. a) `null`; b) `null` (parsing the string `"null"` returns `null`; note that parsing the empty string `""` would throw an error); c) `"string"`.

A5. Bugs: (1) no `event.preventDefault()`, so the page reloads and loses everything; (2) the `event` parameter is missing, so it cannot be prevented; (3) no `isValid` flag / no `trim()`, so a name of spaces passes, and the form does not truly block; (4) the object is saved without `JSON.stringify`, so it becomes `[object Object]`. Corrected:

```
const name = document.querySelector("#name");
form.addEventListener("submit", function (event) {
  event.preventDefault();
  const nameError = document.querySelector("#nameError");
  nameError.textContent = "";
  const value = name.value.trim();
  if (value === "") {
    nameError.textContent = "Name required";
    return;
  }
  localStorage.setItem("user", JSON.stringify({ name: value }));
});
```

A6. On submit: read the trimmed name, save it with `localStorage.setItem("userName", name)`, and show the greeting. On page load: call `localStorage.getItem("userName")`; if it is not null, show `"Welcome back, [name]"`; otherwise show a generic greeting. (To remove, use `removeItem`.)

N. REFLECTION AND RECAP

Plain-English summary. Today your forms learned to think before they act. **Validation** stops bad input — the empty names, the broken emails — before they cause problems. The browser gives us quick checks through HTML5 attributes (`required`, `minlength`, `type="email"`, `pattern`), and JavaScript gives us full control: read the value, `.trim()` it, test it, show or hide an error element, and use an `isValid` flag to decide whether to proceed. Then we met **localStorage**, the browser's small, per-website, persistent, **string-only** filing cabinet. `setItem`, `getItem` and `removeItem` save, read and delete; because it stores only strings, we use `JSON.stringify` to save objects and arrays, and `JSON.parse` (guarded against `null`) to read them back. Our page now greets returning users and remembers their list.

Five important takeaways:

1. Validate before you trust: garbage in, garbage out.
2. HTML5 attributes are the first guard; JavaScript validation is the smarter second guard with clear messages.
3. Use `.trim()` and an `isValid` flag; clear old errors, then only proceed when everything passes.
4. localStorage is per-site, persistent, and **string-only**.
5. `JSON.stringify` to save objects/arrays; `JSON.parse` (with a null check) to read them back. Never store passwords or secrets.

Questions to help you recognise what you can now do:

- Can I stop a form from submitting until the fields are valid?
- Can I explain the difference between `required` and a JavaScript check?
- Can I save a name in localStorage and show it again after a refresh?
- Can I explain why `JSON.stringify` and `JSON.parse` are needed?

Connection to the next session: Our pages can now react, validate and remember — but all the data is typed by hand or hard-coded. Next session we learn to **fetch data from a file or the internet** and display it automatically — the `fetch` command and **JSON** working together. That is how a shop's product list can live in one file and appear on the page without anyone typing each item. That begins Week 3.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner uses one `isValid` flag and sets it to `false` on every failed check without prompting.
- Learner uses `.trim()` on text inputs and can explain why a name of spaces should fail.
- Learner gives specific, kind error messages ("Please enter a valid email address") rather than "Invalid".
- Learner saves with `JSON.stringify` and reads with `JSON.parse`, and guards against `null`.
- Learner checks the Application panel to confirm the data was saved.

Common misconceptions to watch for and correct:

- "HTML `required` is enough." → It can be bypassed; JavaScript is our second guard, and real apps also validate on the server.
- "localStorage can store objects directly." → Show `[object Object]` and explain `stringify`.
- "localStorage is shared / a database / secure." → It is local, per-browser, unencrypted; never store passwords.
- "`getItem` returns the number I saved." → It returns a string; convert with `Number()` if needed.

Suggested remedial exercises for learners who struggle:

1. **Trim drill:** give five values (with leading/trailing spaces, all spaces, empty) and ask which should pass a "not empty" check.
2. **Stringify/parse sorting:** give ten lines and ask learners to mark each as "save" (`stringify`) or "read" (`parse`).
3. **Fix-the-validator drill:** hand out a validator missing `preventDefault`, missing `isValid = false`, and saving a raw object; ask learners to find and fix all three.

Differentiation:

- **Level 1 (supported):** provide `contact-form-partial/` with the HTML and error elements done, and the JavaScript shell with selectors and clear comment markers where each check goes.
- **Level 2 (standard):** the guided and independent exercises as written.
- **Level 3 (challenge):** add a **password** and **confirm password** field; require the password to be at least 6 characters and the two to match (using `===`); save only the username (never the password) and pre-fill the username on return.

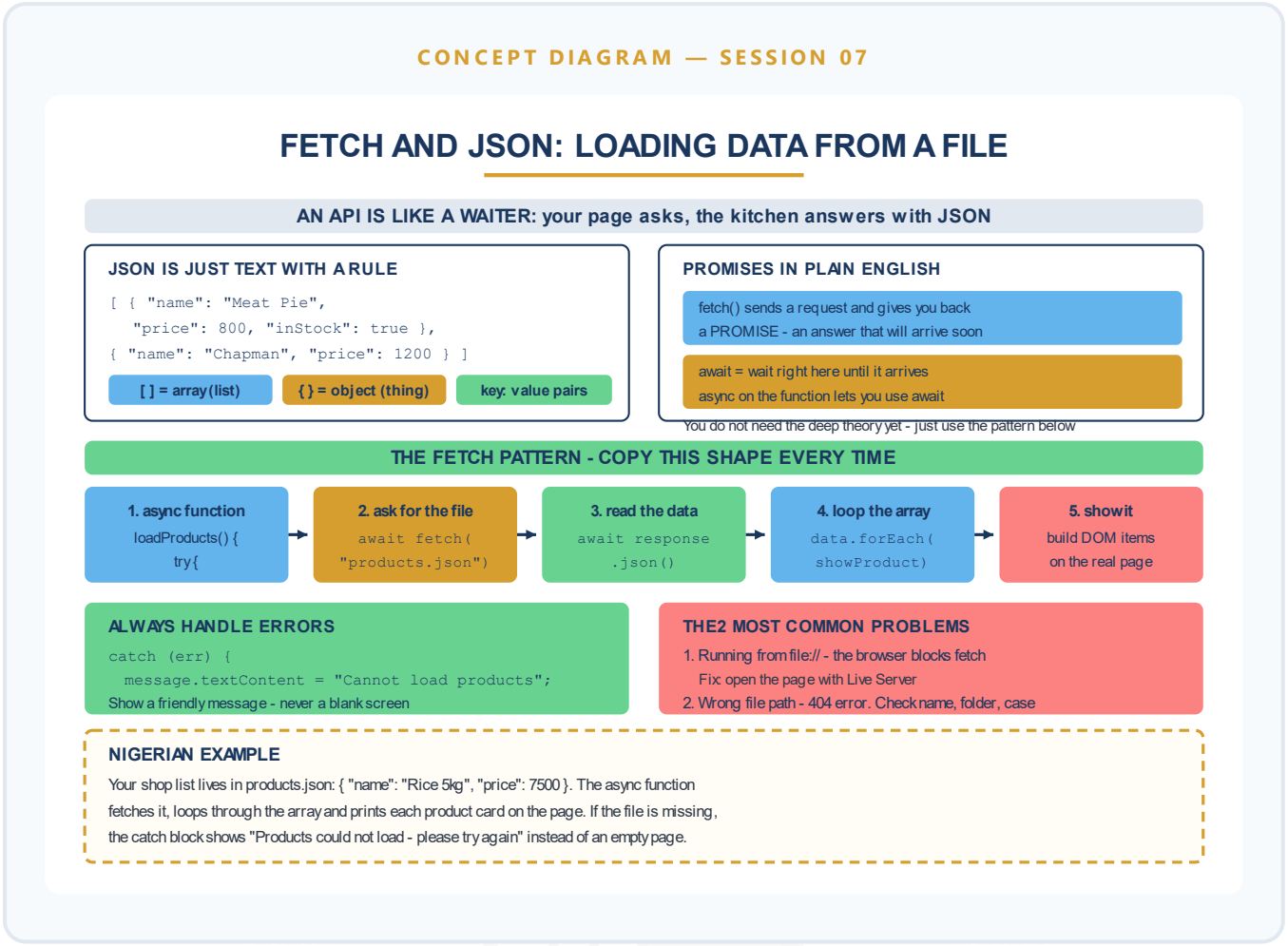
Timing guidance: Validation (Sections F.1–F.4) and localStorage (F.5–F.7) are both essential; do not let one consume the other. If the room is slow, run the localStorage demo but reduce the guided exercise to the validator only (drop the draft feature) and keep the independent exercise. If very fast, extend with the Level 3 challenge and have learners inspect each other's storage in the Application panel.

END OF SESSION 6

Next: Session 7 — Fetching and Displaying Data (fetch and JSON).

SESSION 07 OF 12

FETCHING DATA AND JSON



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 7 — FETCHING DATA AND JSON

A. SESSION IDENTIFICATION

Item	Detail
Session number	7 of 12
Session title	Fetching Data and JSON
Week / Module	Week 3 — Module 3: Data & Real Apps
Recommended duration	105 minutes
Difficulty level	Beginner–Intermediate (introduces asynchronous code)

Item	Detail
Prerequisite knowledge	Sessions 1–6: variables, functions, `if/else`, arrays, objects, loops, DOM selection/creation, events and `addEventListener`, reading input values. Learner must be comfortable building a list with `document.createElement` and `appendChild`.
Connection to previous session	In Session 6 you saved small pieces of data in `localStorage`. That data lived only in the learner's browser. Today the page learns to *ask for data from outside itself* — from a separate file or a web service — and display it. This is how real shops show products that the shop owner can change without editing the page.
Preparation before class	Instructor must install and confirm the VS Code **Live Server** extension works (see G.0 for why this session cannot be run from `file://`). Prepare a `products.json` file and a `shop.html`. Each learner should also have the Live Server extension installed (instructor may need 8 minutes at the start of the week to help).

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Explain in plain English** what an API is and why web pages use them, using a real-world comparison.
2. **Describe what JSON is** and identify its structure (objects, arrays, keys, values) in a sample file.
3. **Explain what a `fetch` call and a Promise are**, in beginner terms, without needing to master the theory.
4. **Write an `async` function** that uses `await fetch(...)` and `await response.json()` to load data from a local `products.json` file.
5. **Render fetched data to the page** by looping the array and building DOM elements (reusing Session 4 skills).
6. **Handle errors** with `try` / `catch`, and display a friendly message when data cannot be loaded.
7. **Diagnose** the two most common fetch problems — running from `file://` (blocked) and a wrong file path (404) — and fix them.

C. REQUIRED MATERIALS

Hardware: One computer per learner; instructor computer with projector. **Software:** VS Code with the **Live Server** extension installed; Google Chrome. No paid tools. **Accounts:** None. This session uses local files only; no internet API required (so it works even if the academy network is down). **Sample files:**

- `products.json` — a catalogue of 10 products (contents given in Section G.1).
- `shop.html` — a page with a `<div id="product-list">` container and the script linked.
- `js/shop.js` — the JavaScript file learners build. **Worksheets:**
- Worksheet 7.1 — "Read the JSON" (a printed JSON snippet; learners list the keys and one value).
- Worksheet 7.2 — "Trace the Await" (order-of-execution questions). **Tools:** Chrome DevTools (Console + Network tab). **Internet:** A live server (Live Server) runs locally, so no internet is needed for the lesson. If the academy has internet, the instructor may optionally demonstrate a public API at the very end as a bonus. **Alternative for learners lacking Live Server:** The instructor can run a simple local server. On Windows, if Python is installed: open a terminal in the project folder and run `python -m http.server 8000`, then open `http://localhost:8000/shop.html`. If neither is available, the instructor demonstrates on the projector and learners observe, then install Live Server next session.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open a well-known Nigerian e-commerce page (or an image of one) showing many products — clothing, phones, shoes. Do not open code.

What the instructor says:

> *"Look at this shop. It shows hundreds of products. Now think about how the shop owner works. When they add a new shoe to the shop, do they go into the website's code and type a new `<div>` for the shoe? If they did, they would need a programmer for every single product change. That is not how it works.*
>
> *Instead, all the products live in one data file — or on the shop's server. The web page asks for the list, receives it, and builds the product cards itself, automatically. Today your shop learns to do exactly that: ask for a list of products and display whatever comes back."*

Then demonstrate the hook: On the projector Console, type a tiny live example against a public API (only if internet is available) OR show a saved screenshot of the Network tab. For example (internet optional):

```
fetch("https://fakestoreapi.com/products/1")
  .then(response => response.json())
  .then(data => console.log(data.title));
```

Say: "The page asked a server far away for product number 1, and the server replied with a title. That reply came as text in a format called JSON. Today you learn to ask and to read that reply."

Expected learner response: Learners often ask "so the products are not typed into the page?" Confirm: correct — the page is a template; the data arrives separately. This is the central idea of the session.

E. PREREQUISITE REVIEW (7 minutes)

Q1. "Write the line that creates a new element and the line that adds it to a container."

- Expected: `const p = document.createElement("p"); container.appendChild(p);`
- Gap: missing `appendChild` — the element exists but is invisible. Remind: creating is not adding.

Q2. "What does `document.querySelector(".price")` return if no element has `class="price"` ?"

- Expected: `null` .
- Gap: learners say "an empty string" or "undefined". Clarify: `null` ; and touching a property of `null` gives "Cannot read properties of null" (this error will appear again with fetch).

Q3. "What is a loop used for, and which loop did we use to render a list of products?"

- Expected: to repeat an action for each item; we used `for...of` or `forEach` .
- Gap: confusing `for` syntax. Do a 30-second reminder.

Bridge: "You can already build a list from an array that is typed into your code. Today the array will arrive from a file *after* the page loads. That timing is the new idea."

F. FULL LECTURE NOTES

F.1 What is an API?

The idea. An **API** (Application Programming Interface) is a way for one program to request data or an action from another program in an agreed format.

Plain-English definition. An API is a **messenger between two systems**. Your web page sends a request in a standard way; the other system sends back a reply.

Analogy. Think of a **restaurant**. You (the web page) do not walk into the kitchen (the server/database) and start cooking. You tell the **waiter** (the API) what you want. The waiter carries your order to the kitchen, and brings back your food (the data). You and the kitchen do not need to know each other's internal workings — you just follow the waiter's standard menu and format.

Why APIs exist. They let websites show fresh data without hard-coding it. A weather page does not know tomorrow's weather when it is built; it asks a weather API at the moment the user opens the page. A shop's product list can be updated by the owner on the server, and every visitor sees the new list automatically.

What it does. Your code calls a **URL** (the API "menu item"), often with the `fetch` function, and receives **JSON** (the "food") back.

Real-life comparison tie-in. When you use a banking app and it shows your balance, the app did not store your balance on your phone. It asked the bank's API, "what is this customer's balance?", and the API replied. That reply is displayed.

Important limitation. Not every API is public, and many require a key or login. This course uses a **local** data file to keep things safe, offline and free.

F.2 What is JSON?

The idea. JSON (JavaScript Object Notation) is a text format for structured data. It looks very much like JavaScript objects and arrays, but it is stored as **plain text**.

Plain-English definition. JSON is a way of writing down data so that any program can read it. It uses `{ }` for objects and `[ ]` for arrays, with **keys** (names) and **values**.

Why it exists. Two different systems — maybe a page written by you and a server written in another language — both need to understand the same data. JSON is a simple, universal language for that exchange.

Example JSON — one product:

```
{
  "id": 1,
  "name": "Ankara Two-Piece Set",
  "price": 15000,
  "category": "clothing",
  "image": "images/ankara-set.jpg"
}
```

Example JSON — a list of products (an array of objects):

```
[
  { "id": 1, "name": "Ankara Two-Piece Set", "price": 15000, "category": "clothing", "image": "images/ankara.jpg" },
  { "id": 2, "name": "Leather Handbag", "price": 22000, "category": "bags", "image": "images/handbag.jpg" }
]
```

Key rules of JSON (important, and different from JavaScript objects):

- Keys **must be in double quotes**: `"name"`, not `name`.
- Strings use double quotes only — no single quotes.
- No trailing commas after the last item.
- No comments allowed inside JSON.
- Values may be: string, number, boolean, `null`, object, or array.

Common misconception. Learners think JSON *is* JavaScript. It is not. JSON is a **text format**. When you receive JSON text, you must convert it into real JavaScript objects before you can use `.name` etc. That conversion is done by `.json()` from fetch, or `JSON.parse()` when reading from `localStorage` (a skill from Session 6).

How a learner recognises it. A `.json` file, or a reply that uses `{ }`, `[ ]`, `:`, and `,` with quoted keys. Open `products.json` in VS Code to see it plainly.

F.3 Asynchronous code and Promises — the timing idea

The idea. `fetch` does not return the data immediately. It has to travel somewhere to get it. So the function returns a **Promise** — an *IOU* for a value that will arrive soon.

Plain-English definition. A **Promise** is an object that represents a value that is not ready yet but will be (or will fail). You attach instructions for "what to do when it arrives".

Analogy. A Promise is like **buying a "PENDING" transfer or ordering food for delivery**. You do not have the food in your hand the instant the rider is dispatched. You have a *promise* it is coming. You then wait for the rider to arrive (await), and then you eat (use the data).

Why it exists. Loading data over a network takes time. If JavaScript froze the whole page while waiting, the page would appear broken. Instead, JavaScript says: "I will not block; here is a Promise. Call me when it is ready." This is called **asynchronous** code.

The two ways to handle a Promise:

Way 1 — `.then()` chaining (the older style):

```
fetch("products.json")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error("Something went wrong:", error);
  });
```

Way 2 — `async` / `await` (the modern, preferred style — what we use):

```
async function loadProducts() {
  const response = await fetch("products.json");
  const data = await response.json();
  console.log(data);
}
```

- `async` marks a function that will do something asynchronous.
- `await` means "pause *this function* here until the result arrives" — without freezing the whole page.
- `response.json()` also returns a Promise, so we `await` it too.

Say to the class: "`await` is the word 'wait for it'. Without `await`, you would try to use the data before it arrived — like eating food that has not been delivered."

Common misconception. Learners forget `await` before `response.json()`, then get `undefined` or a Promise object in the Console. Teach: **every step that comes back from the network needs `await`.**

F.4 Putting it together — fetch, parse, render

The full pattern for this session:

```
const container = document.querySelector("#product-list");

async function loadProducts() {
  try {
    const response = await fetch("products.json");
    const products = await response.json();

    products.forEach(product => {
      const card = document.createElement("div");
      card.className = "card";
      card.innerHTML = `
        <h3>${product.name}</h3>
        <p class="cat">${product.category}</p>
        <p class="price">₦${product.price.toLocaleString()}</p>
      `;
      container.appendChild(card);
    });
  } catch (error) {
    container.textContent = "Sorry, we could not load products right now.";
    console.error("Failed to load products:", error);
  }
}

loadProducts();
```

Line by line:

1. Grab the container element where the cards will go.
2. An `async` function that does the loading.

3. `try` — attempt the risky network work.
4. `await fetch(...)` — ask for the file; wait for the reply.
5. `await response.json()` — convert the JSON text into real JavaScript data.
6. `forEach` over the products; for each one, build a card and append it.
7. `catch` — if anything failed, show a friendly message and log the real error.
8. `loadProducts();` — call the function to start it.

Note on `toLocaleString()`: `15000 .toLocaleString()` gives `"15,000"` — a nicer naira display. It is optional but professional.

F.5 Why `file://` breaks fetch — and the Live Server fix

The problem. When you double-click an HTML file, the browser opens it with a `file:///` address. Modern browsers block `fetch` requests from `file://` pages for security reasons. The Console shows an error like:

```
Access to fetch at 'file:///.../products.json' from origin 'null' has been blocked by CORS policy
```

or

```
Failed to fetch
```

Plain-English reason. The browser treats pages opened from your own disk as "not from a trusted website", so it refuses to let them fetch other files. This is a safety rule to stop malicious files on your computer from reading your private files.

The fix: serve the folder over a local server. The VS Code **Live Server** extension turns your folder into a tiny web server at an address like `http://127.0.0.1:5500/shop.html`. Because the page is now served "as a website", fetch is allowed.

How to use Live Server:

1. Install the **Live Server** extension in VS Code (Extensions panel, search "Live Server", Install).
2. Open your project folder in VS Code.
3. Right-click `shop.html` → **Open with Live Server**.
4. The browser opens `http://127.0.0.1:5500/shop.html`. Fetch now works.

Say to the class: "This is a real-world lesson. Developers rarely open pages by double-clicking; they run a local server. You are now doing it the professional way."

F.6 Handling errors with try/catch

The idea. Sometimes the network fails, the file is missing, or the JSON is broken. `try` / `catch` lets you attempt risky code and gracefully handle failure instead of showing a broken page.

```
try {
  // risky code (fetch, parse)
} catch (error) {
  // what to do if it fails
  console.error(error);
}
```

Why it matters. A professional site never shows a blank white page when data fails. It shows "Unable to load, please try again" — or a fallback. The `catch` block is where you decide that.

What the learner sees on failure: the friendly message from the `catch` block on the page, and the real technical error in the Console for the developer.

F.7 Summary of the session

- An **API** is a messenger between systems; `fetch` is how JavaScript calls one.
- **JSON** is a text format for structured data (`{ }` objects, `[ ]` arrays, quoted keys).
- `fetch` returns a **Promise**; we handle it with `async` / `await`.

- Convert JSON text into data with `await response.json()`.
- Render the data by looping and building DOM elements.
- `try` / `catch` keeps the page friendly when loading fails.
- Run fetch pages from a **local server (Live Server)**, not `file://`.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.0 Before the demo — confirm Live Server works

Open the project folder in VS Code, right-click an HTML file, choose **Open with Live Server**, and confirm the browser address bar shows `http://127.0.0.1:5500/...`. If it does not, fix the extension before the demo. Doing this live in front of the class also teaches the habit.

G.1 The sample `products.json` (create this file first)

```
[
  { "id": 1, "name": "Ankara Two-Piece Set", "price": 15000, "category": "clothing", "image": "images/ankara.jpg" },
  { "id": 2, "name": "Leather Handbag", "price": 22000, "category": "bags", "image": "images/handbag.jpg" },
  { "id": 3, "name": "Beaded Slippers", "price": 6500, "category": "shoes", "image": "images/slippers.jpg" },
  { "id": 4, "name": "Gele Head Wrap", "price": 4000, "category": "accessories", "image": "images/gele.jpg" },
  { "id": 5, "name": "Men's Kaftan", "price": 18000, "category": "clothing", "image": "images/kaftan.jpg" },
  { "id": 6, "name": "Aso-Oke Clutch", "price": 9000, "category": "bags", "image": "images/clutch.jpg" },
  { "id": 7, "name": "Coral Beads Set", "price": 12000, "category": "accessories", "image": "images/coral.jpg" },
  { "id": 8, "name": "Ladies High Heels", "price": 14000, "category": "shoes", "image": "images/heels.jpg" },
  { "id": 9, "name": "Adire Shirt", "price": 8000, "category": "clothing", "image": "images/adire.jpg" },
  { "id": 10, "name": "Jute Tote Bag", "price": 7000, "category": "bags", "image": "images/tote.jpg" }
]
```

The images do not need to exist for this session; the rendered cards focus on name, category and price. (Instructors may add placeholder images or omit the `<img>` tag.)

G.2 Demonstration steps

`shop.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Shop</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Adaeze Fashion</h1>
  <p id="status">Loading products...</p>
  <div id="product-list"></div>

  <script src="js/shop.js"></script>
</body>
</html>
```

`js/shop.js`:

```
const container = document.querySelector("#product-list");
const status = document.querySelector("#status");

async function loadProducts() {
  try {
    const response = await fetch("products.json");
    const products = await response.json();

    status.textContent = `${products.length} products found`;
```

```
    products.forEach(product => {
      const card = document.createElement("div");
      card.className = "card";
      card.innerHTML = `
        <h3>${product.name}</h3>
        <p class="cat">${product.category}</p>
        <p class="price">₦${product.price.toLocaleString()}</p>
      `;
      container.appendChild(card);
    });
  } catch (error) {
    status.textContent = "Sorry, we could not load products right now.";
    console.error("Failed to load products:", error);
  }
}

loadProducts();
```

On the projector:

1. Save all files.
2. Right-click `shop.html` → **Open with Live Server**.
3. Point at "10 products found" and the ten rendered cards.
4. Open DevTools → **Network** tab → refresh → point at the `products.json` request returning 200. **Say:** "This is the request and this is the reply — the waiter bringing the food."

Pause and predict: Before refreshing, ask "What will the status line say if there are ten products?" (`10 products found` .)

G.3 Errors to inject and correct (the teaching gold)

Error A — wrong path. Change `fetch("products.json")` to `fetch("product.json")` (missing 's'). Save, refresh.

- Expected: page shows the friendly message; Console shows a 404 red error; Network tab shows the failed request.
- Fix: correct the file name. **Lesson:** the `catch` protected the user from a blank page.

Error B — missing `await` . Change `const products = await response.json();` to `const products = response.json();` . Save, refresh.

- Expected: `products.forEach is not a function` (because `products` is a Promise, not an array), and the cards do not render.
- Fix: add `await` back. **Lesson:** every network step needs `await` .

Error C — opening with `file://` . Close the Live Server tab and double-click `shop.html` instead.

- Expected: `Failed to fetch` / CORS error; no products.
- Fix: return to Live Server. **Lesson:** always serve fetch pages locally.

Questions to ask: "Which error message told us the file was missing?" (the 404 in Network/Console). "Why did the page still look tidy when it failed?" (the `catch` block).

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: Load a list of services from a JSON file and render them to the page.

Starting materials: A new `services.json` (instructor provides, below), plus `services.html` with `<div id="service-list">` .

Instructor-provided `services.json` :

```
[
  { "id": 1, "name": "Tailoring", "price": 5000, "duration": "3 days" },
  { "id": 2, "name": "Bead Making Class", "price": 3000, "duration": "2 hours" },
  { "id": 3, "name": "Hair Styling", "price": 2500, "duration": "1 hour" },
  { "id": 4, "name": "Makeup", "price": 7000, "duration": "2 hours" }
]
```

Procedure:

1. Create the folder structure `week3/services/` with `services.html` and `js/services.js` .

2. In `services.js`, select the container `#service-list`.
3. Write an `async` function `loadServices()`.
4. Inside, `await fetch("services.json")` and `await response.json()`.
5. Use `forEach` to build a card for each service: name, `₦price` (formatted with `toLocaleString()`), and duration.
6. Append each card to the container.
7. Wrap in `try/catch`; in `catch`, show "Services unavailable, please try again."
8. **Pause and predict:** "How many cards will appear?" (4). "What will Hair Styling display?" (`₦2,500 … 1 hour`).
9. Run via Live Server and compare.

Expected result: Four service cards on the page, each with a name, a naira price and a duration.

Troubleshooting while walking the room:

- `CORS` / `Failed to fetch` → not using Live Server.
- `forEach is not a function` → missing `await` before `response.json()`.
- Empty page, no error → the container selector is wrong (`null`), or cards not appended.
- Prices show as `NaN` → price was quoted as a string; keep numbers unquoted in JSON.

Completion checklist:

- [ ] Fetch uses Live Server URL
- [ ] `await` used twice
- [ ] 4 cards rendered
- [ ] Prices formatted with `toLocaleString()`
- [ ] `try/catch` present
- [ ] No red errors in Console

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Build a "Church Events" loader.

Scenario: *Grace Chapel* wants a small page that lists upcoming events. All event data lives in `events.json`. You must load it and render it.

Instructor-provided `events.json`:

```
[
  { "id": 1, "title": "Youth Vigil", "date": "2026-11-14", "venue": "Main Hall" },
  { "id": 2, "title": "Harvest Thanksgiving", "date": "2026-11-22", "venue": "Church Grounds" },
  { "id": 3, "title": "Choir Practice", "date": "2026-11-08", "venue": "Upper Room" },
  { "id": 4, "title": "Community Outreach", "date": "2026-11-30", "venue": "Mile 3 Market" },
  { "id": 5, "title": "Baptism Service", "date": "2026-12-06", "venue": "Riverside" }
]
```

Requirements:

1. Create `events.html` and `js/events.js`.
2. Load `events.json` with `async/await`.
3. Render each event as a card showing **title**, **date** and **venue**.
4. Show a status line at the top: "X events scheduled" (X = the count).
5. Wrap the loading in `try/catch`; on failure show "Events could not be loaded."

Constraints: Cards must be created with `document.createElement` + `appendChild`. Do not hard-code any event text in the HTML. Run from Live Server.

Time allowance: 15 minutes.

Success criteria:

- Status shows "5 events scheduled".
- Five cards render with correct title/date/venue.

- Removing the `events.json` file and refreshing shows the friendly error message (test this deliberately).

Hints:

- Hint 1: the count is `events.length`.
- Hint 2: to display the date as-is, just insert `${event.date}`.
- Hint 3: the catch block message goes into the same status element.

Instructor answer key:

```
const container = document.querySelector("#event-list");
const status = document.querySelector("#status");

async function loadEvents() {
  try {
    const response = await fetch("events.json");
    const events = await response.json();

    status.textContent = `${events.length} events scheduled`;

    events.forEach(event => {
      const card = document.createElement("div");
      card.className = "event-card";
      card.innerHTML = `
        <h3>${event.title}</h3>
        <p class="date">Date: ${event.date}</p>
        <p class="venue">Venue: ${event.venue}</p>
      `;
      container.appendChild(card);
    });
  } catch (error) {
    status.textContent = "Events could not be loaded.";
    console.error(error);
  }
}

loadEvents();
```

Marking guidance (out of 10): correct fetch + await (2); status count correct (2); five cards render correctly (3); try/catch present and friendly message (2); runs via Live Server with no errors (1).

J. REAL-WORLD CASE STUDY

Situation. *Chidi's Provisions* is a small shop in Port Harcourt. Chidi sells rice, beans, oil and other staples. He wants a simple website so customers can see current prices. His problem: prices change almost every week because of the market. If prices were typed into the page, Chidi would need to call a web developer every time — too slow and too expensive.

Relevant facts.

- Chidi has about 30 products.
- Prices change weekly; product list changes rarely.
- Chidi can edit a text file if it is simple (just names and numbers).
- He has no budget for a database or paid hosting.

Decision point. How can Chidi update prices himself without touching the page's design code?

Possible approaches.

1. Hard-code prices in the HTML (bad — needs a developer each time).
2. Store products in a `products.json` file; the page fetches and renders them. Chidi edits only the JSON.
3. Build a full admin dashboard with a database (overkill now).

Recommended solution. Approach 2. Chidi edits one clearly-structured JSON file. The page always fetches the latest version. This is exactly what the class learned today.

Example (excerpt of Chidi's JSON):

```
[
  { "id": 1, "name": "Long Grain Rice (50kg)", "price": 78000, "category": "grains" },
  { "id": 2, "name": "Beans (Paint Bucket)", "price": 12500, "category": "grains" },
  { "id": 3, "name": "Groundnut Oil (5L)", "price": 9800, "category": "oils" },
  { "id": 4, "name": "Peak Milk Tin", "price": 3500, "category": "dairy" }
]
```

Trade-offs.

- Approach 1 is simplest to build but creates ongoing dependency on a developer — a real business cost.
- Approach 2 gives Chidi independence and costs nothing extra; the only risk is Chidi making a typo in the JSON and breaking it, so a JSON validator (or careful editing) is advised.
- Approach 3 is the long-term answer if Chidi grows to hundreds of products and multiple staff, but it is far beyond this course and unnecessary now.

Professional connection. This "data from outside, rendered by the page" pattern is the foundation of almost every modern website. Junior front-end developers spend much of their time fetching data and rendering it — exactly this skill.

K. COMMON MISTAKES AND TROUBLESHOOTING

K.1 `Failed to fetch` / CORS error, nothing renders

Symptoms: Console shows CORS or `Failed to fetch`; no products. **Likely cause:** The page was opened with `file://` (double-clicked) instead of a local server. (Confirmed by checking the address bar: it starts with `file:///`.) **Investigate:** Look at the browser address bar. **Fix:** Open the page with Live Server (or `http://localhost:8000/...`). **Confirm:** Cards appear and the Network tab shows a 200 for `products.json`. **Prevent:** Always use Live Server for fetch sessions.

K.2 `products.forEach is not a function`

Symptoms: A red TypeError about `forEach` (or `map`, `filter`). **Likely cause:** A missing `await` before `response.json()`, so `products` is a Promise, not an array. (Confirmed by `console.log(products)` showing a Promise.) **Fix:** Add `await` before `response.json()`. **Confirm:** `console.log(products)` shows an array in the Console. **Prevent:** Every network step needs `await`.

K.3 Status says "Loading..." forever / nothing appears, no error

Symptoms: Page stays on the loading text; no error. **Likely causes:** (a) The `loadProducts()` function was never called. (b) The container selector returned `null`. (c) The container `id` in HTML does not match the selector. **Investigate:** Add `console.log(container)` — if it prints `null`, the selector is wrong. Add `console.log("calling now")` before the call to confirm it runs. **Fix:** Correct the selector or add the missing call. **Confirm:** Cards render. **Prevent:** Double-check `id` spelling on both sides.

K.4 `SyntaxError: Unexpected token` when parsing JSON

Symptoms: Error in the `await response.json()` step. **Likely cause:** The JSON file is malformed — a missing comma, a trailing comma after the last item, or keys not in double quotes. (Confirmed by opening `products.json` and validating it.) **Fix:** Correct the JSON. Tools like a JSON validator or VS Code's error underlining help. **Confirm:** The syntax error disappears and data renders. **Prevent:** Follow JSON rules strictly (Section F.2).

K.5 Prices display as `NaN`

Symptoms: Cards show `₦NaN`. **Likely cause:** The `price` value in JSON is a string (e.g., `"15000"`), or a typo like `prices`. (Confirmed with `console.log(typeof product.price)`.) **Fix:** Store numbers unquoted in JSON. **Prevent:** Numbers without quotes; strings with quotes.

K.6 Data loads but the page flickers / duplicate cards on re-run

Symptoms: Cards appear twice. **Likely cause:** The load function is called more than once without clearing the container. **Fix:** Clear the container first: `container.innerHTML = "";` before the loop. **Prevent:** Render functions should build into a fresh container.

Distinguish likely vs confirmed: The CORS error text, the exact TypeError, and `console.log` output are the *evidence* that turns a guess into a confirmed cause. Teach learners to read them before changing code.

L. SESSION ASSIGNMENT

Assignment title: "Load a Real Menu"

Purpose: Practise the full fetch → parse → render → handle-error cycle independently, in a business context.

Realistic scenario: *Mama Ngozi's Kitchen* needs a menu page. The menu changes occasionally, so it must come from a `menu.json` file the kitchen can edit.

Detailed instructions:

1. Create a `menu.json` with at least **6 dishes**, each having: `name`, `price`, `category` (e.g., "rice", "swallow", "drinks"). Design the JSON yourself.
2. Create `menu.html` and `js/menu.js`.
3. Load and render the dishes as cards (name, `₦price`, category).
4. Add a status line showing the number of dishes.
5. Use `try/catch`; on failure, show "Menu is unavailable right now."
6. Deliberately test failure: temporarily rename `menu.json`, refresh, and record what happens; then rename it back.

Required deliverables: `menu.json`, `menu.html`, `js/menu.js`, and a short note (in the workbook) describing what the page showed when the file was missing.

Submission format: Files in the class folder; note in the workbook.

Deadline: Before Session 8.

Marking rubric (20 marks):

Criterion	Marks	Strong answer
Valid JSON with 6+ items and 3 fields	4	Correct JSON syntax, sensible data
Fetch with async/await	4	Both awaits present
Rendering with createElement/appendChild	4	Cards correct and readable
Status count	2	Shows correct number
try/catch + friendly message	4	Graceful failure proven by the test note
Runs via Live Server, no errors	2	Clean Console

Common mistakes to avoid: trailing comma in JSON; missing `await`; opening via `file://`; hard-coding menu items in HTML.

Instructor reference solution:

```
const container = document.querySelector("#menu-list");
const status = document.querySelector("#status");

async function loadMenu() {
  try {
    const response = await fetch("menu.json");
    const menu = await response.json();

    status.textContent = `${menu.length} dishes on the menu`;

    menu.forEach(dish => {
      const card = document.createElement("div");
      card.className = "dish";
      card.innerHTML = `
        <h3>${dish.name}</h3>
        <p>${dish.category}</p>
        <p>₦${dish.price.toLocaleString()}</p>
      `;
```

```
      container.appendChild(card);
    });
  } catch (error) {
    status.textContent = "Menu is unavailable right now.";
    console.error(error);
  }
}

loadMenu();
```

M. KNOWLEDGE CHECK

Q1. Short answer. What is an API, in your own words, and what is the waiter analogy?

Q2. Multiple choice. Which line correctly converts a fetch response into usable data? a) `const data = response.json();` b) `const data = await response.json();` c) `const data = JSON.fetch(response);` d) `const data = response.JSON();`

Q3. Explain in your own words. Why can you not open a fetch page by double-clicking the HTML file?

Q4. Practical identification. Given `const data = await response.json();`, what type is `data` if the JSON file begins with `[`? What about `{`?

Q5. Error detection. This code fails: `const products = response.json(); products.forEach(p => console.log(p.name));` — what is the bug and the fix?

Q6. Scenario. Your fetch works with Live Server but shows the friendly "unavailable" message when you deploy to GitHub Pages because the path to `products.json` is wrong. How would you find the real problem using DevTools?

INSTRUCTOR ANSWER KEY (Q1–Q6)

A1. An API is a messenger that lets one program request data or actions from another in an agreed format. The waiter analogy: your page (customer) orders from the API (waiter) who fetches from the server (kitchen) and returns the data (food). **A2.** (b) `await response.json()`. **Q3.** Because the `file://` protocol is treated as untrusted; browsers block fetch from it for security (CORS). You need a local server so the page is served over `http://`. **A4.** If it begins with `[`, `data` is an **array**; with `{`, an **object**. **A5.** Missing `await`, so `products` is a Promise, not an array; `forEach` fails. Fix: `const products = await response.json();`. **A6.** Open DevTools → Network tab, reload, find the `products.json` request; a 404 (red) confirms the path is wrong. Also check the Console error. Fix the path (case-sensitive on GitHub Pages).

N. REFLECTION AND RECAP

Plain-English summary. Today your page learned to ask for data instead of having it typed in. You met APIs (the messenger), JSON (the data format), Promises and `async/await` (how to wait for data without freezing the page), and you used `fetch` to load a local `products.json` and render cards. You also learned to wrap it in `try/catch` so the page fails gracefully, and why fetch pages must be served with Live Server.

Five important takeaways:

1. An API lets your page request data from outside itself.
2. JSON is text that looks like JavaScript objects and arrays.
3. `fetch` returns a Promise; use `async/await`.
4. Convert with `await response.json()`, then loop and render.
5. Use `try/catch` and always run fetch pages from Live Server.

Questions to help you recognise what you can now do:

- Can I write an `async` function that fetches a JSON file and renders it?
- Can I explain the difference between JSON text and a JavaScript object?
- Can I make the page show a friendly message when loading fails?
- Can I find a failed request in the Network tab?

Connection to the next session: Now that you can load products and render them, next session you will make them buyable. You will build a **shopping cart** — add items, remove items, calculate a total — and keep it alive across page refreshes with `localStorage`. This is the core of your capstone shop.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner uses Live Server automatically, without being reminded.
- Learner writes both `await`s unprompted.
- Learner reads the Console error text and identifies the cause rather than guessing.
- Learner wraps fetch in `try/catch` and can explain what the `catch` does.
- Learner's rendered cards match the JSON data (names/prices correct).

Common misconceptions:

- "JSON is JavaScript." → It is a text format; parse it first.
- "`fetch` gives the data immediately." → It gives a Promise; `await` it.
- "The page must be double-clicked to open." → Only for static pages; fetch needs a server.
- "`console.log` of the response shows the data." → Log after `.json()` / `await` to see real data.

Suggested remedial exercises:

1. Order-of-execution drill: give a snippet with `console.log` before and after an `await` and ask learners to predict the order.
2. JSON-fixing drill: hand out a broken `products.json` (missing commas, single quotes) and ask learners to fix it until the page renders.
3. Error-reading drill: show three Console errors and ask which is CORS, which is a missing `await`, and which is malformed JSON.

Differentiation:

- **Level 1 (supported):** provide `shop.js` with the fetch lines written and blanks for the render loop.
- **Level 2 (standard):** the exercises as written.
- **Level 3 (challenge):** after rendering, add a button that re-fetches the data ("Refresh products"), and (if internet exists) fetch from a public API and render the result.

Timing guidance: The install/Live Server setup can eat time. If needed, pre-install Live Server on all machines before the week, or budget 8 minutes at the start. Protect the guided and independent exercises.

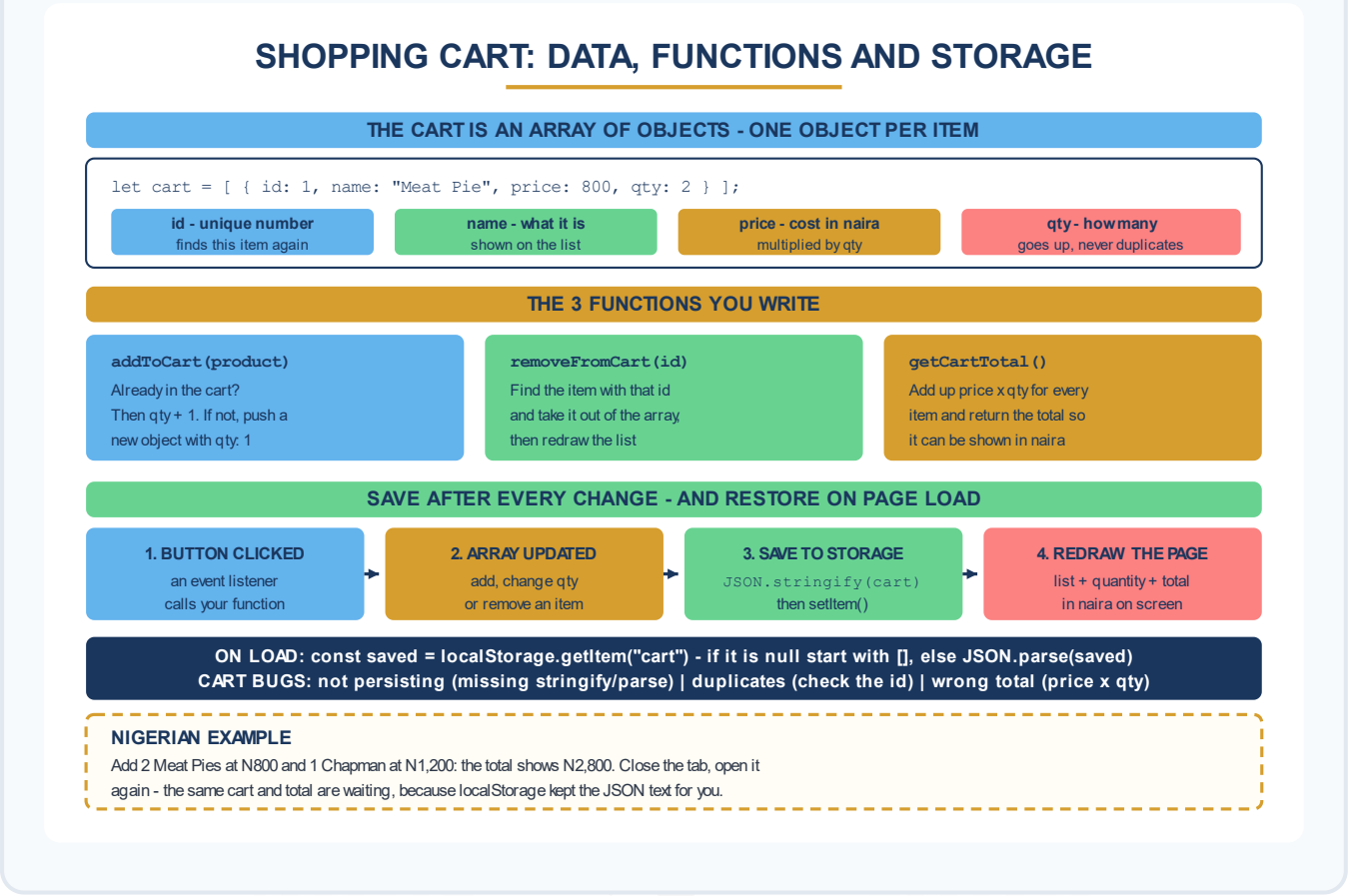
END OF SESSION 7

Next: Session 8 — Building the Shopping Cart with localStorage.

SESSION 08 OF 12

BUILDING THE SHOPPING CART WITH LOCALSTORAGE

CONCEPT DIAGRAM — SESSION 08



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 8 — BUILDING THE SHOPPING CART WITH LOCALSTORAGE

A. SESSION IDENTIFICATION

Item	Detail
Session number	8 of 12
Session title	Building the Shopping Cart with localStorage
Week / Module	Week 3 — Module 3: Data & Real Apps
Recommended duration	105 minutes

Item	Detail
Difficulty level	Intermediate (combines arrays, objects, DOM, events and storage)
Prerequisite knowledge	Sessions 1–7, especially: arrays of objects (S3), creating and appending DOM elements (S4), `addEventListener` and reading input (S5), `localStorage` + `JSON.stringify` / `JSON.parse` (S6), and fetching/rendering a product list (S7).
Connection to previous session	In Session 7 the page *displayed* products. Today each product becomes **buyable**: clicking "Add to Cart" pushes it into a cart array, the cart renders on the page with a running total, and — the key idea — the cart **survives a page refresh** because it is saved in `localStorage`.
Preparation before class	A working shop page from Session 7 (products rendered). A `cart.html` page with an empty `<div id="cart-items">` and a `<span id="cart-total">`. The instructor should confirm Live Server still works.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Design a cart data structure** as an array of objects, each with `id`, `name`, `price` and `qty`.
2. **Write an `addToCart(product)` function** that adds a new item or increases the `qty` of an existing item (avoiding duplicates).
3. **Write a `removeFromCart(id)` function** that removes an item from the cart.
4. **Write a `getCartTotal()` function** that returns the sum of `price × qty` across all cart items.
5. **Render the cart** to the page (item list + quantity + running total in naira).
6. **Persist the cart** to `localStorage` after every change and **restore** it on page load using `JSON.stringify` and `JSON.parse`, handling the empty (`null`) case safely.
7. **Connect product buttons to the cart** using event listeners.
8. **Diagnose** common cart bugs: cart not persisting (missing `JSON.parse` / `stringify`), duplicates appearing, totals wrong, and a `null` cart crashing the page on first load.

C. REQUIRED MATERIALS

Hardware: One computer per learner; instructor computer with projector. **Software:** VS Code + Live Server extension; Google Chrome. **Accounts:** None. All work is local and offline. **Sample files:** The Session 7 `products.json` (reuse it), a `shop.html` page, and a `cart.html` page. **Worksheets:**

- Worksheet 8.1 — "Design the Cart Object" (learners sketch the array of objects for three example items).
- Worksheet 8.2 — "Predict the Total" (given a cart array, compute the total by hand). **Tools:** Chrome DevTools — Console and the **Application** tab (where `localStorage` can be inspected). Note: in Chrome DevTools, `localStorage` is found under **Application → Storage → Local Storage**. **Internet:** Not required (Live Server runs locally). **Alternative for learners lacking Live Server:** As in Session 7, run `python -m http.server 8000` or observe on the projector. This session can also work with a hard-coded product array if fetch is a problem — the cart logic is the focus.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open a familiar online shopping site (or an image) and add three items to the cart, then refresh the page.

What the instructor says:

> *"Watch this. I add a dress, a bag and shoes to the cart. The cart shows three items and a total. Now I refresh the page… and the cart is STILL there — three items, same total. If the cart lived only in the page, refreshing would wipe it. So where is*

it being kept?

Today you build exactly this. A cart that remembers what the customer chose — even after they close the page and come back tomorrow. The tool that lets your page remember is called `localStorage`, and you met it in Session 6. Today we use it for something real."

Expected learner response: Learners may say "a database" or "the server". Clarify: big shops use servers, but many carts are kept in the browser's `localStorage`, and that is what we build today — no server needed.

Then show the Application tab: Open a site, press `F12`, go to **Application** → **Local Storage**, and point at the stored keys. **Say:** "This little drawer in the browser is where we will keep the cart."

E. PREREQUISITE REVIEW (7 minutes)

Q1. "How do you save an array to `localStorage`?"

- Expected: `localStorage.setItem("cart", JSON.stringify(cart));`
- Gap: forgetting `JSON.stringify` — `localStorage` only stores strings. Remind: without it you get `[object Object]` stored.

Q2. "How do you read it back?"

- Expected: `JSON.parse(localStorage.getItem("cart"))`.
- Gap: forgetting `JSON.parse` — you get a string, and `.forEach` fails.

Q3. "Write an event listener that runs `handleClick` when a button with id `add-btn` is clicked."

- Expected: `document.querySelector("#add-btn").addEventListener("click", handleClick);`
- Gap: calling `handleClick()` (with parentheses) instead of passing `handleClick`. Remind: pass the function, do not call it yet.

Q4. "Given `[{price: 100, qty: 2}, {price: 50, qty: 3}]`, what is the total by hand?"

- Expected: `200 + 150 = 350`.
- Gap: adding prices but ignoring qty. Remind: total = price × qty for each, then sum.

Bridge: "You already have every ingredient: arrays, DOM, events and storage. Today we combine them into something a real shop needs — a working cart."

F. FULL LECTURE NOTES

F.1 The cart as an array of objects

The idea. The cart is simply an **array**, and each item in it is an **object** describing one product the customer wants.

Plain-English definition. A cart is a list. Each line in the list says: which product, and how many.

Why represent it this way? Because a customer might add the same product twice, and we need to know the quantity. Storing only product IDs would force us to look the product up again; storing the whole object (`name`, `price`, `qty`) makes rendering easy.

The shape:

```
let cart = [
  { id: 1, name: "Ankara Two-Piece Set", price: 15000, qty: 1 },
  { id: 3, name: "Beaded Slippers", price: 6500, qty: 2 }
];
```

Why `qty` instead of listing the same product twice? If a customer adds the same slippers twice, we prefer one line with `qty: 2` rather than two lines with `qty: 1` each. It is cleaner and the total is the same.

Common misconception. Learners sometimes store only `{id, qty}` and then struggle to show the name and price. Storing the needed fields (`name`, `price`) directly in the cart is simpler for a beginner and is perfectly acceptable at this level. (Advanced apps store IDs and look up data to avoid stale prices — a discussion for later.)

Empty cart. The starting state is `let cart = [];` — an empty list. The page must handle this: "Your cart is empty."

F.2 Adding to the cart — and avoiding duplicates

The idea. When "Add to Cart" is clicked, we check whether that product is already in the cart. If yes, increase its `qty`. If no, add it with `qty: 1`.

```
function addToCart(product) {
  const existing = cart.find(item => item.id === product.id);

  if (existing) {
    existing.qty = existing.qty + 1;
  } else {
    cart.push({ id: product.id, name: product.name, price: product.price, qty: 1 });
  }

  saveCart();
  renderCart();
}
```

Line by line:

1. `cart.find(...)` searches the array and returns the first item whose `id` matches, or `undefined` if none.
2. If found (`existing` is truthy), increase its quantity.
3. Otherwise, `push` a new object with `qty: 1`.
4. `saveCart()` writes the updated cart to `localStorage`.
5. `renderCart()` redraws the visible cart.

Why `find` and not another loop? `find` is a built-in array method that does the search for us — cleaner and less error-prone than a manual `for` loop.

Analogy. Adding to a cart is like adding a line to a handwritten shopping list. If the item is already written, you just change the "1" to "2". You do not write the item again.

common misconception. Learners compare with `==` or compare the whole object. Compare only the **id** with `===`.

F.3 Removing from the cart

The idea. To remove an item, we filter it out of the array by `id`.

```
function removeFromCart(id) {
  cart = cart.filter(item => item.id !== id);
  saveCart();
  renderCart();
}
```

Line by line:

1. `filter` builds a **new array** containing every item whose `id` is **not** the one being removed.
2. Reassign the cart to that new array.
3. Save and re-render.

Why `filter`? It expresses "keep everything except this one" in a single readable line. It also returns a new array, avoiding the pitfalls of editing an array while looping over it.

Removing vs reducing quantity: This simple version removes the whole line. A more advanced version could reduce `qty` by one until it reaches zero — a Level 3 challenge.

F.4 Totalling the cart

The idea. The total is the sum of `price × qty` for every item.

```
function getCartTotal() {
  let total = 0;
  cart.forEach(item => {
    total = total + (item.price * item.qty);
  });
  return total;
}
```

Line by line:

1. Start `total` at `0`.
2. For each item, add `price × qty`.
3. Return the total.

Alternative with `reduce` (optional, show only if the class is strong):

```
function getCartTotal() {
  return cart.reduce((sum, item) => sum + item.price * item.qty, 0);
}
```

Why the manual loop first? For beginners, seeing `total = total + ...` makes the accumulation explicit, which builds understanding. `reduce` is efficient but abstract.

Note on formatting: Display the total with `toLocaleString()` → `₦40,500`. Always show the naira sign.

F.5 Saving and loading the cart (persistence)

The idea. After every change, save the cart; on page load, restore it.

Saving:

```
function saveCart() {
  localStorage.setItem("cart", JSON.stringify(cart));
}
```

Loading (on page load):

```
function loadCart() {
  const saved = localStorage.getItem("cart");
  if (saved) {
    cart = JSON.parse(saved);
  } else {
    cart = [];
  }
}
```

Why the `if (saved)` check? On the very first visit, `getItem` returns `null`. Calling `JSON.parse(null)` returns `null`, and then `cart.forEach` would crash with "Cannot read properties of null". The check guards against the very first load — one of the most common beginner bugs.

Analogy. `localStorage` is the browser's **filing cabinet**, per website. `setItem` is filing a document; `getItem` is taking it out. The cabinet remembers even after you close the browser (until the user clears their browser data).

The string limitation (recap from Session 6). `localStorage` stores only strings. Objects and arrays must be turned into strings with `JSON.stringify` when saving and turned back with `JSON.parse` when loading. Forgetting this is the number-one cart bug.

Important limitation. `localStorage` is specific to **one browser on one device**. A cart saved in Chrome on the academy PC will not appear in the customer's phone browser. Sharing a cart across devices requires a server and user accounts — beyond this course, but worth stating honestly.

F.6 Wiring the buttons

When rendering products (from Session 7), each product card gets an "Add to Cart" button. We attach a listener that calls `addToCart` with that product.

```
  products.forEach(product => {
    const card = document.createElement("div");
    card.className = "card";

    const title = document.createElement("h3");
    title.textContent = product.name;
    card.appendChild(title);

    const priceEl = document.createElement("p");
    priceEl.textContent = `₦${product.price.toLocaleString()}`;
    card.appendChild(priceEl);

    const btn = document.createElement("button");
    btn.textContent = "Add to Cart";
    btn.addEventListener("click", () => addToCart(product));
    card.appendChild(btn);

    container.appendChild(card);
  });
```

Why the arrow function `() => addToCart(product)`? If we wrote `addEventListener("click", addToCart(product))`, it would call `addToCart` immediately (at render time) and pass its return value. Wrapping it in `() => ...` defers the call until the click, and captures the correct `product` for each button. This is a subtle but critical point.

F.7 The full flow

1. Page loads → `loadCart()` restores the cart from `localStorage`.
2. `renderCart()` draws the current cart and total.
3. Products render, each with an "Add to Cart" button.
4. Click → `addToCart` → update array → `saveCart` → `renderCart`.
5. Remove click → `removeFromCart` → update array → `saveCart` → `renderCart`.
6. Refresh → step 1 restores everything. The cart persists.

F.8 Summary of the session

- The cart is an **array of objects** (`{id, name, price, qty}`).
- `addToCart` avoids duplicates with `find`; `removeFromCart` uses `filter`.
- `getCartTotal` sums `price × qty`.
- Save with `JSON.stringify` + `setItem`; load with `getItem` + `JSON.parse`, guarding against `null`.
- Wire buttons with a closure `() => addToCart(product)`.
- `localStorage` is per-browser, per-device.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 Prepare `cart.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Cart</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Your Cart</h1>
  <p id="cart-status">Your cart is empty.</p>
  <div id="cart-items"></div>
  <h2>Total: <span id="cart-total">₦0</span></h2>
  <p><button id="clear-cart">Clear Cart</button></p>
```

```
  <script src="js/cart.js"></script>
</body>
</html>
```

G.2 `js/cart.js` — a self-contained demo with sample products

For the demo (and to keep it independent of fetch timing), the instructor uses a small hard-coded `products` array. This keeps focus on the cart logic.

```
const products = [
  { id: 1, name: "Ankara Two-Piece Set", price: 15000, category: "clothing" },
  { id: 2, name: "Leather Handbag", price: 22000, category: "bags" },
  { id: 3, name: "Beaded Slippers", price: 6500, category: "shoes" }
];

let cart = [];

const itemsEl = document.querySelector("#cart-items");
const totalEl = document.querySelector("#cart-total");
const statusEl = document.querySelector("#cart-status");
const listEl = document.querySelector("#product-list");

function saveCart() {
  localStorage.setItem("cart", JSON.stringify(cart));
}

function loadCart() {
  const saved = localStorage.getItem("cart");
  cart = saved ? JSON.parse(saved) : [];
}

function addToCart(product) {
  const existing = cart.find(item => item.id === product.id);
  if (existing) {
    existing.qty += 1;
  } else {
    cart.push({ id: product.id, name: product.name, price: product.price, qty: 1 });
  }
  saveCart();
  renderCart();
}

function removeFromCart(id) {
  cart = cart.filter(item => item.id !== id);
  saveCart();
  renderCart();
}

function getCartTotal() {
  let total = 0;
  cart.forEach(item => {
    total += item.price * item.qty;
  });
  return total;
}

function renderCart() {
  itemsEl.innerHTML = "";

  if (cart.length === 0) {
    statusEl.textContent = "Your cart is empty.";
    totalEl.textContent = "₦0";
    return;
  }

  statusEl.textContent = `${cart.length} item type(s) in your cart`;

  cart.forEach(item => {
    const row = document.createElement("div");
    row.className = "cart-row";
    row.innerHTML = `
```

```
      <span>${item.name} × ${item.qty} — ₦${(item.price * item.qty).toLocaleString()}</span>
    `;
    const removeBtn = document.createElement("button");
    removeBtn.textContent = "Remove";
    removeBtn.addEventListener("click", () => removeFromCart(item.id));
    row.appendChild(removeBtn);
    itemsEl.appendChild(row);
  });

  totalEl.textContent = `₦${getCartTotal().toLocaleString()}`;
}

// --- Load cart on start ---
loadCart();
renderCart();
```

G.3 Demonstration steps

1. Save, open `cart.html` via **Live Server**.
2. In the Console (for the demo, since there are no product buttons on this page) run `addToCart(products[0])` and watch the cart line and total appear.
3. Add the slippers twice (`addToCart(products[2])` twice) → one line with `× 2`.
4. **Pause and predict:** "What is the total after one ankara set and two slippers?" (`15000 + 13000 = 28000`.)
5. Point at the total: `₦28,000`.
6. **Refresh the page.** The cart is still there. **Say:** "That is `localStorage` doing its job."
7. Open DevTools → **Application** → **Local Storage** and show the `cart` key holding the JSON string.
8. Click Remove on one item → it disappears, total updates, save happens.
9. Click Clear Cart → cart empties (see the listener below in `cart.html`).

Add this near the end of `cart.js` for the clear button:

```
document.querySelector("#clear-cart").addEventListener("click", () => {
  cart = [];
  saveCart();
  renderCart();
});
```

G.4 Errors to inject and correct

Error A — no `JSON.parse`. Change `cart = saved ? JSON.parse(saved) : [];` to `cart = saved ? saved : [];`. Refresh.

- Expected: `cart.forEach is not a function` (the cart is a string).
- Fix: restore `JSON.parse`. **Lesson:** storage holds strings; parse to use.

Error B — no `null` guard. Change `loadCart` to just `cart = JSON.parse(localStorage.getItem("cart"));` on a fresh browser (clear storage first).

- Expected: `Cannot read properties of null` at `renderCart`. **Lesson:** guard the first-load `null`.

Error C — immediate call bug. Change the product button listener to `addEventListener("click", addToCart(product))` (in the full shop).

- Expected: the item is added immediately on render, before any click. **Lesson:** wrap in `() => addToCart(product)`.

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: Build the cart on the shop page from Session 7, with "Add to Cart" buttons connected to products loaded from `products.json`.

Starting materials: Session 7's `shop.html`, `js/shop.js`, and `products.json`. Add a cart area to `shop.html`.

Procedure:

1. Add to `shop.html` (above the product list):

```
<p id="cart-status">Your cart is empty.</p>
<div id="cart-items"></div>
<h2>Total: <span id="cart-total">₦0</span></h2>
```

2. Copy the cart functions (`saveCart` , `loadCart` , `addToCart` , `removeFromCart` , `getCartTotal` , `renderCart`) into `shop.js` .
3. In the product render loop, add an "Add to Cart" button to each card with `() => addToCart(product)` .
4. Call `loadCart()` and `renderCart()` at the start.
5. Save, run via Live Server.
6. **Pause and predict:** "Add two different products. What will the total be?" Compute by hand before clicking.
7. Refresh the page → cart persists.
8. Remove one item → total reduces.

Expected result: A shop page where products have Add to Cart buttons; the cart area updates live; the total is correct; refreshing keeps the cart.

Troubleshooting while walking the room:

- Items appear twice → missing `find` duplicate check.
- `forEach is not a function` → cart loaded as a string (missing `JSON.parse`).
- Add button adds on page load → forgot to wrap the call in an arrow function.
- Total shows `NaN` → price stored as string; ensure numbers.

Completion checklist:

- ☐ Products render with Add to Cart buttons
- ☐ Adding a product updates the cart and total
- ☐ Adding the same product twice increases qty (no duplicate line)
- ☐ Remove works
- ☐ Cart persists after refresh
- ☐ Empty cart shows "Your cart is empty."

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Build a "Simple Shopping List" app (a personal version of the cart idea).

Scenario: A student wants a small list to track items to buy at the market, that stays saved.

Requirements:

1. A page `list.html` with a text input `#item-name` , a number input `#item-price` , an "Add Item" button, a container `#list` , and a total `#total` .
2. Items are stored as an array of objects `{ name, price }` in `localStorage` under the key `"marketList"` .
3. Add an item → append to the array → save → render. (Duplicates are allowed here; no qty needed.)
4. Each rendered item has a "Bought" (remove) button.
5. Show the running total in naira.
6. On load, restore the list from `localStorage` .

Constraints: Use `JSON.stringify` / `JSON.parse` ; guard against `null` on first load; run from Live Server.

Time allowance: 15 minutes.

Success criteria:

- Adding "Rice 5000" and "Beans 3000" shows total `₦8,000` .
- Refreshing keeps the list and total.
- Removing an item updates the total.
- Empty list shows a friendly message.

Hints:

- Hint 1: read the input with `document.querySelector("#item-price").value` and convert with `Number(...)`.
- Hint 2: save under `"marketList"`, not `"cart"`, to avoid clashing with the demo.
- Hint 3: clear the input after adding so the user is not confused.

Instructor answer key:

```
let list = [];
const listEl = document.querySelector("#list");
const totalEl = document.querySelector("#total");
const nameInput = document.querySelector("#item-name");
const priceInput = document.querySelector("#item-price");

function save() {
  localStorage.setItem("marketList", JSON.stringify(list));
}
function load() {
  const saved = localStorage.getItem("marketList");
  list = saved ? JSON.parse(saved) : [];
}
function render() {
  listEl.innerHTML = "";
  if (list.length === 0) {
    listEl.textContent = "Your list is empty. Add an item above.";
    totalEl.textContent = "₦0";
    return;
  }
  let total = 0;
  list.forEach((item, index) => {
    total += item.price;
    const row = document.createElement("div");
    row.innerHTML = `${item.name} — ₦${item.price.toLocaleString()} `;
    const btn = document.createElement("button");
    btn.textContent = "Bought";
    btn.addEventListener("click", () => {
      list.splice(index, 1);
      save();
      render();
    });
    row.appendChild(btn);
    listEl.appendChild(row);
  });
  totalEl.textContent = `₦${total.toLocaleString()}`;
}
document.querySelector("#add-item").addEventListener("click", () => {
  const name = nameInput.value.trim();
  const price = Number(priceInput.value);
  if (!name || !price || price <= 0) {
    alert("Please enter a valid item name and price.");
    return;
  }
  list.push({ name, price });
  nameInput.value = "";
  priceInput.value = "";
  save();
  render();
});

load();
render();
```

Marking guidance (out of 10): correct add + input reading (3); correct remove via `splice` (2); running total correct (2); persistence with stringify/parse + null guard (2); clean UI, no errors (1).

J. REAL-WORLD CASE STUDY

Situation. *Ada's Cosmetics* sells skincare products online. Customers browse, add items to a cart, then continue browsing other pages before checking out. Early on, Ada's site had a problem: when a customer clicked to another page, the cart emptied. Customers got frustrated and abandoned orders.

Relevant facts.

- The site has multiple pages (Shop, About, Contact).
- The cart was held in a plain JavaScript variable — which resets every time a new page loads.
- The shop has no server-side accounts; it just needs the cart to survive page changes and refreshes.

Decision point. How should the cart be stored so it survives navigation between pages?

Possible approaches.

1. Keep the cart in a variable (breaks on navigation).
2. Save the cart in `localStorage`, and load it on every page.
3. Store the cart on a server with a customer account (requires backend and login).

Recommended solution. Approach 2 for this stage. On every page, the script runs `loadCart()` from `localStorage` and `renderCart()`. The cart appears consistently across Shop, About and Contact. Checkout can later move the data to a server.

Trade-offs.

- Approach 1 is the easiest to write but fails the moment the customer leaves the page — a real business loss.
- Approach 2 fixes persistence with a few lines and no backend cost; the limitation is that the cart is per-device and per-browser (a customer who switches from phone to laptop loses it).
- Approach 3 is the professional long-term answer (carts tied to accounts) but requires a server, a database and login — far beyond this course.

Professional connection. The "load state on every page" pattern is standard in real front-end work. Junior developers are frequently asked to make a widget (cart, filters, preferences) remember its state — and `localStorage` is the first tool they reach for in small projects.

K. COMMON MISTAKES AND TROUBLESHOOTING

K.1 Cart does not persist after refresh

Symptoms: Cart empties on refresh. **Likely causes:** (a) `saveCart()` never called after changes. (b) `JSON.stringify` missing on save. (c) `loadCart()` not called on load. (Confirmed by inspecting the `cart` key in DevTools → Application → Local Storage.) **Investigate:** Open Application tab; is there a `cart` key? Is its value valid JSON? **Fix:** Ensure every change calls `saveCart()`, and the page start calls `loadCart()` + `renderCart()`. **Confirm:** Cart survives refresh; key is present. **Prevent:** A single choke-point function that saves and renders after every mutation.

K.2 `cart.forEach is not a function`

Symptoms: Red TypeError. **Likely cause:** The cart was loaded as a string (missing `JSON.parse`). (Confirmed by `console.log(typeof cart)` → `"string"`.) **Fix:** `cart = saved ? JSON.parse(saved) : [];`. **Prevent:** Always parse on load, stringify on save.

K.3 Duplicate lines for the same product

Symptoms: Adding the slippers twice shows two separate lines. **Likely cause:** `addToCart` always `push`es; it never checks with `find`. **Fix:** Add the `find` duplicate check and increment `qty`. **Prevent:** Design `addToCart` with the find-or-create pattern from the start.

K.4 Total is wrong or `NaN`

Symptoms: Total shows `NaN` or too much/too little. **Likely causes:** (a) `price` stored as a string. (b) Multiplying by a missing `qty`. (c) Using `==` / `+` inconsistently. **Investigate:** `console.log(cart)` and `typeof item.price`. **Fix:** Ensure numbers; ensure `qty` always set. **Prevent:** Create cart items with a numeric `price` and a `qty` of at least 1.

K.5 `Cannot read properties of null`

Symptoms: Page crashes on first ever load. **Likely cause:** `getItem` returned `null` and code tried to iterate it. (Confirmed on a fresh browser with empty storage.) **Fix:** Guard with `if (saved)` before parsing. **Prevent:** Always handle the empty-storage case.

K.6 Item added immediately on page load

Symptoms: Every product jumps into the cart as soon as the page renders. **Likely cause:** `addEventListener("click", addToCart(product))` — the product is called at render time. **Fix:** `addEventListener("click", () => addToCart(product))`. **Prevent:** Never call the handler while attaching it.

Distinguish likely vs confirmed: The Application tab (is `cart` present and valid JSON?), the `typeof` of the cart, and the Console error are the evidence that confirms the cause. Never guess—inspect.

L. SESSION ASSIGNMENT

Assignment title: "A Working Mini-Shop Cart"

Purpose: Combine rendering, events, arrays/objects and persistence into one realistic feature.

Realistic scenario: A small online store needs a cart that persists across refreshes, allows removing items, prevents duplicates and shows a running total.

Detailed instructions:

1. Start from your Session 7 shop page (products fetched from `products.json`).
2. Add the cart area (status, items container, total) and the cart functions.
3. Each product card must have an "Add to Cart" button.
4. Requirements:
   - Adding the same product twice increases its quantity (one line).
   - A "Remove" button next to each cart line.
   - A "Clear Cart" button.
   - The cart persists after refresh.
   - The total is shown in naira with `toLocaleString()`.
5. Test the full cycle: add, add again, remove, refresh, clear.

Required deliverables: `shop.html`, `js/shop.js`, `products.json`, and a short test log in the workbook (what you did, what you saw, whether persistence worked).

Submission format: Files in the class folder; test log in the workbook.

Deadline: Before Session 9.

Marking rubric (20 marks):

Criterion	Marks	Strong answer
Cart data structure correct	3	Array of objects with id/name/price/qty
Add with duplicate handling	3	Increasing qty, not duplicating
Remove works	3	Line removed, total updates
Total correct in naira	3	`price × qty` summed, formatted
Persistence (stringify/parse + null guard)	4	Survives refresh; no crash on first load
Buttons wired correctly (closure)	2	No add-on-load bug
Clean UI and no console errors	2	Friendly empty message

Common mistakes to avoid: missing `JSON.parse`; missing null guard; calling the handler immediately; storing prices as strings.

Instructor reference solution: Use the `cart.js` from Section G.2, adapted so `products` come from the fetch in Session 7 and the cart area is added to the shop page. The cart functions are identical; only the product source changes.

M. KNOWLEDGE CHECK

Q1. Short answer. Why do we store the cart as an array of objects rather than a single number or string?

Q2. Multiple choice. Which correctly saves the cart array? a) `localStorage.setItem("cart", cart);` b) `localStorage.setItem("cart", JSON.stringify(cart));` c) `localStorage.save("cart", cart);` d) `localStorage.setItem(JSON.stringify(cart), "cart");`

Q3. Explain in your own words. Why is the `if (saved)` check important in `loadCart`?

Q4. Practical identification. Given `cart = [{id:1, price:5000, qty:2}, {id:2, price:3000, qty:1}]`, what is `getCartTotal()`?

Q5. Error detection. This button adds on page load: `btn.addEventListener("click", addToCart(product));`. Fix it and explain.

Q6. Scenario. A customer adds items in Chrome on the shop's laptop and then opens the site on their phone — the cart is empty. Explain why, and what would be required to share the cart across devices.

INSTRUCTOR ANSWER KEY (Q1–Q6)

A1. Because we need to remember multiple products and the quantity of each; a single value cannot represent that. **A2.** (b). **A3.** On the first visit `getItem` returns `null`; parsing `null` gives `null`, and iterating it would crash. The check lets us substitute an empty array. **A4.** `5000*2 + 3000*1 = 13000` → `₦13,000`. **A5.** Fix: `btn.addEventListener("click", () => addToCart(product));`. Without the arrow wrapper, `addToCart(product)` runs immediately during rendering and passes its result as the handler. **A6.** `localStorage` is per-browser and per-device. To share a cart across devices you would need a server to store the cart against a customer account (login).

N. REFLECTION AND RECAP

Plain-English summary. Today products became buyable. You built a cart as an array of objects with quantities, added an `addToCart` that avoids duplicates, a `removeFromCart` using `filter`, and a `getCartTotal` that sums `price × qty`. You wired "Add to Cart" buttons with closures, and — the big idea — you persisted the cart to `localStorage` so it survives a refresh, guarding against the empty first load.

Five important takeaways:

1. The cart is an array of objects with a `qty`.
2. `find` prevents duplicates; `filter` removes items.
3. Total = sum of `price × qty`.
4. Save with `JSON.stringify`; load with `JSON.parse`; guard against `null`.
5. Wire buttons with `() => addToCart(product)`.

Questions to help you recognise what you can now do:

- Can I build a cart that survives a refresh?
- Can I explain why `JSON.stringify` is needed for storage?
- Can I fix the add-on-load bug?
- Can I calculate and display a running total?

Connection to the next session: You now have products, a filter-to-come and a working cart. Next session ties it all together into a **multi-page, responsive shop** — a search/filter feature, a consistent navigation bar, and a layout that works on a phone. This is the dress rehearsal for your capstone.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner's cart persists correctly across refreshes, on the first serious attempt.
- Learner inspects `localStorage` in the Application tab to verify saving.
- Learner handles the empty-cart case without being told.
- Learner explains the difference between `find` and `filter`.
- Learner's total is always correct, including for multiple quantities.

Common misconceptions:

- "`localStorage` is shared between users/devices." → It is local to one browser/device.
- "I can store the array directly." → Only strings; use `JSON.stringify`.
- "The first load will just work." → Must guard against `null`.
- "Attaching the handler and passing the product is fine." → Needs the arrow closure.

Suggested remedial exercises:

1. Persistence drill: given a broken `loadCart` without `JSON.parse`, ask learners to find and fix it and prove it with the Application tab.
2. Manual-total drill: five cart arrays on paper; compute the totals by hand, then check with code.
3. Duplicate drill: give an `addToCart` that always pushes; ask learners to add the duplicate check.

Differentiation:

- **Level 1 (supported):** provide the cart functions with the `find` and `filter` lines pre-written; learners wire the buttons and rendering.
- **Level 2 (standard):** the exercise as written.
- **Level 3 (challenge):** implement quantity controls (+ / −) that reduce `qty` before removing, and add a small product search box on the shop page.

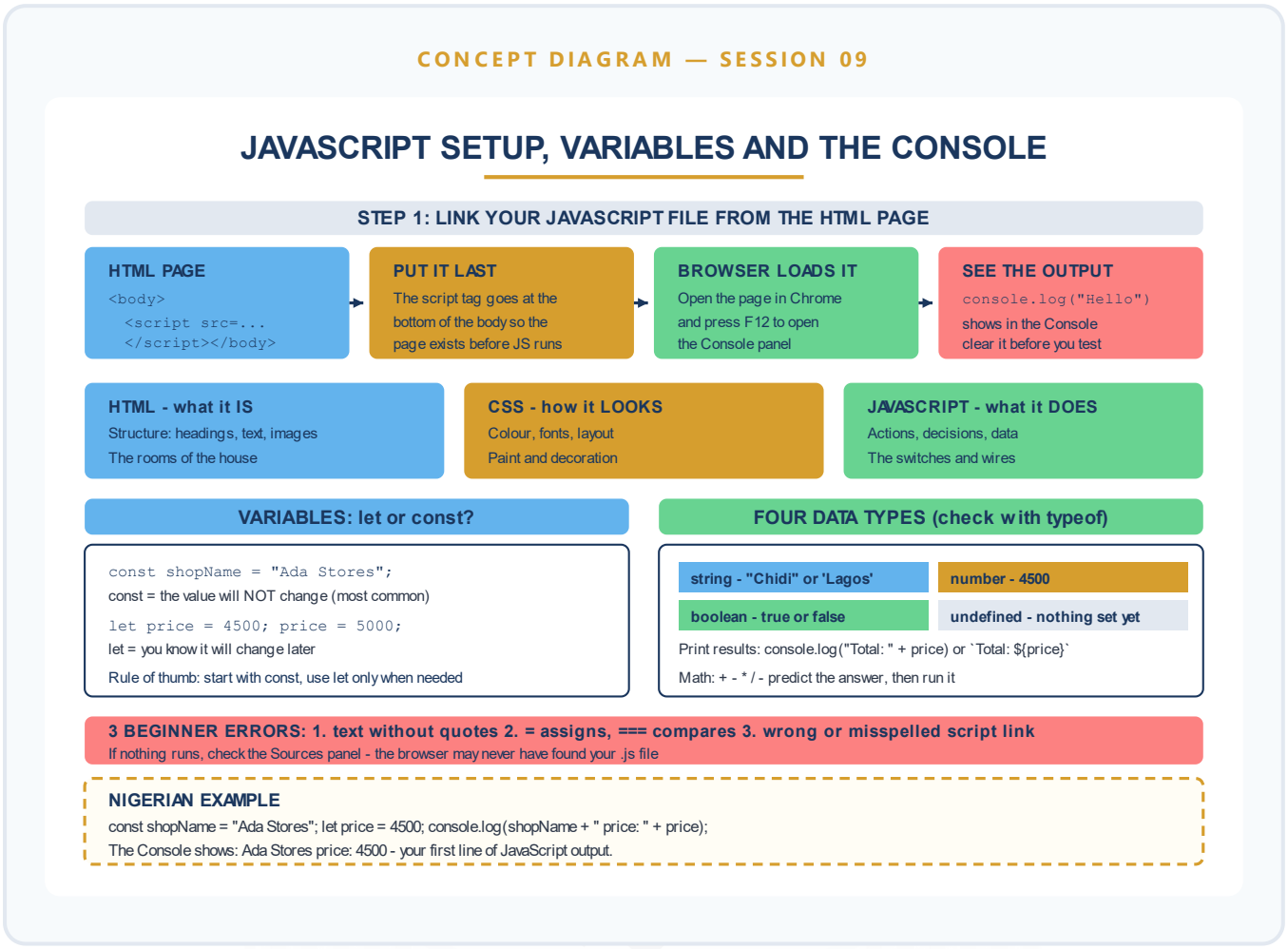
Timing guidance: The duplicate-check and the `null` guard are the two hardest concepts; slow down there. If the class is fast, introduce `reduce` for the total and a simple checkout summary.

END OF SESSION 8

Next: Session 9 — Multi-page Integration and Responsive Design.

SESSION 09 OF 12

MULTI-PAGE INTEGRATION AND RESPONSIVE DESIGN



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 9 — MULTI-PAGE INTEGRATION AND RESPONSIVE DESIGN

A. SESSION IDENTIFICATION

Item	Detail
Session number	9 of 12
Session title	Multi-page Integration and Responsive Design
Week / Module	Week 3 — Module 3: Data and App
Recommended duration	105 minutes

Item	Detail
Difficulty level	Intermediate beginner (this is the first "build a real site" session)
Prerequisite knowledge	Sessions 1–8. Learners must be able to: declare variables and functions; use arrays, objects and loops; select and change the DOM; handle the `click`, `submit` and `input` events; render a list of products from data; and save/load a cart in `localStorage`. They must also remember the Web Design basics: the `<meta name="viewport">` tag, flexbox/grid and media queries.
Connection to previous work	In Session 7 learners rendered products from data. In Session 8 they built a cart that survives a page refresh using `localStorage`. But so far everything lived on **one page**. A real shop has several pages, and the cart must not disappear when the customer clicks "About". Today we join those pieces into a small multi-page site, add the capstone's headline feature — a live product **search/filter** — and make sure it all works on a phone.
Preparation before class	Academy PCs on. Chrome and VS Code installed. Each learner has their `webdev-practice` folder and their Session 8 cart work. Instructor prepares the complete `demo-week3-s9/` example (given in Section F) and has Live Server installed in VS Code. Projector shows VS Code plus Chrome. A phone (or Chrome's device toolbar) is available to demonstrate mobile widths.
Capstone link	The product search/filter and the consistent, cart-aware navigation built today are **directly required** by the capstone shop (Home/Shop, About, Contact, product filter, persisted cart, responsive).

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Structure a small multi-page website** (`index.html`, `about.html`, `contact.html`) that shares one CSS file and one JavaScript file, using **relative paths**.
2. **Write a consistent header/nav/footer** that appears on every page, and link between pages with `<a href="about.html">`.
3. **Explain what "relative" versus "absolute" paths are**, and why relative paths are required for a site that will be deployed later.
4. **Keep the cart consistent across pages** by loading the cart from `localStorage` and redrawing the cart badge on every page load.
5. **Build a live product filter** using the `input` event, `String.prototype.toLowerCase()` and `String.prototype.includes()`, that updates the rendered products as the user types.
6. **Recap and apply responsive design** — the `viewport` meta tag, CSS Grid/Flexbox layouts, and media queries — and explain how they interact with content that JavaScript renders *after* the page loads.
7. **Test a site at 360px, 768px and 1024px** using Chrome DevTools device toolbar, and identify and fix at least three common mobile problems (overflow, oversized images, navigation that does not collapse).

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). Pairing is fine with the swap method from Session 1.
- Instructor computer connected to a projector or large screen.
- A smartphone is helpful for the testimonial moment at the end, but not required (DevTools device toolbar does the job offline).

Software

- Google Chrome (for DevTools device toolbar — the phone/tablet icon).
- Visual Studio Code.

- The VS Code **Live Server** extension (installed in Session 7). Live Server serves the site over `http://127.0.0.1:5500`, which we need so that `localStorage` and page navigation behave like a real hosted site.
- Windows 10 or 11.

Accounts

- None required this session.

Sample files

- Instructor's finished `demo-week3-s9/` folder (all files given in Section F).
- Optional: a small `images/` folder with 2–3 sample photos for the "large image" discussion. Learners without photos can use the placeholder images generated by the demo code.

Worksheets

- Worksheet 9.1 — "Map My Site" (blank boxes for the three pages; learners draw the shared header/footer).
- Worksheet 9.2 — "Break It on Mobile" (a checklist of widths and problems to test).

Tools

- Chrome DevTools: Console, Elements and the device toolbar (`Ctrl + Shift + M`).
- W3C validator not needed today (that is Session 10).

Internet

- Not required, but if available the instructor may show one real Nigerian online shop and resize it in DevTools to compare.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open a real online shop (any Nigerian fashion or electronics store). Set the browser window narrow, roughly the width of a phone. Scroll and point out: the menu button, the two-column product grid, and the prices that still fit the screen.

What the instructor says:

> *"Look at this shop on a narrow screen. Does it look like a website designed for a laptop squeezed into a phone? No — the layout changed. The grid became two columns, the menu hid behind a button, the text still fits. That is called **responsive design**, and you learned the tools for it in the Web Design course.*
>
> *Now here is the part nobody tells beginners: this shop has an 'Add to cart' button on the product page, a basket number at the top, and when you click 'About us', the basket number **stays**. How? The pages are separate files! So how does the basket follow you?*
>
> *That is today's job. We turn your one-page product app into a **real multi-page site**, add a **search box** that filters products as you type, and make sure the whole thing survives on a phone."*

Expected learner response: Learners may say "the server remembers the cart" or "they use a database". Accept all answers, then reveal: the browser itself remembers, using `localStorage`, and each page redraws the badge from that saved data. This is the key idea of the session.

Then demonstrate the hook: In DevTools, press `Ctrl + Shift + M` to open the device toolbar. Choose "iPhone SE" or set width to 360px. Then drag the ruler to 768px and 1024px. Say:

> *"Three widths are our friends today: 360px (small phone), 768px (tablet), 1024px (small laptop). If your site looks good at all three, it will look good almost anywhere."*

E. PREREQUISITE REVIEW (7 minutes)

Run this as a quick oral round-robin with the questions on the projector. Call on different learners.

Question 1: "In Session 7, how did we turn `data/products.json` into visible product cards on the page?"

- Expected answer: we `fetch`ed the data, converted it with `.json()`, then looped over the array and created/inserted elements (or built an HTML string and set `innerHTML`).
- Common gap: learners forget that `fetch` needs Live Server (it fails on `file://`). Remind them: today we will use a plain JavaScript array so the session works even offline, but the loop-and-render idea is exactly the same.

Question 2: "In Session 8, where did the cart live so it survived a refresh, and how did we store it?"

- Expected answer: in `localStorage`, stored as a string using `JSON.stringify(...)`, and read back with `JSON.parse(localStorage.getItem(...))`.
- Common gap: storing an array directly without `JSON.stringify`. `localStorage` can only store text. Remind them.

Question 3: "What does `localStorage.setItem("cart", data)` do differently from a normal variable?"

- Expected answer: a normal variable dies when the page is closed or refreshed; `localStorage` keeps the value in the browser until it is cleared.
- Common gap: thinking `localStorage` is shared between different websites. Clarify: it is private to the site (the same origin), and persists across pages of that site — which is exactly what we need today.

Question 4: "From Web Design — what tag tells a phone not to zoom out and shrink your whole page?"

- Expected answer: `<meta name="viewport" content="width=device-width, initial-scale=1.0">`.
- Common gap: learners omit it entirely, then wonder why their site looks tiny with big white margins on a phone. This is the single most common responsive bug.

Bridge to today (say this):

> *"You can render products, you can save a cart, and you know the tools of responsive design. Today we combine all three — one shared CSS file, one shared cart, three pages, a search filter, and a mobile check. This is exactly the capstone."*

If more than a third of the class cannot answer Question 2, spend five extra minutes re-running the small `localStorage` demo from Session 8 before continuing.

F. FULL LECTURE NOTES

F.1 From one page to many pages

The idea. A **multi-page website** is a collection of separate HTML files in one folder, where each file is a full page and they link to each other with the `<a>` tag. Our site will have three pages:

File	Page	Purpose
`index.html`	Home / Shop	The main page: hero, search box, product grid, cart summary. `index.html` is special — it is the page the browser opens by default when someone visits your site folder, and later it will be your GitHub Pages home.
`about.html`	About	Tells the shop's story (Adaeze's business).
`contact.html`	Contact	A contact form and business details.

Plain-English definition. A website is not one big file; it is a **folder of files** that reference each other. This is exactly like a house: `index.html` is the sitting room (where visitors arrive), `about.html` is the family room, `contact.html` is the office. The front door always opens into the sitting room.

Why we split pages. Imagine one giant page where "About" and "Contact" sat below 200 products. The customer would scroll forever. Splitting keeps each page short and focused, and each page loads faster because it is smaller.

How pages link to each other. With the **anchor** tag:

```
<a href="about.html">About Us</a>
<a href="contact.html">Contact</a>
```

```
<a href="index.html">Home</a>
```

- `<a>` is the *anchor* / link element.
- `href` means **h**ypertext **ref**erence — the destination.
- `about.html` is the **relative path**: a file in the *same folder* as the page doing the linking.

Real-life comparison. Relative paths are like giving directions from where you are standing: "go to the next door on this same street." Absolute paths are like a full postal address from outer space: `C:\Users\Ada\Desktop\shop\about.html`. Relative is what professionals use because it keeps working when the folder is moved (and moving is exactly what happens when you deploy).

Worked example — the same navigation on every page. Every page starts and ends with the *same* header and footer. Copying the same markup into all three files is how beginner multi-page sites are built. (More advanced tools can "include" a shared file automatically, but that needs a build step or a server; we keep it simple and copy carefully.)

F.2 The shared header, nav and footer

The idea. A professional site repeats the same header (logo + navigation) and footer on every page so the visitor never gets lost. We copy the identical block into each HTML file.

Why consistency matters. If the nav is different on each page, the customer gets confused and loses trust. A study of your own experience will confirm it: when a shop's menu suddenly changes, you wonder if you are still in the same website. Consistency is not decoration — it is **usability** and **professionalism**. It is also the reason the cart badge must appear in the header on *every* page.

Full working header/nav markup (put this at the top of the `<body>` of every page):

```
<header class="site-header">
  <div class="header-inner">
    <a class="brand" href="index.html">Adaeze Fashion</a>

    <input type="checkbox" id="navToggle" class="nav-toggle">
    <label for="navToggle" class="nav-toggle-label">Menu</label>

    <nav class="site-nav" aria-label="Main navigation">
      <a href="index.html" class="active">Shop</a>
      <a href="about.html">About</a>
      <a href="contact.html">Contact</a>
      <a href="index.html" class="cart-link">Cart <span id="cartCount" class="cart-count">0</span></a>
    </nav>
  </div>
</header>
```

Line by line:

- `<a class="brand" href="index.html">` — the shop name, always linking home.
- `<input type="checkbox" id="navToggle">` — a hidden checkbox. We use it to make a **CSS-only mobile menu**: on small screens the checkbox is the switch that opens the menu, so we do not even need JavaScript for the menu itself.
- `<label for="navToggle">` — the visible "Menu" button. Clicking the label toggles the checkbox (this is a built-in browser behaviour: a label controls the input it is `for`). On desktop we hide both the checkbox and the "Menu" label.
- `<nav ... aria-label="Main navigation">` — the navigation block. The `aria-label` is a small accessibility help: a screen-reader will announce "Main navigation".
- Each `<a>` goes to a relative `.html` file.
- `class="active"` marks the current page. On `index.html` the Shop link gets `active`; on `about.html` the About link gets it, and so on.
- The cart link contains `<span id="cartCount">0</span>` — the **cart badge**. JavaScript will fill this number from `localStorage` on every page.

The shared footer (put this just before the closing `</body>`, above the `<script>` tags):

```
<footer class="site-footer">
  <p>© 2026 Adaeze Fashion, Port Harcourt. Built by CEA students.</p>
  <p>WhatsApp: 0803 000 0000</p>
</footer>
```

Common misconception. Beginners change the nav on one page "just for now" and forget. Rule: if you edit the header in one file, edit it in **all** files immediately. A quick trick: keep the header block in a text file called `_header.txt` and paste from it.

Limitation. Copying markup means a change must be made in every file. That is acceptable for three pages. With twenty pages, professionals use a server-side include or a framework; that is a topic for a later course, not today.

F.3 Keeping the cart consistent across pages

The idea. Each page is a fresh page load. When `about.html` loads, the product page's variables are long gone from memory. The only thing that survives is what we put in `localStorage`. So the plan is:

1. On **every** page, read the cart from `localStorage`.
2. Add up the quantities.
3. Write that number into the badge `<span id="cartCount">`.

Why this works. `localStorage` belongs to the whole **origin** (the same site/folder), not to one page. So `index.html`, `about.html` and `contact.html` all see the same saved cart. This is the magic that makes the cart "follow" the customer.

Shared cart code (`js/cart.js`):

```
// cart.js — shared by every page
const CART_KEY = "adaezeCart";

// Read the cart array from localStorage (returns [] if nothing is saved).
function getCart() {
  const saved = localStorage.getItem(CART_KEY);
  if (!saved) {
    return [];
  }
  try {
    return JSON.parse(saved);
  } catch (error) {
    console.warn("Cart data was corrupt and has been reset.", error);
    return [];
  }
}

// Save the cart array as text.
function saveCart(cart) {
  localStorage.setItem(CART_KEY, JSON.stringify(cart));
}

// The total number of items (add up the qty of every line).
function cartCount() {
  return getCart().reduce(function (total, item) {
    return total + item.qty;
  }, 0);
}

// Put the current count into the header badge on this page.
function updateCartBadge() {
  const badge = document.getElementById("cartCount");
  if (!badge) {
    return;   // this page has no badge — do nothing
  }
  badge.textContent = cartCount();
}

// Add one of a product id to the cart, then save and refresh the badge.
function addToCart(id) {
  const cart = getCart();
  const existing = cart.find(function (item) {
    return item.id === id;
  });
  if (existing) {
    existing.qty = existing.qty + 1;
  } else {
    cart.push({ id: id, qty: 1 });
  }
  saveCart(cart);
```

```
  updateCartBadge();
}
```

Line by line the important parts:

- `CART_KEY` is the *name* under which we save. It must be **exactly the same** on every page, or the pages will not find each other's cart. A typo here is a classic bug (see Section K).
- `getCart()` wraps `JSON.parse` in `try...catch`. If the saved text is corrupt, we reset to an empty array instead of crashing. `try...catch` means "attempt this; if it throws an error, run the catch block."
- `reduce` walks the array and adds up `item.qty`. We start from `0`.
- `updateCartBadge()` guards against a missing badge (`if (!badge) return;`) so pages without a badge do not crash.
- `addToCart` either increases the quantity of an existing line or pushes a new line.

The key line for today's outcome: every page's own script calls `updateCartBadge()` as soon as the page loads. That is what makes the number appear everywhere.

Worked example of the flow.

1. On `index.html` the customer clicks "Add to cart" for the Ankara Dress. `addToCart(1)` runs, `localStorage` now holds `[{"id":1,"qty":1}]`, and the badge shows `1`.
2. The customer clicks the "About" link. The browser loads `about.html` fresh. The badge would show `0` — *unless* `about.html` also loads `cart.js` and calls `updateCartBadge()`.
3. Because it does, `getCart()` reads the same saved data and the badge shows `1` again. The cart "followed" the customer.

Misconception. "The cart is in memory, so it stays." No — memory is wiped on every navigation. Only `localStorage` persists. Repeat this until it is clear.

F.4 The live product search/filter (the capstone feature)

The idea. A **search box** is an `<input>`. As the customer types, we filter the product array so only matching products are shown. This is the single most impressive feature in the capstone and it is surprisingly short.

Plain-English definition. A **filter** is a rule that keeps only some items from a list. A **search** filters by matching text.

Why we use the `input` event (not `click` or `keyup`). `input` fires after the value of a text field changes for *any* reason — typing, pasting, cutting, even using the phone's voice input. `keyup` only fires on keys and misses paste-with-mouse. So `input` is the correct, modern choice.

The three string tools we need:

Tool	What it does	Example
`.toLowerCase()`	Makes text all lowercase	`"Ankara".toLowerCase()` → `"ankara"`
`.includes(text)`	Returns `true` if the text appears somewhere inside	`"ankara dress".includes("ankara")` → `true`
`.trim()`	Removes spaces from the start and end	`" shoes ".trim()` → `"shoes"`

Why `toLowerCase` on both sides. Computers compare letters exactly. `"Ankara"` and `"ankara"` are *different* strings because `A` and `a` are different characters. So if we compare the raw strings, a customer typing "ankara" would find nothing. We lowercase **both** the search term and the product name so the comparison ignores capitalisation. This is one of the most common real search bugs, and we deliberately plant it in Session 10's broken example.

The filter code:

```
function applyFilter() {
  const searchBox = document.getElementById("searchBox");
  const term = searchBox.value.toLowerCase().trim();

  const filtered = products.filter(function (product) {
    return product.name.toLowerCase().includes(term);
  });

  renderProducts(filtered);
}
```

Line by line:

- `searchBox.value` — the text the customer has typed.
- `.toLowerCase().trim()` — normalise it: lowercase and no stray spaces.
- `products.filter(...)` — `Array.prototype.filter` calls the function once per product and keeps only the ones where the function returns `true`.
- `product.name.toLowerCase().includes(term)` — the test: does this product's lowercased name contain the search term? If `term` is empty (`""`), every string `.includes("")` is `true`, so **all** products show — which is exactly the behaviour we want (empty box = show everything).
- `renderProducts(filtered)` — redraw the grid with only the surviving products.

Wiring it up with the `input` event:

```
const searchBox = document.getElementById("searchBox");
searchBox.addEventListener("input", applyFilter);
```

Offering a nice empty state. If nothing matches, the grid would be blank and the customer might think the page is broken. We handle it inside `renderProducts`:

```
if (list.length === 0) {
  grid.innerHTML = "<p>No products match your search. Try another word.</p>";
  return;
}
```

Worked example.

- Products: Ankara Dress, Aso Oke Gele, Beaded Necklace.
- Customer types `d`. `filtered` keeps any name containing "d": "Ankara Dress" (has a 'd') and "Beaded Necklace" (has 'd') — two shown.
- Customer types `gele`. Only "Aso Oke Gele" survives — one shown.
- Customer types `xyz`. Nothing survives — the empty-state message shows.

Misconception. "filter changes the original array." No — `.filter` returns a **new** array and leaves `products` untouched. That is why, when the customer clears the box, all products come back.

Limitation. Our search matches the product **name** only. A real shop might also match the category or a description. As a challenge, learners can extend the test to `product.category` too.

F.5 Responsive design recap — and how it interacts with JavaScript

What "responsive" means. A **responsive** website changes its layout to suit the screen size: one column on a phone, two on a tablet, three on a laptop. The layout "responds" to the device.

The four tools (from Web Design):

1. **The viewport meta tag** — always in the `<head>`:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Without it, a phone pretends to be a 980px-wide desktop and shrinks your page into a tiny strip.

2. **Flexible units** — use `%`, `fr`, `rem` and `vw` instead of fixed pixel widths like `width: 1100px`. A fixed width causes horizontal scrolling on small screens — the number-one "overflow" bug.
3. **CSS Grid and Flexbox** — modern layout systems. We use Grid for the product cards and Flexbox for the header.

```
.product-grid {
  display: grid;
  grid-template-columns: 1fr;      /* one column by default (phones) */
  gap: 1rem;
}
```


4. **Media queries** — rules that apply only at certain widths:

```
@media (min-width: 600px) {
  .product-grid { grid-template-columns: repeat(2, 1fr); }   /* tablets */
}
@media (min-width: 900px) {
  .product-grid { grid-template-columns: repeat(3, 1fr); }   /* laptops */
}
```

How this interacts with JavaScript-rendered content — the important new lesson. The CSS rules do **not** care whether an element was typed in the HTML by hand or created later by JavaScript. A `.product-card` created by `document.createElement` in the browser is a normal element, and the same media queries style it.

> ***So:*** *as long as JavaScript gives each created element the correct* ***class names****, it will be responsive automatically. This is why, in our render function, we always set* `card.className = "product-card"` *— that one line is what connects our JavaScript to all the responsive CSS.*

The trap: if JavaScript sets the wrong class (or forgets it), the cards render as unstyled stacked blocks and the layout looks broken **only on some widths**. When a learner says "it works on my laptop but breaks on my phone", suspect a missing or misspelled class name before suspecting the CSS.

Where mobile problems really come from (previewed; Section K goes deeper):

- **Overflow:** a fixed-width element (or a very long unbroken word) wider than the screen pushes the page sideways.
- **Oversized images:** a 4000px photo displayed on a 360px screen. Fix with `img { max-width: 100%; height: auto; }` plus compressed image files.
- **Nav that does not collapse:** the horizontal menu runs off the screen. Fix with our checkbox menu and a media query.

F.6 Testing responsiveness with Chrome DevTools device toolbar

What it is. The **device toolbar** lets you simulate phone and tablet screens inside your desktop Chrome, without owning those devices.

How to open it: Press `Ctrl + Shift + M`, or click the small phone/tablet icon at the top-left of DevTools. Then choose a device from the dropdown (e.g. "iPhone SE"), or type a custom width.

The three widths we test every time:

Width	Represents	What to expect
360px	Small phone	One-column product grid; the "Menu" button appears; text is readable with no sideways scroll.
768px	Tablet	Two-column product grid; horizontal nav visible.
1024px	Small laptop	Three-column product grid; full desktop layout.

How to check for the classic overflow bug: in the device toolbar, look at the right edge. If a horizontal scrollbar appears, something is wider than the screen. A quick way to find the culprit: in the Console, type `document.documentElement.scrollWidth` — if it is larger than the width you set, something overflows. Then in the **Elements** panel, hover over elements and watch the page highlight to find the wide one. (A useful pro tip: add `* { outline: 1px solid red; }` temporarily to see every box.)

Why we test in the toolbar and not only by resizing the window. The toolbar also tells the page the correct device width, mimicking the `viewport` meta tag's real behaviour, and it shows touch events. It is the closest thing to a real phone without leaving your desk.

Important honesty note: the device toolbar is a *simulation*; it is not a real phone. It does not test real network speed or real device fonts perfectly. Later, in Session 11, we deploy and open the real URL on an actual phone as the final proof.

F.7 Summary of the session

- A multi-page site is a folder of files linked with `<a href="page.html">` using **relative paths**.
- Every page shares the **same header/nav/footer** and the **same CSS/JS files** for consistency.
- The cart follows the customer because every page loads the cart from `localStorage` and redraws the badge.

- A live **search filter** uses the `input` event, `toLowerCase()`, `includes()` and `Array.filter()`.
- **Responsive design** (viewport tag, flexible units, Grid/Flexbox, media queries) styles JavaScript-created elements too — as long as the class names are correct.
- Always test at **360px, 768px and 1024px**.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- A folder `demo-week3-s9/` containing: `index.html`, `about.html`, `contact.html`, `css/style.css`, `js/products.js`, `js/cart.js`, `js/app.js` (all contents in Appendix 9A).
- Open the whole folder in VS Code. Start Live Server on `index.html`.
- DevTools open; device toolbar ready (`Ctrl + Shift + M`).
- A marker and the whiteboard for drawing the "shared block copied into each file" idea.

G.2 Demonstration 1 — click through the pages and watch the cart follow (5 minutes)

Say: "Watch the small number next to 'Cart' in the top corner. I will add a product, then leave this page entirely, and you tell me what happens."

Do this on the projector:

1. On `index.html`, click **Add to cart** on one product. The badge changes from `0` to `1`.
2. Click the **About** link. The URL in the address bar changes to `.../about.html`. Point out the badge: it still shows `1`.
3. Click **Contact**, then back to **Shop**. The badge stays `1`.

Ask: "The About page never displayed a product. How did it know the cart had one item?"

- Answer: `about.html` loads `js/cart.js`, calls `updateCartBadge()`, which reads the same `localStorage` the shop page wrote to.

Then show the failure deliberately: In `about.html`, remove the `<script src="js/cart.js"></script>` line, save, refresh About.

- Expected result: the badge shows `0` even though the cart has an item.
- **Say:** "There it is — the cart looks lost. Put the line back and it returns. Every page needs the shared cart script."

G.3 Demonstration 2 — the live search filter (5 minutes)

Say: "Now the capstone feature. Type with me."

Do this:

1. On `index.html`, click into the search box.
2. Type `an`. Point out the grid shrinking to matching products *as you type*, with no button pressed.
3. Clear the box — all products return.
4. Type `gele` — only the matching product remains.
5. Type `zzz` — the "No products match your search" message appears.

Reasoning to explain aloud: the `input` event fires on every keystroke; `.filter` makes a new array; `renderProducts` redraws the grid; the empty term shows everything because every string includes `""`.

Inject an error on purpose: change `product.name.toLowerCase().includes(term)` to `product.name.includes(term)`. Save, refresh, type `ankara` (lowercase).

- Expected result: no products show, because `"Ankara Dress".includes("ankara")` is `false` — capital A versus small a.
- **Say:** "Silent bug! There is no red error. That is why we lowercase both sides. Never trust your eyes on capitalisation."
- Fix it back.

G.4 Demonstration 3 — the mobile check (5 minutes)

Say: "A shop that only works on a laptop is only half a shop. Let's break it and fix it."

Do this:

1. Press `Ctrl + Shift + M`. Set width to **360px**.
2. Point out the one-column grid and the **Menu** button.
3. Click **Menu** — the nav drops down (pure CSS, using the checkbox).
4. Widen to **768px** — two columns, menu gone, links visible.
5. Widen to **1024px** — three columns.
6. **Inject the bug:** in `css/style.css`, delete the `<meta name="viewport">` line from `index.html`'s head (or comment it out). Save, refresh at 360px.
   - Expected result: the page looks zoomed-out and tiny with big margins — as if the phone is pretending to be a desktop.
   - **Say:** "This is the number-one responsive mistake in Nigeria, because many learners copy old starter templates that lack this line. Always include the viewport tag."
   - Put it back.
7. **Inject a second bug:** add `width: 1200px;` to `.product-grid` in the CSS. Refresh at 360px.
   - Expected result: a horizontal scrollbar appears; content is cut off.
   - **Say:** "Fixed width equals overflow. Use `fr` columns and `max-width`, never a hard `width` on a layout container." Undo the change.

Questions to ask during the demo:

- "If I add a new product to the array, do I need to touch the CSS?" (No — the grid handles any number of cards.)
- "Why does the badge not update if I only change the CSS?" (The badge is data, driven by JavaScript and `localStorage`, not CSS.)
- "What are our three test widths?" (360, 768, 1024.)

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding each step, convert the learner's Session 8 product page into a **three-page site** with a shared header/cart badge and a **live search filter**, then verify it at three widths.

Starting materials: The learner's `webdev-practice` folder, containing at least a working product page and `localStorage` cart from Session 8 (or the instructor's starter given in Appendix 9A).

Procedure (instructor calls out each numbered step; learners follow on their machines; instructor walks the room):

1. **Create the structure.** Inside `webdev-practice`, create folders `css` and `js`. You should have:

```
webdev-practice/
├── index.html
├── about.html
├── contact.html
├── css/
│   └── style.css
└── js/
    ├── products.js
    ├── cart.js
    └── app.js
```

2. **Move all styling into `css/style.css`.** Cut the `<style>` block out of `index.html` and paste it into `style.css`, then link it in every page's `<head>`: `<link rel="stylesheet" href="css/style.css">`
3. **Add the shared header** (from Section F.2) to the top of the `<body>` of **all three** pages.
4. **Add the shared footer** just before the closing `</body>` of all three pages.
5. **Create `js/cart.js`** and type the code from Section F.3.
6. **Create `js/products.js`** with your product array. Use your own shop's products; each product needs `id`, `name`, `price` and `category`.
7. **Create `js/app.js`** with a `renderProducts(list)` function and an `applyFilter()` function (the code is in Appendix 9A). At the bottom, add:

```
document.addEventListener("DOMContentLoaded", function () {
  updateCartBadge();
  const searchBox = document.getElementById("searchBox");
  if (searchBox) {
    renderProducts(products);
    searchBox.addEventListener("input", applyFilter);
  }
});
```

8. **Link the scripts** at the bottom of `index.html` (before `</body>`):

```
<script src="js/products.js"></script>
<script src="js/cart.js"></script>
<script src="js/app.js"></script>
```

Link only `js/cart.js` and `js/app.js` on `about.html` and `contact.html` (no products needed).

9. **On `about.html` and `contact.html`,** mark the correct nav link with `class="active"` and give the pages real content (an About paragraph; a Contact form and WhatsApp number).
10. **Test the cart follows you.** Add two products on Shop, then visit About and Contact. The badge should read `2` everywhere.
11. **Test the search.** Type `a`, then a full product word, then nonsense. Watch the grid respond.
12. **Pause and predict.** Before opening the device toolbar, ask learners to write down: "How many columns will I see at 360px? At 768px? At 1024px?" Then test and compare.
13. **Responsive test.** Open Live Server, then DevTools device toolbar. Check 360px, 768px and 1024px. Look for horizontal scroll.

Expected result:

- All three pages share one identical header and footer.
- The cart badge shows the same number on every page.
- Typing in the search box filters the grid live; clearing it restores all products.
- At 360px: one column + Menu button; at 768px: two columns; at 1024px: three columns; no horizontal scrollbar.

Points to pause on and predict:

- After step 10: "What will the badge show on About if I added two items?" (2.)
- After step 11: "If the box is empty, how many products show?" (All of them, because every name includes `""`.)
- After step 12: the three column counts.

Troubleshooting while walking the room:

- Badge always `0` → the page is missing `<script src="js/cart.js"></script>`, or the `CART_KEY` differs from the shop page. Or the learner is running via `file://` instead of Live Server for some browsers' stricter rules.
- Search does nothing → the `input` listener was never attached, or the id is `search` in HTML but `searchBox` in JS. Confirm with `console.log(searchBox)`.
- Nothing renders at all → a red error in the Console (Session 10 will teach reading it). Common cause: `renderProducts(products)` runs before `products.js` is loaded — check the script order.
- Layout does not change with width → the viewport meta tag is missing, or the media queries are in the wrong file (they must be in `style.css`).
- Two columns never appear → a typo in `grid-template-columns`, or the media query is below a later rule that overrides it.

Completion checklist (learner ticks each):

- ☐ `index.html`, `about.html`, `contact.html` exist and link to each other
- ☐ All pages share the identical header/nav/footer
- ☐ One shared `css/style.css` linked on all pages with **relative** paths
- ☐ `js/cart.js` loaded on all pages; cart badge matches on all three
- ☐ `js/products.js` holds at least 4 products with `id`, `name`, `price`, `category`
- ☐ `renderProducts` draws the grid and sets `class="product-card"`
- ☐ Search box filters live using `input`, `toLowerCase`, `includes`

- [ ] Empty search shows all products; no matches shows the empty-state message
- [ ] Tested at 360px, 768px, 1024px with no horizontal scroll
- [ ] Nav collapses to a Menu button on the phone width

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Extend the shop with a **category filter bar** and an **active-link highlight** — a new task building on what you just did, not a copy of it.

Scenario: *Adaeze* sells dresses, gele (head-ties) and jewellery. Customers want to click a category to see only that type. She also wants the current page's menu link to be clearly highlighted so shoppers never feel lost.

Starting information: the three-page site you just built (`index.html` , `about.html` , `contact.html`), with the shared header, cart badge and search box.

Requirements:

1. Add a **category bar** to `index.html` above the product grid, containing buttons: **All**, **Dresses**, **Gele**, **Jewellery**. Give the bar the class `category-bar` .
2. Clicking a category button shows **only** products whose `category` matches; clicking **All** shows everything.
3. Highlight the **active category button** so the customer can see which one is selected (for example, keep a `selected` class on it).
4. On all three pages, ensure the current page's nav link has the `active` class so it is visually underlined.
5. A comment in the HTML, for each nav block, is **not** required — but each page's `active` class **must** be on the correct link.

Constraints:

- Use the `click` event and `Array.filter()` ; do **not** reload the page.
- The search box must **still work** together with the category filter (if a category is chosen and the customer types, both rules apply).
- No page reloads; no duplicated CSS (keep styles in `style.css`).
- Time allowance: **15 minutes.**

Success criteria:

- Clicking **Gele** shows only gele products.
- Clicking **All** restores every product.
- The chosen category button looks selected.
- Typing a search term while a category is active refines further (no crash, sensible results).
- Each page's own link is highlighted in the nav.
- No red errors in the Console.

Hints:

- Hint 1: keep one variable, e.g. `let activeCategory = "All";` . A category button sets it, then re-runs the filter.
- Hint 2: make one function `refreshView()` that applies **both** the category and the search term, and call it from both the button clicks and the search `input` .
- Hint 3: give each category button `data-category="Dresses"` and read it with `button.dataset.category` .

Instructor answer key (do not show learners):

Update `index.html` to add the bar (between the search box and the grid):

```
<div class="category-bar" id="categoryBar">
  <button class="category-btn selected" data-category="All">All</button>
  <button class="category-btn" data-category="Dresses">Dresses</button>
  <button class="category-btn" data-category="Gele">Gele</button>
  <button class="category-btn" data-category="Jewellery">Jewellery</button>
</div>
```

Add to `js/app.js` :

```
let activeCategory = "All";

// Apply BOTH the category and the search term together.
function refreshView() {
  const searchBox = document.getElementById("searchBox");
  const term = searchBox ? searchBox.value.toLowerCase().trim() : "";

  let result = products.filter(function (product) {
    const matchesCategory =
      activeCategory === "All" || product.category === activeCategory;
    const matchesSearch =
      product.name.toLowerCase().includes(term);
    return matchesCategory && matchesSearch;
  });

  renderProducts(result);
}

// Category buttons.
const categoryBar = document.getElementById("categoryBar");
if (categoryBar) {
  categoryBar.addEventListener("click", function (event) {
    const button = event.target.closest(".category-btn");
    if (!button) {
      return;
    }
    activeCategory = button.dataset.category;

    // Move the 'selected' highlight.
    const allButtons = categoryBar.querySelectorAll(".category-btn");
    allButtons.forEach(function (b) {
      b.classList.remove("selected");
    });
    button.classList.add("selected");

    refreshView();
  });
}

// Search box now calls refreshView (not applyFilter).
const searchBox = document.getElementById("searchBox");
if (searchBox) {
  searchBox.addEventListener("input", refreshView);
}
```

Add to `style.css`:

```
.category-bar {
  display: flex;
  gap: 0.5rem;
  flex-wrap: wrap;
  padding: 0 1rem 1rem;
  max-width: 1100px;
  margin: 0 auto;
}
.category-btn {
  border: 1px solid #8e44ad;
  background: #fff;
  color: #8e44ad;
  padding: 0.4rem 0.9rem;
  border-radius: 999px;
  cursor: pointer;
}
.category-btn.selected {
  background: #8e44ad;
  color: #fff;
}
```

Active nav link (already in base CSS):

```
.site-nav a.active {
  border-bottom: 2px solid #fff;
  font-weight: 700;
}
```

Marking guidance (out of 10): category bar present and styled (2); correct filtering logic (3); active category highlight toggles correctly (2); search and category combine without crashing (2); correct nav `active` link on each of the three pages (1).

J. REAL-WORLD CASE STUDY

Situation. *Tunde* runs an online phone-accessory shop called *Tunde Gadgets* in Port Harcourt. His site started as one long page: about 60 accessories, then the About text, then the contact form, all stacked. Customers complained they could not find a charger quickly, and some said they "lost" items because after ordering they scrolled away and could not find the cart again.

Relevant facts.

- The single page was hard to use on a phone: 60 products in one column, then a wall of text.
- There was no search box.
- The cart was a JavaScript variable that reset on refresh, so customers who refreshed lost everything.
- Tunde's page had no `<meta name="viewport">` tag, so on phones it appeared tiny with big white margins.

Decision point. Should Tunde (a) hire a developer to rebuild everything, (b) keep one page but add a search box, or (c) split the site into a proper multi-page shop with a persistent cart and responsive layout?

Possible approaches.

1. **Hire a full rebuild.** Expensive and slow; overkill for a small accessories shop.
2. **One page + search.** Cheaper, but the cart still resets and the About/Contact content still clutters the shop.
3. **Multi-page integration (today's skill).** `index.html` (shop), `about.html`, `contact.html`; a shared responsive header with a cart badge; a live search filter; the cart saved in `localStorage`.

Recommended solution. Approach 3. It uses exactly the skills from Sessions 4–9, costs Tunde nothing but time, and fixes all four complaints at once: search finds products fast, separate pages keep each page short, the badge shows the cart on every page, and the viewport tag makes it usable on phones.

Example solution (the essential pieces):

```
// Shared cart that every page can read.
const CART_KEY = "tundeCart";

function getCart() {
  return JSON.parse(localStorage.getItem(CART_KEY)) || [];
}

function updateCartBadge() {
  const badge = document.getElementById("cartCount");
  if (!badge) return;
  const count = getCart().reduce(function (n, item) { return n + item.qty; }, 0);
  badge.textContent = count;
}

// Live search on the shop page.
function refreshView() {
  const term = document.getElementById("searchBox").value.toLowerCase().trim();
  const shown = products.filter(function (p) {
    return p.name.toLowerCase().includes(term);
  });
  renderProducts(shown);
}
```

Trade-offs.

- Approach 1 gives the best result but is not affordable at this size and slows the business.
- Approach 2 is quick but does not fix the lost cart or the clutter.

- Approach 3 is free, matches Tunde's scale, and teaches reusable skills — but it means Tunde must keep the shared header/footer identical across pages when he adds products (a manual discipline). That is acceptable for a small shop and is exactly what the capstone asks for.

Professional connection. This is the daily work of a junior front-end developer: taking a growing single-page app and **integrating** it into a coherent, responsive, multi-page product. The skills of shared layout, `localStorage` persistence, live filtering and responsive testing are named on entry-level job adverts every day in Nigeria and abroad.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this table as the live troubleshooting guide. When a learner is stuck, ask them to check the relevant row before giving the answer.

K.1 The cart badge shows 0 on other pages

Symptoms: The badge updates on the shop but resets to `0` on About/Contact. **Likely causes:** (a) `js/cart.js` is not linked on that page. (b) `CART_KEY` is spelled differently in different files. (c) `updateCartBadge()` is never called on that page. **Investigate:** In the Console on the About page, type `localStorage.getItem("adaezeCart")`. If it returns the saved cart text but the badge is `0`, the page is not reading it. Check the `<script>` tags and the spelling of the key. **Corrective steps:** Add `<script src="js/cart.js"></script>` and `<script src="js/app.js"></script>` to the page; ensure `updateCartBadge()` runs on `DOMContentLoaded`; make the key identical everywhere. **Confirm:** Badge matches on all three pages. **Prevent:** Treat the header block and script tags as one unit — copy them identically.

K.2 Navigation links go to the wrong place (or nowhere)

Symptoms: Clicking "About" opens a blank page or a file-not-found error. **Likely causes:** (a) A wrong relative path, e.g. `href="pages/about.html"` when the file is in the main folder. (b) A misspelt file name, e.g. `abut.html`. (c) Case mismatch (`About.html` vs `about.html`). **Investigate:** Look at the file list in VS Code; compare spelling and folder exactly. **Corrective steps:** Fix the `href` to match the real file name and location. **Prevent:** Create the file first, then write the link; use VS Code's autocomplete inside `href="..."`.

K.3 The site works locally but breaks after deployment (absolute paths)

Symptoms: After uploading to GitHub Pages, images and links are broken. **Likely cause:** Absolute paths such as `href="C:\Users\Ada\shop\about.html"` or `src="/Users/Ada/img/x.jpg"`. A web server never has your `C:\` drive. **Investigate:** Search your files for `C:\` or a leading `/` in paths. **Corrective steps:** Replace every absolute path with a **relative** path: `about.html`, `css/style.css`, `images/x.jpg`. **Prevent:** Never type drive letters in HTML. (This connects to Session 11.)

K.4 The search filter returns nothing for a word you can see

Symptoms: You type a product name in lowercase and the grid empties. **Likely cause:** Missing `.toLowerCase()` on the product name (case-sensitive comparison). **Investigate:** `console.log(product.name, term)` inside the filter to see the raw values. **Corrective steps:** Lowercase **both** sides: `product.name.toLowerCase().includes(term)`. **Prevent:** Always normalise text before comparing.

K.5 The whole page scrolls sideways on a phone (overflow)

Symptoms: A horizontal scrollbar; content cut off at 360px. **Likely causes:** (a) A fixed `width: 1000px` on a layout container. (b) A large image without `max-width: 100%`. (c) A long unbroken word or URL. **Investigate:** In the Console, run `document.documentElement.scrollWidth` and compare with 360. In Elements, hover boxes to find the wide one. **Corrective steps:** Replace fixed widths with `max-width: 100%` or `fr` columns; add `img { max-width: 100%; height: auto; }`; add `overflow-wrap: break-word;` where needed. **Prevent:** Never use a fixed pixel width on the main layout container.

K.6 The site looks tiny on a phone with big white margins

Symptoms: On a real phone, the whole page is zoomed out into a narrow strip. **Likely cause:** The `<meta name="viewport">` tag is missing from the `<head>`. **Investigate:** View the page source and check the `<head>`. **Corrective steps:** Add `<meta name="viewport" content="width=device-width, initial-scale=1.0">`. **Prevent:** Start every new page from a template that already has it.

K.7 The nav runs off the screen / does not collapse

Symptoms: On a phone, menu links spill off the right edge. **Likely causes:** (a) No media query to switch the nav to a vertical menu. (b) The checkbox-menu CSS selector is wrong. **Investigate:** At 360px, inspect the `<nav>` and see whether `.site-nav { display:none }` is being applied. **Corrective steps:** Add the mobile media query and ensure the label/checkbox are siblings: `.nav-toggle:checked ~ .site-nav { display: flex; }`. **Prevent:** Test at 360px as soon as you build any nav.

Distinguish likely vs confirmed. In each case, confirm before fixing: read the Console, run a `console.log`, or inspect the element. Never guess-and-change blindly.

L. SESSION ASSIGNMENT

Assignment title: "My Three-Page Shop — Integrated and Responsive"

Purpose: Practise building a consistent multi-page site with a shared cart, a live search filter and a responsive layout — the exact structure of the capstone.

Realistic scenario: You are building the front end for a small Port Harcourt business (your own idea, or Adaeze Fashion). It must have three pages, a working product search and a cart that follows the customer between pages.

Detailed instructions:

1. Create a folder `assignment9/` with this structure:

```
assignment9/
├── index.html        (Home / Shop)
├── about.html
├── contact.html
├── css/style.css     (one shared file)
└── js/
    ├── products.js   (your array, at least 6 products)
    ├── cart.js       (the shared cart helper)
    └── app.js        (render + search + badge)
```

2. Every page must share the **same** header (with cart badge), the **same** nav and the **same** footer.
3. `index.html` shows: a product grid rendered by JavaScript, a search box that filters live, and an "Add to cart" button on each product.
4. `about.html` tells the business story; `contact.html` shows a form (name, email, message) and a WhatsApp number. Both pages must show the live cart badge.
5. Use **only relative paths**.
6. Test at **360px, 768px and 1024px**; the grid must go 1 → 2 → 3 columns and the nav must collapse to a Menu button on the phone width.

Required deliverables:

- The full `assignment9/` folder (all three pages + css/js).
- At least two screenshots: one at 360px and one at 1024px (paste into your workbook or submit as image files).
- A short note (3–5 lines) in your workbook describing one problem you hit and how you fixed it.

Submission format: Zip the folder or place it in the class shared folder; screenshots in the workbook. **Deadline:** Before Session 10.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
Three pages, correct structure	3	All files present; `index.html` is the shop; relative paths only
Consistent shared header/nav/footer	3	Visually identical on all pages; correct `active` link per page
Cart persists across pages	4	Badge shows the same count on Shop, About and Contact
Live search filter	4	Filters on `input`; case-insensitive; empty state handled; empty box shows all

Criterion	Marks	Description of a strong answer
Responsive layout	4	1/2/3 columns at 360/768/1024; viewport tag present; no horizontal scroll
Presentation / no errors	2	Clean look; no red Console errors

Common mistakes to avoid: absolute paths; missing `cart.js` on a page; forgetting `.toLowerCase()`; forgetting the viewport tag; a fixed-width container causing overflow.

Instructor reference solution: the complete code in **Appendix 9A** below is the reference (all files). A strong learner answer will match its structure, though the products and styling will differ.

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. What is the difference between a *relative* path and an *absolute* path, and which must we use for a deployed site?

Q2. Multiple choice. Which event should a search box use to filter as the user types, including when the user pastes with the mouse? a) `click` b) `keyup` c) `input` d) `change`

Q3. Explain in your own words. Why does the cart badge stay the same number when you move from `index.html` to `about.html`?

Q4. Practical identification. What does each piece of code do? a) `"Ankara".toLowerCase()` b) `"ankara dress".includes("ankara")` c) `products.filter(p => p.price > 5000)` d) `<meta name="viewport" content="width=device-width, initial-scale=1.0">`

Q5. Error detection. A learner's search only works when typing an exact match with capital letters. What single character-gobbling mistake is the most likely cause, and how do you fix it? Give the fixed line.

Q6. Scenario. In `app.js`, a learner writes the search listener, but the page throws `Cannot read properties of null (reading 'addEventListener')`. Name two likely causes and the quickest way to confirm which one it is.

INSTRUCTOR ANSWER KEY (Q1–Q6)

A1. A relative path points to a file based on the current page's folder (e.g. `about.html`, `css/style.css`); an absolute path gives a full location from the drive or server root (e.g. `C:\Users\Ada\site\about.html`). We must use **relative** paths, because a web server does not have your local drive; relative paths keep working when the site is moved to GitHub Pages.

A2. (c) `input` — it fires for typing and pasting. `keyup` misses paste-via-mouse; `change` fires only when the field loses focus.

A3. Because every page reads the cart from the same `localStorage` key (`adaezeCart`) and calls `updateCartBadge()` on load. `localStorage` belongs to the whole site (origin), not to a single page, so the value survives navigation; the badge is simply redrawn on each page.

A4. a) Returns the lowercase string `"ankara"`. b) Returns `true` because "ankara" appears inside the name. c) Returns a new array of only products priced above ₦5,000. d) Tells the phone browser to use the device's actual width and not to zoom out — the foundation of responsive design.

A5. The product name is compared without lowercasing, so `"Ankara Dress".includes("ankara")` is `false` (capital A). Fix by lowercasing both sides:

```
return product.name.toLowerCase().includes(term);
```

A6. Two likely causes: (1) the element's `id` in the HTML is different from the one used in `getElementById` (typo or wrong id), or (2) the `<script>` is placed in the `<head>` / above the element, so the element does not exist yet when the code runs. Quickest confirmation: in the Console type `document.getElementById("searchBox")` — if it returns `null`, the selector is wrong or runs too early; check the exact `id` in the Elements panel. (Fix by matching the id, or by putting the script at the bottom of the body / using `DOMContentLoaded`.)

N. REFLECTION AND RECAP

Plain-English summary. Today your one-page app grew into a real **three-page site**. You learned that a website is a folder of files linked with relative paths, that every page shares one CSS file and one JavaScript file for consistency, and that the cart follows the customer because each page reloads it from `localStorage` and redraws the badge. You built the capstone's headline feature — a live **search filter** using the `input` event, `toLowerCase()` and `includes()` — and you revisited **responsive design**: the viewport tag, flexible units, Grid/Flexbox and media queries. Crucially, you learned that JS-created elements are styled by the same responsive CSS, as long as the class names are right. Finally, you tested at 360px, 768px and 1024px.

Five important takeaways:

1. A website is a folder of files linked with relative paths; `index.html` is the front door.
2. Every page repeats the same header/nav/footer and links the same CSS/JS.
3. The cart persists across pages only because `localStorage` is read and the badge redrawn on every page.
4. A live search filter = `input` event + `Array.filter` + `toLowerCase()` + `includes()`.
5. Responsive design styles JavaScript-created elements too — test at 360, 768 and 1024.

Questions to help you recognise what you can now do:

- Can I create a second page and link to it correctly with a relative path?
- Can I make the cart badge show the same number on three different pages?
- Can I write a search filter from memory?
- Can I explain why a phone shows my page tiny if the viewport tag is missing?
- Can I check my layout at three widths and fix an overflow?

Connection to the next session: Our site now works, but does it work *correctly*? When something breaks, how do we find out quickly? Next week we move into **Module 4: Polish and Deploy**, starting with **Session 10 — Debugging, Validating and Optimising**. You will learn to read error messages in the Console, pause code with breakpoints, validate your HTML/CSS, and make your pages accessible and fast.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner builds the second and third page without prompting and links them with correct relative paths.
- Learner adds `cart.js` to every page and can explain *why* the badge must be redrawn per page.
- Learner writes the filter using `input` (not `keyup`) and lowercases both sides without being told.
- Learner tests 360/768/1024 without being reminded and spots overflow themselves.
- Learner keeps the shared header identical across files.

Common misconceptions to watch for and correct:

- "The cart is remembered in memory." → It is wiped on navigation; only `localStorage` persists.
- "A relative path and an absolute path are the same on my computer." → They differ the moment you deploy; insist on relative.
- "CSS is only for HTML I typed." → JS-created elements are styled by the same CSS if class names match.
- "filter changes the original array." → It returns a new array.
- "Responsive design is only about the browser window." → The viewport tag and device behaviour matter; always test in the device toolbar.

Suggested remedial exercises for learners who struggle:

1. **Two-page drill:** create just `index.html` and `about.html` with identical headers and a working link, then view both in the device toolbar at 360px.
2. **Filter drill:** given an array and a fixed search box, write only the `applyFilter` function and test it with three typed words.
3. **Path drill:** hand out five links, some relative and some absolute, and ask the learner to correct the bad ones.

Differentiation:

- **Level 1 (supported):** provide the complete three-file starter from Appendix 9A with one page and the search left blank; the learner completes the search and the About page and copies the header.
- **Level 2 (standard):** the guided exercise plus the independent category filter.
- **Level 3 (challenge):** also filter by category **and** price range together; add a "sort by price" dropdown; add a small "(x results)" count that updates live; and make the nav a proper hamburger with an animated icon.

Timing guidance: The guided exercise and the search feature are the core of the session. If the class is slow, reduce the case study (Section J) to a two-minute talk and shorten Section G.3, but protect time for the guided exercise steps 8–13 (linking scripts and the search/filter) and the independent exercise. The mobile test must not be skipped — it is the "Responsive" half of the session title.

APPENDIX 9A — COMPLETE RUNNABLE EXAMPLE

The following files together form the working demo. Save them in one folder, open the folder in VS Code, and use **Live Server** on `index.html`.

`css/style.css`

```
:root {
  --brand: #8e44ad;
  --ink: #222222;
  --muted: #666666;
  --bg: #faf7fb;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, Arial, sans-serif;
  color: var(--ink);
  background: var(--bg);
  line-height: 1.5;
}

img {
  max-width: 100%;
  height: auto;
  display: block;
}

/* ---------- Header / nav ---------- */
.site-header {
  background: var(--brand);
  color: #ffffff;
}

.header-inner {
  max-width: 1100px;
  margin: 0 auto;
  padding: 0.75rem 1rem;
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
}

.brand {
  color: #ffffff;
  font-weight: 700;
  text-decoration: none;
  font-size: 1.2rem;
}

.site-nav {
```

```
  display: flex;
  gap: 1rem;
  flex-wrap: wrap;
}

.site-nav a {
  color: #ffffff;
  text-decoration: none;
  padding: 0.25rem 0;
  border-bottom: 2px solid transparent;
}

.site-nav a.active {
  border-bottom-color: #ffffff;
  font-weight: 700;
}

.cart-count {
  background: #ffffff;
  color: var(--brand);
  border-radius: 999px;
  padding: 0 0.5rem;
  font-weight: 700;
}

/* Hidden on desktop: the checkbox and its label */
.nav-toggle {
  display: none;
}
.nav-toggle-label {
  display: none;
}

/* ---------- Content ---------- */
.hero {
  max-width: 1100px;
  margin: 0 auto;
  padding: 1.5rem 1rem 0.5rem;
}

.search-wrap {
  max-width: 1100px;
  margin: 0 auto;
  padding: 0 1rem 1rem;
}

#searchBox {
  width: 100%;
  padding: 0.75rem;
  font-size: 1rem;
  border: 1px solid #cccccc;
  border-radius: 8px;
}

.product-grid {
  max-width: 1100px;
  margin: 0 auto;
  padding: 0 1rem 2rem;
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
}

.product-card {
  background: #ffffff;
  border: 1px solid #eeeeee;
  border-radius: 10px;
  overflow: hidden;
  display: flex;
  flex-direction: column;
}

.product-img {
  width: 100%;
```

```
  aspect-ratio: 4 / 3;
  object-fit: cover;
}

.product-body {
  padding: 0.75rem 1rem 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.product-name {
  font-weight: 700;
  margin: 0;
}

.product-price {
  color: var(--brand);
  font-weight: 700;
  margin: 0;
}

button.add {
  background: var(--brand);
  color: #ffffff;
  border: 0;
  padding: 0.6rem;
  border-radius: 8px;
  cursor: pointer;
  font-size: 1rem;
}

button.add:hover {
  opacity: 0.9;
}

.cart-summary {
  max-width: 1100px;
  margin: 0 auto 2rem;
  padding: 0 1rem;
}

.site-footer {
  background: #2c2c34;
  color: #dddddd;
  text-align: center;
  padding: 1rem;
  margin-top: 2rem;
}

/* ---------- Responsive breakpoints ---------- */
@media (min-width: 600px) {
  .product-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 900px) {
  .product-grid {
    grid-template-columns: repeat(3, 1fr);
  }
}

@media (max-width: 640px) {
  .nav-toggle-label {
    display: block;
    cursor: pointer;
    color: #ffffff;
    font-weight: 700;
    border: 1px solid #ffffff;
    border-radius: 6px;
    padding: 0.25rem 0.6rem;
  }
  .site-nav {
```

```
    display: none;
    width: 100%;
    flex-direction: column;
    padding-top: 0.5rem;
  }
  .nav-toggle:checked ~ .site-nav {
    display: flex;
  }
}
```

js/products.js

```
// products.js — the shop's data.

// Generates a coloured placeholder image so the demo runs with no image files.
// In a real shop you would replace this with real <img src="images/..."> files.
function placeholderImage(label, color) {
  const svg =
    '<svg xmlns="http://www.w3.org/2000/svg" width="400" height="300">' +
    '<rect width="400" height="300" fill="' + color + '"/>' +
    '<text x="200" y="165" font-family="Arial" font-size="32" fill="#ffffff" text-anchor="middle">' +
    label +
    '</text>' +
    '</svg>';
  return "data:image/svg+xml," + encodeURIComponent(svg);
}

const products = [
  { id: 1, name: "Ankara Dress", price: 18000, category: "Dresses", image: placeholderImage("Ankara", "#8e44ad"), alt: "Ankara print
  { id: 2, name: "Lace Gown", price: 35000, category: "Dresses", image: placeholderImage("Lace", "#c0392b"), alt: "Navy lace evening
  { id: 3, name: "Aso Oke Gele", price: 7500, category: "Gele", image: placeholderImage("Gele", "#16a085"), alt: "Aso oke head-tie"
  { id: 4, name: "Silk Gele", price: 5500, category: "Gele", image: placeholderImage("Silk", "#2980b9"), alt: "Silk head-tie" },
  { id: 5, name: "Beaded Necklace", price: 4500, category: "Jewellery", image: placeholderImage("Beads", "#d35400"), alt: "Handmade
  { id: 6, name: "Gold Earrings", price: 9000, category: "Jewellery", image: placeholderImage("Gold", "#b8860b"), alt: "Gold drop ea
  { id: 7, name: "Kente Wrap", price: 22000, category: "Dresses", image: placeholderImage("Kente", "#27ae60"), alt: "Kente cloth wra
  { id: 8, name: "Coral Beads Set", price: 15000, category: "Jewellery", image: placeholderImage("Coral", "#e74c3c"), alt: "Coral be
];
```

js/cart.js

```
// cart.js — shared by every page.
const CART_KEY = "adaezeCart";

function getCart() {
  const saved = localStorage.getItem(CART_KEY);
  if (!saved) {
    return [];
  }
  try {
    return JSON.parse(saved);
  } catch (error) {
    console.warn("Cart data was corrupt and has been reset.", error);
    return [];
  }
}

function saveCart(cart) {
  localStorage.setItem(CART_KEY, JSON.stringify(cart));
}

function cartCount() {
  return getCart().reduce(function (total, item) {
    return total + item.qty;
  }, 0);
}

function updateCartBadge() {
  const badge = document.getElementById("cartCount");
  if (!badge) {
    return;
  }
```

```
  badge.textContent = cartCount();
}

function addToCart(id) {
  const cart = getCart();
  const existing = cart.find(function (item) {
    return item.id === id;
  });
  if (existing) {
    existing.qty = existing.qty + 1;
  } else {
    cart.push({ id: id, qty: 1 });
  }
  saveCart(cart);
  updateCartBadge();
}
```

`js/app.js`

```
// app.js — shared by every page.

function renderProducts(list) {
  const grid = document.getElementById("productGrid");
  if (!grid) {
    return;
  }

  if (list.length === 0) {
    grid.innerHTML = "<p>No products match your search. Try another word.</p>";
    return;
  }

  const html = list
    .map(function (product) {
      return (
        '<article class="product-card">' +
        '<img class="product-img" src="' + product.image + '" alt="' + product.alt + '">' +
        '<div class="product-body">' +
        '<h3 class="product-name">' + product.name + "</h3>" +
        '<p class="product-price">₦' + product.price.toLocaleString("en-NG") + "</p>" +
        '<button class="add" data-id="' + product.id + '">Add to cart</button>' +
        "</div>" +
        "</article>"
      );
    })
    .join("");

  grid.innerHTML = html;
}

function applyFilter() {
  const searchBox = document.getElementById("searchBox");
  const term = searchBox.value.toLowerCase().trim();
  const filtered = products.filter(function (product) {
    return product.name.toLowerCase().includes(term);
  });
  renderProducts(filtered);
}

function renderCartSummary() {
  const box = document.getElementById("cartSummary");
  if (!box) {
    return;
  }
  const cart = getCart();
  if (cart.length === 0) {
    box.innerHTML = "<p>Your cart is empty.</p>";
    return;
  }
  let total = 0;
  let html = "<ul>";
  cart.forEach(function (item) {
    const product = products.find(function (p) {
```

```
      return p.id === item.id;
    });
    if (!product) {
      return;
    }
    const line = product.price * item.qty;
    total += line;
    html += "<li>" + product.name + " × " + item.qty + " — ₦" + line.toLocaleString("en-NG") + "</li>";
  });
  html += "</ul><p><strong>Total: ₦" + total.toLocaleString("en-NG") + "</strong></p>";
  box.innerHTML = html;
}

document.addEventListener("DOMContentLoaded", function () {
  updateCartBadge();

  const searchBox = document.getElementById("searchBox");
  if (searchBox) {
    renderProducts(products);
    searchBox.addEventListener("input", applyFilter);
  }

  const grid = document.getElementById("productGrid");
  if (grid) {
    grid.addEventListener("click", function (event) {
      if (event.target.classList.contains("add")) {
        const id = Number(event.target.dataset.id);
        addToCart(id);
        renderCartSummary();
      }
    });
    renderCartSummary();
  }
});
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze Fashion — Shop</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="site-header">
    <div class="header-inner">
      <a class="brand" href="index.html">Adaeze Fashion</a>

      <input type="checkbox" id="navToggle" class="nav-toggle">
      <label for="navToggle" class="nav-toggle-label">Menu</label>

      <nav class="site-nav" aria-label="Main navigation">
        <a href="index.html" class="active">Shop</a>
        <a href="about.html">About</a>
        <a href="contact.html">Contact</a>
        <a href="index.html" class="cart-link">Cart <span id="cartCount" class="cart-count">0</span></a>
      </nav>
    </div>
  </header>

  <main>
    <section class="hero">
      <h1>Welcome to Adaeze Fashion</h1>
      <p>Beautiful Nigerian fashion, delivered from Port Harcourt.</p>
    </section>

    <div class="search-wrap">
      <label for="searchBox">Search products</label>
      <input id="searchBox" type="text" placeholder="e.g. Ankara, Gele, Beads">
    </div>
```

```
    <div class="product-grid" id="productGrid"></div>

    <section class="cart-summary" id="cartSummary"></section>
  </main>

  <footer class="site-footer">
    <p>© 2026 Adaeze Fashion, Port Harcourt. Built by CEA students.</p>
    <p>WhatsApp: 0803 000 0000</p>
  </footer>

  <script src="js/products.js"></script>
  <script src="js/cart.js"></script>
  <script src="js/app.js"></script>
</body>
</html>
```

`about.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>About — Adaeze Fashion</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="site-header">
    <div class="header-inner">
      <a class="brand" href="index.html">Adaeze Fashion</a>

      <input type="checkbox" id="navToggle" class="nav-toggle">
      <label for="navToggle" class="nav-toggle-label">Menu</label>

      <nav class="site-nav" aria-label="Main navigation">
        <a href="index.html">Shop</a>
        <a href="about.html" class="active">About</a>
        <a href="contact.html">Contact</a>
        <a href="index.html" class="cart-link">Cart <span id="cartCount" class="cart-count">0</span></a>
      </nav>
    </div>
  </header>

  <main class="hero">
    <h1>About Adaeze Fashion</h1>
    <p>Adaeze started sewing in 2019 from a small shop on Ebony Road, Port Harcourt.
       Today she styles brides, graduates and professionals across Rivers State.</p>
    <p>Every piece is made to order and delivered with care.</p>
  </main>

  <footer class="site-footer">
    <p>© 2026 Adaeze Fashion, Port Harcourt. Built by CEA students.</p>
    <p>WhatsApp: 0803 000 0000</p>
  </footer>

  <script src="js/cart.js"></script>
  <script src="js/app.js"></script>
</body>
</html>
```

`contact.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contact — Adaeze Fashion</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
```

```
  <header class="site-header">
    <div class="header-inner">
      <a class="brand" href="index.html">Adaeze Fashion</a>

      <input type="checkbox" id="navToggle" class="nav-toggle">
      <label for="navToggle" class="nav-toggle-label">Menu</label>

      <nav class="site-nav" aria-label="Main navigation">
        <a href="index.html">Shop</a>
        <a href="about.html">About</a>
        <a href="contact.html" class="active">Contact</a>
        <a href="index.html" class="cart-link">Cart <span id="cartCount" class="cart-count">0</span></a>
      </nav>
    </div>
  </header>

  <main class="hero">
    <h1>Contact Us</h1>
    <p>Call or WhatsApp: 0803 000 0000. Visit us at 24/26 Ebony Road, Port Harcourt.</p>

    <form>
      <p>
        <label for="contactName">Your name</label><br>
        <input id="contactName" type="text" name="name">
      </p>
      <p>
        <label for="contactEmail">Email</label><br>
        <input id="contactEmail" type="email" name="email">
      </p>
      <p>
        <label for="contactMessage">Message</label><br>
        <textarea id="contactMessage" name="message" rows="4"></textarea>
      </p>
      <button type="submit">Send message</button>
    </form>
  </main>

  <footer class="site-footer">
    <p>© 2026 Adaeze Fashion, Port Harcourt. Built by CEA students.</p>
    <p>WhatsApp: 0803 000 0000</p>
  </footer>

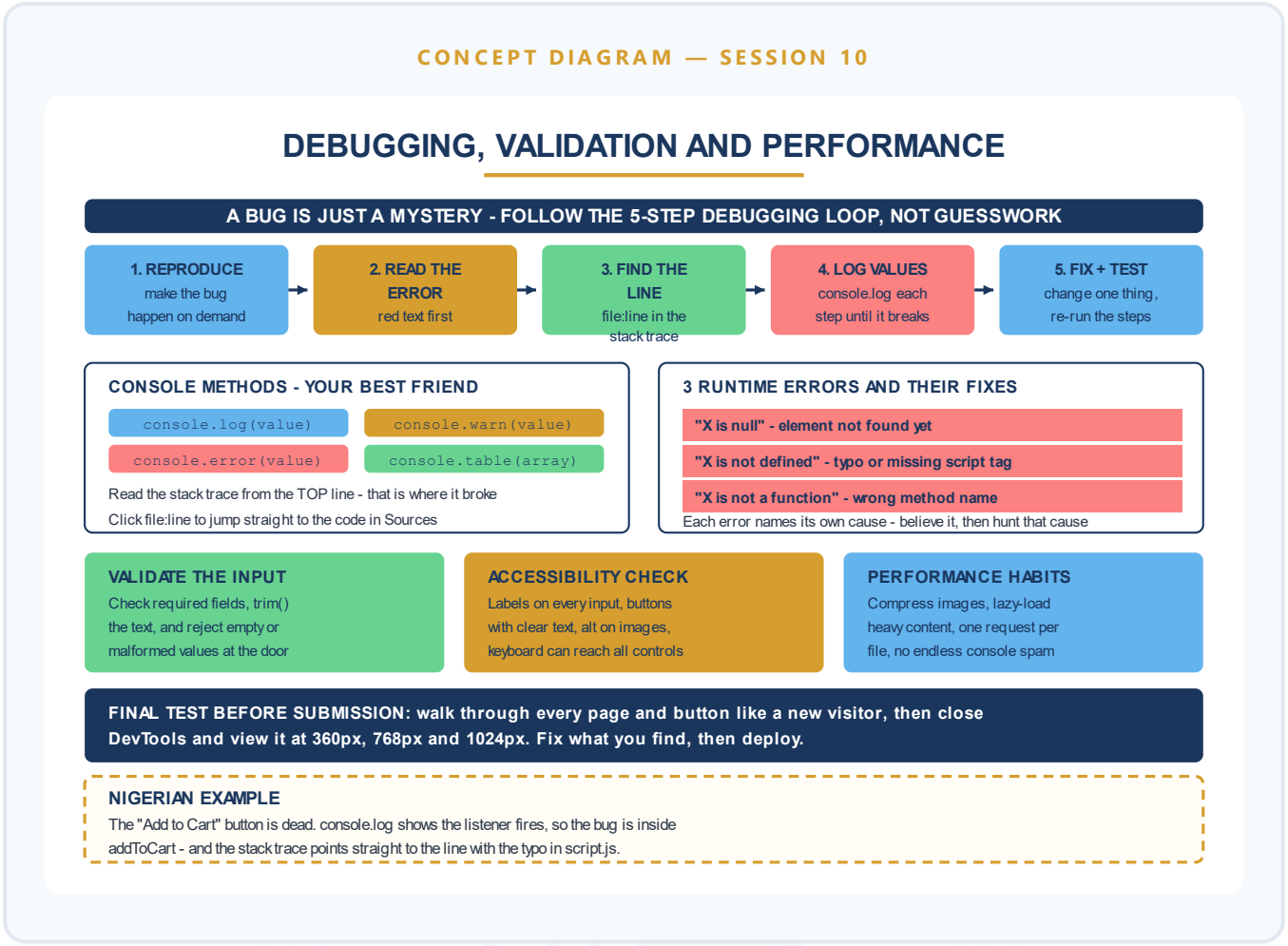
  <script src="js/cart.js"></script>
  <script src="js/app.js"></script>
</body>
</html>
```

END OF SESSION 9

Next: Session 10 — Debugging, Validating and Optimising.

SESSION 10 OF 12

DEBUGGING, VALIDATING AND OPTIMISING



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 10 — DEBUGGING, VALIDATING AND OPTIMISING

A. SESSION IDENTIFICATION

Item	Detail
Session number	10 of 12
Session title	Debugging, Validating and Optimising
Week / Module	Week 4 — Module 4: Polish and Deploy
Recommended duration	105 minutes
Difficulty level	Intermediate beginner

Item	Detail
Prerequisite knowledge	Sessions 1–9. Learners must be able to: select DOM elements, render lists, handle events, use `localStorage`, build a three-page site and run it with Live Server. They must be comfortable opening the Console, because today we use it seriously.
Connection to previous work	In Session 9 the three-page shop worked — on your machine, when nothing went wrong. But real projects *always* have mistakes. Today we learn the professional habit that separates a beginner from someone employable: finding and fixing errors **quickly and calmly**, then validating and optimising so the site is correct, accessible and fast. This directly prepares the capstone and the Session 11 deployment.
Preparation before class	Academy PCs on. Chrome and VS Code installed. The instructor prepares two folders: `demo-week4-s10/broken/` (with the deliberate bugs) and `demo-week4-s10/fixed/` (the corrected version). DevTools open and ready. If the academy has internet, the W3C validator pages can be shown; if not, explain using the saved example (Section F.7).
Capstone link	The capstone is marked partly on "presentation / no errors" and on a working deployed app. Debugging, validation, accessibility and performance are exactly what make the difference between a project that merely runs and a project that earns full marks.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Apply the debugging procedure** — read the error, locate the line, form a hypothesis, test, repeat — to a broken page without panicking.
2. **Use the Chrome DevTools Console, Sources panel and breakpoints** to pause code, inspect variables, and step over/into lines.
3. **Use the console methods** `console.log`, `console.warn`, `console.error`, `console.table` and `console.assert` appropriately, and explain when each is best.
4. **Read a stack trace** and identify the top line as the location where the error occurred (`file:line`).
5. **Diagnose and fix the three most common runtime errors**: "Cannot read properties of null/undefined", "is not defined", and "is not a function".
6. **Validate HTML and CSS** using the W3C validators and explain why validation matters even for beginners.
7. **Apply basic accessibility** — `alt` text, `<label>` tied to inputs, sufficient colour contrast and keyboard-accessible buttons — and explain why it is professional and legally relevant.
8. **Apply basic performance improvements** — compress images, avoid repeated DOM work inside loops, and remove unnecessary `console.log` calls from finished code.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). Pairing is fine with the swap method.
- Instructor computer connected to a projector or large screen.

Software

- Google Chrome (Console, Sources panel, breakpoints, device toolbar).
- Visual Studio Code with the **Live Server** extension (used since Session 7).

Accounts

- None required. The W3C validators (`validator.w3.org` and `jigsaw.w3.org/css-validator`) are free and need no login, but they do need internet. If there is no internet, the instructor demonstrates the concept using the saved example in Section F.7.

Sample files

- Instructor's two prepared folders: `demo-week4-s10/broken/` and `demo-week4-s10/fixed/` (all code in Appendix 10A). These contain **four deliberate bugs** used throughout the session.

Worksheets

- Worksheet 10.1 — "Find the Bug" (the broken code printed with space in the margin to write the diagnosis for each bug).
- Worksheet 10.2 — "Accessibility Audit" (a checklist: every image has `alt`? every input has a `<label>`? contrast good? buttons are real buttons?).

Tools

- Chrome DevTools: Console, Sources, breakpoints, `console.table`, device toolbar.
- W3C HTML validator and W3C CSS validator.

Internet

- Needed only for the validators. Everything else works offline.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open the `broken/index.html` page from the prepared folder with Live Server. Open DevTools (`F12`). The page will be nearly blank and the Console will show a **red error**.

What the instructor says:

> *"Last session, everything worked. Today I am going to show you failure — on purpose. This shop page is broken. Look at the screen: almost nothing. Look at the Console: red text. And here is the truth nobody says out loud: **every** developer, every day, sees screens like this. The difference between a professional and a beginner is not that professionals never get errors. It is that they know how to find the error and stay calm.*
>
> *Today you will learn a procedure — a recipe — for finding bugs, so you never stare at a red screen not knowing where to start. By the end, you will take this broken page and fix every bug in it with me."*

Expected learner response: Learners will feel relieved that errors are normal, and curious about the red text. Some may already guess "the id is wrong" from Session 9.

Then demonstrate the hook: Point to the red error in the Console and read it aloud, slowly:

> *"`Uncaught TypeError: Cannot read properties of null (reading 'innerHTML')` at `shop.js:11`. We are going to learn exactly what every word of that means. By the end of this session, that sentence will not scare you — it will be a map to the fix."*

E. PREREQUISITE REVIEW (7 minutes)

Run this quickly as an oral round-robin with the questions on the projector.

Question 1: "In Session 9, why does the cart badge show the same number on About and Contact as on the Shop?"

- Expected answer: because every page reads the cart from `localStorage` and calls `updateCartBadge()` on load.
- Common gap: learners forget that `localStorage` is the only thing that survives navigation. Remind them: today, if the badge ever shows `0` wrongly, one of the first things to check is whether `cart.js` is loaded.

Question 2: "What file did we put the shared cart code in, and how many pages link it?"

- Expected answer: `js/cart.js`, linked on all pages.
- Common gap: learners link it on some pages only. Remind them a missing script tag produces a runtime error like "is not defined".

Question 3: "From Session 9, why must the search box compare lowercased text on both sides?"

- Expected answer: because `"Ankara"` and `"ankara"` are different to the computer; lowercasing both makes the comparison case-insensitive.
- Common gap: exactly this bug is planted in today's broken page. Tell them to watch for it.

Question 4: "Where do we put the `<script>` tag, and why?"

- Expected answer: at the bottom of the `<body>`, so the HTML elements exist before the script tries to find them. (Or in the head with `defer`.)
- Common gap: this is directly related to today's "Cannot read properties of null" error — if the script runs too early, `getElementById` returns `null`. Connect the two.

Bridge to today (say this):

> *"Reading the error, knowing where to look, and staying calm — that is the skill of debugging. Today it becomes a procedure you can repeat for the rest of your career."*

If more than a third of the class cannot answer Question 4, spend five extra minutes re-running the "script at the bottom" demonstration before continuing.

F. FULL LECTURE NOTES

F.1 The debugging mindset — a procedure, not a talent

The idea. Debugging is the work of finding and fixing mistakes ("bugs") in code. It is not luck and it is not magic. It is a **procedure** you can follow every single time.

Why it matters. Beginners often panic, change many lines at once, then cannot tell which change fixed or broke what. Professionals change **one thing at a time** and test after each change. This is the single most important habit in this session.

The five-step procedure (write this on the board and repeat it all session):

1. **Read the error first.** Do not guess. The browser usually tells you *what* went wrong and *where*.
2. **Find the line.** The error names a file and a line number (for example `shop.js:11`). Go to that line.
3. **Form a hypothesis.** Say the cause out loud as a sentence: *"I think `grid` is `null` because the id is misspelled."*
4. **Test the hypothesis.** Change **one** thing that would confirm or disprove it, then refresh and read the error again.
5. **Repeat** until the error is gone, then test the feature manually (click, type, resize).

Plain-English definition. The error message is like a doctor's note. It tells you the symptom and the body part. Your job is to go and look at that body part, not to take every medicine in the cupboard.

Analogy. Debugging is like tracing water from a leaking pipe. You do not rip out the whole wall. You follow the wet trail to the exact spot, fix that spot, and check again.

Real-life comparison. A mechanic listening to an engine: a professional does not replace the whole engine. They listen, form a theory ("that knock is a loose belt"), test it, and fix only that. Good debugging is professional listening.

Common misconception. "I should change many things at once so it gets fixed faster." Actually, changing many things at once makes it impossible to know what fixed it — and risks adding new bugs. One change, then test.

Limitation. Not every bug shows an error. Some bugs are **silent** — the page looks fine but the result is wrong (for example, a search that finds nothing because of capitalisation). For silent bugs you need `console.log`, `console.table` and `console.assert` to *see* what your variables actually contain (Section F.3).

F.2 Chrome DevTools deep-dive

The idea. DevTools (Developer Tools) is the set of tools built into Chrome for inspecting and fixing a web page. We open it with `F12`. Three panels are essential today.

1. The Console. Already familiar from Session 1. It shows:

- `console.log` output (your own messages)

- errors in **red**,
- warnings in **yellow**,
- and lets you type commands directly.

2. The Elements panel. Shows the live HTML (the DOM) after JavaScript has run. Right-click any element on the page → **Inspect** to jump to it. Very useful to confirm an element's real `id` or `class` when a selector "cannot find" it.

3. The Sources panel — and breakpoints. This is new and powerful.

What a breakpoint is. A **breakpoint** is a marker you place on a line of code that tells Chrome: *"pause the program here before running this line."* When paused, the whole program freezes and you can inspect every variable at that exact moment.

Why breakpoints exist. `console.log` shows you a value at the moment it runs, but it clutters the code and only shows what you thought to print. A breakpoint lets you look at *everything* and step through the code line by line.

How to set a breakpoint (the steps):

1. Open DevTools → **Sources** tab.
2. On the left, open your file (for example `shop.js`). If you cannot see it, open the **Page** section (it lists the files the page loaded).
3. Click the **line number** in the left margin next to a line you want to pause on. A blue marker appears.
4. Refresh the page. Chrome stops at that line and the panel highlights it.

The controls once paused:

- **Step over** — run the current line and stop at the next line (does not go *into* functions). The most used button.
- **Step into** — if the current line calls a function, go *inside* that function.
- **Step out** — finish the current function and return to where it was called.
- **Resume** — continue running normally until the next breakpoint.

Watching a variable. While paused, the **Scope** pane (or hovering over a variable in the code) shows its current value. To keep an eye on one specific variable as you step, use the **Watch** pane: click "+", type the variable name (for example `term`), and its value updates after every step. This is how professionals find the exact line where a value becomes wrong.

Say to the class: "A breakpoint is a pause button for your code. When you pause a footballer on video, you can see where every leg is. A breakpoint lets you see where every variable is."

Important tip. Breakpoints only appear and pause if the correct file is loaded. If your JavaScript is inside the HTML (`<script>` block) rather than a separate file, look under the page's HTML document in the **Page** list.

F.3 The console methods — five tools, five jobs

`console.log` is only one tool. Professional code uses the right tool for the job.

1. `console.log` — general information.

```
console.log("Products loaded:", products.length);
```

2. `console.warn` — a warning, not a crash, shown in yellow.

```
console.warn("Cart is empty — nothing to total.");
```

Use it when something is unusual but not fatal.

3. `console.error` — a real problem, shown in red.

```
console.error("Product grid not found. Check the id in index.html.");
```

Use it when the code cannot do its job. It also gives a stack trace (Section F.4).

4. `console.table` — the best way to look at an array of objects (like products). This is the star of today.

```
const products = [
  { id: 1, name: "Ankara Dress", price: 18000, category: "Dresses" },
  { id: 2, name: "Aso Oke Gele", price: 7500, category: "Gele" },
```

```
  { id: 3, name: "Beaded Necklace", price: 4500, category: "Jewellery" }
];
console.table(products);
```

The Console prints a neat **table** with one row per product and one column per property — instantly readable. When a filter behaves strangely, `console.table(products)` lets you *see* the exact data.

5. `console.assert` — check an assumption and only shout if it is wrong.

```
console.assert(products.length === 3, "Expected 3 products but found " + products.length);
```

`console.assert(condition, message)` does nothing if the condition is `true`; if it is `false`, it prints an **error** with your message. It is a quiet guard for assumptions.

Which to use when:

Method	Colour	Use it for
`console.log`	normal	General values and progress messages while developing
`console.warn`	yellow	Something odd but not fatal
`console.error`	red	Something that stops the code working
`console.table`	normal	Looking at arrays of objects (products, cart items)
`console.assert`	red on failure	Checking an assumption without cluttering success

A serious warning about leaving logs in. `console.log` is a **development** tool. Before you deploy (Session 11), remove the debugging logs. They are not visible to normal users, but: (a) they make your Console noisy so real errors get lost, (b) printing a large array in a loop is genuinely slow, and (c) on a real site, careless logs can leak information you did not mean to share. We return to this in F.9.

F.4 Reading a stack trace

The idea. When an error happens, the Console prints not just one line but a **stack trace** — a list of the function calls that led to the error, from the outermost call down to the exact failing line.

A real example:

```
Uncaught TypeError: Cannot read properties of null (reading 'innerHTML')
    at render (shop.js:11)
    at shop.js:16
```

How to read it:

- The **first line** tells you the *type* of error (`TypeError`) and the *message* (`Cannot read properties of null`).
- The **first `at ...` line** is the **most important**: it is the exact place the error happened — file `shop.js`, line `11`, inside the function `render`.
- The lines below show who *called* that function — here, line 16 of `shop.js` called `render`.

Rule to remember: top line = where it broke. Always start at the top `at` line, not the bottom.

Plain-English definition. A stack trace is a breadcrumb trail. It shows you the path the program took, ending at the spot where it fell over.

How to use it in the debugging procedure. Step 2 of our procedure ("find the line") is almost always answered by the top line of the stack trace. Click it in Chrome and it opens the file at that line automatically.

Common misconception. "The bottom line is the problem." No — the bottom is where execution *started*; the top is where it *stopped*. Read from the top.

F.5 The three most common runtime errors

A **runtime error** is an error that happens *while the program is running* (as opposed to a syntax error, which stops the file from loading at all). These three cause the vast majority of beginner JavaScript pain.

Error 1 — `Cannot read properties of null (reading '...')` (or "of undefined").

- **What it means:** your code tried to use a value that is `null` (nothing) or `undefined` (no value yet). Most often, `getElementById` found no element.
- **The three usual causes:**
  1. **Typo in the selector/id** — the HTML has `id="productGrid"` but the JS asks for `"product-grid"`.
  2. **The element does not exist** — you are on the About page but your script tries to find the shop's grid.
  3. **The script runs too early** — the script is in the `<head>` (or the element is created later) so the element is not there yet.
- **Fix:** match the id exactly (check the Elements panel), guard with `if (!element) return;`, and put the script at the bottom of the body (or wrap in `DOMContentLoaded` / use `defer`).

Error 2 — `ReferenceError: produts is not defined`.

- **What it means:** you used a name that JavaScript does not know. Almost always a **spelling mistake** or a **scope** problem (the variable was declared inside a function and used outside).
- **Fix:** check the spelling letter by letter; confirm where the variable was declared; make sure it is declared *before* it is used.
- **Analogy:** calling a person by a name they do not have. The compiler says "I do not know anyone called that."

Error 3 — `TypeError: products.foreach is not a function`.

- **What it means:** you tried to *call* something as a function, but that thing is not a function. Usually a **typo in a method name** (`foreach` instead of `forEach`) or a **wrong type** (for example calling `.toUpperCase()` on a number).
- **Fix:** spelling (`forEach`, `toString`, `filter` are case-sensitive); check the value's type with `typeof` or `console.log` before calling a method on it.

Say to the class: "Learn these three sentences by heart. If you can recognise them, you can fix maybe eight out of ten beginner bugs."

F.6 Validating HTML and CSS

The idea. Validation is checking your code against the official rules of the language. For HTML the rules come from the W3C (World Wide Web Consortium). For CSS there is a separate validator. A **validator** is a free online tool that reads your code and lists anything the rules do not allow.

The two validators:

- HTML: `validator.w3.org` ("Markup Validation Service").
- CSS: `jigsaw.w3.org/css-validator`.

How to use the HTML validator:

1. Open `validator.w3.org`.
2. Choose the **"Validate by Direct Input"** tab.
3. Paste your whole HTML file, then click **Check**.
4. Read the results: **Errors** (must fix) and **Warnings** (should consider).

What it commonly catches:

- **Unclosed elements** — you opened `<div>` and never closed it. The browser often *guesses* and your layout looks almost right, which is why this is dangerous: it can break differently in different browsers.
- **A missing `alt` on an image** — flagged for accessibility.
- **Nesting mistakes** — a `<section>` closed while a `<div>` inside it is still open.
- **A missing `charset`** — the validator reminds you to declare the character encoding.

Why validation matters even for beginners (the honest reasons):

1. **The browser is forgiving; the next browser may not be.** Browsers "fix" broken HTML silently in their own way. Your page may look fine in Chrome and wrong in Safari or on a phone. Validation removes the guesswork.
2. **Broken markup hurts accessibility.** Screen readers rely on correct structure. An unclosed element can confuse them.

3. **It is the professional baseline.** "Does it validate?" is a standard, easy quality check a client or employer can ask. Passing it shows discipline.
4. **It often reveals a bug you have not seen yet.** An unclosed `<div>` is frequently the cause of a mysterious layout problem.

Important limitation. Validation checks whether your code follows the *rules* — it does **not** check whether your design is good, your JavaScript works, or your content is correct. A page can validate perfectly and still be an ugly, broken app. Validation is necessary, not sufficient.

Say to the class: "Validation is like an inspector checking that your building obeys the building code. It does not tell you if the building is beautiful, but it tells you it will not collapse."

F.7 Basic accessibility — building for everyone

The idea. Accessibility (often shortened to "a11y") means building so that people with disabilities — visual, motor, hearing, cognitive — can still use your site. It is both a professional standard and, increasingly, a legal requirement.

Why it matters (professional and legal).

- A blind user navigates with a **screen reader** that reads the page aloud. That reader depends on your markup.
- A user with a motor disability may use only a keyboard, not a mouse.
- A user with low vision needs enough contrast.
- **Legal relevance:** many countries require accessible public services and commercial sites. In Nigeria, the **Discrimination Against Persons with Disabilities (Prohibition) Act 2018** establishes the rights of persons with disabilities — building a site that excludes them is against the spirit and letter of that law. Globally, businesses have faced legal action over inaccessible sites. Accessibility is not a favour; it is part of doing business honestly.

The four beginner essentials:

1. `alt` text on every image.

```
<img src="images/ankara-dress.jpg" alt="Green and gold Ankara print dress on a mannequin">
```

- **What it is:** a short text description of the image.
- **Why:** a screen reader reads the `alt` aloud; if the image fails to load, sighted users see the text.
- **Rule:** describe the *purpose* of the image briefly. Decorative images (pure decoration) can use `alt=""` (empty), which tells the screen reader to skip them.

2. A `<label>` tied to every input.

```
<label for="searchBox">Search products</label>
<input id="searchBox" type="text">
```

- The `for` attribute on the label must match the `id` of the input. Then clicking the label focuses the input, and the screen reader announces the label with the field.
- Never rely on a `placeholder` alone as a label — placeholders disappear when the user types and are poorly announced.

3. Sufficient colour contrast.

- **Contrast** is how different the text colour is from the background. Light grey text on white is hard or impossible to read.
- The widely used accessibility guideline (WCAG) sets a minimum contrast ratio of **4.5:1** for normal-sized text and **3:1** for large text. You do not need to memorise the maths; the practical rule is: **dark text on light background, or light text on dark background, and test it.** We already use white on a dark purple header, which has good contrast.
- **Common failure:** pink or light-grey text on white. Fix by darkening the text.

4. Keyboard-accessible buttons — use a real `<button>`.

```
<!-- Good: a real button -->
<button class="add" data-id="1">Add to cart</button>

<!-- Bad: a clickable div pretending to be a button -->
<div class="add" onclick="addToCart(1)">Add to cart</div>
```

- A real `<button>` can be reached with the **Tab** key, activated with **Enter/Space**, and is announced as a button by screen readers — all for free.
- A clickable `<div>` gets none of this unless you manually add `tabindex`, `role` and keyboard handlers. So: **use a real button.** Our product cards already do.

Bonus: focus visibility. When a keyboard user tabs onto a link or button, you must be able to **see** where the focus is. Never write `outline: none` on interactive elements without providing a visible replacement.

Say to the class: "Accessibility is not extra work on top of the site. It is part of building the site correctly. A button that only works with a mouse is an unfinished button."

F.8 Basic performance — make it fast

Performance means how quickly a page loads and responds. Slow sites lose customers. Three beginner-level wins:

1. Compress images.

- A photo straight from a phone camera can be 4–8 MB. On a slow Nigerian mobile network that is many seconds of waiting.
- **Fix:** resize images to the largest size you will actually display (often 1200px wide is plenty) and compress them. Free online tools such as TinyPNG reduce the file size dramatically with almost no visible loss. Save images as JPEG for photos and PNG/SVG for logos and icons.
- In the code, always allow them to shrink: `img { max-width: 100%; height: auto; }`.
- **Optional advanced:** add `loading="lazy"` to `<img>` tags below the fold, so the browser only loads them when the user scrolls near them.

2. Avoid repeating DOM work inside loops.

- Every time you touch the DOM (change `innerHTML`, append a child), the browser does work. Doing it once is fast; doing it inside a loop over 60 products is slow.

The slow way (repeated DOM writes):

```
list.forEach(function (p) {
  grid.innerHTML += "<p>" + p.name + "</p>";   // rewrites the WHOLE grid each time — slow
});
```

The fast way (build the string, write once):

```
const html = list.map(function (p) {
  return "<p>" + p.name + "</p>";
}).join("");
grid.innerHTML = html;                          // one single DOM write
```

The `.map(...).join("")` builds one complete string in memory; we touch the DOM **once**. On a small list the difference is invisible, but the *habit* is what professionals use, and on large lists it is the difference between smooth and janky.

3. Remove unnecessary `console.log` calls.

- Logging is fast for one line, but a `console.log` inside a loop that runs thousands of times — or logging a large array repeatedly — adds real work and clutters your error view. When you finish debugging, **delete the debugging logs.**

Bonus: trust the browser's cache and keep files small. Splitting CSS and JS once (as we did) lets the browser cache them for return visits. Minifying (removing spaces from) final files is a later, automated step.

Say to the class: "A shop that loads in three seconds sells more than a shop that loads in ten. Performance is not vanity — it is money."

F.9 Summary of the session

- Debug with a **procedure**: read the error, find the line, hypothesise, test one change, repeat.
- DevTools: **Console** (messages/errors), **Elements** (live HTML), **Sources + breakpoints** (pause and inspect).
- Console methods: `log`, `warn`, `error`, `table`, `assert` — each with a purpose.
- The **stack trace's top line** is where it broke (`Uncaught TypeError: ...`).

- The big three errors: **null/undefined**, **is not defined**, **is not a function**.
- **Validate** HTML/CSS with the W3C tools; it is the professional baseline.
- **Accessibility:** `alt`, tied `<label>`, contrast, real `<button>` — professional and legally relevant.
- **Performance:** compress images, write to the DOM once, remove stray logs.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- `demo-week4-s10/broken/` containing `index.html` and `js/shop.js` (from Appendix 10A — the buggy version).
- `demo-week4-s10/fixed/` containing the corrected `index.html` and `js/shop.js`.
- Live Server running on the **broken** folder. DevTools open.
- A marker to write the five-step debugging procedure on the board (keep it visible all session).

G.2 Demonstration 1 — fix bug 1 (null error) and read the stack trace (5 minutes)

Say: "Let's follow the procedure, out loud, together. Step one: read the error."

Do this:

1. Refresh `broken/index.html`. The Console shows: `Uncaught TypeError: Cannot read properties of null (reading 'innerHTML')` `at render (shop.js:11)`.
2. **Read it aloud.** Point to "TypeError" (the type), "Cannot read properties of null" (the message), and "shop.js:11" (the location).
3. Click the top `at render (shop.js:11)` line in the Console — Chrome opens `shop.js` at line 11.
4. **Hypothesis:** "`grid` is `null`, so `getElementById` found nothing. I think the id in the HTML does not match." Open the Elements panel, find the shop grid, and read its real id: `productGrid`.
5. Compare with the JS: it asks for `"product-grid"`. **Confirmed cause.**
6. **Fix:** change `getElementById("product-grid")` to `getElementById("productGrid")`. Save, refresh.

- Expected result: the products now render; the red error disappears.

Ask: "Which line of the stack trace did we start from, and why?" (The top `at` line — that is where it broke.)

G.3 Demonstration 2 — fix bug 2 (is not defined) (3 minutes)

Say: "The page loads now — but the Console still has a red error. Read it."

1. Console shows: `ReferenceError: produts is not defined` `at shop.js:16`.
2. **Hypothesis:** a spelling mistake. Look at line 16: `console.log("Total products:", produts.length);`.
3. Compare with the declaration at the top: `const products = [...]`. **Confirmed:** `produts` vs `products`.
4. **Fix** the spelling. Save, refresh.

- Expected result: `Total products: 3` prints with no error.

Ask: "What are the two usual causes of 'is not defined'?" (A spelling mistake, or using a variable outside the scope where it was declared.)

G.4 Demonstration 3 — fix bug 3 (is not a function) with a breakpoint (4 minutes)

Say: "The page works, but only until I use the search. Watch what happens when I type."

1. Type `a` in the search box. Console shows: `TypeError: products.foreach is not a function` `at shop.js:21`.
2. **Explain:** `forEach` is spelled with a capital **E**. `foreach` (all lowercase) does not exist, so JavaScript says "that is not a function."
3. **Show a breakpoint:** in Sources → `shop.js`, click the line number next to `const filtered = products.foreach(...)`. Refresh and type. Chrome pauses. Hover over `products` in the **Watch** pane — it shows the array. Step over — the error occurs exactly on that line.
4. **Fix:** change `.foreach` to `.filter` (correcting both the spelling *and* the logic — `filter` is what keeps matching items). Save, refresh, type `a`.

- Expected result: two products show (names containing "a"). No error.

Ask: "Why did `filter` work where `foreach` did not?" (`foreach` is not a real method; `filter` is the array method that returns matching items.)

G.5 Demonstration 4 — find bug 4 (silent logic bug) using breakpoints and console.table (3 minutes)

Say: "No red error now. But is the search correct? Type a product name in lowercase."

1. Type `ankara` (lowercase). The grid goes **empty** — but "Ankara Dress" should match! **No error message.** This is a silent bug.
2. **This is when we need `console.table` and breakpoints.** Open the Console and type:

   ```
   console.table(products);
   ```

   Point at the table — the names have capital letters.
3. **Hypothesis:** the comparison is case-sensitive. `"Ankara Dress".includes("ankara")` is `false` because of the capital A.
4. Set a breakpoint on the `return p.name.includes(term);` line, refresh, type a letter. When paused, hover `p.name` (shows `"Ankara Dress"`) and `term` (shows `"ankara"`). **Confirmed.**
5. **Fix:** change to `p.name.toLowerCase().includes(term)`. Save, refresh, type `ankara`.

- Expected result: "Ankara Dress" now shows.

Ask: "Was there any red error for bug 4? How did we find it?" (No error; we found it by testing the feature and using `console.table` plus a breakpoint to inspect values.)

G.6 Demonstration 5 — validate and audit (wrap-up, on the fixed page)

1. Paste the **broken** `index.html` into `validator.w3.org` (Direct Input). Point out the reported errors: an unclosed `<div>`, and "An img element must have an alt attribute", plus the missing charset suggestion.
2. Paste the **fixed** `index.html` and click Check: **"Document checking completed. No errors or warnings to show."** (or at most a benign warning).
3. Point at the fixed file: `<label for="search">`, `<img ... alt="...">`, real `<button>`s. Say: "These are the accessibility fixes, and they cost us nothing."

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: In pairs, fix a fresh broken page using the five-step debugging procedure and DevTools, then validate and audit it.

Starting materials: the instructor copies a **second** broken page (Worksheet 10.1) to each pair. This one has the same *types* of bugs but different names, so learners cannot simply copy the demo answers. The second broken page is given in **Appendix 10B** (instructor-only until the review).

Procedure (instructor calls each step; learners work in pairs and write their diagnosis on the worksheet before changing any code):

1. Open the broken page with Live Server and press `F12`.
2. **Do not change anything yet.** Read the Console error aloud to your partner and write, on the worksheet, the error type and the file:line.
3. Following the procedure (read → find line → hypothesise → test one change → repeat), fix **bug A** (the null error). Confirm the fix by refreshing and seeing the products appear.
4. Fix **bug B** (the "is not defined" error). Confirm by refreshing with no red error.
5. Trigger the search; fix **bug C** (the "is not a function" error). Confirm by typing a letter.
6. Find **bug D** (the silent logic bug) by testing the search with a lowercase word, then use `console.table()` or a breakpoint to prove the cause. Fix it.
7. **Pause and predict** before each fix: "What do I think will change when I make this one edit?" Write it down, then test.
8. Run the page through `validator.w3.org` and fix at least one validation error (likely the unclosed tag and the missing `alt`).
9. Do a quick accessibility audit with Worksheet 10.2: is every input labelled? does the image have `alt`? are the buttons real `<button>`s?

Expected result:

- All four bugs fixed; the page renders products, the search works case-insensitively, and there are no red Console errors.
- At least one validation error fixed; the page re-validates cleanly (or with only benign warnings).

Points to pause on and predict:

- Before fixing bug A: "Will the page reload and still show an error, or not? Why?" (The next error, bug B, will surface.)
- Before fixing bug C: "What will the search do after this fix?" (It will run, but may still return wrong results because of the silent bug.)
- Before fixing bug D: "Why does typing `Ankara` work but `ankara` fail?" (Capitalisation.)

Troubleshooting while walking the room:

- Learner changes three lines at once → stop them; insist on one change per test.
- Learner cannot find the error line → show them how to click the top `at` line in the Console to jump to the source.
- Learner fixes the spelling but not the logic (`.foreach` → `.filter` needed, not just case) → ask them what `filter` returns that `forEach` does not.
- Learner cannot find a silent bug → direct them to `console.table(products)` and a breakpoint.

Completion checklist (learner ticks each):

- ☐ Bug A (null) fixed; products render
- ☐ Bug B (is not defined) fixed; no red error
- ☐ Bug C (is not a function) fixed; typing works
- ☐ Bug D (silent case bug) fixed; lowercase search matches
- ☐ Used Console, and at least once used `console.table` or a breakpoint
- ☐ Ran the W3C validator and fixed one HTML error
- ☐ Completed the accessibility audit (Worksheet 10.2)
- ☐ No red errors remain in the Console

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Take a small broken "Todo" app and fix a single reported `TypeError`, then improve it.

Scenario: *Emeka* built a quick to-do list for his cyber café to track repair jobs. He says: "When I click Add, nothing happens, and the Console shows a red error. Also I want it to look professional." Your job is to fix it and polish it.

Starting information: the broken file below (also provided as `todo-broken/index.html`):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Emeka's Jobs</title>
</head>
<body>
  <h1>Repair Jobs</h1>
  <input id="jobInput" type="text" placeholder="Describe the job">
  <button id="addBtn">Add</button>
  <ul id="jobList"></ul>

  <script>
    const jobs = [];

    document.getElementById("addBtn").addEventListener("click", function () {
      const input = document.getElementById("jobInput");
      const text = input.value.trim();
      if (text === "") return;
      jobs.push(text);
      renderJobs();
      input.value = "";
    });
```

```
    function renderJobs() {
      const list = document.getElementById("joblist");
      list.innerHTML = jobs.map(function (job) {
        return "<li>" + job + "</li>";
      }).join("");
    }
  </script>
</body>
</html>
```

Requirements:

1. Find and fix the **single runtime bug** so that clicking **Add** adds a job to the list.
2. Write down, in your own words, the error type and the exact cause.
3. Add accessibility: a `<label for="jobInput">` and make sure the button is a real `<button>` (it already is — confirm).
4. Run the fixed file through the W3C HTML validator and confirm no errors.
5. **Optional (Level 3):** make the input case-cleaned (trim is already there) and give each job a "Remove" button.

Constraints: change **one** thing at a time and re-test; do not rewrite the whole file; keep the code readable.

Time allowance: 15 minutes.

Success criteria:

- Clicking **Add** with text adds an item to the list.
- The Console shows no red errors.
- The cause is written down in the learner's own words.
- The page has a `<label>` tied to the input.
- The file validates with no errors.

Hints:

- Hint 1: read the top `at ...` line of the error and go to that line.
- Hint 2: `getElementById` is case-sensitive. Compare the id in the HTML with the id in the JavaScript, character by character.
- Hint 3: `document.getElementById("jobList")` versus `document.getElementById("joblist")` — look carefully.

Instructor answer key (do not show learners):

Bug: `renderJobs()` calls `document.getElementById("joblist")`, but the `<ul>` in the HTML has `id="jobList"` (capital L). `getElementById` is case-sensitive, so it returns `null`, and the next line `list.innerHTML = ...` throws:

```
Uncaught TypeError: Cannot read properties of null (reading 'innerHTML')
    at renderJobs (todo.html:22)
```

Fix:

```
const list = document.getElementById("jobList");   // capital L, matching the HTML
```

Accessibility fix:

```
<label for="jobInput">New job</label>
<input id="jobInput" type="text" placeholder="Describe the job">
<button id="addBtn">Add</button>
```

Optional Level 3 (remove + safe markup):

```
function renderJobs() {
  const list = document.getElementById("jobList");
  list.innerHTML = jobs.map(function (job, index) {
    return '<li>' + job + ' <button class="remove" data-index="' + index + '">Remove</button></li>';
  }).join("");
}
// add a delegated listener on the list:
document.getElementById("jobList").addEventListener("click", function (event) {
  if (event.target.classList.contains("remove")) {
```

```
    jobs.splice(Number(event.target.dataset.index), 1);
    renderJobs();
  }
});
```

Marking guidance (out of 10): correct id fix (4); error correctly explained (2); `<label>` tied to input (1); validation clean (1); no red errors and Add works (2).

J. REAL-WORLD CASE STUDY

Situation. *Chinaza* runs *Chinaza Couture*, a fashion label with an online shop. She is ready to launch. On the morning of launch, she clicks "Buy" on her own site and **nothing happens**. Her developer is travelling and offline. She has a radio interview in two hours and a link she promised to share on Instagram.

Relevant facts.

- The page loads and looks perfect.
- Clicking "Buy" does nothing.
- She can open the site in Chrome and is willing to look at the Console if someone tells her what to do.
- The site's JavaScript is in one file, `js/cart.js`.

Decision point. Should she (a) cancel the launch, (b) tell customers to order by WhatsApp and lose the online sales, or (c) try to find and fix the bug quickly using DevTools?

Possible approaches.

1. **Cancel/fallback.** Safe but costly; the whole point of the launch was the online shop.
2. **Open the Console and read the error.** Follow the five-step procedure, form a hypothesis, test one change, and — if it works — keep the launch.
3. **Guess and randomly change code.** Dangerous: risks breaking more than it fixes, just before a public launch.

Recommended solution. Approach 2. The Console will almost certainly name the file and line. In a typical launch-day bug, the "Buy" handler is trying to read from an element that does not exist (a renamed id after a last-minute edit), producing a `Cannot read properties of null` error. Reading the top line of the stack trace, checking the id in the Elements panel, and matching it fixes it in minutes.

Example solution (the kind of fix this procedure finds):

```
// BEFORE (bug): the button id was renamed in HTML but not in JS
const buyButton = document.getElementById("buyButton");   // returns null after rename
buyButton.addEventListener("click", addToCart);           // <-- error here

// AFTER (fixed): match the real id shown in the Elements panel
const buyButton = document.getElementById("addToCart");
if (!buyButton) {
  console.error("Buy button not found — check the id in the HTML.");
} else {
  buyButton.addEventListener("click", addToCart);
}
```

Trade-offs.

- Approach 1 is safe but abandons the launch goal and the effort already spent.
- Approach 2 risks nothing extra: reading errors is free, and testing one change at a time cannot make things worse than they already are. It turns a crisis into a five-minute fix.
- Approach 3 is the worst option: changing many lines blindly before a launch is how small bugs become outages.

Professional connection. "Production is broken, find it fast" is one of the most valued skills in software teams. Senior developers are not people who never see errors — they are people who can *read* an error, form a hypothesis, and test calmly under pressure. That is precisely the procedure you learned today.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this table as the live troubleshooting guide. When a learner is stuck, ask them to check the relevant row before giving the answer, and always insist on **one change at a time**.

K.1 Red error: `Cannot read properties of null (reading '...')`

Symptoms: A red error mentioning `null` or `undefined`; a feature does nothing. **Likely causes:** (a) A typo or case mismatch in an `id` in `getElementById`. (b) The element does not exist on this page. (c) The script runs before the element exists (script in the head, or above the element). **Investigate:** Click the top `at` line to jump to the code. In the Elements panel, confirm the real id. In the Console type `document.getElementById("yourId")` and see if it returns `null`. Check `typeof element`. **Corrective steps:** Match the id exactly (case-sensitive); guard with `if (!element) return;`; move the script to the bottom of the body or use `DOMContentLoaded` / `defer`. **Confirm:** The error disappears and the feature works. **Prevent:** Copy-paste ids between HTML and JS; test each feature as you build it.

K.2 `ReferenceError: something is not defined`

Symptoms: Red error naming a word. **Likely causes:** (a) Spelling mistake in a variable/function name. (b) The variable was declared inside a function but used outside (scope). (c) The declaration never ran because an earlier line threw. **Investigate:** Look at the named word; search your file for where it was declared; check the Console for an earlier error. **Corrective steps:** Fix the spelling; move the declaration to the right scope; fix any earlier error (errors cascade — the first is usually the real one). **Confirm:** Error gone. **Prevent:** Consistent `camelCase` naming; declare variables before use.

K.3 `TypeError: x.y is not a function`

Symptoms: Red error when a method is called. **Likely causes:** (a) A typo/case error in a method name (`foreach`, `tolowercase`). (b) Calling a method on the wrong type (e.g. `.toUpperCase()` on a number). **Investigate:** `console.log(typeof x)` to confirm the type; compare the method name with the correct spelling. **Corrective steps:** Spell the method correctly (`forEach`, `toLowerCase`); convert the type first (`String(x)`, `Number(x)`). **Confirm:** Error gone. **Prevent:** Method names are case-sensitive; check the type before calling.

K.4 The layout breaks on one browser or device but not another

Symptoms: Looks fine in Chrome on the laptop; wrong in Safari or on a phone. **Likely cause:** Invalid HTML — often an unclosed element that the browser "guesses" how to fix differently. **Investigate:** Run the page through `validator.w3.org`. **Corrective steps:** Fix the reported errors (usually unclosed tags); re-validate. **Confirm:** The validator reports no errors and the layout is stable. **Prevent:** Validate before you consider a page finished.

K.5 A silent bug — right code, wrong result

Symptoms: No red error, but the result is wrong (empty search, wrong total). **Likely causes:** Logical mistakes: missing `.toLowerCase()`, wrong comparison operator, off-by-one, comparing string to number. **Investigate:** `console.table(products)` or a breakpoint with the **Watch** pane to inspect the actual values at the moment of the comparison. **Corrective steps:** Fix the logic (e.g. lowercase both sides; use `===`; convert types). **Confirm:** The feature gives the expected result. **Prevent:** Test features with tricky input (mixed case, empty, boundaries).

K.6 The Console is full of old messages and you cannot find the error

Symptoms: Hundreds of `console.log` lines hide the one red error. **Likely cause:** Debugging logs left in the code, especially inside loops. **Investigate:** Scroll to the very top; errors usually appear first. Use the Console's **filter** box and the "Errors only" toggle (the default-level dropdown). **Corrective steps:** Use the clear button (🚫) on the Console toolbar; use the level filter; remove debugging logs from the code. **Confirm:** The Console shows only real errors. **Prevent:** Delete logs when a feature is finished; never log inside a tight loop.

K.7 Images make the page slow / huge

Symptoms: The page takes many seconds to load on a phone. **Likely cause:** Multi-megabyte images straight from a camera, displayed small. **Investigate:** DevTools → **Network** tab → refresh → look at the image file sizes. **Corrective steps:** Compress with a tool such as TinyPNG, resize to the needed dimensions, and add `loading="lazy"` for below-the-fold images. **Confirm:** Image sizes drop sharply and the page loads faster. **Prevent:** Compress before adding any image to the project.

Distinguish likely vs confirmed. In every case, confirm before fixing: read the exact red error, run `console.table` or set a breakpoint, or run the validator. Never change code on a hunch.

L. SESSION ASSIGNMENT

Assignment title: "Debug, Validate and Optimise Report"

Purpose: Prove you can take a broken page, fix it with a documented procedure, validate it, and improve its accessibility and performance — the professional quality pass before deployment.

Realistic scenario: A client sends you a page that "does not work". You must find and fix the bugs, validate the code, and deliver a short written report explaining each fix (clients love to see that you understand *why*, not just *what*).

Detailed instructions:

1. Use the broken page provided by your instructor (or the one in Appendix 10B, if released by your instructor).
2. Produce the fixed `index.html` and `js/app.js` (or `shop.js`).
3. Write a **debugging report** with one row per bug:

#	Error shown (type + message)	File:line	Cause	Fix	How I confirmed it
1					
2					
3					
4					

4. Run the fixed HTML through `validator.w3.org` and paste the result line ("No errors or warnings" or the list).
5. Complete the accessibility audit (Worksheet 10.2) and note any fixes you made (`alt` , `<label>` , contrast, real buttons).
6. Note two performance improvements you made (for example, DOM written once, images compressed, logs removed).

Required deliverables:

- The fixed HTML and JS files.
- The completed debugging report table.
- The validator result (screenshot or copied text).
- The completed accessibility audit.

Submission format: Files plus the report in the class folder (or zipped). **Deadline:** Before Session 11.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
Bugs correctly identified	5	All four bugs found, with accurate error type and file:line
Bugs correctly fixed	5	One-change-at-a-time approach; page works; no red errors
Debugging report clarity	3	Cause and fix explained in the learner's own words
Validation passed	3	HTML validates with no errors
Accessibility fixes	2	`alt`, tied `<label>`, real buttons, adequate contrast
Performance note	2	At least two sensible optimisations identified

Common mistakes to avoid: changing many lines at once; fixing the symptom by deleting the feature; leaving debugging logs in; forgetting `alt`/labels; reporting "it works now" without explaining the cause.

Instructor reference solution: Appendix 10A (the `fixed/` folder) is the reference for bugs A–D; the report should identify each bug as shown in Section G. A strong report names the error type, the file:line from the stack trace, the cause in plain English, and the single-line fix.

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. List the five steps of the debugging procedure in order.

Q2. Multiple choice. In a stack trace, which line tells you *where* the error happened? a) The bottom-most line b) The first line after "Uncaught" c) The top-most `at ...` line d) The last line of the trace

Q3. Explain in your own words. What does the error `Cannot read properties of null (reading 'innerHTML')` usually mean, and name two common causes.

Q4. Practical identification. Which console method would you use for each job? a) Print an array of product objects as a neat table b) Print a warning in yellow about an empty cart c) Check an assumption and only shout if it is false d) Print a red, real problem

Q5. Error detection. Find and explain the bug:

```
const title = document.getElementById("titel");
title.textContent = "Welcome";
```

Q6. Scenario. A learner's search works when they type `Ankara` but finds nothing when they type `ankara`. There is no red error. What is the bug, how would you prove it with DevTools, and what is the fix?

INSTRUCTOR ANSWER KEY (Q1–Q6)

A1. (1) Read the error first. (2) Find the line (from the stack trace). (3) Form a hypothesis. (4) Test one change. (5) Repeat until fixed, then test the feature.

A2. (c) The top-most `at ...` line.

A3. The code tried to use a value that is `null` (nothing) — usually because `getElementById` found no element. Two common causes: (a) a typo/case mismatch between the HTML `id` and the id used in JavaScript; (b) the script runs before the element exists (script in the head, or above the element, without `DOMContentLoaded` / `defer`).

A4. a) `console.table`; b) `console.warn`; c) `console.assert`; d) `console.error`.

A5. The id in the HTML is likely `title` (or some other spelling), but the JS asks for `"titel"` — a typo. `getElementById` returns `null`, so `title.textContent = ...` throws `Cannot read properties of null (reading 'textContent')`. Fix: match the id exactly. (Also guard with an `if`.)

A6. The bug is a missing `.toLowerCase()` on the product name, so the comparison is case-sensitive ("Ankara Dress" does not include "ankara"). Prove it by setting a breakpoint on the comparison line (or `console.log`) to see that `p.name` is `"Ankara Dress"` while `term` is `"ankara"`; `console.table(products)` also shows the capital letters. Fix: `return p.name.toLowerCase().includes(term);`.

N. REFLECTION AND RECAP

Plain-English summary. Today you learned that errors are normal and that debugging is a **procedure**, not a talent: read the error, find the line, form a hypothesis, test one change, repeat. You went deep into DevTools — the Console, the Elements panel, and the Sources panel with **breakpoints**, where you can pause code, inspect variables and step through it line by line (step over, step into, step out). You learned the five console methods and when to use each, how to read a **stack trace** (top line = where it broke), and how to recognise the big three errors: null/undefined, "is not defined", and "is not a function". You validated your HTML/CSS with the W3C tools, made your pages accessible (`alt`, tied `<label>`, contrast, real `<button>`), and improved performance (compressed images, one DOM write, no stray logs).

Five important takeaways:

1. Debug with a procedure: read the error, find the line, hypothesise, test one change, repeat.
2. The **top line** of a stack trace is where it broke (`file:line`).
3. Learn the big three errors by name: null/undefined, is not defined, is not a function.
4. Validate your HTML/CSS, and build accessible pages (alt, labels, contrast, real buttons).

5. Write to the DOM once, compress images, and remove debugging logs before deploying.

Questions to help you recognise what you can now do:

- Can I read an error and go straight to the failing line?
- Can I set a breakpoint and inspect a variable while paused?
- Can I tell the difference between a silent logic bug and a runtime error?
- Can I validate a page and fix a validation error?
- Can I name two accessibility fixes and two performance fixes I actually made?

Connection to the next session: Your site now works, is valid, accessible and faster — and you can fix it when it breaks. It is polished. The only thing left is to put it on the internet so the whole world (and your client) can see it. Next session: **Session 11 — Deploying to GitHub Pages and Writing Documentation.** You will create a GitHub account, upload your site, turn on GitHub Pages, and share a real public link.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner reads the Console error before touching any code.
- Learner makes **one** change per test and refreshes to check.
- Learner can interpret a stack trace's top line and jump to the source.
- Learner uses `console.table` (not dozens of `console.log` s) to inspect arrays.
- Learner distinguishes a silent logic bug from a runtime error and knows it needs inspection tools.
- Learner runs the validator and corrects the reported errors.

Common misconceptions to watch for and correct:

- "Errors mean I am bad at this." → Errors are normal; the skill is the procedure.
- "Change many things at once to fix it faster." → One change per test, always.
- "The bottom of the stack trace is the problem." → The top `at` line is.
- "If there is no red error, the code is correct." → Silent logic bugs exist; test features with awkward input.
- "Accessibility is extra." → It is part of building correctly and is legally relevant (Disability Act 2018).
- "Validation makes my site good." → It makes it rule-compliant; it does not fix bad design or broken logic.

Suggested remedial exercises for learners who struggle:

1. **Error naming drill:** give ten one-line errors and ask learners to name each type and the first place to look.
2. **Breakpoint drill:** a small script with a wrong total; learner sets one breakpoint and uses the Watch pane to find the bad variable.
3. **Validator drill:** hand out a file with two unclosed tags and one missing `alt` ; learner fixes and re-validates.

Differentiation:

- **Level 1 (supported):** provide the broken page with the four bug *locations* marked in comments (but not the fixes); learner fixes one at a time with the procedure sheet.
- **Level 2 (standard):** the guided exercise and independent exercise as written.
- **Level 3 (challenge):** add a fifth bug (a wrong comparison operator causing a silent wrong total), and require a written debugging report; also require `loading="lazy"` and a compressed-image note.

Timing guidance: The live demo (Section G) and the guided bug hunt (Section H) are the heart of the session — protect them. If the class is slow, compress the validation walkthrough (G.6) and the performance discussion (F.8) into short talks, but do not skip: (a) reading the stack trace, (b) the breakpoint demonstration, and (c) the silent bug found with `console.table` . Those three are the transferable skills of the session.

APPENDIX 10A — BROKEN AND FIXED SHOP PAGE

BROKEN VERSION: `broken/index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Adaeze Fashion — Shop (Broken)</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Adaeze Fashion Shop</h1>
  <section>
    <input id="search" type="text" placeholder="Search products">
    <div class="product-grid" id="productGrid">
  </section>
  <img src="images/ankara-dress.jpg">
  <script src="js/shop.js"></script>
</body>
</html>
```

Validation problems: unclosed `<div>` (error), `<img>` missing `alt` (error), missing `<meta charset>` (reminder).

BROKEN VERSION: `broken/js/shop.js`

```
const products = [
  { id: 1, name: "Ankara Dress", price: 18000 },
  { id: 2, name: "Aso Oke Gele", price: 7500 },
  { id: 3, name: "Beaded Necklace", price: 4500 }
];

const searchBox = document.getElementById("search");

function render(list) {
  const grid = document.getElementById("product-grid");   // BUG 1: wrong id
  grid.innerHTML = "";                                     // throws: grid is null
  list.forEach(function (p) {
    grid.innerHTML += "<p>" + p.name + " — ₦" + p.price + "</p>";
  });
}

render(products);

console.log("Total products:", produts.length);            // BUG 2: typo (not defined)

searchBox.addEventListener("input", function () {
  const term = searchBox.value.toLowerCase();
  const filtered = products.foreach(function (p) {         // BUG 3: foreach (not a function)
    return p.name.includes(term);                          // BUG 4: missing toLowerCase (silent)
  });
  render(filtered);
});
```

FIXED VERSION: `fixed/index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze Fashion — Shop</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Adaeze Fashion Shop</h1>
  <section>
    <label for="search">Search products</label>
    <input id="search" type="text" placeholder="e.g. Ankara">
    <div class="product-grid" id="productGrid"></div>
  </section>
  <img src="images/ankara-dress.jpg" alt="Green and gold Ankara print dress on a mannequin" width="600" height="400">
  <script src="js/shop.js"></script>
```

```
</body>
</html>
```

FIXED VERSION: `fixed/js/shop.js`

```
const products = [
  { id: 1, name: "Ankara Dress", price: 18000 },
  { id: 2, name: "Aso Oke Gele", price: 7500 },
  { id: 3, name: "Beaded Necklace", price: 4500 }
];

console.table(products);
console.assert(products.length === 3, "Expected 3 products but found " + products.length);

const searchBox = document.getElementById("search");

function render(list) {
  const grid = document.getElementById("productGrid");     // FIX 1: correct id
  if (!grid) {
    console.error("Product grid not found. Check the id in index.html.");
    return;
  }
  if (list.length === 0) {
    grid.innerHTML = "<p>No products match your search.</p>";
    return;
  }
  const html = list.map(function (p) {                     // one DOM write (performance)
    return "<p>" + p.name + " — ₦" + p.price.toLocaleString("en-NG") + "</p>";
  }).join("");
  grid.innerHTML = html;
}

render(products);

console.log("Total products:", products.length);           // FIX 2: correct name

searchBox.addEventListener("input", function () {
  const term = searchBox.value.toLowerCase().trim();
  const filtered = products.filter(function (p) {          // FIX 3: correct method
    return p.name.toLowerCase().includes(term);            // FIX 4: lowercase both sides
  });
  render(filtered);
});
```

Note: once this page is validated and audited, the debugging `console.log`, `console.table` and `console.assert` lines should be removed before deploying in Session 11 (keeping `console.error` in the guard is optional and harmless).

APPENDIX 10B — SECOND BROKEN PAGE (GUIDED EXERCISE, ANSWER KEY)

Instructor-only. Distribute to learners as `guided/index.html`.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Emeka's Mini Shop</title>
</head>
<body>
  <h1>Emeka's Mini Shop</h1>
  <input id="find" type="text" placeholder="Find a product">
  <div id="items"></div>
  <img src="images/phone.jpg">
  <script>
    const item = [
      { id: 1, name: "Phone Charger", price: 1800 },
      { id: 2, name: "Power Bank", price: 12000 },
```

```
      { id: 3, name: "Earbuds", price: 6500 }
    ];

    function show(list) {
      const box = document.getElementById("itemList");   // Bug A: wrong id (real id is "items")
      box.innerHTML = "";
      list.forEach(function (x) {
        box.innerHTML += "<p>" + x.name + " — ₦" + x.price + "</p>";
      });
    }

    show(items);

    console.log("Count:", items.length);                 // Bug B: "items" is not defined (array is "item")

    const findBox = document.getElementById("find");
    findBox.addEventListener("input", function () {
      const term = findBox.value.toLowerCase();
      const result = item.map(function (x) {              // Bug C: uses map (returns all, unchanged) instead of filter
        return x.name.includes(term);                     // Bug D: missing toLowerCase on x.name (silent)
      });
      show(result);
    });
  </script>
</body>
</html>
```

Answer key for Appendix 10B:

- **Bug A** (null): `getElementById("itemList")` but the real id is `items`. → `Uncaught TypeError: Cannot read properties of null (reading 'innerHTML')` at `show`. Fix: `getElementById("items")`.
- **Bug B** (is not defined): `show(items)` and `items.length` reference `items`, but the array is named `item`. → `ReferenceError: items is not defined`. Fix: rename consistently (prefer renaming the array to `items`, which reads better).
- **Bug C** (wrong method / silent logic): `item.map(...)` returns an array of booleans, not the matching products, so `show` receives booleans and prints `undefined` names. Fix: `item.filter(...)`.
- **Bug D** (silent): missing `.toLowerCase()` on `x.name`. Fix: `x.name.toLowerCase().includes(term)`.
- **Validation:** missing `<meta charset="UTF-8">` is not present here, so the validator's main complaint will be the `<img>` missing `alt` and a recommendation; also note the `<input>` has no `<label>` (accessibility, not a validator error).

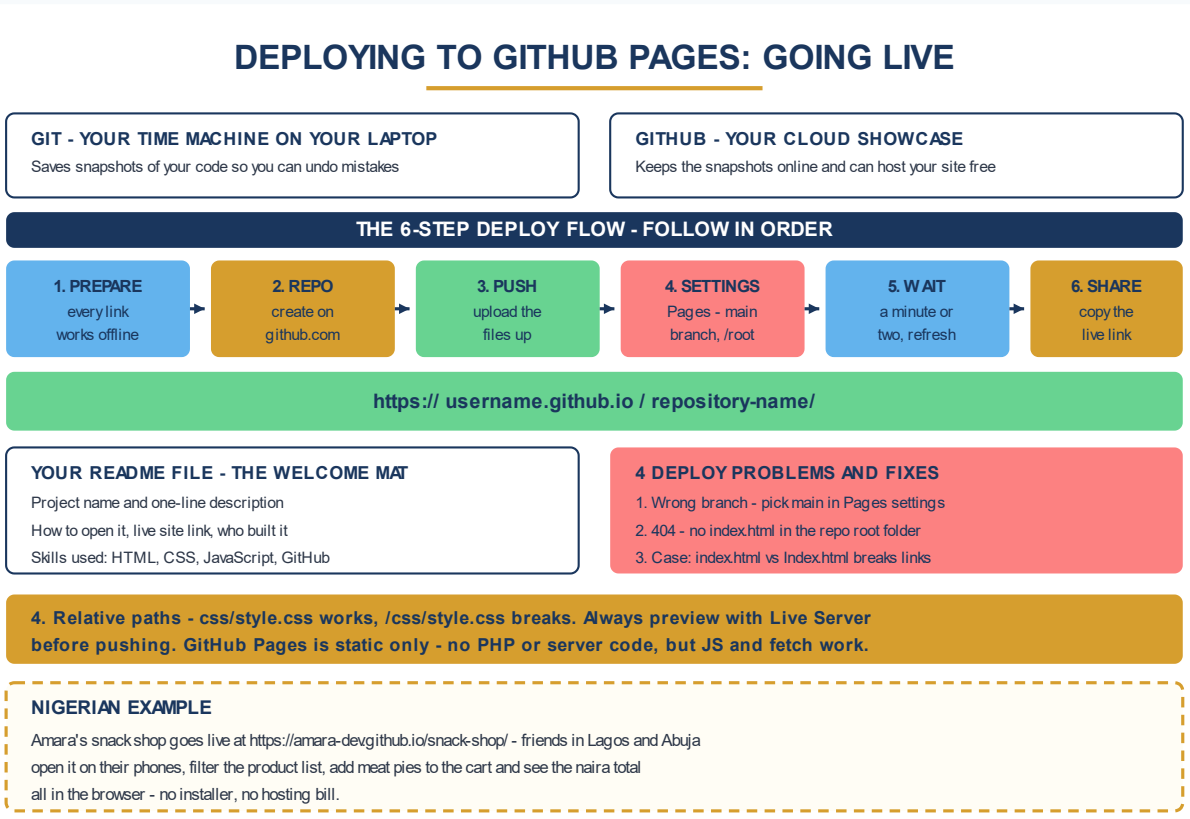
END OF SESSION 10

Next: Session 11 — Deploying to GitHub Pages and Writing Documentation.

SESSION 11 OF 12

PUBLISHING YOUR SITE WITH GITHUB PAGES

CONCEPT DIAGRAM — SESSION 11



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 11 — PUBLISHING YOUR SITE WITH GITHUB PAGES

A. SESSION IDENTIFICATION

Item	Detail
Session number	11 of 12
Session title	Publishing Your Site with GitHub Pages
Week / Module	Week 4 — Module 4: Polish and Deploy
Recommended duration	105 minutes
Difficulty level	Beginner (first contact with Git and GitHub)

Item	Detail
Prerequisite knowledge	Sessions 1–10. The learner must have a working multi-page site (or at least a working `index.html` with linked `css/style.css` and `js/app.js`), know the Chrome DevTools Console, understand relative file paths, and be able to run the site locally (double-click or Live Server).
Connection to previous work	In Session 10 the learner fixed bugs and optimised the site, but it still lived only on one computer. This session puts it on the internet so anyone — a customer, a relative, a future employer — can open it with a real link.
Preparation before class	Academy PCs on and logged in; Google Chrome and VS Code installed; internet access **is required this session** (unlike most earlier sessions); each learner's `CEA-WebDev-Name` project folder ready; instructor GitHub account already created for the demo; a phone with mobile data for the "test on a phone" step; the projector able to show the instructor's GitHub browser tab.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Explain in their own words** what Git is and what GitHub is, and state the difference between the two without confusing them.
2. **Create a free GitHub account** (or log in to an existing one) with a verified email address.
3. **Create a public repository** on GitHub and upload a website's files using the web interface.
4. **Enable GitHub Pages** for a repository and locate the published URL (`https://username.github.io/repo-name/`).
5. **Describe the structure of the published URL** and explain how to update the live site after editing a file.
6. **Write a simple** `README.md` containing a project name, description, feature list, how to run the site, and the author's name.
7. **Test a live site** on both a desktop browser and a mobile phone, and confirm that assets (CSS, JavaScript, images) load.
8. **Diagnose and fix** the four common deployment problems: Pages set to the wrong branch/folder, a 404 error, a case-sensitive file name (`index.html` vs `Index.html`), and assets missing because of wrong relative paths.
9. **Use the Git command line** (for learners who are ready) to `init`, `add`, `commit`, add a remote and `push` a project to GitHub.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). As before, two learners may share with the pair-swap method.
- Instructor computer connected to a projector or large screen.
- **One smartphone** (instructor's is enough, but learners may use their own if they have data) for the "test on a phone" step.

Software

- Google Chrome (any recent version).
- Visual Studio Code (VS Code).
- Optional: Git for Windows (the command-line tool) — only needed for the command-line deployment path (Section F.5). If it is not installed and a learner wants to use it, download the free installer from `https://git-scm.com/download/win`. Ask the academy administrator before installing.

Accounts

- A **free GitHub account**. This is required this session. No paid plan is needed — GitHub Pages is free for public repositories. No card or payment details are required.
- Internet access is required on every machine for this session.

Sample files

- The instructor's demo site (`demo-deploy/index.html`, `demo-deploy/css/style.css`, `demo-deploy/js/app.js`).

- A completed `README.md` template (given in Section F.8).

Worksheets

- Worksheet 11.1 — "Button-by-Button" (a printed checklist of every GitHub click, with a blank line for the learner to tick).
- Worksheet 11.2 — "My README Draft" (sentence starters for the README).

Tools

- Chrome DevTools (still useful for checking that CSS and JavaScript loaded on the live site).
- The GitHub website in a browser (`https://github.com`).
- Optional: Git for Windows (command line).

Internet

- **Required.** If the academy internet fails, give the whole class the fallback described at the end of Section O (paired offline README work and a re-run of the upload next session).

Alternative for learners who lack a facility

- A learner without a GitHub account may pair with a neighbour and perform the clicks on the neighbour's account while the neighbour watches, then swap. Every learner must still end the session able to *describe* each step. The learner completes their own repository as the assignment.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open two browser tabs side by side. In the left tab, open the learner-style local file `file:///C:/Users/.../CEA-WebDev-Name/index.html` . In the right tab, open a real, already-published site (for example the instructor's own demonstration page at `https://<instructor-username>.github.io/demo-deploy/`).

What the instructor says:

> *"Look at the left tab. This is the site we have been building for four weeks. It works — but look at the address bar: it starts with `file:///C:/Users/...` . That address only works on this one computer. If I send it to your phone right now, it will not open. If I paste it in a WhatsApp message to a customer, they will see nothing.*
>
> *Now look at the right tab. This address starts with `https://` . I can send this link to anybody in the world — your customer in Aba, your aunt in Lagos, a recruiter in Abuja — and they open it on any phone or computer. Nothing to install.*
>
> *So the question for today is simple: how do we move our site from the left tab to the right tab? How do we get a real link?"*

Expected learner response: Learners often say "you need to pay for a website" or "you need to buy a domain." Accept the answers, then correct the biggest myth:

> *"You do **not** need to pay to get a real link. Today you will each publish a working site to the internet, for free, using a service called GitHub Pages. By the end of this session, every one of you will have a link you can send to anyone."*

Why this opener works: It shows the exact gap (a private file versus a public link) with two visible tabs, kills the "it costs money" fear, and promises a concrete reward — a link — that every learner wants before the session is over.

E. PREREQUISITE REVIEW (7 minutes)

Run this as a fast oral round-robin. Put the questions on the projector.

Question 1: "In your project folder, what is the special file name that a website's home page must have?"

- Expected answer: `index.html` .
- Common gap: learners call it `home.html` or `Home.html` . Correct firmly: for GitHub Pages, the home page file at the top level **must** be named `index.html` , all lowercase

Question 2: "Your CSS is in a folder called `css`. Write the line that links your stylesheet from `index.html`."

- Expected answer: `<link rel="stylesheet" href="css/style.css">`.
- Common gap: writing an absolute path like `C:\Users\...\css\style.css`. Remind them: website files always use **relative paths** (paths relative to the current file), never Windows drive paths.

Question 3: "Your site uses `fetch('data/products.json')`. Why did we need Live Server for that in Session 7, and will it work on a published site?"

- Expected answer: `fetch` does not work from `file://` because of browser security (CORS); it needs a server. On a published GitHub Pages site, the site **is** served from a web server over `https`, so `fetch` works correctly.
- Common gap: learners think the published site will have the same problem. Reassure them: publishing *fixes* the `fetch` problem.

Question 4: "Name two things that must be in the same folder for your `index.html` to work normally."

- Expected answer: the `css` folder and the `js` folder (and any `images` or `data` folders, and any other `.html` pages like `about.html`, `contact.html`).
- Common gap: learners forget sub-pages. Remind them: **all** pages and folders must be uploaded, not only `index.html`.

Bridge to today (say this):

> *"You now have a complete site. It has structure, styling, behaviour, a cart that remembers, and a validated form. The only thing missing is an audience. Today we get the audience. We will learn two words first — **Git** and **GitHub** — and then we will publish."*

Gap handling: If more than a third of the class cannot answer Question 1 and Question 2, spend five extra minutes opening each learner's project folder together and confirming that `index.html` exists at the top level and that the stylesheet link uses a relative path, before starting Section F.

F. FULL LECTURE NOTES

F.1 Why deploy at all

The idea. To **deploy** means to put your website on a computer that is always switched on and connected to the internet, so that anybody with the link can open it. That always-on computer is called a **web server**.

Plain-English definition. Right now your site lives on your laptop. A laptop sleeps, runs out of battery, and is only reachable by you. A deployed site lives on a server that never sleeps, so the whole world can reach it.

Why this matters to a business. Imagine *Adaeze Fashion* has a beautiful shop built in this course. It sits on Adaeze's laptop. A customer in Warri asks for her website and she says "come and see it on my laptop." That is not a business — that is a hobby. A real URL that anyone can open turns a practice exercise into a **shop window**. Adaeze can put the link in her WhatsApp status, her Instagram bio and her business card. Orders can begin.

Real-life comparison. Think of a school play. The **rehearsal** happens in a closed hall — only the cast sees it (your local `file://` site). The **performance** happens on a stage with an audience and tickets — anyone can come (your deployed site). Deployment is the night of the performance.

Limitation to state honestly. GitHub Pages hosts **static** sites: HTML, CSS, JavaScript and files like JSON and images. It does **not** run a backend (no database, no logins on a server, no taking real payments). Our site is fully static — the cart is stored in the visitor's own browser with `localStorage` — so GitHub Pages is a perfect fit.

F.2 Git and GitHub — two different things

These two words confuse beginners more than almost anything else in web development, so we separate them clearly.

Git is a **version control tool** that runs on your own computer. It records a history of every change you make to your files. Each saved snapshot is called a **commit**. Git lets you see what changed, when, and go back to an earlier version if you break something.

GitHub is a **website** (`https://github.com`) that hosts Git projects online. When you upload your Git project to GitHub, other people (or a machine like GitHub Pages) can see and use it.

Plain-English comparison. Git is the **notebook that remembers every draft** of your work — page 1, page 2, and so on. GitHub is the **library shelf** where you place the finished notebook so the whole world can read it.

Another way to say it:

- Git = the tool. It lives on your computer. It works offline.
- GitHub = the website. It lives on the internet. It needs network access.

Important misconception. "Git and GitHub are the same thing." They are not. You can use Git with no internet and never touch GitHub. You can use GitHub without ever learning Git commands by uploading through the website — and that is exactly the first method we will learn today.

Definition of a repository. A **repository** (often shortened to **repo**) is a project folder that is tracked, either by Git or by GitHub. When we say "create a repository," we mean "create an online folder on GitHub to hold your website's files."

F.3 Creating a free GitHub account

The whole course rule applies here: **no paid tools**. GitHub accounts are free. Follow these steps exactly.

1. Open Chrome and go to `https://github.com`.
2. Click the **Sign up** button at the top right.
3. Enter your **email address**. Use an address you can open right now, because GitHub will send a verification code. A Gmail address is fine.
4. Create a **password** (at least 8 characters, mix letters and numbers).
5. Choose a **username**. This is important: your username becomes part of your public web address. Choose something professional and simple, for example `adaeze-fashion-ph` or `blessingokoro-dev`. Avoid nicknames with numbers you will be embarrassed by later. Usernames may contain letters, numbers and hyphens only.
6. Solve the puzzle / verify you are human, then click **Create account**.
7. GitHub emails you a **code** (usually 6 digits). Type it into the box on screen. This verifies the account.
8. If asked whether you want the free or paid plan, choose **Free**.

Expected screen: After verification you land on your GitHub home dashboard with a welcome message and a **Create repository** button.

What the learner should see: Their chosen username in the top-right corner. Write that username on Worksheet 11.1 — it is part of your live URL later.

Common gap: Learners pick a username they later dislike, or one that is already taken (GitHub shows "username is not available"). If taken, add a hyphen and a short word, e.g. `adaeze-fashion-ph2`. Usernames can be changed later in Settings, but it is easier to choose well now.

F.4 Deployment Path A — the web upload method (best for beginners)

This method uses only the GitHub website. No commands, no installation. It is the method we recommend for this session.

Step 1 — Create the repository.

1. Log in to GitHub.
2. Click the `+` icon at the top right, then click **New repository**.
3. In **Repository name**, type a short, lowercase name with hyphens, e.g. `adaeze-fashion`.
4. In **Description**, type a short sentence: "My shop website for Adaeze Fashion."
5. Select **Public**. (GitHub Pages requires a Public repo on the free plan. A Private repo cannot serve Pages for free.)
6. Tick **Add a README file**. (We will edit it later.)
7. Click the green **Create repository** button.

Expected screen: You are taken to the new repository page. You can see the repo name at the top and a single file, `README.md`.

Step 2 — Upload the website files.

1. On the repository page, click the **Add file** button (near the top right), then choose **Upload files**.

2. A drop zone appears that says "Drag files here to add them to your repository."
3. Open your `CEA-WebDev-Name` folder in Windows Explorer. Select **all** your site files and folders — `index.html`, `about.html`, `contact.html`, the `css` folder, the `js` folder, the `images` folder and the `data` folder (with `products.json`).
4. Drag them into the browser drop zone, or click **choose your files** and select them. **Upload the folders too** — modern GitHub keeps the folder structure when you drag folders in.
5. Wait for the file list to appear. It should show each file with a green icon.
6. Scroll down to **Commit changes**. Leave the default message ("Add files via upload"). Click the green **Commit changes** button.

Expected screen: A blue banner says "Uploaded..." and the repository page now lists all your files: `index.html`, `css/`, `js/`, `images/`, `data/`, etc.

Critical fact to say aloud: "The `index.html` file **must be in the top level** — directly in the repository, not inside another folder. If your `index.html` is inside a folder called `website`, GitHub Pages will show a 404 page. The home page must sit at the root."

Step 3 — Enable GitHub Pages.

1. On the repository page, click the **Settings** tab (top row, with a small gear icon).
2. In the left sidebar, under the heading **Code and automation**, click **Pages**.
3. Under **Build and deployment**, find **Source**. It is a dropdown; choose **Deploy from a branch**.
4. Under **Branch**, choose **main** from the dropdown, and leave the folder dropdown as **/ (root)**. Click **Save**.
5. A notification appears: "GitHub Pages source saved." Now wait. GitHub needs **1 to 3 minutes** to build your site the first time.

Step 4 — Get and test the URL.

1. Refresh the **Pages** settings screen every minute. When it is ready, a green message appears: **"Your site is live at https://username.github.io/repo-name/"**.
2. Click that link, or copy it and paste it into a new tab.
3. Your site should appear exactly as it did on your laptop.

Expected screen: The live page with your heading, your styling, your images and a working cart. The address bar shows `https://username.github.io/adaeze-fashion/`.

Say to the class: "Congratulations — you are now published. Send that link to one person before you leave this room."

F.5 Deployment Path B — the Git command-line method (for learners who are ready)

This method is faster once you know it, and it is what professional developers use. It requires **Git for Windows** installed on your machine. These are the exact commands, with what each one does.

Open a terminal in your project folder: in VS Code, open your project, then press **Ctrl + `** (the backtick key) to open the integrated terminal. Type the commands one at a time.

```
# 1. Tell Git who you are (do this once per computer)
git config --global user.name "Blessing Okoro"
git config --global user.email "blessing@example.com"

# 2. Start tracking this folder with Git
git init

# 3. Stage all your files (tell Git which changes to save)
git add .

# 4. Save a snapshot with a message
git commit -m "First version of my shop website"

# 5. Name the main branch 'main' (GitHub's default)
git branch -M main

# 6. Connect your local folder to the GitHub repository
git remote add origin https://github.com/blessingokoro-dev/adaeze-fashion.git

# 7. Upload your files to GitHub
git push -u origin main
```

Line by line, in plain English:

- `git config --global user.name "..."` — stores your name on every commit. Run once.
- `git config --global user.email "..."` — stores your email. Use the **same email** as your GitHub account.
- `git init` — creates a hidden `.git` folder that turns your project folder into a tracked repository.
- `git add .` — the dot means "everything in this folder." Stage all files for the next commit.
- `git commit -m "..."` — saves a labelled snapshot. The message should describe what changed.
- `git branch -M main` — names the branch `main` (GitHub's default branch name).
- `git remote add origin <URL>` — links your local folder to the online repository. Use the URL GitHub shows under the green **Code** button (HTTPS tab). It ends with `.git`.
- `git push -u origin main` — uploads your commits to GitHub.

First-time authentication: GitHub will ask you to sign in or, more likely, to authenticate with a browser pop-up. Follow the pop-up and approve. Older tutorials mention typing a password — GitHub removed password authentication, so if it asks, use the browser sign-in route.

How to recognise success: After `git push`, the terminal prints lines that end with `main -> main` and `branch 'main' set up to track ...`. Refresh your GitHub repository page — your files are now there. Then enable Pages exactly as in Section F.4, Step 3.

Useful follow-up commands:

```
git status        # what has changed since the last commit?
git log --oneline # a short list of your saved snapshots
git add .         # stage new changes
git commit -m "Updated the price of the ankara dress"
git push          # upload the new commit (no need for -u the second time)
```

Limitation: this path is optional. If a learner finds the command line stressful, Path A is completely acceptable and produces the same public result.

F.6 The published URL — its structure and how to update the site

The structure of the URL. For a repository named something other than your username:

```
https://username.github.io/repo-name/
        └── your GitHub username
                  └──────── the fixed GitHub Pages domain
                            └── your repository name
```

So for username `adaeze-fashion-ph` and repository `adaeze-fashion`, the URL is:

```
https://adaeze-fashion-ph.github.io/adaeze-fashion/
```

Special case: if you name a repository exactly `username.github.io` (for example `adaeze-fashion-ph.github.io`), then the site is at the shorter address `https://adaeze-fashion-ph.github.io/`. We will not use this special case in this course; use a normal repo name.

How to update the live site after editing a file.

- **Path A (web upload):** go back to the repository, click **Add file → Upload files**, drag the edited file(s) in, and click **Commit changes**. GitHub Pages rebuilds automatically, usually within a minute. Changes are **not** instant — refresh after a short wait, and use a hard refresh (`Ctrl + F5`) if you still see the old version.
- **Path B (Git):** in your terminal run `git add .`, then `git commit -m "describe the change"`, then `git push`. The live site updates within a minute.

Say this clearly: "Editing the file on your laptop does **nothing** to the live site until you upload or push. GitHub only shows what you have sent it."

F.7 Testing the live site

After publishing, you must test like a real visitor would.

On a desktop:

1. Open the live URL in Chrome.
2. Click through every page: Home/Shop, About, Contact.
3. Add something to the cart, then refresh the page — the cart should still be there (this proves `localStorage` works on the live site).
4. Right-click the page and choose **Inspect → Console**. There should be **no red errors**. If you see red, something did not upload or a path is wrong.
5. Click **Inspect → Network**, refresh, and check that `style.css`, `app.js` and `products.json` all show a status of **200** (success). A status of **404** means that file is missing or the path is wrong.

On a phone:

1. Open WhatsApp or your email on the phone.
2. Send yourself the live URL (or type it carefully into the phone's browser).
3. Open the link. The site should load without any app.
4. Pinch and zoom; rotate the phone; check that the layout is still usable (this tests your responsive work from Session 9).
5. Test the cart and the contact form on the phone too.

Why the phone test matters: Most of your customers will open your site on a phone, not a laptop. A site that works only on a big screen is not finished.

F.8 Writing a README.md

The idea. A **README** is the front page of your repository. The name means "read me." It is written in a simple formatting language called **Markdown**, and the file is named `README.md`. GitHub displays it automatically below your file list.

Why every repository should have one. When a visitor, a classmate or an employer opens your repo, the README tells them what the project is, what it does, and how to use it — without asking you. A project with no README looks unfinished.

A README template for this course (learners may copy this into their own `README.md`):

```
# Adaeze Fashion — Online Shop

A responsive shop website for Adaeze Fashion, a clothing business in Port Harcourt.

## Features
- Product list loaded from `data/products.json`
- Live search and category filter
- Add to cart with quantity and running total (₦)
- Cart saved in the browser with localStorage
- Validated contact form
- Responsive layout for phone and desktop

## How to run
1. Download or clone this repository.
2. Open `index.html` in a browser, or run it with the VS Code Live Server extension.

## Built with
HTML, CSS and JavaScript (no frameworks, no paid tools).

## Author
Blessing Okoro — Cyber Elias Academy Web Development student.

## Live site
https://adaeze-fashion-ph.github.io/adaeze-fashion/
```

Line-by-line notes:

- `#` makes a big heading (the project name). `##` makes a smaller sub-heading.
- A line starting with `-` becomes a bullet point.
- Backticks `` ` `` around a file or code name make it look like code.
- A link in the form `[text](url)` becomes clickable. For the live-site line, learners may write the address plainly.

Worked example — editing the README on GitHub:

1. On the repository page, click the `README.md` file name.
2. Click the **pencil icon** (Edit this file) at the top right.
3. Replace the text with your own, using the template above.
4. Scroll down, and click **Commit changes**.

Expected result: The repository's front page now shows your README, formatted with a title, bullet points, and a link.

F.9 Summary of the session

- **Deploying** means putting your site on an always-on server so anyone can open it.
- **Git** = the tool that records versions, on your computer. **GitHub** = the website that hosts projects online.
- **Path A (web upload):** create repo → **Add file** → **Upload files** → **Commit changes** → **Settings** → **Pages** → **Deploy from a branch** → **main** → **/(root)** → **Save**.
- **Path B (Git):** `git init` → `git add .` → `git commit -m "..."` → `git remote add origin <url>` → `git push -u origin main`.
- The live URL is `https://username.github.io/repo-name/`, and `index.html` must be at the top level.
- A **README.md** describes the project: name, description, features, how to run, author.
- Test on **desktop and phone**, and check the Console and Network tabs for errors.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- A demonstration folder `demo-deploy/` containing `index.html`, `css/style.css` and `js/app.js`.
- A GitHub account already logged in, on the projector.
- A phone with data for the live test.

G.2 Demonstration 1 — publish a tiny site live (8 minutes)

Say: "Watch every click with me. I am going to create a repository and put a real page on the internet in five minutes. Do not touch your keyboard yet — just watch, then you will do it yourself."

Do, narrating each click:

1. Go to `https://github.com`, click `+` → **New repository**.
2. Name it `cea-demo`. Description: "Instructor demonstration." Select **Public**. Tick **Add a README file**. Click **Create repository**.
3. Click **Add file** → **Upload files**. Drag in `index.html`, then drag the `css` and `js` folders.
4. Click **Commit changes**.
5. Click **Settings** → **Pages**. Source = **Deploy from a branch**. Branch = **main**, folder = **/ (root)**. Click **Save**.
6. **Pause and predict:** "What do you think the web address will be?" Take guesses. (Answer: `https://<instructor-username>.github.io/cea-demo/`.)
7. Wait ~1–2 minutes, refresh. Point at the green message **"Your site is live at ..."**.
8. Open the URL in a new tab. The demo page appears.
9. Open the URL on the phone and hold it up to the class.

Injected error #1 (wrong folder): Go back to the repository, create a folder called `site`, and move `index.html` inside it using **Add file**. Wait, then refresh the live URL.

- **Expected result:** The live page now shows **404 — File not found**.
- **Say:** "This is the number one Pages mistake — the home page is not at the top level. Let's fix it." Move `index.html` back to the root. The site returns.

Injected error #2 (case sensitivity): Rename `index.html` to `Index.html` (with a capital I). Refresh the live URL.

- **Expected result:** On GitHub Pages (a Linux server) this now shows **404**, even though the same file name would work on Windows. Say: "GitHub Pages is **case-sensitive**. `Index.html` is not the same as `index.html`."
- Fix by renaming back to lowercase `index.html`.

Questions to ask:

- "Where do the files actually live now — my laptop or the internet?" (Both; GitHub has a copy.)
- "If I change a file on my laptop, does the live site change?" (No — only after upload/push.)
- "What must the home page file be named?" (`index.html` , lowercase, at the root.)

G.3 Demonstration 2 — update the site and write a README (7 minutes)

Say: "A site is never finished. Let's change a price and watch the live site update."

1. In the repo, click `index.html` , click the **pencil** icon, change a visible heading or price.
2. Scroll down and click **Commit changes**.
3. Wait ~1 minute, then hard-refresh (`Ctrl + F5`) the live URL.
4. **Expected result:** The change appears. **Say:** "This is why we wait — the change is not instant, but it is automatic."

Then write the README: click `README.md` , click the **pencil**, paste the SEction F.8 template, and **Commit changes**. Refresh the repo home page.

- **Expected result:** The README is now displayed, nicely formatted, at the bottom of the repository page.

Ask: "Why do we bother with a README?" (So anyone who opens the project understands it; it looks professional; it links to the live site.)

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: Each learner publishes their own project's home page (at minimum `index.html` plus its `css` and `js`) to GitHub Pages, then tests the live link.

Starting materials: The learner's `CEA-WebDev-Name` folder; internet; their GitHub account (created in Section F.3 or already existing).

Procedure (instructor calls each numbered step; learners follow; instructor walks the room):

1. Log in to `https://github.com` .
2. Click `+` → **New repository**. Name it `cea-webdev-<yourname>` (all lowercase, hyphens, no spaces).
3. Select **Public**. Tick **Add a README file**. Click **Create repository**.
4. Click **Add file** → **Upload files**.
5. From your project folder, drag in `index.html` **and** your `css` and `js` folders (and any other pages or assets you have).
6. Click **Commit changes**.
7. Click **Settings** → **Pages**.
8. Under **Source**, choose **Deploy from a branch**; choose branch **main** and folder **/ (root)**; click **Save**.
9. **Pause and predict:** "Write down what you think your live URL will be." Learners write it on Worksheet 11.1, then wait and refresh until **"Your site is live at ..."** appears.
10. Open the live URL in a new browser tab.
11. **Pause and predict:** "Before you press refresh — will your cart items still be there after a page reload?" (Yes, because they are saved in `localStorage` on the visitor's device.)
12. Add an item to the cart, refresh, and confirm it persists.
13. Press `F12` , open the **Console**, and confirm there are no red errors. Open the **Network** tab and confirm `style.css` and `app.js` show status **200**.
14. Send the live URL to the instructor (WhatsApp or the class group) and, if you have a phone with data, open it on the phone.

Expected result:

- Your site loads at `https://<your-username>.github.io/<repo-name>/` .
- Styling and JavaScript work; the cart persists across a refresh; no red errors in the Console.

Points to pause and predict:

- After step 8: "How long will it take before the site is live?" (1–3 minutes).

- After step 12: "Does clearing the browser's site data remove the cart?" (Yes — that is expected; `localStorage` lives in the browser.)

Troubleshooting while walking the room:

- **404 page** → `index.html` is not at the top level, or the file is named `Index.html`, or Pages is set to the wrong branch/folder.
- **Page loads but is unstyled** → `css/style.css` did not upload, or the path is wrong (check Network tab for a 404 on the CSS file).
- **"Your site is live" does not appear** → wait; if it still does not appear after 5 minutes, re-check that the branch is `main` and the folder is `/ (root)`.
- **Login trouble** → confirm the email was verified.

Completion checklist (learner ticks each):

- ☐ Repository created and set to Public
- ☐ `index.html` uploaded **at the top level**
- ☐ `css` and `js` folders uploaded
- ☐ GitHub Pages enabled from branch `main`, folder `/ (root)`
- ☐ Live URL opens and shows the site
- ☐ Cart persists after refresh on the live site
- ☐ No red errors in the Console; `style.css` and `app.js` show 200 in Network
- ☐ Live URL written on Worksheet 11.1

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Write a complete `README.md` for your deployed repository and publish it.

Scenario: You have just published your project and you want a visitor to understand it in ten seconds. A recruiter, a classmate or a customer should be able to read your README and know what the site is, what it does and who made it.

Requirements:

1. On your repository's home page, click `README.md`, then the **pencil (Edit this file)** icon.
2. Replace the content with a README that contains **all five** of the following, each labelled:
   - A project **title** (`# ...`).
   - A one- or two-sentence **description**.
   - A **Features** list of at least **four** bullet points.
   - A **How to run** section with at least two numbered steps.
   - An **Author** line with your name.
3. Add a **Live site** section with your real published URL.
4. Click **Commit changes**.
5. Refresh your repository home page and confirm the README displays with your headings and bullets.

Constraints:

- Use Markdown only (`#`, `##`, `-`, numbered lists). Do not paste a screenshot.
- Do not leave the default GitHub text ("Add a README file").
- The live-site URL must be the real one that opens.

Time allowance: 15 minutes.

Success criteria:

- The README displays with a title, description, a Features list of at least four items, a How-to-run section, an Author line and the live URL.
- All five required parts are clearly present.
- The URL in the README actually opens the site.

Hints:

- Hint 1: Use `## Features` before the bullet list so it appears as a heading.
- Hint 2: A bullet is a hyphen followed by a space: `- Cart saved with localStorage`.
- Hint 3: If you forget your live URL, go to Settings → Pages and copy it from the green message.

Instructor answer key (do not show learners before they attempt it):

```
# Chinedu Electronics — Product Showcase

A simple responsive product page for Chinedu Electronics, a phone-accessory shop in Port Harcourt.

## Features
- Products loaded from `data/products.json`
- Live search box that filters products as you type
- Add to cart with quantity and naira running total
- Cart saved in the browser with localStorage
- Validated contact form

## How to run
1. Download this repository as a ZIP, or clone it with Git.
2. Open `index.html` in Chrome, or use the VS Code Live Server extension.

## Author
Chinedu Okafor — Cyber Elias Academy Web Development student.

## Live site
https://chinedu-okafo.github.io/cea-webdev-chinedu/
```

Marking guidance (out of 10): title (1); clear description (2); Features list with 4+ items (3); How-to-run with steps (2); Author line (1); correct and working live URL (1).

J. REAL-WORLD CASE STUDY

Situation. *Mrs. Ngozi* sells handbags from her shop and posts photos on Instagram. In her Instagram bio she has room for exactly **one link**. She has been sending customers her WhatsApp number typed as text, but people keep mistyping it, and she cannot show all her bags in a photo feed.

Relevant facts.

- Instagram allows only one clickable link in the bio.
- Mrs. Ngozi has 5 main bag designs and wants customers to see prices and a photo of each.
- She does not have a developer and does not want to pay monthly fees.
- She has heard that "websites cost money."

Decision point. How can one free link in her Instagram bio show all her products and let customers contact her?

Possible approaches.

1. Keep sending her WhatsApp number as text and hope customers type it correctly.
2. Pay a developer a large fee for a full e-commerce site.
3. Deploy a simple static product page to GitHub Pages for free, and put that URL in her Instagram bio.

Recommended solution (for this stage). Approach 3. A GitHub Pages site is free, gives her one clean link, shows all her products with prices, and includes a contact form. A visitor taps the link, sees the bags, taps a bag, and fills the contact form — which (in this course version) can be set to open the customer's email or WhatsApp with the message pre-filled.

Example of how it helps: The bio link becomes `https://ngozi-bags.github.io/shop/`. Customers see photos, prices in naira, and a "Contact me" button. No typo, no missed messages.

Trade-offs.

- Approach 1 is free but loses sales to typos and looks unprofessional.
- Approach 2 gives a full system (payments, stock, orders) but is expensive and overkill for five bags; and it still needs maintenance.
- Approach 3 is free, takes about an hour, and is live worldwide — but it is **static**: it cannot take automatic online payments. For Mrs. Ngozi, contact-by-form or WhatsApp is enough for now, and she can upgrade later.

Professional connection. Being able to take an existing folder of files and turn it into a free public URL — with a README so others understand it — is a core junior-developer skill. Many small businesses have exactly this need, and a freelancer who can deploy to GitHub Pages can charge a fair fee for a task that costs the business nothing in hosting.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this table as the live troubleshooting guide. Ask learners to check the relevant row before you give the answer.

K.1 The live site shows "404 — There isn't a GitHub Pages site here"

Symptoms: The URL opens but GitHub shows a 404 page. **Likely/confirmed causes:** (a) `index.html` is not at the top level of the repository (it is inside a folder). (b) The file is named `Index.html` with a capital I. (c) GitHub Pages was never enabled, or was set to the wrong branch/folder. **How to investigate:** Look at the repository file list — is `index.html` visible at the very top level? Check **Settings** → **Pages** for the branch and folder. Confirm the exact spelling and case of the file name on GitHub. **Corrective steps:** Move `index.html` to the top level; rename it to all lowercase; set branch `main` and folder `/ (root)`; click **Save**; wait 1–3 minutes. **How to confirm resolved:** The URL loads your site instead of the 404 page. **Prevention:** Always name the home page `index.html` (lowercase) and keep it at the repository root.

K.2 "Your site is live" never appears

Symptoms: After saving Pages settings, no live message appears after several minutes. **Likely causes:** Pages not enabled; the wrong branch selected; a build still in progress. **Investigate:** Refresh the Pages settings page; check the **Actions** tab for a running or failed build. On the free plan, the first build can take a few minutes. **Fix:** Re-select **Deploy from a branch**, branch **main**, folder **/ (root)**, and **Save** again. Wait, then refresh. **Confirm:** Green "Your site is live at …" message appears. **Prevent:** Upload the files **before** enabling Pages.

K.3 The page loads but has no styling (and no JavaScript behaviour)

Symptoms: Plain text, no colours; the cart button does nothing. **Likely cause:** `css/style.css` or `js/app.js` did not upload, or the `<link>` / `<script>` path is wrong. **Investigate:** Press `F12` → **Network**, refresh, and look for a red **404** next to `style.css` or `app.js`. Also check the **Console** for red errors. **Fix:** Upload the missing files, or correct the path in the HTML to the correct relative path. **Confirm:** Network shows 200 for both; styling returns. **Prevent:** Always upload the `css` and `js` folders, and use relative paths.

K.4 Images do not load on the live site (but did on the laptop)

Symptoms: Broken-image icons; captions with no picture. **Likely causes:** (a) The `images` folder was not uploaded. (b) The file name case differs (`Photo.JPG` vs `photo.jpg`) — Windows is not case-sensitive but GitHub Pages is. (c) The path is a Windows absolute path like `C:\...`. **Investigate:** Network tab → find the image request → it shows 404. Compare the exact case of the file name on GitHub with the path in your code. **Fix:** Upload the images; rename to match exactly (use all-lowercase file names); change the path to a relative path such as `images/bag-one.jpg`. **Confirm:** Images appear. **Prevention:** Use lowercase names for all files and folders; never use `C:\` paths in web code.

K.5 The site shows an old version after I changed a file

Symptoms: You updated a file but the live site looks the same. **Likely cause:** The browser cached the old CSS/JS, or you did not upload/push the change. **Investigate:** Hard-refresh with `Ctrl + F5`. Check the repository to see whether the changed file is actually there. **Fix:** If the file on GitHub is old, upload/push the change. If it is new, hard-refresh. **Confirm:** The new content appears. **Prevention:** Hard refresh after every update; remember edits are not live until uploaded.

K.6 `fetch` of `products.json` fails on the live site

Symptoms: The product list is empty and the Console shows a red fetch error. **Likely cause:** The `data` folder (with `products.json`) was not uploaded, or the path is wrong. **Investigate:** In the Console, read the URL that failed. Open that URL directly — you will get a 404 if the file is missing. **Fix:** Upload `data/products.json`; correct the path in `fetch(...)`. **Confirm:** Products appear. **Prevention:** Upload every folder your site uses.

Distinguish likely vs confirmed: A cause becomes confirmed only when you see the evidence — the 404 in the Network tab, the wrong file name on GitHub, or the red Console error. Teach learners to confirm before changing code.

L. SESSION ASSIGNMENT

Assignment title: "Publish Your Project and Document It"

Purpose: Produce a professional, publicly accessible version of your project with a proper README, ready for the capstone.

Realistic scenario: You have built a shop site for a small business. The owner now wants a public link to share on WhatsApp and Instagram, and a short document describing what the site does. Your job is to deliver both.

Detailed instructions:

1. Ensure your project's `index.html` (and any other pages you have) is at the top level of your project folder, and that `css/`, `js/` and any `data/`, `images/` folders are present.
2. Create a new **Public** GitHub repository named `cea-webdev-<yourname>`.
3. Upload your files (Path A) or push them (Path B).
4. Enable GitHub Pages from branch `main`, folder `/ (root)`.
5. Confirm the live URL opens your site.
6. Write a complete `README.md` containing: title, description, Features list (4+ items), How to run, Author, and the live-site URL.
7. Test the live site on a phone as well as a desktop.
8. Write, in your workbook, your **live URL** and one thing you had to fix during deployment.

Required deliverables:

- A live public URL (`https://username.github.io/repo-name/`).
- A repository containing all site files with `index.html` at the top level.
- A completed `README.md`.
- A short note in the workbook describing one problem you met and how you fixed it.

Submission format: Post the live URL and the repository URL in the class group and/or the class folder. Paste the live URL into the workbook.

Deadline: Before Session 12 (the capstone session). Bring the live URL to the capstone — it is part of the presentation.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
Repository created & Public	3	Correct name; set to Public so Pages works
Files uploaded correctly	4	`index.html` at top level; `css` / `js` /assets all present
GitHub Pages enabled correctly	3	Branch `main`, folder `/ (root)`; site builds
Live URL works	4	Opens the real site on desktop and phone; no 404
README complete	4	Title, description, 4+ features, how-to-run, author, live URL
Problem note & no errors	2	Console clean; learner can describe a fix

Common mistakes to avoid: leaving `index.html` inside a folder; naming it `Index.html`; forgetting to upload `css` / `js`; setting Pages to the wrong branch; writing a README that is just the default text.

Instructor reference solution: A correct submission has a repository whose file list shows `index.html`, `about.html`, `contact.html`, `README.md`, `css/`, `js/` and `data/` at the top level; Pages enabled from `main`, `/ (root)`; the README matching the template in Section F.8; and a live URL that opens with working styling, a working cart that persists on refresh, and no red Console errors. The workbook note names a real problem (for example, "my CSS did not load because I forgot to upload the `css` folder") and the fix.

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. In one sentence each: what is Git, and what is GitHub?

Q2. Multiple choice. Which URL is most likely correct for a repository called `adaeze-fashion` owned by user `adaeze-ph`? a) `https://github.com/adaeze-ph/adaeze-fashion/live` b) `https://adaeze-ph.github.io/adaeze-fashion/` c) `https://adaeze-fashion.adaeze-ph.com/` d) `https://adaeze-ph.gitwebsite.io/adaeze-fashion/`

Q3. Explain in your own words. Why must the home page be named `index.html` and kept at the top level of the repository?

Q4. Practical ordering. Put these GitHub Pages steps in the correct order: a) Click **Save** under Settings → Pages. b) Click **Add file** → **Upload files** and commit. c) Click `+` → **New repository** and set it to Public. d) Copy the live URL from the green message.

Q5. Error detection. A learner's live site shows a 404. They uploaded their files, but the repository file list shows `website/index.html` and `website/css/style.css`. What is wrong, and how do they fix it?

Q6. Scenario. A classmate says, "I changed my price on my laptop but the live site still shows the old price. My site is broken." Is the site broken? What should they check first?

INSTRUCTOR ANSWER KEY (Q1–Q6)

A1. Git is a version-control tool that runs on your computer and records snapshots (commits) of your files. GitHub is a website that hosts Git projects online so they can be shared and served. **A2.** (b) `https://adaeze-ph.github.io/adaeze-fashion/`. **A3.** `index.html` is the default home page a web server looks for in a folder. For GitHub Pages, that file must be at the top level of the repository; if it is inside a sub-folder, the server finds nothing at the root and returns a 404. **A4.** Correct order: **c → b → a → d**. (Create the repo, upload files, enable Pages and Save, then copy the URL.) **A5.** The files are inside a `website` sub-folder, so there is no `index.html` at the repository root. Fix: move `index.html` (and the `css` / `js` folders) to the top level — or, if keeping the folder, set the Pages folder appropriately — then re-check. The simplest fix is to upload the contents of `website` directly to the repository root. **A6.** The site is probably **not** broken. Editing a file on the laptop does not change the live site until it is uploaded or pushed. Check first whether the change is actually in the GitHub repository (open the file on GitHub); if not, upload/push it. Then hard-refresh (`Ctrl + F5`) because the browser may be caching the old CSS/JS.

N. REFLECTION AND RECAP

Plain-English summary. Today your work left your laptop and reached the world. You learned that **Git** records versions on your computer and **GitHub** hosts projects online, and that a **repository** is a project folder on GitHub. You created a free account, made a public repository, uploaded your files, enabled **GitHub Pages**, and got a real URL of the form `https://username.github.io/repo-name/`. You wrote a **README.md** so others understand the project, and you tested the live site on a desktop and a phone. You also learned how to update the site and how to fix the four classic deployment problems: wrong folder/branch, 404, case-sensitive file names, and wrong relative paths.

Five important takeaways:

1. Git is the version tool on your computer; GitHub is the hosting website. They are different.
2. `index.html` must be lowercase and at the **top level** of the repository, or you get a 404.
3. Enable Pages with **Settings → Pages → Deploy from a branch → main → / (root) → Save**; wait 1–3 minutes.
4. Editing a file does nothing to the live site until you upload or push it.
5. Every repository needs a **README** and should be tested on a phone.

Questions to help you recognise what you can now do:

- Can I create a GitHub repository and upload my files without help?
- Can I enable GitHub Pages and find my live URL?
- Can I explain the difference between Git and GitHub to a friend?
- Can I write a README with a title, description, features and author?
- Can I diagnose a 404 on Pages?

Connection to the next session: You now have everything: a working, responsive, interactive site that is live on the internet with a README. In the **final session (Session 12)** you will assemble your full **capstone** — the 3–4 page Adaeze Fashion shop with product filtering, a persistent cart and a validated contact form — deploy it, and **present** it, showing your live URL and explaining your own code line by line. Bring your live URL today's assignment.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner creates the repository and uploads files **without** being told which button to press.
- Learner keeps `index.html` at the top level on the first try and knows why it matters.
- Learner can state the difference between Git and GitHub in their own words.
- Learner reads the **Network** tab to find a 404 instead of guessing.
- Learner writes a README that a stranger could understand.
- Learner tests the live site on a phone and reports the result.

Common misconceptions to watch for and correct:

- "Git and GitHub are the same thing." → Separate them: tool on your machine vs website online.
- "I need to pay for a web address." → GitHub Pages is free for public repos.
- "My changes appear instantly." → They do not; you must upload/push and wait, then hard-refresh.
- "If it works on my laptop it will work live." → Case-sensitivity and relative paths differ on the live server.
- "The README is just for me." → It is the first thing any visitor or employer reads.

Suggested remedial exercises for learners who struggle:

1. **Button rehearsal:** Have the learner narrate every GitHub click aloud while a partner watches, before touching the keyboard.
2. **404 drill:** Give the learner a repository with `index.html` inside a sub-folder and ask them to fix it.
3. **Case drill:** Provide a repo where `Style.css` is linked as `style.css`; ask them to find and fix the mismatch.
4. **README skeleton:** Provide half a README and ask the learner to complete the missing sections.

Differentiation:

- **Level 1 (supported):** Provide the exact repository name and a filled-in README skeleton; the learner fills in their name and URL; run Path A only.
- **Level 2 (standard):** The full assignment as written, Path A.
- **Level 3 (challenge):** Use Path B (Git command line), then create a second update commit and push it. Optionally, add a custom `404.html` page, or set up the repo as `username.github.io` for a shorter URL.

Timing guidance. Internet speed is the main risk this session. Start Section G early. If the build is slow, let learners begin writing their README (Section F.8/I) while their Pages site builds in the background. Protect the Guided Practical — a learner who leaves without a live URL has lost the main point of the session.

Fallback if the internet fails completely: Run the loopback-paced lesson: have learners write the exact URL they *would* get, draft their README in VS Code, and rehearse the GitHub click sequence on Worksheet 11.1. Collect the repositories and run the upload in a short re-run at the start of Session 12.

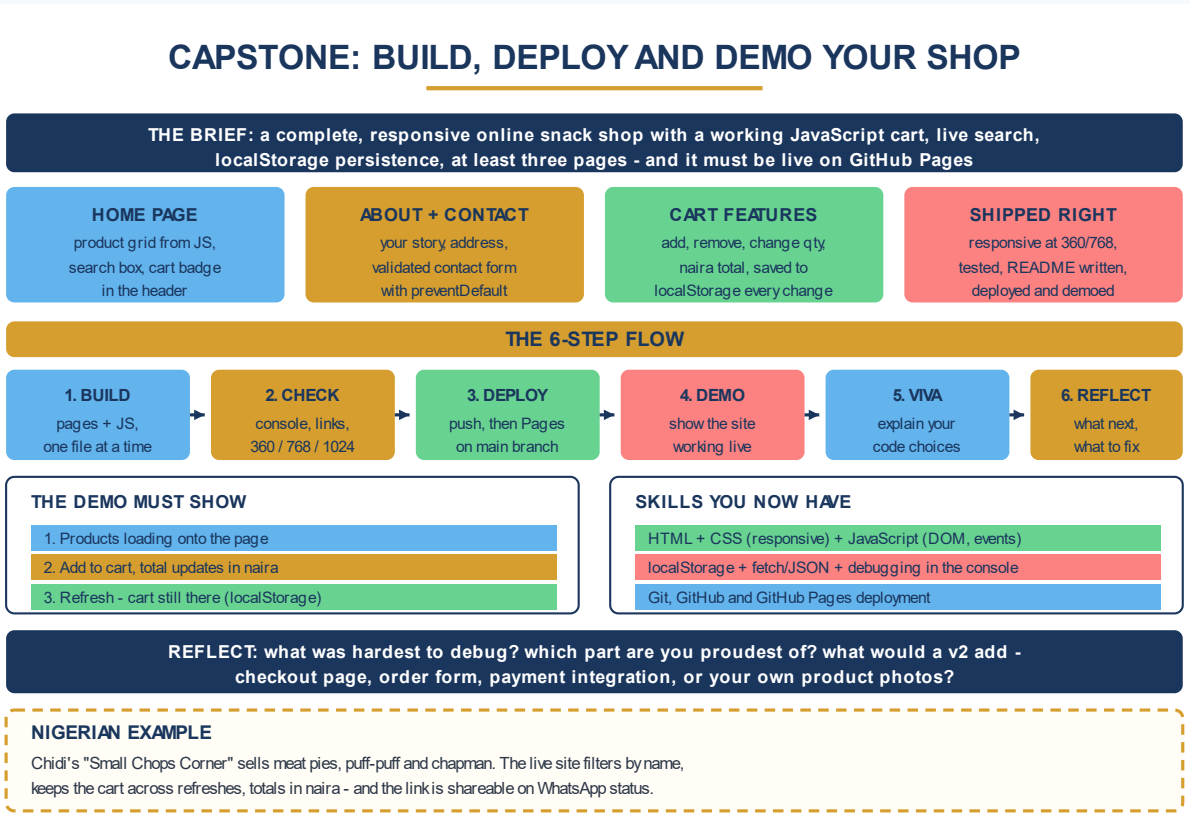
END OF SESSION 11

Next: Session 12 — Capstone Build, Deploy and Present (the final session).

SESSION 12 OF 12

CAPSTONE BUILD, DEPLOY AND PRESENT

CONCEPT DIAGRAM — SESSION 12



CYBER ELIAS ACADEMY — WEB DEVELOPMENT

SESSION 12 — CAPSTONE BUILD, DEPLOY AND PRESENT

A. SESSION IDENTIFICATION

Item	Detail
Session number	12 of 12 (final session)
Session title	Capstone Build, Deploy and Present
Week / Module	Week 4 — Module 4: Polish and Deploy
Recommended duration	105 minutes
Difficulty level	Beginner, integrating all course skills

Item	Detail
Prerequisite knowledge	Sessions 1–11. The learner must already have a multi-page site in progress, a product list from a JS array or `products.json` (Session 7), a cart with `localStorage` (Session 8), a validated contact form (Session 6), a responsive layout (Session 9), and a working GitHub Pages link (Session 11).
Connection to previous work	Every session built one piece of the capstone. This session assembles those pieces into the final **Adaeze Fashion** shop, checks each required feature against a rubric, deploys the finished site, and presents it (live demo + viva). It is the graduation session.
Preparation before class	Each learner's `CEA-WebDev-Name` folder (with capstone files) open; projectors for presentations; the rubric printed (one per learner and one per assessor); the instructor's own completed reference capstone deployed and ready to show; a score sheet for each presenter; the certificate-verification note ready (see the course-completion note at the end).

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Assemble** a complete 3–4 page interactive shop site (Home/Shop, About, Contact) that satisfies the capstone brief.
2. **Verify** their own work against a written checklist of required features before presenting.
3. **Deploy** the finished capstone to GitHub Pages and confirm a working public URL.
4. **Demonstrate** the live site, including add-to-cart, quantity change, removal and **refresh persistence**.
5. **Explain one piece of their own code line by line** in plain English, naming the file and the line.
6. **Answer viva questions** about variables, functions, the DOM, events, `fetch`, `localStorage`, validation, responsiveness and deployment.
7. **Reflect** on what they have learned and state honestly what they can now do (a junior front-end beginner) and what they should learn next.

C. REQUIRED MATERIALS

Hardware

- One computer per learner; instructor computer with projector; a phone with data for live testing.

Software

- VS Code; Google Chrome; Git for Windows (optional).

Accounts

- GitHub account (from Session 11) with the capstone repository already created and Pages enabled.

Sample files

- The instructor's completed reference capstone (`cef-adefash/` folder structure and code, given in Section L).
- The printed **Capstone Rubric** (Section L.4) and the **Viva Question Sheet** (Section L.6).

Worksheets

- Worksheet 12.1 — "Capstone Feature Checklist" (the learner ticks each required feature).
- Worksheet 12.2 — "My Code Walkthrough" (the learner writes, in advance, the code snippet they will explain).
- Worksheet 12.3 — "Presentation Order" (the instructor's running order and score sheet).

Tools

- Chrome DevTools (Console and Network) for the final live check.

Internet

- **Required** for the presentation and deployment check. If the internet is down, learners present from their local site with Live Server and re-confirm the live URL when the connection returns; marks for deployment are held and confirmed the same day.

Alternative for learners who lack a facility

- A learner whose machine fails presents from a classmate's machine using their own repository (log in and open the live URL). The repository id and URL identify the owner, so marks are unaffected.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open the instructor's own finished capstone live URL (`https://<instructor-username>.github.io/<capstone-repo>/`) on the Home/Shop page. Do not open any code.

What the instructor says:

> *"For twelve sessions you have learned one tool at a time — variables, functions, the DOM, events, `fetch` , `localStorage` , validation, responsive design, debugging, deployment. Each of those was a single brick. Today, look at this."*

Then demonstrate quickly on the projector: type in the search box and watch products filter; click **Add to cart** two or three times; change a quantity; show the running total in naira; then **refresh the whole page** and point at the cart still holding its items. Say:

> *"Everything you see here is the work of the same skills you each have. This is not magic — it is the eleven sessions joined together. Today is your day. Every one of you will show a live site like this, explain your own code, and answer a few questions. Nobody leaves empty-handed, because the build work is almost done — today is about finishing, checking, deploying and presenting."*

Expected learner response: A mix of excitement and nervousness. Normalise the nerves:

> *"You are not being tested like an examination hall. You are presenting the thing you built. If something does not work, we will fix it together, and that honesty is part of being a professional."*

Why this opener works: It shows the finished destination, reframes the session as assembly rather than a new topic, and reduces presentation fear by making the work already largely complete.

E. PREREQUISITE REVIEW (7 minutes)

Run this fast, as the capstone depends on every one of these. Put the questions on the projector and take quick answers.

Question 1: "What method loads `products.json` and turns the response into usable data?"

- Expected answer: `fetch('data/products.json').then(response => response.json()).then(data => renderProducts(data))` , with a `.catch(...)` for errors.
- Gap: learners forget `.json()` . Remind them: the first step gets the raw response; `.json()` converts it.

Question 2: "How do you open the browser's saved cart when the page loads?"

- Expected answer: `const cart = JSON.parse(localStorage.getItem('cart')) || [];` — read the string and convert it back to an array, with a fallback empty array if nothing is saved.
- Gap: forgetting `JSON.parse` (cart comes back as a string) or forgetting the `|| []` fallback (first visit crashes).

Question 3: "A form should not reload the page when submitted. What stops it?"

- Expected answer: call `event.preventDefault()` inside the submit handler.
- Gap: learners read input values before the handler runs. Remind them the handler runs on submit.

Question 4: "Name the four function behaviours the cart must support."

- Expected answer: add an item, change a quantity, remove an item, and show the running total.
- Gap: forgetting quantity and remove. These are in the brief.

Question 5: "At which screen widths do we check responsiveness?"

- Expected answer: a phone width (about 360px), a tablet width (about 768px), and a desktop width (about 1200px).

Bridge to today (say this):

> *"Those five answers are the heart of the capstone. Today we make sure each one is present, working, and deployed. Then you present."*

Gap handling: If the room is shaky on Questions 2 and 3, spend five minutes at the whiteboard writing `JSON.parse(localStorage.getItem('cart')) || []` and a short submit handler, because these two are the most common capstone failure points.

F. FULL LECTURE NOTES

F.1 What the capstone is

The idea. A **capstone** is the final project that proves you can combine everything you have learned into one working product. It is not a new topic — it is the gathering-up of all twelve sessions.

Plain-English definition. The capstone is a **3–4 page interactive shop website** for a small business. We use **Adaeze Fashion**, a clothing shop in Port Harcourt, as the running example, but a learner may use their own chosen small business.

Why it exists. Employers and clients do not care how many lessons you watched. They care what you can build. The capstone is the single artefact — a live URL — that you can show to a customer, a relative or a future employer to say: "I built this."

What "interactive" means here. The site is not a poster. It loads products from a file, filters them as you type, remembers a cart that survives a refresh, checks a form before sending, and adapts to the screen size.

Real-life comparison. The capstone is like the **final year project** of a trade apprentice: not a test paper, but a finished piece of furniture the carpenter can point to and say, "I made that." The customer can sit on the chair.

F.2 The pages of the capstone

The capstone has three to four pages, each served as an `.html` file:

1. **Home / Shop (`index.html`)** — the main page. It shows a heading and banner for Adaeze Fashion, a **search box**, a **category filter**, and the **product grid** rendered from `data/products.json`. Each product card has a picture, a name, a price in naira, an **Add to cart** button, and a **quantity** control.
2. **About (`about.html`)** — who Adaeze Fashion is: a short story, a photo, the business values. Static content is fine here; the point is that the **same navigation and footer** appear on every page.
3. **Contact (`contact.html`)** — a **validated contact form** (name, email, message) with error messages and a success message on valid submit.
4. *(Optional fourth page)* **Cart (`cart.html`)** — a dedicated page listing the cart items, quantities, subtotal and total. If you do not make a separate page, show the cart in a panel on the shop page. Either is acceptable.

Consistency rule: every page must share the same header navigation and the same footer, and links must work between all pages. A visitor must never get lost.

F.3 The required features (the capstone checklist)

Read this aloud and have learners tick Worksheet 12.1. These are non-negotiable for a passing capstone:

- ☐ **Product list** rendered from a JavaScript array **or** from `products.json` using `fetch`.
- ☐ **Live search** that filters products as the user types.

- [ ] **Category filter** (at least two categories, e.g. Dresses, Bags, Shoes).
- [ ] **Add to cart** with **quantity** for each product.
- [ ] **Cart persisted in** `localStorage` so it survives a page refresh.
- [ ] **Running total** displayed in **naira (₦)**.
- [ ] **Validated contact form** (required fields, email check, error messages, success message).
- [ ] **Responsive layout** that works on phone, tablet and desktop.
- [ ] **A public deployed URL** (GitHub Pages) that opens the finished site.
- [ ] **A README.md** describing the project, features, how to run it, and the author.

Why these exactly. Each one maps to a session: the product list to Session 7 (fetch/JSON); search/filter to Session 7; add/quantity/cart/total/persistence to Session 8; the validated form to Session 6; responsiveness to Session 9; deployment and README to Session 11. The capstone is the course, made visible.

F.4 How the product list is built (recap of the core technique)

The shop page fetches the product data and renders cards. A minimal `data/products.json`:

```
[
  { "id": 1, "name": "Ankara Dress", "price": 15000, "category": "Dresses", "image": "images/ankara-dress.jpg" },
  { "id": 2, "name": "Aso-Oke Set", "price": 42000, "category": "Dresses", "image": "images/aso-oke.jpg" },
  { "id": 3, "name": "Leather Handbag", "price": 18000, "category": "Bags", "image": "images/handbag.jpg" },
  { "id": 4, "name": "Beaded Sandals", "price": 9500, "category": "Shoes", "image": "images/sandals.jpg" }
]
```

The JavaScript that loads and renders it:

```
let allProducts = [];
let currentCart = [];

fetch('data/products.json')
  .then(response => response.json())
  .then(data => {
    allProducts = data;
    renderProducts(allProducts);
  })
  .catch(error => {
    console.error('Could not load products:', error);
  });

function renderProducts(products) {
  const grid = document.getElementById('product-grid');
  grid.innerHTML = '';
  products.forEach(product => {
    const card = document.createElement('div');
    card.className = 'card';
    card.innerHTML = `
      <img src="${product.image}" alt="${product.name}">
      <h3>${product.name}</h3>
      <p class="price">₦${product.price.toLocaleString()}</p>
      <button onclick="addToCart(${product.id})">Add to cart</button>
    `;
    grid.appendChild(card);
  });
}
```

Line-by-line notes:

- `fetch('data/products.json')` asks the server for the file. This only works from a server (Live Server or GitHub Pages), not from a double-clicked file.
- `.then(response => response.json())` converts the response into a JavaScript array.
- `.then(data => { allProducts = data; renderProducts(data); })` keeps a copy in `allProducts` (so the filter can search it) and draws the cards.
- `.catch(...)` handles errors so the page does not fail silently.
- `toLocaleString()` turns `15000` into `15,000`, so prices read nicely in naira.

F.5 Search and category filter (recap of Session 7)

```
const searchBox = document.getElementById('search');
const categorySelect = document.getElementById('category');

function applyFilters() {
  const text = searchBox.value.toLowerCase().trim();
  const category = categorySelect.value;

  const filtered = allProducts.filter(product => {
    const matchesText = product.name.toLowerCase().includes(text);
    const matchesCategory = category === 'all' || product.category === category;
    return matchesText && matchesCategory;
  });

  renderProducts(filtered);
}

searchBox.addEventListener('input', applyFilters);
categorySelect.addEventListener('change', applyFilters);
```

Why `.toLowerCase()` on both sides: the search must ignore capitals. "dress" should find "Ankara Dress". This is one of the most common capstone bugs — a filter that only works when the case matches exactly.

Why we keep `allProducts`: so that when the user clears the search, we filter from the **full** list, not from an already-filtered list. Filtering the filtered list slowly loses items.

F.6 The cart: add, quantity, remove, total, and persistence (recap of Session 8)

The rule: always store **data** (id, quantity), never the HTML. Store the cart as an array of objects and re-render it after every change.

```
const CART_KEY = 'adaeze-cart';

function loadCart() {
  const saved = localStorage.getItem(CART_KEY);
  currentCart = saved ? JSON.parse(saved) : [];
}

function saveCart() {
  localStorage.setItem(CART_KEY, JSON.stringify(currentCart));
  renderCart();
}

function addToCart(id) {
  const existing = currentCart.find(item => item.id === id);
  if (existing) {
    existing.qty += 1;
  } else {
    currentCart.push({ id: id, qty: 1 });
  }
  saveCart();
}

function changeQty(id, delta) {
  const item = currentCart.find(item => item.id === id);
  if (!item) return;
  item.qty += delta;
  if (item.qty <= 0) {
    removeFromCart(id);
    return;
  }
  saveCart();
}

function removeFromCart(id) {
  currentCart = currentCart.filter(item => item.id !== id);
  saveCart();
}
```

```
function cartTotal() {
  let total = 0;
  currentCart.forEach(item => {
    const product = allProducts.find(p => p.id === item.id);
    if (product) total += product.price * item.qty;
  });
  return total;
}
```

Line-by-line notes:

- `localStorage.getItem(CART_KEY)` reads a **string**. `JSON.parse` turns that string back into an array. If nothing is saved, we fall back to `[]`.
- `JSON.stringify` turns the array into a string before `localStorage.setItem`.
- `addToCart` checks whether the item is already in the cart. If yes, it increases `qty` by 1; if no, it adds a new `{ id, qty }` entry. This is why clicking "Add to cart" twice should show quantity 2, not two separate rows.
- `changeQty` uses `delta` (+1 or -1). If quantity drops to 0, the item is removed.
- `cartTotal` looks up the price in `allProducts` and multiplies by the quantity, then adds everything up.
- **Every change calls** `saveCart()`, which writes to `localStorage` **and** re-renders the cart badge and list.

Why prices are stored in naira as whole numbers: storing naira as integers avoids floating-point rounding errors (the "₦0.30000000000000004" problem). If a shop ever uses kobo, store the smallest unit (kobo) as integers and divide only when displaying.

F.7 The validated contact form (recap of Session 6)

```
const form = document.getElementById('contact-form');

form.addEventListener('submit', function (event) {
  event.preventDefault();

  const name = document.getElementById('name').value.trim();
  const email = document.getElementById('email').value.trim();
  const message = document.getElementById('message').value.trim();
  const emailPattern = /.+@.+\..+/;

  let errors = [];
  if (name === '') errors.push('Please enter your name.');
  if (!emailPattern.test(email)) errors.push('Please enter a valid email address.');
  if (message.length < 5) errors.push('Your message is too short.');

  const feedback = document.getElementById('feedback');
  if (errors.length > 0) {
    feedback.style.color = 'red';
    feedback.textContent = errors.join(' ');
  } else {
    feedback.style.color = 'green';
    feedback.textContent = `Thank you, ${name}. Your message has been received.`;
    form.reset();
  }
});
```

Why `preventDefault()` **first:** without it, the browser reloads the page on submit and the JavaScript runs on a fresh, empty page.

Why `.trim()` **:** so a message of only spaces is treated as empty.

Why we collect all errors before showing them: the user sees every problem at once instead of fixing one, submitting, and discovering another.

F.8 Responsive layout (recap of Session 9)

Responsiveness comes from three things:

1. The `<meta name="viewport" content="width=device-width, initial-scale=1">` line in the `<head>` of every page.

2. A flexible layout (CSS `flex` or `grid`) that wraps product cards.
3. **Media queries** that adjust for smaller screens.

```
.product-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 16px;
}

@media (max-width: 600px) {
  .product-grid { grid-template-columns: 1fr; }
  nav ul { flex-direction: column; }
  body { font-size: 16px; }
}
```

How to verify: open the live site on a phone (about 360px wide), then on a tablet (about 768px) and a desktop (about 1200px). The grid should show one column on the phone and more columns on wider screens, and the navigation should remain usable.

F.9 What "presenting" means

The idea. A **presentation** (also called a **demo**) is a short, spoken walkthrough of your finished work. In this course it lasts about **3 minutes** per learner, followed by **2 minutes** of **viva** (short questions).

Plain-English definition. A viva is an oral question-and-answer session where the instructor checks that you understand the code you wrote — not just that it runs.

Why it matters professionally. Clients do not buy code; they buy confidence. A developer who can show a live site and explain how it works wins the job. Being able to explain your own code line by line is the difference between "I copied a tutorial" and "I can build for you."

What each learner does in the 3 minutes:

1. Open the **live URL** on the projector (not the local file).
2. Show the shop, type in the search box, use the category filter.
3. Add an item to the cart, change its quantity, show the **running total in naira**.
4. **Refresh the page** and show that the cart is still there (persistence).
5. Submit the contact form once with an error and once successfully.
6. Open one file in VS Code and **explain one function line by line** in plain English.
7. Answer the instructor's viva questions.

Honest expectation to set: the site does not have to be beautiful. It has to **work** and you have to **understand** it. A plain, working, well-explained shop beats a fancy, broken one every time.

F.10 Summary of the session

- The capstone is a 3–4 page interactive shop: Home/Shop, About, Contact (and optional Cart).
- Required features: product list, live search, category filter, add-to-cart with quantity, `localStorage` persistence, naira total, validated form, responsive layout, public URL, README.
- The cart stores **data** (id + qty), never HTML, and saves after every change.
- `.toLowerCase()` on both sides makes search case-insensitive; keep `allProducts` for filtering.
- Presentation = live demo + refresh persistence + line-by-line code explanation + viva.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- The instructor's completed reference capstone, **already deployed** to GitHub Pages.
- The same files open in VS Code.

- The printed **Capstone Rubric** (Section L.4) on the whiteboard or projector.
- A stopwatch.

G.2 Demonstration 1 — a model 3-minute presentation (7 minutes)

Say: "I will now do exactly what each of you will do, so you know what to expect. Watch the clock and notice the order."

The instructor performs, narrating:

1. Opens the **live URL** (not a local file). Points out that the address is `https://...github.io/...`.
2. Types `dress` in the search box → the grid filters to dresses only. Presses the category dropdown to `Bags` → the grid changes.
3. Clicks **Add to cart** on a handbag; clicks it again → quantity shows 2. Changes the quantity with the **+** and **−** buttons and shows the total update to, say, `₦36,000`.
4. **Refreshes the page** in front of the class. The cart still shows the handbag ×2 and the total is the same. **Say:** "This is the persistence mark — `localStorage`. If it vanished on refresh, that mark would be lost."
5. Submits the contact form with an empty name → a red error appears. Then fills it correctly → a green success message appears.
6. Opens `js/app.js` in VS Code and explains `addToCart` **line by line** (the `find`, the `if`, the push, the `saveCart()`).
7. **Say:** "Three minutes. Live URL, search, cart with refresh, form, and one function explained. That is the whole presentation."

Questions to ask: "Which feature did I prove with the refresh?" (persistence) ; "Why did I show the form failing first?" (to prove validation actually works).

G.3 Demonstration 2 — injecting and fixing a capstone-class bug (8 minutes)

Say: "Now I will break the cart the way many of you accidentally will, and we will fix it together."

1. In `saveCart()`, remove `JSON.stringify` so it saves the array directly:

```
localStorage.setItem(CART_KEY, currentCart);   // WRONG
```

2. Add an item and refresh.
3. **Expected result:** The cart shows `[object Object]` or fails to load; the Console may show a red error about `JSON.parse`. **Say:** "This is the number one capstone bug — forgetting `JSON.stringify` / `JSON.parse`. `localStorage` only stores **strings**."
4. Fix it back to `localStorage.setItem(CART_KEY, JSON.stringify(currentCart));` and confirm the cart works again.

Second injected bug — the case-sensitive filter:

1. In `applyFilters`, remove `.toLowerCase()` from the search comparison, so it reads `product.name.includes(text)`.
2. Type `Dress` (capital D). **Expected result:** nothing is found, even though "Ankara Dress" exists. **Say:** "Search must ignore case — put `.toLowerCase()` on both the typed text and the product name."
3. Fix, and confirm `dress`, `Dress` and `DRESS` all find the product.

Ask: "How would you have spotted the cart bug without me showing you?" (Open the Console and read the red error; check the Application tab → Local Storage to see what was saved.)

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: Finish and self-check the capstone against the required-features list, using the instructor's step order, so that every learner has a complete deployable site by the end of the exercise.

Starting materials: The learner's `CEA-WebDev-Name` folder with whatever capstone work they already have; the feature checklist (Worksheet 12.1).

Procedure (the "finishing sprint" — instructor calls each step; learners work):

1. Open your project in VS Code with **File → Open Folder**.
2. On `index.html`, confirm the **viewport** line is present and that `products.json` is fetched from `data/products.json`.
3. Run the site with **Live Server** (right-click `index.html` → **Open with Live Server**) so `fetch` works.

4. **Pause and predict:** "Will the products load if you open the file by double-clicking instead of Live Server?" (No — `fetch` is blocked from `file://`. This is why we use a server, and it works on GitHub Pages.)
5. Confirm the product grid renders **all** products and that each card shows name, naira price and an **Add to cart** button.
6. Test **search** with lower and upper case; test the **category** filter.
7. Add an item to the cart; add it again; confirm the **quantity** becomes 2 and the **total** updates in naira.
8. **Pause and predict:** "Now refresh the page — will the cart survive?" Refresh and confirm. If it does not, check that `JSON.stringify` is used when saving and `JSON.parse` when loading.
9. Change a quantity with + and −, then remove an item; confirm the total recalculates.
10. Go to `contact.html`; submit the form empty (expect a red error), then correctly (expect a green success).
11. Open DevTools `F12` → **Console**; confirm **no red errors**. Open the **Network** tab; confirm `products.json`, `style.css` and `app.js` all show **200**.
12. Tick every item on Worksheet 12.1. Anything unticked is what you fix in the independent sprint next.

Expected result: A running local capstone with every required feature working, no red Console errors, and all checklist items ticked.

Points to pause and predict: after step 4 (fetch needs a server); after step 8 (persistence); after step 10 (validation should block, not accept).

Troubleshooting while walking the room:

- Products do not load → `fetch` path wrong, or opened with `file://` instead of Live Server.
- Cart empty after refresh → missing `JSON.parse` / `JSON.stringify`.
- Search finds nothing for a correct word → missing `.toLowerCase()`.
- Form reloads the page → missing `event.preventDefault()`.
- "Cannot read properties of null" → an element `id` in the HTML does not match the `getElementById` in JS (a typo).

Completion checklist: the same ten items as Worksheet 12.1 (Section F.3).

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Complete, deploy and rehearse the capstone independently, then write your code-walkthrough notes.

Scenario: This is presentation day. You must walk on stage with a **live URL** and be ready to explain one function. Use these 15 minutes to fix the last missing feature, re-deploy, and prepare.

Requirements:

1. Fix any feature still unticked on Worksheet 12.1.
2. Re-deploy: if your changes are local, upload/push them (Session 11, Path A or B), then confirm the **live URL** shows the updated site (hard-refresh with `Ctrl + F5`).
3. On the deployed site, run the full demo sequence once: search, filter, add-to-cart, quantity, refresh persistence, form validation.
4. On Worksheet 12.2, write down **the exact function you will explain** and the plain-English meaning of each of its lines.
5. Write one sentence naming the feature you are most proud of and one thing you would improve with more time.

Constraints:

- The presentation must use the **live URL**, not a local file.
- The function you explain must be **your own work** in your project (`addToCart`, `applyFilters`, `cartTotal`, or your form handler).
- Maximum of **two instructor hints** during the sprint — try, investigate the Console, then ask.

Time allowance: 15 minutes.

Success criteria:

- Every required feature works on the live URL.
- The cart persists across a refresh on the live site.
- Your Worksheet 12.2 contains a real function from your code with line-by-line notes.

- You can state your live URL from memory.

Hints:

- Hint 1: If the live site is old, first check whether the change is on GitHub; if not, upload/push it, then hard-refresh.
- Hint 2: If a feature is half-done, do the smallest working version — a simple search box and a simple cart are worth full marks; a fancy broken one is not.
- Hint 3: Read the Console; it usually tells you the exact line of the error.

Instructor answer key: the reference capstone in Section L fully satisfies this brief. A learner's work is acceptable if all ten checklist items are present and working on the live URL, even if the design is plain.

J. REAL-WORLD CASE STUDY

Situation. *Emmanuel* finished this course three months ago. A small bakery, *Sweet Crumb*, asked him to build a website. He built a 3-page site with a product list of cakes, a search box, a simple cart, a validated order form, and deployed it free on GitHub Pages. He charged ₦45,000 for the build and showed the owner how to update prices by editing `products.json`.

Relevant facts.

- The bakery had been taking orders only by phone and losing track of custom cake requests.
- Emmanuel's site is static; it does not take online payment.
- The owner can edit `products.json` and the order form opens the customer's email app with the order message pre-filled.
- Within a month, the bakery reported more enquiries because customers could see prices and cake photos before calling.

Decision point. The owner now wants to (a) accept online payments and (b) let staff log in to mark orders as "ready". Emmanuel must decide whether he can do this himself or must learn more first.

Possible approaches.

1. Tell the owner the site "cannot do that" and stop.
2. Try to fake payments and logins with JavaScript alone.
3. Explain honestly that payments and logins need a **backend** (a server with a database), and either learn backend next or bring in a partner who knows it.

Recommended solution. Approach 3. This is the honest professional answer and the one that protects Emmanuel's reputation. Static sites and GitHub Pages cannot securely take payments or verify a login, because anything in front-end JavaScript can be seen and changed by the visitor. Emmanuel should say what the current site **can** do (show products, collect order messages, filter, cart) and what needs a next step (a server).

Example — what Emmanuel says to the owner: "The current site is perfect for showing your cakes and collecting order messages, and it costs nothing to host. To take payments safely and manage staff logins, we need a real server and database — that is a different skill. I can learn it, or work with a partner. I will not fake it on the front end, because that would put your customers' money at risk."

Trade-offs.

- Approach 1 loses a client and is dishonest about what is possible.
- Approach 2 is dangerous: a fake payment system can look real and lose the bakery money.
- Approach 3 is honest, keeps trust, and gives Emmanuel a clear learning path (backend: Node.js, PHP, or a payment provider like Paystack).

Professional connection. Knowing the boundary of what you can build — and communicating it honestly — is a mark of a professional. Junior developers who pretend to have skills they lack damage their reputation; those who say "I can do this now and learn that next" get rehired.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this as the capstone's live troubleshooting guide.

K.1 The cart disappears after refreshing

Symptoms: Add to cart works, but after a page refresh the cart is empty. **Likely/confirmed cause:** The cart is not saved with `JSON.stringify`, or not loaded with `JSON.parse`. **Investigate:** Open DevTools → **Application** → **Local Storage** and look at the cart key. If it holds `[object Object]` or is missing, the save is wrong. If the Console shows a `JSON.parse` error, the load is wrong. **Fix:** Save with `localStorage.setItem(key, JSON.stringify(cart))`; load with `JSON.parse(localStorage.getItem(key)) || []`. **Confirm:** The cart survives a refresh. **Prevent:** Always save after every cart change and load once at start-up.

K.2 The search or filter stops working (or "loses" items)

Symptoms: Typing a capital letter finds nothing, or after clearing the search some products are missing. **Likely cause:** Missing `.toLowerCase()` on the typed text/product name, or filtering the already-filtered list instead of the full list. **Investigate:** Type the product name in the exact same case as the data — if it works then, it is a case problem. **Fix:** Apply `.toLowerCase()` to both sides; keep an untouched `allProducts` array and filter from it every time. **Confirm:** Any case finds the product; clearing search restores all products. **Prevent:** Never overwrite `allProducts` when filtering.

K.3 Products do not appear on the live site (but appear locally)

Symptoms: The grid is empty online; the Console shows a red `fetch` error. **Likely causes:** `products.json` was not uploaded; the path is wrong; or the page was opened without a server locally. **Investigate:** In the Console, read the failed URL; open it directly (404 = missing file). Locally, confirm you are using Live Server, not a double-clicked file. **Fix:** Upload the `data` folder; correct the path; always use a server for `fetch`. **Confirm:** Products appear locally and online. **Prevent:** Upload every folder; test on the live URL.

K.4 "Cannot read properties of null (reading 'value')"

Symptoms: Red Console error; a feature does nothing. **Likely cause:** A typo between the HTML `id` and the `getElementById('...')` in JavaScript, or the script runs before the element exists. **Investigate:** Copy the `id` from the HTML and compare it letter-for-letter with the JS. Check that the `<script>` is at the bottom of the body. **Fix:** Match the ids exactly; keep the script at the bottom. **Confirm:** The feature works and the error is gone. **Prevent:** Name ids carefully; test in the Console with `document.getElementById('theId')`.

K.5 The contact form accepts empty inputs or reloads the page

Symptoms: Clicking submit reloads the page, or an empty form is "accepted". **Likely cause:** Missing `event.preventDefault()`, or validation not checking `.trim()`. **Investigate:** Look at the submit handler — is `preventDefault()` the first line? **Fix:** Call `event.preventDefault()` first; validate each field after trimming. **Confirm:** Empty submit shows a red error and does not reload. **Prevent:** Always start a submit handler with `preventDefault()`.

K.6 The total is wrong or shows odd decimals

Symptoms: The total is off by small fractions (e.g. `₦300.00000000000006`) or does not update. **Likely cause:** Floating-point maths on prices, or the total is not recalculated after a change. **Investigate:** Log the individual line totals. **Fix:** Store naira as whole numbers (or kobo as integers); recalculate the total inside `saveCart()` / `renderCart()` after every change. **Confirm:** The total is a clean naira figure. **Prevent:** Avoid decimals in prices; re-render after every change.

K.7 The live URL shows an old version or a 404

Symptoms: The presentation link shows the previous version, or GitHub's 404 page. **Likely cause:** Changes were not uploaded/pushed, the browser cached the old CSS/JS, or `index.html` is not at the repository root. **Investigate:** Open the repository on GitHub and check the file is there; confirm `index.html` is at the top level. **Fix:** Upload/push; hard-refresh with `Ctrl + F5`; fix the file location. **Confirm:** The live URL shows the current site. **Prevent:** Re-deploy at the end of the sprint and test before presenting.

Distinguish likely vs confirmed: check the Console, the Network tab or the Application tab and see the actual evidence before changing code.

L. SESSION ASSIGNMENT — THE CAPSTONE

L.1 Assignment title and purpose

Title: "Adaeze Fashion — Interactive Shop (Capstone)."

Purpose: To prove, in one deployed artefact, that the learner can combine variables, functions, arrays/objects, the DOM, events, `fetch`, `localStorage`, form validation, responsive design, debugging and deployment, and can explain their own code.

L.2 Scenario and brief

You are a junior web developer hired by **Adaeze Fashion**, a small clothing shop in Port Harcourt. The owner wants a simple, free-to-host website that shows her products and lets customers build a cart before contacting her. Build a **3–4 page** site and deploy it.

Pages: Home/Shop (`index.html`), About (`about.html`), Contact (`contact.html`) — and optionally Cart (`cart.html`). All pages share the same navigation and footer.

L.3 Detailed instructions and deliverables

1. Build the pages and the shared navigation.
2. Store products in `data/products.json` (at least **8 items**, each with `id`, `name`, `price`, `category`, `image`).
3. Render the product grid with `fetch` and `.forEach`, sorted or grouped as you like.
4. Add **live search** and a **category filter**.
5. Implement **add to cart**, **quantity change**, **remove**, and a **running total in naira**.
6. **Persist the cart in** `localStorage` so it survives a refresh.
7. Build a **validated contact form** with error and success messages.
8. Make the layout **responsive** and test at 360px, 768px and 1200px.
9. Write a **README.md**.
10. **Deploy** to GitHub Pages and confirm a working **live URL**.

Deliverables: the live URL; the repository (all files, `index.html` at the root); the `README.md`; a **3-minute live demo**; and a **viva**.

Submission format: live URL posted in the class group and repository link submitted; the demo is performed in class.

Deadline: Presented during Session 12; final live URL confirmed the same day.

L.4 CAPSTONE RUBRIC (30 marks)

Mark each row from the performance descriptions. Total out of 30. (The course also records a separate 5-mark viva/demo component; the Presentation/viva row below contributes to it.)

#	Criterion	Marks	Excellent (full marks)	Satisfactory (about half)	Needs work (low/zero)
1	**Functionality** (pages, nav, product list)	5	3–4 pages; consistent nav/footer; all 8+ products render from JSON with images and prices	Pages exist but nav inconsistent or some products missing	One page only, or products hard-coded/broken
2	**Cart logic** (add, quantity, remove)	4	Add, +/− quantity, remove all work; no duplicate rows; total recalculates each time	Add works but quantity or remove missing/buggy	Add does not work or cart is broken
3	**Persistence** (`localStorage`)	4	Cart fully survives a refresh; correct `JSON.stringify` / `parse`	Cart saves but not always loads, or only sometimes persists	No persistence; cart lost on refresh
4	**Validation** (contact form)	3	`preventDefault`; required/email checks; clear errors + success message	Some checks but weak messages or missing `preventDefault`	No validation, or form does nothing
5	**Responsiveness**	3	Works cleanly at ~360/768/1200px; grid and nav adapt; viewport meta present	Mostly usable but breaks or overflows on phone	Fixed-width; unusable on phone
6	**Code quality**	3	Clear names, organised files, comments where useful, no Console	Works but messy names or minor errors	Unreadable, large errors, or code copied

#	Criterion	Marks	Excellent (full marks)	Satisfactory (about half)	Needs work (low/zero)
			errors	left in	without understanding
7	**Documentation / README**	2	Title, description, 4+ features, how-run, author, live URL	Partial README missing sections	No README or default text only
8	**Deployment**	3	Public GitHub Pages URL opens the finished site, `index.html` at root	URL works but old version or minor 404 on an asset	No working URL / 404 page
9	**Presentation / viva**	4	Live demo fully shown; refresh persistence demonstrated; one function explained line by line; viva answers correct	Demo mostly shown; explanation thin; some viva gaps	No live demo, or cannot explain own code
	TOTAL	**30**			

Pass condition (course design): 50% or more overall **and** a deployed, functional app. The capstone is a required component.

L.5 Viva/presentation — separate 5-mark component

The course records a **5-mark viva/demo** component in addition to the 30-mark capstone. Assess it during the same 3-minute demo:

Item	Marks
Uses the **live URL** (not local)	1
Demonstrates cart **persistence across refresh**	1
Explains **one code function line by line**	2
Answers viva questions correctly	1

L.6 VIVA QUESTIONS WITH MODEL ANSWERS (instructor-only)

1. **"What does `localStorage` store, and why do we convert with `JSON.stringify` and `JSON.parse`?"** → It stores strings only. Our cart is an array of objects, so we `stringify` it to a string when saving and `parse` it back to an array when loading.
2. **"Why does the search work whether I type 'dress' or 'Dress'?"** → Because we call `.toLowerCase()` on both the typed text and the product name before comparing.
3. **"Show me where you add 1 to the cart quantity."** → Points to the `addToCart` branch that does `existing.qty += 1`, and explains the `find` that checks whether the item already exists.
4. **"Why do we need `event.preventDefault()` in the form?"** → Without it, the browser reloads the page on submit and our JavaScript runs on an empty page; `preventDefault` stops the default reload.
5. **"Why must `index.html` be at the top level for GitHub Pages?"** → The server looks for `index.html` as the default home page in the root folder of the site; if it is inside a sub-folder, the root has no home page and returns a 404.
6. **"What would happen if I opened your site by double-clicking the file instead of using a server?"** → `fetch` would fail due to browser security (CORS); that is why we use Live Server locally and GitHub Pages online.
7. **"How did you make the site work on a phone?"** → Viewport meta tag, a flexible grid/flex layout, and media queries tested at phone width, tablet width and desktop width.
8. **"If you had one more week, what would you add, and what would it require?"** → E.g. online payments or customer logins, which need a **backend** (server + database), because front-end code cannot keep secrets safely.

L.7 Common mistakes to avoid

Missing `preventDefault`; forgetting `JSON.stringify` / `parse`; forgetting `.toLowerCase()` in search; filtering the filtered list; wrong `id` spelling causing `null` errors; not uploading folders so assets 404; and presenting from the local file instead of the live URL.

L.8 Instructor reference solution

A representative, complete structure that satisfies the brief:

```
CEA-WebDev-Blessing/
├── index.html          (Home / Shop)
├── about.html          (About)
├── contact.html        (Contact + validated form)
├── README.md
├── css/
│   └── style.css       (layout, cards, nav, media queries)
├── js/
│   └── app.js          (fetch, render, filter, cart, validation)
├── data/
│   └── products.json   (8 products)
└── images/
    ├── ankara-dress.jpg
    ├── aso-oke.jpg
    ├── handbag.jpg
    └── ... (one per product)
```

Key snippets:

`index.html` (shop page skeleton):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Adaeze Fashion — Shop</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    <h1>Adaeze Fashion</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="about.html">About</a>
      <a href="contact.html">Contact</a>
      <a href="index.html#cart">Cart (<span id="cart-count">0</span>)</a>
    </nav>
  </header>

  <main>
    <input id="search" type="text" placeholder="Search products...">
    <select id="category">
      <option value="all">All</option>
      <option value="Dresses">Dresses</option>
      <option value="Bags">Bags</option>
      <option value="Shoes">Shoes</option>
    </select>

    <section id="product-grid" class="product-grid"></section>

    <section id="cart">
      <h2>Your Cart</h2>
      <ul id="cart-items"></ul>
      <p>Total: <strong id="cart-total">₦0</strong></p>
    </section>
  </main>

  <footer>&copy; 2026 Adaeze Fashion, Port Harcourt.</footer>

  <script src="js/app.js"></script>
</body>
</html>
```

`js/app.js` (core logic — abridged but complete in behaviour):

```
let allProducts = [];
let currentCart = [];
const CART_KEY = 'adaeze-cart';

// Load products
```

```
fetch('data/products.json')
  .then(r => r.json())
  .then(data => { allProducts = data; renderProducts(allProducts); })
  .catch(err => console.error('Product load failed:', err));

// Load saved cart
currentCart = JSON.parse(localStorage.getItem(CART_KEY)) || [];

function renderProducts(products) {
  const grid = document.getElementById('product-grid');
  grid.innerHTML = '';
  products.forEach(p => {
    const card = document.createElement('div');
    card.className = 'card';
    card.innerHTML = `
      <img src="${p.image}" alt="${p.name}">
      <h3>${p.name}</h3>
      <p class="price">₦${p.price.toLocaleString()}</p>
      <button onclick="addToCart(${p.id})">Add to cart</button>`;
    grid.appendChild(card);
  });
}

function applyFilters() {
  const text = document.getElementById('search').value.toLowerCase().trim();
  const cat = document.getElementById('category').value;
  const filtered = allProducts.filter(p =>
    p.name.toLowerCase().includes(text) &&
    (cat === 'all' || p.category === cat)
  );
  renderProducts(filtered);
}
document.getElementById('search').addEventListener('input', applyFilters);
document.getElementById('category').addEventListener('change', applyFilters);

function addToCart(id) {
  const existing = currentCart.find(i => i.id === id);
  if (existing) existing.qty += 1;
  else currentCart.push({ id, qty: 1 });
  saveCart();
}

function changeQty(id, delta) {
  const item = currentCart.find(i => i.id === id);
  if (!item) return;
  item.qty += delta;
  if (item.qty <= 0) { removeFromCart(id); return; }
  saveCart();
}

function removeFromCart(id) {
  currentCart = currentCart.filter(i => i.id !== id);
  saveCart();
}

function cartTotal() {
  return currentCart.reduce((sum, item) => {
    const p = allProducts.find(x => x.id === item.id);
    return p ? sum + p.price * item.qty : sum;
  }, 0);
}

function saveCart() {
  localStorage.setItem(CART_KEY, JSON.stringify(currentCart));
  renderCart();
}

function renderCart() {
  const list = document.getElementById('cart-items');
  list.innerHTML = '';
  currentCart.forEach(item => {
    const p = allProducts.find(x => x.id === item.id);
    if (!p) return;
    const li = document.createElement('li');
```

```
    li.innerHTML = `
      ${p.name} — ₦${p.price.toLocaleString()}
      <button onclick="changeQty(${item.id}, -1)">−</button>
      ${item.qty}
      <button onclick="changeQty(${item.id}, 1)">+</button>
      <button onclick="removeFromCart(${item.id})">Remove</button>`;
    list.appendChild(li);
  });
  const count = currentCart.reduce((n, i) => n + i.qty, 0);
  document.getElementById('cart-count').textContent = count;
  document.getElementById('cart-total').textContent = `₦${cartTotal().toLocaleString()}`;
}

// Initial render once products arrive
renderCart();
```

Note for the instructor: `renderCart()` is called at the end and again inside `saveCart()`. Because products load asynchronously, if the cart renders before products arrive, prices may briefly show `₦0`; the `renderProducts` step should also call `renderCart()` after `allProducts` is set. Point this out to Level 3 learners as a real-world async-ordering detail.

`contact.html` form + handler (inside `js/app.js` or its own file):

```
const form = document.getElementById('contact-form');
if (form) {
  form.addEventListener('submit', function (event) {
    event.preventDefault();
    const name = document.getElementById('name').value.trim();
    const email = document.getElementById('email').value.trim();
    const message = document.getElementById('message').value.trim();
    const emailOk = /.+@.+\..+/.test(email);
    const feedback = document.getElementById('feedback');

    if (name === '' || !emailOk || message.length < 5) {
      feedback.style.color = 'red';
      feedback.textContent = 'Please fill all fields with a valid name, email and message.';
    } else {
      feedback.style.color = 'green';
      feedback.textContent = `Thank you, ${name}. We will reply to ${email} soon.`;
      form.reset();
    }
  });
}
```

`css/style.css` (essentials):

```
* { box-sizing: border-box; }
body { font-family: Arial, sans-serif; margin: 0; color: #222; }
header { background: #7b2d26; color: #fff; padding: 12px 16px; }
nav a { color: #fff; margin-right: 12px; text-decoration: none; }
.product-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 16px; padding: 16px;
}
.card { border: 1px solid #ddd; border-radius: 8px; padding: 10px; text-align: center; }
.card img { width: 100%; height: 180px; object-fit: cover; border-radius: 6px; }
.price { font-weight: bold; color: #7b2d26; }
@media (max-width: 600px) {
  .product-grid { grid-template-columns: 1fr; }
  nav a { display: block; margin: 6px 0; }
}
```

`README.md`: use the template from Session 11, Section F.8, adapted to Adaeze Fashion.

Deployment: follow Session 11, Section F.4 (upload `index.html`, `about.html`, `contact.html`, `css/`, `js/`, `data/`, `images/` with `index.html` at the root) or Section F.5 (Git push), then enable Pages from `main`, `/ (root)`.

M. KNOWLEDGE CHECK (Course Review)

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. What is the difference between `let` and `const`? Give one example of each from your capstone.

Q2. Multiple choice. Which line correctly loads a saved cart from `localStorage`? a) `let cart = localStorage.getItem('cart');` b) `let cart = JSON.parse(localStorage.getItem('cart')) || [];` c) `let cart = JSON.stringify(localStorage.getItem('cart'));` d) `let cart = localStorage.cart;`

Q3. Explain in your own words. Why do we use `.toLowerCase()` on both the search text and the product name?

Q4. Practical identification. What does each do? a) `document.getElementById('search').addEventListener('input', applyFilters);` b) `event.preventDefault();` c) `localStorage.setItem('adaeze-cart', JSON.stringify(cart));`

Q5. Error detection. Find and fix the bug:

```
function addToCart(id) {
  cart.push({ id: id, qty: 1 });
  renderCart();
}
```

(Given: clicking the same product twice should increase the quantity, and the cart should persist after refresh.)

Q6. Scenario. A learner's live site shows products but no styling. The repository lists `index.html`, `js/app.js` and `data/products.json` but no `css` folder. What happened, and what are the two things to check?

Q7. Deployment. Why must `index.html` be at the top level of the repository for GitHub Pages?

INSTRUCTOR ANSWER KEY (Q1–Q7)

A1. `let` can be reassigned; `const` cannot. Examples: `let currentCart = []` (changes as items are added) and `const CART_KEY = 'adaeze-cart'` (never changes). **A2.** (b). **A3.** Search must ignore capitalisation so that "Dress", "dress" and "DRESS" all match. `toLowerCase()` on both sides makes the comparison case-insensitive. **A4.** a) Attaches a function that runs every time the user types in the search box; b) stops the form's default page reload on submit; c) converts the cart array to a string and saves it in the browser under the key `adaeze-cart`. **A5.** The bug adds a new row every time instead of increasing the existing item's quantity, and it never persists. Fix:

```
function addToCart(id) {
  const existing = cart.find(item => item.id === id);
  if (existing) {
    existing.qty += 1;
  } else {
    cart.push({ id: id, qty: 1 });
  }
  saveCart();   // which does JSON.stringify + localStorage.setItem + renderCart
}
```

A6. The `css` folder (with `style.css`) was not uploaded. Check (1) whether `style.css` exists in the repository, and (2) whether the `<link rel="stylesheet" href="css/style.css">` path in the HTML matches the uploaded location. Upload the folder and correct the path. **A7.** A web server treats `index.html` in a folder as the default home page. GitHub Pages serves the repository root, so the root must contain `index.html`; otherwise the root has no home page and shows a 404.

N. REFLECTION AND RECAP

Plain-English summary. Today you assembled, finished, deployed and presented your **capstone** — the Adaeze Fashion shop. It loads products from a JSON file with `fetch`, filters them with live search and a category dropdown, lets customers add items and change quantities, keeps the cart alive across a refresh using `localStorage`, shows a running total in naira, validates the contact form, and adapts to any screen size. It is live on the internet, documented with a README, and you can explain your own code. You have completed the full arc from "what is JavaScript?" to a real, public web application.

Five important takeaways (and what you can now do):

1. You can build a **multi-page, interactive website** from scratch with HTML, CSS and JavaScript.
2. You can **load data** from a file and **render** it, and **filter** it in real time.
3. You can build a **cart** that **remembers** the customer with `localStorage`.
4. You can **validate** a form and give clear feedback.
5. You can **debug, deploy** to a public URL, **document** your project, and **explain** your code — the skills that turn a learner into a junior developer.

Reflection questions (write short answers):

- Which feature are you most proud of, and why?
- Which feature gave you the most trouble, and how did you solve it?
- If you showed this site to a real shop owner tomorrow, what would you improve first?
- Can you explain one function from your code without looking?

Honest statement about where you are: Finishing this course makes you a **junior front-end beginner** — someone who can build and deploy interactive, static websites. It does **not** yet make you a full software engineer. You are not yet doing backend development (servers and databases), advanced architecture, testing at scale, or production security. That is the honest picture, and it is a strong, employable starting point.

What to learn next (choose a path):

- **More JavaScript (deepen the front end):** asynchronous JavaScript, modules, error handling, then a framework such as **React** or **Vue**.
- **Backend development (full-stack): Node.js** (with Express) or **PHP**, plus a database (**MongoDB** or **MySQL**), REST APIs, and authentication — this is what unlocks online payments and logins.
- **A payment/backend service:** learn **Paystack** or **Flutterwave** APIs once you have a backend, to add real online payments to a shop like Adaeze Fashion.
- **Freelancing skills:** portfolio building, pricing, and client communication — you already have a deployable product to show.

Course wrap-up. Twelve sessions ago, a website was something other people made. Today you built one, published it, and can explain how it works. Cyber Elias Academy: *From Zero to Expert, Together.* Come back for the advanced tracks, and bring the sites you build next.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner's **live URL** opens the finished site, and the cart survives a refresh on the live URL.
- Learner explains a function **in their own words**, not by reading the code aloud.
- Learner can point to where persistence happens (`JSON.stringify` / `parse`) and where validation happens (`preventDefault` + checks).
- Learner reads the Console/Network tab to diagnose an issue during the sprint.
- Learner honestly describes which features are complete and which are partial.

Common misconceptions to watch for and correct:

- "The cart is stored on the server." → It is stored in each visitor's own browser (`localStorage`); it is per-device.
- " `localStorage` can store the array directly." → It stores strings only; use `JSON.stringify` / `parse`.
- "The site is only done if it looks beautiful." → Working and explainable beats pretty; design can improve later.
- "Static sites can take payments." → They cannot securely; payments need a backend.
- "Deploying is the end." → Deployment plus being able to explain the code is the real finish.

Suggested remedial exercises for learners who struggle:

1. **Persistence drill:** give a cart that saves but does not load, and ask the learner to fix `JSON.parse`.
2. **Filter drill:** give a search that is case-sensitive and ask them to fix it with `.toLowerCase()`.
3. **Trace drill:** give a `fetch` → `render` → `filter` chain and ask them to trace which function runs first.
4. **Explain-aloud drill:** have them teach one function to a partner before presenting to the class.

Differentiation:

- **Level 1 (supported):** Provide a working reference capstone with `products.json` and the cart pre-filled; the learner works on one or two features (e.g. render the grid, or fix validation) and presents those with support.
- **Level 2 (standard):** The full brief as written.
- **Level 3 (challenge):** Add a **product detail modal**, a **sort by price** control, a **separate** `cart.html` page, or a **"clear cart"** button; store prices in **kobo** to avoid rounding; or set up a custom `username.github.io` repo for a shorter URL.

Timing guidance. The 105 minutes are tight on presentation day. Suggested split: A–F (30 min), G (15 min), H (18 min), I (15 min, sprint + prepare), presentations and viva (as many learners as time allows, ~5 min each), then N and the certificate note (7 min). If the class is large, run presentations in **two parallel rooms** with a second assessor, or carry remaining presentations into a short follow-up slot. **Protect the live demo** — a learner who cannot present has an incomplete capstone. Mark deployment strictly: a missing live URL caps the deployment row at zero regardless of how good the code is.

If time runs short: cut Section J (case study) to a two-minute mention and Section M to a take-home review, and spend the time on presentations and fixing broken deployments.

COURSE-COMPLETION NOTE

Congratulations. Every learner who presents a working, deployed capstone and answers the viva has completed the Cyber Elias Academy **Web Development** course. Certificates are issued after the academy confirms: at least **10 of 12 sessions** attended, assignments completed, a passing capstone (**50% or more**, with the 30-mark capstone and the 5-mark viva/demo recorded), and a functional deployed app. Certificates can be verified at **cea.ng/certificates/verify**. Record each learner's **name, live URL, deliverables and signature** on the class sign-off sheet before they leave. This certificate recognises a **junior front-end beginner** — a strong foundation, and a clear path forward to backend, more JavaScript, and frameworks.

END OF SESSION 12

End of the Web Development course. From Zero to Expert, Together — Cyber Elias Academy, Port Harcourt.